University Name

Faculty of Computer Science and Engineering

Department of Artificial Intelligence

Financial Fraud Detection Using Multi-Paradigm Deep Learning: Graph Neural Networks, Spiking Neural Networks, and Unsupervised Autoencoders

Graduation Project

Student:

Supervisor:

Co-Supervisor:

Academic Year 2024–2025

Declaration of Originality

I hereby declare that this thesis, entitled *Financial Fraud Detection Using Multi-Paradigm Deep Learning: Graph Neural Networks, Spiking Neural Networks, and Unsupervised Autoencoders*, is the result of my own original work. This thesis has not been submitted for any degree or examination at any other university. All sources of information used have been properly acknowledged and referenced.

I confirm that the research presented in this thesis was conducted in accordance with the principles of research integrity and honesty. No part of this thesis has been previously published, except where explicit reference is made to the work of others.

Signature: _____

Name:

Date:

Abstract

Financial fraud detection remains a critical challenge in the banking and financial services industry, with fraudsters continuously evolving their tactics to circumvent traditional rule-based detection systems. This thesis presents a comprehensive multi-paradigm investigation of deep learning architectures for financial fraud detection, leveraging the complementary strengths of graph neural networks, spiking neural networks, and unsupervised autoencoders to identify fraudulent patterns that are invisible to conventional tabular approaches.

We develop FraudGraphBuilder, a specialized framework that transforms flat tabular applicant data into a rich heterogeneous graph comprising over one million nodes and 14.5 million edges across 17 distinct node types. This graph representation captures relational patterns such as shared devices, overlapping behavioral profiles, and coordinated temporal activity that are indicative of fraudulent networks.

Three complementary deep learning pipelines are designed, implemented, and systematically evaluated. The **GNN pipeline** investigates three graph neural network architectures: a GraphSAGE-based model employing neighborhood sampling and aggregation across three convolutional layers, achieving a test ROC AUC of 0.8859 with a true positive rate of 52.54% at 5% false positive rate; a Quantum Graph Neural Network (QGNN) introducing a hybrid quantum-classical paradigm with angle encoding and variational quantum circuit layers, achieving a peak AUC of 0.8769; and a Split GNN architecture introducing a novel graph-partitioning mechanism with an edge classifier and dual-branch Beta wavelet filtering, achieving a competitive AUC of up to 0.8806. A comprehensive 45-experiment hyperparameter tuning study for the Split GNN reveals that simpler spectral filtering ($C = 1$) consistently outperforms more complex alternatives, and that incorporating same-class edge information ($K = 0$) is essential for effective fraud detection.

The **SNN pipeline** explores the application of spiking neural networks to fraud detection, employing Leaky Integrate-and-Fire (LIF) neuron models with direct rate coding (Bernoulli sampling) for temporal feature representation. Unlike prior SNN approaches to fraud detection that employ population coding [23], our pipeline uses a one-to-one feature-to-spike-channel mapping, avoiding population expansion while maintaining competitive performance. Surrogate gradient descent enables effective training through non-differentiable spike thresholds, while ensemble methods combining multiple model configurations enhance detection stability and robustness. While SNNs offer theoretical advantages for event-driven computation on dedicated neuromorphic hardware, their software-based simulation introduces significant computational overhead that negates energy efficiency claims in software-only deployments, as discussed in Chapter 6.

The **Autoencoder pipeline** leverages unsupervised anomaly detection through two complementary architectures: a Variational Autoencoder (VAE) that models the

latent distribution of legitimate transactions using the evidence lower bound (ELBO) objective with the reparameterization trick, and a Denoising Autoencoder (DAE) that learns robust feature representations by reconstructing inputs corrupted with swap noise. Score-level stacking of VAE and DAE anomaly scores with gradient-boosted classifiers (XGBoost and LightGBM) produces a powerful ensemble that effectively separates fraudulent from legitimate patterns without requiring labeled training data.

Cross-version fairness analysis across six data distributions reveals significant performance variability (AUC range: 0.7587–0.9454 for GNN, 0.8101–0.9587 for SNN, 0.7893–0.9421 for AE), highlighting the critical importance of monitoring for distributional shift in production systems. The SNN pipeline demonstrates the highest stability across data versions with a coefficient of variation of only 5.53%, compared to 7.21% for GraphSAGE and 6.13% for the VAE-Stacked autoencoder. A controlled resampling experiment demonstrates that inverse-frequency class weighting outperforms ADASYN-based resampling strategies for graph-based fraud detection, providing better-calibrated probability outputs and superior overall performance, while synthetic oversampling within autoencoder training loops causes severe probability miscalibration and uniform performance drops [20]. Cross-pipeline comparison reveals complementary strengths: the GNN pipeline excels at exploiting relational structure, the SNN pipeline offers temporal processing advantages through spike-based feature representation and achieves the highest overall detection performance, and the Autoencoder pipeline provides robust anomaly detection without label dependence.

The contributions of this thesis include: (1) a robust graph construction pipeline for financial fraud data, (2) a systematic comparison of three distinct GNN paradigms, (3) the first application of direct rate-coded SNNs—using Bernoulli spike encoding with a one-to-one feature mapping rather than population coding—to financial fraud detection on the BAF benchmark, (4) a dual-autoencoder stacking framework with gradient-boosted meta-learners, (5) comprehensive hyperparameter sensitivity analysis for spectral graph filtering, (6) fairness and robustness evaluation across data distributions, and (7) a cross-pipeline comparison providing practical recommendations for production deployment of multi-paradigm fraud detection systems.

Keywords: Financial Fraud Detection, Graph Neural Networks, Spiking Neural Networks, Variational Autoencoder, Denoising Autoencoder, GraphSAGE, Quantum Machine Learning, Split GNN, Rate Coding, Ensemble Methods, Class Imbalance, Fairness Evaluation

Acknowledgment

First and foremost, I would like to express my sincere gratitude to my supervisor for their invaluable guidance, patience, and constructive feedback throughout this research. Their expertise and insights have been instrumental in shaping the direction and quality of this work.

I would also like to thank the faculty members of the Department of Artificial Intelligence for providing a stimulating academic environment and for their commitment to research excellence.

Special thanks to my colleagues and classmates for their support, collaboration, and stimulating discussions that enriched my understanding of graph neural networks, spiking neural networks, autoencoders, and financial fraud detection.

Finally, I am deeply grateful to my family for their unwavering support, encouragement, and understanding throughout this challenging journey.

Dedication

To my family, whose love and support made this possible.

Contents

List of Figures

List of Tables

List of Algorithms

List of Abbreviations and Acronyms

Abbreviation	Definition
ADASYN	Adaptive Synthetic Sampling
AUC	Area Under the Curve
AUC-PR	Area Under the Precision-Recall Curve
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BAF	Bank Account Fraud
BCE	Binary Cross-Entropy
BPTT	Backpropagation Through Time
DAE	Denoising Autoencoder
ELBO	Evidence Lower Bound
FPR	False Positive Rate
GAT	Graph Attention Network
GELU	Gaussian Error Linear Unit
GNN	Graph Neural Network
GP	Graph Partitioning
KL	Kullback-Leibler Divergence
LIF	Leaky Integrate-and-Fire
MLP	Multi-Layer Perceptron
QGNN	Quantum Graph Neural Network
QML	Quantum Machine Learning
ROC	Receiver Operating Characteristic
SAGEConv	SAGE Convolution Layer
SNN	Spiking Neural Network
TPR	True Positive Rate
VAE	Variational Autoencoder
VQC	Variational Quantum Circuit

Glossary

Term	Definition
Angle Encoding	A quantum feature encoding method that maps classical data to rotation angles of quantum gates
Barren Plateaus	A phenomenon in variational quantum circuits where the gradient vanishes exponentially with system size
Beta Wavelet	A spectral graph wavelet constructed from Beta polynomial coefficients for multi-scale graph filtering
Edge Classifier	A neural module that predicts whether a graph edge connects nodes of the same or different classes
Evidence Lower Bound	A lower bound on the log-likelihood of data used as the training objective for variational autoencoders, comprising reconstruction quality and KL divergence terms
Focal Loss	A loss function that down-weights well-classified examples to focus learning on hard samples
Graph Splitting	The process of partitioning graph edges into subsets based on learned edge semantics
Heterogeneous Graph	A graph containing multiple types of nodes and/or edges
Hinge Loss	A margin-based loss function used for training the edge classifier
Laplacian	A matrix representation of graph connectivity used in spectral graph theory
Latent Space	A lower-dimensional learned representation capturing essential data structure, used by autoencoders to encode input distributions
Membrane Potential	The internal state variable of a spiking neuron that accumulates incoming signals and triggers a spike when a threshold is reached
Message Passing	The mechanism by which GNN nodes exchange information with their neighbors
Over-smoothing	A phenomenon in deep GNNs where node embeddings become indistinguishable after many layers
Population Coding	A neural coding scheme where information is represented by the collective activity pattern of a group of neurons rather than a single neuron

Chapter 1

Introduction

1.1 Problem Statement

Financial fraud detection is a critical challenge facing the global banking and financial services industry. With the rise of digital banking and online transactions, fraudsters have developed increasingly sophisticated methods to circumvent traditional detection systems. According to industry estimates, financial fraud costs the global economy hundreds of billions of dollars annually [80], with credit card fraud alone accounting for over $30 billion in losses each year. The fundamental challenge lies in identifying the small fraction of fraudulent transactions (typically less than 1% of all transactions) among millions of legitimate ones, a problem characterized by extreme class imbalance, evolving fraud patterns, and the need for real-time detection with minimal false alarms.

Traditional fraud detection systems rely on rule-based approaches and tabular machine learning models—which, despite strong performance on structured data [28]—that treat each transaction independently. These methods fail to capture the relational patterns that are often the strongest indicators of organized fraud [69]: shared devices between multiple applicants, coordinated application bursts from the same geographic area, and overlapping behavioral profiles that emerge only when the connections between entities are examined. Moreover, even the best classical machine learning models exhibit misleadingly high accuracy metrics that mask devastatingly poor fraud detection: Ahmed and Shamsuddin [2] reported decision tree accuracy of 97.17% on banking fraud data, yet Dong [20] demonstrated that Random Forest achieves only 0.16% fraud recall despite 98.93% overall accuracy, confirming that conventional metrics are inadequate under extreme class imbalance. Single-paradigm deep learning approaches—whether based solely on graph neural networks, standard feedforward architectures, or isolated anomaly detection models—suffer from complementary limitations. Purely graph-based methods depend heavily on graph construction quality and struggle with temporal dynamics [34]; standard neural networks lack relational reasoning; and unsupervised anomaly detectors cannot exploit labeled fraud patterns effectively. Standalone autoencoders achieve near-random performance using reconstruction error alone, with Dong [20] reporting a DAE AUC of only 0.5210 without supervised stacking. Spiking neural networks face calibration instability and temporal generalization failure on distributional shift scenarios [3]. No single learning paradigm fully addresses the multifaceted nature of financial fraud, motivating the exploration of a multi-paradigm approach that combines the strengths of graph-based, neuromorphic, and unsupervised deep learning.

1.2 Motivation and Significance

The motivation for this research stems from five key observations. First, financial fraud is inherently relational: fraud rings operate through shared resources (devices, IP addresses, email domains) that create detectable network patterns when represented as a graph. Second, graph neural networks (GNNs) have demonstrated state-of-the-art performance in domains where relational structure is informative [14], [69], including social network analysis, drug discovery, and recommendation systems. Third, the application of GNNs to financial fraud detection is still an emerging field, with significant gaps in understanding the relative strengths of different GNN architectures, the impact of graph construction strategies, and the robustness of these models under distributional shift. Fourth, spiking neural networks (SNNs) offer a biologically inspired computing paradigm with inherent advantages for temporal pattern recognition and event-driven processing [3], [54], yet their application to financial fraud detection remains virtually unexplored and faces calibration challenges under extreme class imbalance. Fifth, unsupervised autoencoders—particularly Variational Autoencoders (VAEs) [42] and Denoising Autoencoders (DAEs) [92]—provide a powerful anomaly detection framework that does not require labeled fraud examples, making them highly suitable for domains where fraud labels are scarce or delayed. However, standalone autoencoders achieve near-random anomaly detection performance on highly imbalanced tabular data [20], [62], necessitating hybrid stacking approaches to achieve competitive results.

The significance of this work lies in its potential to advance both the theoretical understanding and practical deployment of multi-paradigm fraud detection systems. By systematically comparing and combining three fundamentally different deep learning paradigms—graph-based relational reasoning (GNN), neuromorphic temporal processing (SNN), and unsupervised anomaly detection (Autoencoder)—this thesis provides actionable insights for practitioners seeking to deploy robust, multi-faceted fraud detection in production environments. Each paradigm addresses a distinct aspect of the fraud detection challenge: GNNs exploit relational structure, SNNs capture temporal dynamics through spike-based processing, and autoencoders enable label-independent anomaly scoring.

1.3 Research Gaps, Questions, and Objectives

1.3.1 Research Gaps

Despite the growing interest in deep learning-based fraud detection, several critical gaps remain in the literature:

1. **Graph construction methodology:** Existing works often treat graph construction as a preprocessing afterthought, without systematic analysis of how node type selection, feature bucketing strategies, and edge construction policies affect downstream model performance [36].

2. **GNN architecture comparison:** Most studies evaluate a single GNN architecture without systematic comparison across fundamentally different paradigms (spatial vs. quantum-inspired vs. spectral approaches).

3. **Hyperparameter sensitivity:** The sensitivity of spectral GNN architectures to hyperparameters such as wavelet order and graph splitting configurations is poorly understood.

4. **Fairness and robustness:** The performance stability of GNN-based fraud detectors under distributional shift and across different data versions has received limited attention [15], [44].

5. **Class imbalance handling:** The effectiveness of resampling strategies [12], [31] in the context of graph-based learning, where neighborhood structure is critical, remains unexplored.

6. **SNNs for fraud detection:** The application of spiking neural networks to financial fraud detection is virtually unexplored [54]. Key questions remain regarding the suitability of LIF neuron models, rate coding strategies, surrogate gradient descent training, and ensemble methods for the extreme class imbalance and high-dimensional feature spaces characteristic of fraud data.

7. **Autoencoder ensembles for fraud:** While individual autoencoder architectures (VAE [42], DAE [92]) have been applied to anomaly detection, standalone autoencoders achieve near-random performance on imbalanced tabular fraud data, with DAEs reporting AUC as low as 0.5210 [20] and standard autoencoders achieving AUC of approximately 0.55 [62]. The systematic combination of complementary autoencoder paradigms through score-level stacking with gradient-boosted meta-learners [13], [41] has not been fully investigated for financial fraud, and the resampling paradox—where synthetic oversampling degrades autoencoder performance—requires further investigation [20]. The optimal design of swap noise corruption, latent space dimensionality, and evidence lower bound optimization [23] for fraud-specific anomaly detection remains open.

8. **Cross-paradigm comparison:** No existing work systematically compares GNN, SNN, and autoencoder-based approaches on the same fraud detection task [14], [69], limiting practitioners' ability to select or combine paradigms based on their complementary strengths and weaknesses.

1.3.2 Research Questions

RQ1. How can tabular financial fraud data be effectively transformed into a heterogeneous graph representation that captures relational fraud patterns?

RQ2. Which GNN architecture paradigm (spatial aggregation, quantum-inspired, or spectral splitting) provides the best trade-off between fraud detection performance and computational efficiency?

RQ3. How sensitive is the Split GNN architecture to its key hyperparameters (wavelet order C, split point K, edge loss weight γ_{edge}), and what is the optimal configuration?

RQ4. How robust are GNN-based fraud detectors to distributional shifts across different data versions, and what are the failure modes?

RQ5. Does resampling-based class balancing improve graph-based fraud detection compared to loss-based class weighting [23]?

RQ6. Can spiking neural networks with Leaky Integrate-and-Fire neurons and rate coding achieve competitive fraud detection performance [54], and how does surrogate gradient descent training perform under extreme class imbalance?

RQ7. How do Variational Autoencoders [42] and Denoising Autoencoders [92] compare as unsupervised anomaly detectors for fraud, and does score-level stacking with XGBoost/LightGBM [13], [41] improve detection over individual models?

RQ8. What are the complementary strengths and weaknesses of the GNN, SNN, and Autoencoder pipelines, and how can they be effectively combined or selected for production deployment?

1.3.3 Research Objectives

O1. Design and implement a graph construction pipeline (FraudGraphBuilder) that transforms tabular fraud data into a heterogeneous graph with 17 node types.

O2. Implement and evaluate three GNN architectures: GraphSAGE, Quantum GNN, and Split GNN.

O3. Conduct a comprehensive 45-experiment hyperparameter tuning study for the Split GNN.

O4. Evaluate model fairness and robustness across six data distribution variants.

O5. Compare class weighting and resampling strategies for handling extreme class imbalance in graph-based fraud detection.

O6. Design and implement an SNN pipeline using Leaky Integrate-and-Fire neurons, rate coding, and surrogate gradient descent, and evaluate ensemble methods for improving SNN-based fraud detection.

O7. Design and implement an Autoencoder pipeline combining a Variational Autoencoder (VAE) and a Denoising Autoencoder (DAE) with score-level stacking using XGBoost and LightGBM meta-learners.

O8. Conduct a systematic cross-pipeline comparison of GNN, SNN, and Autoencoder approaches, identifying complementary strengths and providing practical deployment recommendations.

1.4 Proposed Solution Overview

This thesis proposes a comprehensive multi-paradigm fraud detection framework that addresses the identified research gaps. The framework consists of five interconnected components:

Graph Construction: The FraudGraphBuilder transforms flat tabular applicant data into a heterogeneous graph with 17 distinct node types (categorical, bucketed

continuous, and binary/flag), creating shared attribute nodes that enable the detection of relational fraud patterns through indirect user connections.

GNN Pipeline: Three GNN architectures are implemented and compared [14]. GraphSAGE provides a strong classical baseline through neighborhood sampling and aggregation. The Quantum GNN explores quantum-inspired feature transformations through angle encoding and variational quantum circuits. The Split GNN introduces a novel graph-partitioning mechanism that combines edge classification with dual-branch Beta wavelet filtering for frequency-domain feature extraction. A 45-experiment grid search over wavelet order ($C \in \{1, 2, 3\}$), split point ($K \in \{-1, 0, 1, 2\}$), and edge loss weight ($\gamma_{\text{edge}} \in \{0.2, 0.4, 0.6, 0.8, 1.0\}$) reveals the sensitivity and optimal configuration of the Split GNN architecture.

SNN Pipeline: A spiking neural network pipeline employs Leaky Integrate-and-Fire (LIF) neurons with rate coding to process tabular fraud features as temporal spike trains [54]. Surrogate gradient descent enables effective backpropagation through the non-differentiable spike threshold, while ensemble methods combining multiple SNN configurations enhance detection stability. The SNN paradigm offers inherent advantages for event-driven transaction processing on neuromorphic hardware, though software-based simulation introduces computational overhead.

Autoencoder Pipeline: An unsupervised anomaly detection pipeline combines a Variational Autoencoder (VAE) [42], which models the latent distribution of legitimate transactions using the ELBO objective with the reparameterization trick, and a Denoising Autoencoder (DAE) [92], which learns robust representations by reconstructing inputs corrupted with swap noise. Score-level stacking of VAE and DAE anomaly scores with gradient-boosted classifiers (XGBoost [13] and LightGBM [41]) produces a powerful ensemble that effectively separates fraudulent from legitimate patterns without requiring labeled training data.

Cross-Pipeline Comparison and Robustness Evaluation: Cross-version validation across six data distributions quantifies model stability under distributional shift, a controlled resampling experiment evaluates the effectiveness of ADASYN-based class balancing [31] versus inverse-frequency weighting, and a systematic cross-pipeline comparison identifies the complementary strengths and weaknesses of each paradigm for practical deployment.

1.5 Thesis Organisation

The remainder of this thesis is organised as follows:

Chapter 2 presents a comprehensive literature review covering the domain background of financial fraud detection, the state-of-the-art in software solutions, a detailed survey of traditional, classical machine learning, graph neural network, spiking neural network, and autoencoder approaches, a comparative analysis across paradigms, and the identification of research gaps that motivate this work.

Chapter 3 presents the analysis and requirements engineering, including stakeholder analysis, functional and non-functional requirements, and data requirements for the proposed system.

Chapter 4 describes the system architecture and design, including the high-level system architecture, component design, the AI pipeline architecture, and the technology stack selection.

Chapter 3 details the ML methodology and implementation, covering data preprocessing techniques, model selection and justification, feature engineering and selection, hyperparameter optimisation strategy, and software implementation details.

Chapter 5 presents the GNN pipeline, including the design and evaluation of GraphSAGE, Quantum GNN, and Split GNN architectures, hyperparameter sensitivity analysis, and cross-version fairness evaluation.

Chapter 6 presents the SNN pipeline, covering the design and implementation of Leaky Integrate-and-Fire neurons, rate coding strategies, surrogate gradient descent training, and ensemble methods for spiking neural network-based fraud detection.

Chapter 7 presents the Autoencoder pipeline, including the design and evaluation of Variational Autoencoder and Denoising Autoencoder models, anomaly score computation, and score-level stacking with XGBoost/LightGBM meta-learners.

Chapter 8 provides a cross-pipeline comparison, analysing the complementary strengths and weaknesses of the GNN, SNN, and Autoencoder approaches, and offering practical recommendations for multi-paradigm deployment.

Chapter 9 concludes the thesis with a summary of contributions, critical reflection, and recommendations for future research.

Appendices provide the AI Ethics & Bias Statement, Reproducibility Report, Generative AI Disclosure, Extended Hyperparameter Configurations, Denoising Autoencoder Architecture and Objective Formulations, Spiking Neural Network Mathematical Derivations, and Graph Neural Network Algorithmic Procedures.

Chapter 2

Literature Review

This chapter provides a comprehensive survey of the literature relevant to financial fraud detection using multi-paradigm deep learning. We begin by establishing the domain background, including the nature of financial fraud, the characteristics of benchmark datasets, and the fairness considerations that motivate responsible model design. We then review the state-of-the-art in software solutions and frameworks that support fraud detection research. The core of the chapter presents a detailed survey of approaches, spanning traditional rule-based systems, classical machine learning, graph neural networks, spiking neural networks, and autoencoder-based methods. For each paradigm, we examine the foundational theories, architectural innovations, and empirical findings with specific attention to performance on the Bank Account Fraud (BAF) benchmark suite. Finally, we identify five critical research gaps that this thesis addresses, demonstrating the need for a unified multi-paradigm comparison under controlled experimental conditions and providing a roadmap for the contributions that follow in subsequent chapters.

2.1 Domain Background

Financial fraud constitutes one of the most pervasive and economically damaging categories of cybercrime in the modern digital economy. The scope of financial fraud is vast, encompassing credit card fraud, identity theft, insurance fraud, money laundering, and securities manipulation, among other illicit activities. In the context of credit card fraud alone, global losses exceed $30 billion annually, a figure that continues to grow as digital payment systems proliferate and fraudsters adopt increasingly sophisticated techniques [80]. Credit card application fraud—the specific focus of this thesis—occurs when criminals submit applications using stolen or synthetic identities to obtain credit facilities, which are then exploited for immediate financial gain before the fraud is detected. The economic impact of application fraud is particularly severe because each successful fraudulent application can result in losses ranging from thousands to tens of thousands of dollars, and organized fraud rings may submit hundreds of coordinated applications simultaneously [37].

The challenge of detecting financial fraud is compounded by several interrelated factors that distinguish it from conventional classification problems. First, the extremely low base rate of fraud means that even highly accurate classifiers can produce an overwhelming number of false positives when deployed at scale, overwhelming human investigators and eroding confidence in the detection system. Second, fraud patterns

are inherently adversarial: as detection systems evolve, fraudsters adapt their strategies to circumvent new defenses, creating a continuous arms race between attackers and defenders that renders static models obsolete over time. Third, the cost asymmetry between false positives and false negatives is severe: false positives inconvenience legitimate customers and erode institutional trust, while false negatives result in direct financial losses that may be irreversible once the fraudulent activity is discovered. Fourth, the temporal nature of financial data introduces distributional shift, as the statistical properties of both legitimate and fraudulent applications evolve over time due to changes in customer behavior, economic conditions, and fraudster tactics. These factors collectively make fraud detection one of the most challenging problems in applied machine learning [4].

The detection of credit card application fraud presents unique challenges even within the broader domain of financial fraud detection. Unlike transaction-level fraud, where each transaction can be evaluated in the context of a known customer's historical behavior, application fraud involves making decisions about previously unknown entities with limited historical data. The absence of a behavioral baseline for new applicants means that detection must rely on the intrinsic properties of the application itself and its relationships to other applications and entities. This relational aspect—the fact that fraudulent applications often share devices, IP addresses, email domains, and other attributes with other fraudulent applications—is the key insight that motivates graph-based approaches to fraud detection, which we explore in detail in Section 2.

2.1.1 The Bank Account Fraud Dataset Suite

The Bank Account Fraud (BAF) dataset suite [24] has emerged as the primary benchmark for evaluating fraud detection methods under realistic and challenging conditions. Developed by Feedzai and released in 2022, the BAF suite was designed to address the limitations of earlier fraud detection benchmarks, which often used synthetic data, lacked temporal structure, or failed to represent the extreme class imbalance characteristic of real-world fraud detection. The base version of the dataset contains approximately one million application instances, with a fraud prevalence of 1.10%, yielding a class imbalance ratio of approximately 89:1. This extreme imbalance is representative of real-world fraud detection scenarios and poses significant challenges for standard classification algorithms that assume balanced class distributions. The BAF dataset is derived from real-world credit card application data that has been anonymized and preprocessed to protect sensitive information while preserving the statistical properties relevant to fraud detection research.

A critical feature of the BAF dataset is its temporal structure, which is essential for realistic evaluation of fraud detection models. The data is partitioned into training (months 0–4), validation (month 5), and test (months 6–7) splits, reflecting the temporal reality of fraud detection deployments where models must generalize to future data that may exhibit distributional shift. This temporal split design stands in sharp contrast to the random stratified splits commonly used in machine learning research, which tend to overestimate model performance by allowing information leakage between temporally adjacent observations [89]. The temporal separation ensures that models are evaluated on their ability to detect fraud patterns that may have shifted from the training period, a scenario that is practically inevitable given the adversarial

evolution of fraud strategies and the natural drift in customer application patterns over time.

The BAF suite comprises six distinct versions, each designed to test a specific challenge relevant to real-world fraud detection. The **Base** version provides the standard evaluation setting with the full feature set and natural class distribution, serving as the primary benchmark for comparing detection methods. **Variant I** (Group Size Disparity) introduces significant differences in the number of applications across demographic groups, testing fairness under population imbalance and challenging models to maintain equitable performance when some groups are severely underrepresented. **Variant II** (Prevalence Disparity) manipulates the fraud rate across demographic groups, creating a scenario where the base rate differs substantially across protected attributes and directly testing the Chouldechova impossibility result [15]. **Variant III** (Prevalence Shift) introduces a temporal shift in the overall fraud rate between training and test periods, challenging models to adapt to changing class priors without retraining. **Variant IV** (Temporal Prevalence Shift) combines temporal drift with group-specific prevalence changes, representing the most realistic and challenging scenario where both the overall and group-conditional fraud rates evolve over time. **Variant V** (Temporal Separability Shift) introduces a temporal shift in the separability of legitimate and fraudulent applications, testing whether models can maintain detection performance when the feature distributions of the two classes become more similar over time—a particularly challenging form of distributional shift that reduces the signal available for discrimination. This comprehensive suite of variants enables systematic investigation of model behavior under a range of realistic distributional challenges that are rarely addressed in the fraud detection literature.

The BAF dataset features comprise 30 attributes, which can be decomposed into 17 continuous or numerical features, 5 categorical features, and 13 binary flags or indicator variables. The continuous features capture quantitative application properties such as income, credit history metrics, and application timing, and require appropriate normalization or scaling before use in neural network architectures. The categorical features encode discrete attributes such as employment status and housing type, and must be encoded through techniques such as one-hot encoding or embedding layers to be processed by deep learning models. The binary flags indicate the presence or absence of specific risk indicators, and their sparse, high-dimensional nature presents challenges for models that assume dense feature representations. This heterogeneous feature space presents challenges for models that assume homogeneous input distributions, motivating the exploration of architectures capable of processing mixed data types effectively. The combination of continuous, categorical, and binary features in a single dataset reflects the complexity of real-world financial data and makes the BAF suite a particularly demanding benchmark for deep learning methods that are typically optimized for homogeneous input types.

2.1.2 Fairness Considerations in Fraud Detection

The deployment of fraud detection systems raises important fairness concerns, as models may exhibit disparate performance across demographic groups defined by protected attributes such as age, gender, or ethnicity. Chouldechova [15] demonstrated the impossibility of simultaneously achieving equalized odds (equal true positive and false positive rates across groups) and calibration (predictive parity) when base rates differ

across groups, a condition that is virtually guaranteed in fraud detection due to the varying prevalence of fraud across demographic segments. This impossibility result has profound implications: any fraud detection system will necessarily exhibit some form of unfairness, and the choice of which fairness metric to optimize involves normative judgments about the relative costs of different types of errors across groups. The practical consequence is that fraud detection system designers must make explicit trade-offs between different fairness criteria, and these trade-offs should be informed by domain-specific considerations about the relative harms of different error types.

Empirical studies on the BAF dataset have revealed true positive rate (TPR) gaps of up to 5.5 percentage points across age groups, indicating that certain demographic segments are systematically less well-served by standard fraud detection models [44]. These disparities persist even when models are trained without explicit access to protected attributes, as proxy features correlated with group membership can encode discriminatory patterns that the model learns to exploit. The persistence of fairness disparities in nominally "blind" models underscores the difficulty of achieving equitable outcomes through data omission alone and motivates the development of fairness- aware training methods that explicitly constrain model behavior across demographic groups. Dai et al. [17] have proposed fairness-aware training objectives for graph-based fraud detection, demonstrating that it is possible to reduce TPR disparities without substantial degradation in overall detection performance. However, the interplay between fairness constraints and the architectural innovations explored in this thesis—including graph neural networks, spiking neural networks, and autoencoder-based methods—remains largely unexplored. The tension between detection performance and fairness underscores the need for multi-metric evaluation frameworks that go beyond aggregate AUC to assess model behavior across demographic subgroups, a principle that guides the evaluation methodology of this thesis.

2.2 State-of-the-Art in Software Solutions

Modern financial fraud detection systems increasingly incorporate advanced analytics alongside traditional tabular processing methods, driven by the recognition that fraud patterns exploit relational structures that are invisible to feature-based classifiers. Commercial platforms such as Featurespace, ThetaRay, and DataVisor have developed proprietary systems that leverage network analysis and unsupervised anomaly detection for real-time fraud screening, though the specific algorithms and architectures employed are often not publicly disclosed. The commercial landscape reflects a growing consensus that single- paradigm approaches are insufficient for the complexity of modern fraud, and that effective detection requires the integration of multiple analytical perspectives. In the academic and open-source domain, several notable frameworks and benchmarks have emerged that facilitate reproducible fraud detection research and enable systematic comparison of competing approaches.

The IEEE CIS Fraud Detection competition hosted on Kaggle popularized the use of rich tabular features for fraud detection, generating a large corpus of comparative studies on gradient-boosted methods and establishing important baselines for the field. However, the competition dataset lacked explicit relational structure, limiting the exploration of graph-based approaches and focusing research attention on feature engineering and ensemble methods for tabular data. More recently, graph-based benchmarks such as the Anti-Money Laundering (AML) dataset and the BAF suite [24]

have provided transaction-level data with entity relationships, enabling the systematic evaluation of graph-based methods. Arulkumaran et al. [5] proposed BankGuard, a framework integrating graph analytics with traditional fraud detection pipelines, demonstrating the practical value of combining relational and feature-based reasoning in production systems and providing a template for the multi-pipeline architecture explored in this thesis.

For graph neural network research, PyTorch Geometric (PyG) provides efficient implementations of various GNN layers including SAGEConv, GATConv, and general message-passing abstractions that support custom aggregation schemes and heterogeneous graph processing. The Deep Graph Library (DGL) offers an alternative framework with native support for heterogeneous graphs, which is particularly relevant for financial fraud detection where nodes of different types (applicants, devices, IP addresses) interact through typed edges. For quantum machine learning, PennyLane and Qiskit provide simulators for variational quantum circuits that can be integrated with classical neural network components, enabling research on quantum-inspired architectures without requiring quantum hardware. For spiking neural networks, frameworks such as Norse and snnTorch [22] provide differentiable Leaky Integrate-and-Fire (LIF) neuron implementations compatible with PyTorch, enabling gradient-based training through surrogate gradient methods and providing the computational infrastructure upon which the SNN pipeline of this thesis is built. For autoencoder-based anomaly detection, PyTorch and TensorFlow provide flexible architectures for building VAEs and DAEs, while XGBoost [13] and LightGBM [41] offer efficient gradient-boosted decision tree implementations that serve as meta- learners for score-level stacking in hybrid detection pipelines. The availability of these mature, well-documented frameworks has substantially lowered the barrier to entry for multi-paradigm fraud detection research and enables the systematic comparison conducted in this thesis.

2.3 Survey of Approaches

This section presents a detailed survey of approaches to financial fraud detection, organized by paradigm. We progress from traditional rule-based systems through classical machine learning to the three deep learning paradigms central to this thesis: graph neural networks, spiking neural networks, and autoencoder-based methods. For each paradigm, we review the foundational methods, recent advances, and specific applications to financial fraud detection, with emphasis on performance characteristics on the BAF dataset where available. This progressive organization reflects the historical development of the field while establishing the context for the multi-paradigm comparison that is the central contribution of this thesis.

2.3.1 Traditional and Rule-Based Approaches

Traditional fraud detection systems rely on expert-defined rules, heuristic thresholds, and pattern-matching techniques to identify suspicious applications. These systems operate by encoding domain knowledge into explicit decision criteria, such as flagging applications originating from high-risk geographic regions, those submitted within unusual time windows, or those associated with known fraudulent devices. Rule-based systems offer several practical advantages that have sustained their use in the financial industry for decades: they are fully interpretable, allowing compliance officers and

regulators to understand exactly why an application was flagged; they can be rapidly deployed and modified in response to emerging fraud patterns without requiring retraining; and they require no training data, making them immediately applicable in cold- start scenarios where historical fraud data is unavailable [4]. In the early days of digital banking, rule- based systems formed the backbone of fraud detection infrastructure and continue to serve as a first line of defense in many financial institutions, particularly for detecting obvious fraud patterns that conform to known templates.

However, rule-based approaches suffer from fundamental limitations that constrain their effectiveness in modern fraud detection environments. First, they are inherently reactive, as rules must be crafted in response to previously observed fraud patterns, leaving them vulnerable to novel attack strategies that do not match existing rule templates. Sophisticated fraud rings deliberately vary their tactics to avoid triggering known rules, employing techniques such as device rotation, geographic distribution, and temporal spacing to evade detection. Second, the maintenance burden grows combinatorially as the number of rules increases, leading to rule conflicts, redundancy, and inconsistency that are difficult to diagnose and resolve. Large financial institutions may maintain thousands of fraud detection rules, many of which interact in complex and unpredictable ways. Third, rule-based systems lack the statistical rigor to handle the probabilistic nature of fraud signals, often producing binary decisions that fail to account for the uncertainty inherent in fraud assessment. A rule that flags all applications from a particular IP range, for example, makes no distinction between high-confidence and low-confidence signals within that range. Fourth, sophisticated fraudsters can systematically probe and circumvent rule-based defenses through trial-and-error attacks, as the deterministic nature of rules makes them predictable once their logic is inferred. These limitations have motivated the transition toward data-driven machine learning approaches that can learn complex, non-linear decision boundaries from historical data while maintaining the flexibility to adapt to evolving fraud patterns.

Despite their limitations, rule-based systems continue to play a complementary role in modern fraud detection architectures. Hybrid systems that combine rule- based pre-screening with machine learning scoring can leverage the strengths of both approaches: rules provide fast, interpretable filtering for obvious cases, while machine learning models handle the ambiguous cases that require nuanced statistical reasoning. This hybrid philosophy extends to the multi-paradigm approach explored in this thesis, where different models may be deployed in different stages of a detection pipeline, each contributing its unique strengths to the overall detection capability. The persistence of rule-based systems in production environments also serves as a reminder that practical fraud detection requires not only statistical power but also interpretability, maintainability, and regulatory compliance—considerations that influence the design choices of all paradigms examined in this thesis.

2.3.2 Classical Machine Learning Approaches

Classical machine learning methods—including logistic regression, decision trees, random forests, and gradient-boosted models—have been the workhorses of fraud detection for over two decades. These methods operate on tabular feature representations of individual applications, treating each application as an independent observation and learning decision boundaries that separate legitimate from fraudulent instances based on feature values. The extensive literature on classical approaches provides impor-

tant baselines and insights that inform the design of more sophisticated deep learning methods, and the performance of classical models on the BAF dataset establishes the benchmark against which the deep learning paradigms explored in this thesis must be measured. The enduring popularity of classical methods in production fraud detection systems reflects their robustness, interpretability, and strong performance on tabular data, properties that have been extensively documented in the machine learning literature [28].

Performance of Classical Models on BAF

Ahmed and Shamsuddin [2] conducted a comparative study of classical classifiers on the BAF dataset, reporting accuracy values of 87.91% for logistic regression, 90.78% for random forest, and 97.17% for decision trees. While these accuracy figures appear impressive at first glance, they are profoundly misleading due to the extreme class imbalance: a naive classifier that labels all instances as legitimate would achieve approximately 98.9% accuracy on the BAF base dataset, rendering accuracy an inappropriate and deceptive metric for fraud detection evaluation. The high accuracy of the decision tree in particular reflects overfitting to the majority class rather than genuine fraud detection capability, as the tree learns to predict the majority class for most leaf nodes while creating deep, spurious branches that memorize noise in the minority class. This observation underscores the critical importance of using class-sensitive metrics such as area under the receiver operating characteristic curve (AUC), true positive rate at fixed false positive rate (TPR@FPR), and precision-recall curves when evaluating fraud detection models, a principle that is consistently applied throughout this thesis.

Islam et al. [36] provided a more nuanced evaluation of classical methods, reporting that gradient-boosted models (GBM) achieve an AUC of 0.98 on the BAF dataset, but with a true positive rate below 50% at operationally relevant false positive rates. This striking disparity between AUC and TPR highlights a crucial practical consideration: while the AUC summarizes detection performance across all possible decision thresholds, operational fraud detection systems must operate at very low false positive rates (typically 1– 5%) to avoid overwhelming human investigators with false alerts and degrading the customer experience for legitimate applicants. At these low FPR operating points, the detection rate of even the best classical models can be alarmingly low, leaving a substantial proportion of fraudulent applications undetected. This finding motivates the exploration of deep learning approaches that may capture more discriminative patterns, particularly relational patterns that are invisible to tabular classifiers.

Dong [20] further demonstrated the limitations of classical approaches in a comprehensive evaluation, showing that a random forest model achieving 98.93% accuracy on a fraud detection task obtained a fraud recall of only 0.16%, meaning that the model detected fewer than 2 in 1,000 fraudulent applications. This catastrophic failure at the minority class illustrates the fundamental challenge of class imbalance for classical methods and the inadequacy of accuracy as an evaluation metric. The results of Dong's study also highlighted that tree-based ensembles, while generally more robust to imbalance than single decision trees due to their bagging and boosting mechanisms, still require explicit imbalance handling strategies to achieve acceptable fraud detection rates. The consistent pattern across these studies—high accuracy masking catastrophic minority-class performance—underscores the need for both improved

evaluation methodology and more powerful detection architectures.

Class Imbalance Handling Strategies

The extreme class imbalance characteristic of fraud detection datasets has motivated the development and application of various resampling and loss- modification strategies, each with distinct mechanisms and trade-offs. The most widely used resampling methods include SMOTE [12], which generates synthetic minority class examples by interpolating between existing minority instances and their nearest neighbors in feature space, and ADASYN [31], which adaptively focuses synthetic sample generation on harder-to-learn minority instances based on their local class distribution. Loss-based approaches include class weighting [11], which assigns higher misclassification costs to minority class examples to shift the decision boundary toward better minority-class coverage, and focal loss [45], which dynamically down-weights the loss contribution of well-classified easy examples to focus training on hard, misclassified instances that are typically concentrated near the decision boundary. Each of these strategies modifies the effective training distribution in different ways, and their interaction with different model architectures is an active area of research.

Uwaoma [90] conducted a systematic evaluation of five sampling strategies (SMOTE, ADASYN, random oversampling, random undersampling, and no sampling) across four model architectures on the BAF dataset, providing the most comprehensive assessment of imbalance handling strategies for this benchmark to date. The study revealed that the effectiveness of resampling strategies is highly architecture-dependent: while resampling can improve the performance of classical models by providing a more balanced training signal, it can paradoxically degrade the performance of graph-based methods. Specifically, ADASYN was found to reduce GraphSAGE AUC from 0.8859 to 0.8748, a counterintuitive result that highlights the tension between modifying the label distribution and preserving the relational structure upon which GNNs depend. When synthetic minority instances are added to the training set, they are assigned feature vectors that may not correspond to realistic nodes in the graph, creating artificial neighborhoods that distort the topology and confuse the message-passing mechanism. This "resampling paradox" has significant implications for the design of multi-paradigm systems, as a preprocessing strategy that benefits one model type may actively harm another.

Farahat et al. [23] extended the analysis of imbalance handling by proposing objective-aware evaluation frameworks that explicitly account for the operational cost structure of fraud detection, arguing that standard metrics such as AUC fail to capture the asymmetric costs of false positives and false negatives in deployment scenarios. Their work motivates the use of cost-sensitive evaluation metrics such as TPR@5%FPR, which directly measures the detection rate at an operationally acceptable false positive rate, providing a more realistic assessment of model utility. Parkar et al. [65] provided a broader comparative analysis of preprocessing strategies for fraud detection, confirming that no single resampling strategy consistently outperforms others across all model architectures and evaluation metrics, further emphasizing the need for paradigm- specific tuning and the careful selection of imbalance handling strategies based on the model architecture and evaluation criteria.

2.3.3 Graph Neural Network Approaches

Graph neural networks (GNNs) have emerged as a powerful paradigm for financial fraud detection, motivated by the insight that financial fraud is inherently relational [69]. Fraud rings share resources—devices, IP addresses, email domains, and phone numbers—creating network patterns that become detectable when applicants and their attributes are represented as interconnected nodes in a graph. This relational perspective transforms the fraud detection problem from a classification task on isolated feature vectors to a node classification task on a graph, where the topology itself carries diagnostic information that is invisible to tabular methods. Recent surveys by Cheng et al. [14] and Vanderhulst et al. [34] have documented the rapid growth of GNN-based fraud detection methods, highlighting both the promise and the open challenges of this approach. The GNN paradigm encompasses a diverse family of architectures, each with distinct mechanisms for aggregating neighborhood information and different assumptions about the graph structure, motivating the systematic comparison conducted in this thesis.

Graph Construction for Financial Data

The construction of the graph representation is a critical preprocessing step that significantly impacts downstream GNN performance and is often overlooked in comparative studies that focus exclusively on model architecture. For the BAF dataset, the graph is constructed as a heterogeneous network with approximately 1,000,047 nodes and roughly 14.5 million directed edges spanning 17 attribute types. Nodes represent both applicants and their associated attributes (devices, IP addresses, email domains, phone numbers, etc.), while edges encode the relationships between applicants and their shared attributes. This bipartite- like structure means that applicants are connected indirectly through shared attribute nodes, and the resulting graph topology encodes the relational patterns that are diagnostic of organized fraud. The scale and heterogeneity of this graph present computational challenges for GNN training, including memory- efficient neighborhood sampling, mini-batch construction, and handling of variable-degree nodes.

The design choices involved in graph construction have received increasing attention in the literature, as researchers have recognized that the graph representation can be as important as the model architecture in determining detection performance. Liu et al. [48] demonstrated that heterogeneous graph representations, which preserve the type information of nodes and edges, consistently outperform homogeneous graphs that collapse all node and edge types into a single category. The type information allows heterogeneous GNNs to apply type-specific transformations to different edge categories, enabling the model to distinguish between, for example, a device- sharing connection and an email-sharing connection, which may carry very different fraud implications. Pandey et al. [64] investigated the impact of feature discretization on graph construction, showing that continuous features must be binned into discrete categories to serve as attribute nodes, and that the granularity of discretization affects both graph density and detection performance. Coarse discretization produces dense graphs with many shared attribute nodes but limited discriminative power, while fine discretization produces sparse graphs that may not provide sufficient connectivity for effective message passing. The interaction between graph construction strategies and downstream GNN architectures remains an active area of research, as different architectures may benefit

from different graph representations, and understanding the sensitivity of detection performance to graph construction choices is essential for the practical deployment of GNN- based fraud detection systems.

Spatial GNN Architectures: GraphSAGE

GraphSAGE [29] introduced the inductive representation learning paradigm for large-scale graphs, generating node embeddings by sampling and aggregating features from a node's local neighborhood. Unlike transductive methods such as DeepWalk and node2vec, which require retraining when new nodes are added to the graph, GraphSAGE learns an aggregation function that generalizes to unseen nodes, making it well-suited for the dynamic nature of financial fraud detection where new applications arrive continuously and must be classified without retraining the entire model. The original GraphSAGE paper proposed three aggregation schemes—mean, LSTM, and pooling—each providing different trade-offs between expressiveness and computational cost. The mean aggregator computes the element-wise average of neighbor representations, providing a simple and robust baseline. The LSTM aggregator applies a recurrent network to a randomly permuted sequence of neighbor features, capturing sequential dependencies at the cost of order sensitivity. The pooling aggregator applies a max-pooling operation across neighbor features, capturing the most salient dimensions of the neighborhood representation.

In the fraud detection domain, GraphSAGE has been applied to capture neighborhood-level fraud signals through multi-hop information propagation, allowing the model to detect fraud patterns that extend beyond an individual applicant's features to include the features of related entities. On the BAF base dataset, GraphSAGE achieves an AUC of 0.8859 and a TPR@5%FPR of 52.54%, demonstrating that relational information provides substantial discriminative power beyond what is available from tabular features alone. The mean aggregator, which computes the element-wise average of neighbor representations, has been found to be the most effective for fraud detection, likely because the uniform weighting avoids overfitting to individual neighbor signals in sparse regions of the graph where the number of neighbors may be small and unrepresentative.

Graph Attention Networks (GAT) [91] extend GraphSAGE by learning attention weights over neighbor contributions, allowing the model to assign higher importance to more informative neighbors. GAT-based fraud detection models can learn to focus on high-risk neighbors (e.g., known fraudsters sharing a device) while down-weighting less informative connections (e.g., connections through popular email providers shared by many legitimate users). However, the attention mechanism introduces additional parameters that can be prone to overfitting in the sparse, high-dimensional graphs typical of fraud detection, and the performance gains of GAT over GraphSAGE have been inconsistent across benchmark datasets. The attention mechanism also adds computational overhead during both training and inference, which may be significant for large-scale graphs with millions of edges. These considerations motivate the use of GraphSAGE as the primary spatial GNN baseline in this thesis, while acknowledging that attention-based architectures may offer advantages in specific deployment scenarios where interpretability of neighbor importance is valued.

Spectral GNN Architectures: Split GNN with Wavelet Filtering

Spectral graph theory provides a powerful framework for analyzing graph structure through the eigendecomposition of the graph Laplacian, enabling the design of GNN architectures that operate in the frequency domain rather than the spatial domain. Spectral GNNs apply learnable filters in the frequency domain, enabling the model to capture both local and global structural patterns through the spectral properties of the graph. ChebNet [19] approximates spectral filters using Chebyshev polynomials of the graph Laplacian, providing a computationally efficient alternative to exact spectral filtering that avoids the cost of eigendecomposition. Graph Convolutional Networks (GCN) [43] simplify ChebNet to first-order approximations, resulting in a scalable architecture that aggregates neighbor features with learned weights but sacrifices the multi-scale filtering capability of higher-order polynomial approximations. These foundational works established the viability of spectral filtering for graph representation learning, but their application to fraud detection has been limited by the computational cost of spectral decomposition on large-scale graphs and the difficulty of designing spectral filters that capture fraud-relevant patterns.

The Split GNN architecture [95] represents a significant advance in spectral GNN design for fraud detection, addressing the limitations of prior spectral methods through a novel dual-branch filtering mechanism. The key innovation is the combination of edge classification with dual-branch spectral filtering using Beta polynomial wavelets [30]. The edge classifier partitions the graph into subgraphs based on learned edge semantics, effectively separating same-class connections (homophilic edges, where connected nodes tend to share the same label) from cross-class connections (heterophilic edges, where connected nodes tend to have different labels). Each subgraph is then processed by an independent wavelet filter branch, allowing the model to apply different spectral transformations to different types of structural patterns. The Beta wavelet framework enables multi-scale graph signal decomposition, capturing both fine-grained local patterns and coarse-grained global structures through a family of polynomial wavelet filters of varying order. The wavelet order parameter controls the scale of analysis: low-order wavelets capture local neighborhood patterns, while high-order wavelets capture broader community structures that may indicate organized fraud rings.

On the BAF base dataset, Split GNN achieves a best AUC of 0.8806, slightly below GraphSAGE but with different performance characteristics across the precision-recall trade-off. The dual-branch architecture allows Split GNN to separately model homophilic connections (links between nodes of the same class, which are common in shared-device patterns where fraudulent applicants reuse the same resources) and heterophilic connections (links between nodes of different classes, which may indicate fraudsters sharing attributes with legitimate users through popular email providers or widely used IP addresses). This separation is particularly valuable in fraud detection, where the graph exhibits mixed homophily: fraudulent applicants are connected to each other through shared fraudulent resources but also connected to legitimate applicants through common infrastructure. The wavelet filtering approach also provides natural multi-scale analysis, with lower-order wavelets capturing local neighborhood structure and higher-order wavelets capturing broader community patterns that may indicate organized fraud rings operating across multiple shared resources.

Quantum-Inspired GNN Architectures

Quantum machine learning (QML) has emerged as a promising paradigm for capturing complex patterns through high-dimensional quantum feature spaces, offering the potential to represent data distributions that are difficult to capture with classical architectures. Variational quantum circuits (VQCs) parameterized by trainable rotation angles can encode classical data into quantum states, where quantum entanglement creates correlations that are difficult to represent classically and may capture subtle relational patterns in graph-structured data. Recent work by Herrman et al. [32] demonstrated the application of quantum graph neural networks to classification tasks, showing that quantum feature maps can capture subtle patterns in graph-structured data through quantum kernel methods and parameterized quantum circuits. The theoretical appeal of quantum-enhanced representations lies in the exponential size of the quantum Hilbert space, which can in principle represent exponentially many feature combinations with a linear number of qubits.

Innan and Finance [35] proposed the Quantum Graph Neural Network (QGNN) architecture, which integrates variational quantum circuits with classical graph neural network layers for financial fraud detection. The QGNN uses parameterized quantum circuits to process node features, generating quantum-enhanced representations that are then propagated through the graph using classical message-passing operations. This hybrid quantum-classical design allows the quantum circuits to focus on feature transformation while the classical GNN handles graph topology, combining the potential advantages of both paradigms. On the BAF base dataset, QGNN achieves an AUC of 0.8769, underperforming both GraphSAGE and Split GNN. This result suggests that the current generation of quantum-inspired architectures, while theoretically promising, has not yet reached the practical maturity needed to outperform well- tuned classical GNN methods on large-scale fraud detection tasks. The underperformance may be attributable to several factors, including the limited expressiveness of shallow quantum circuits, the difficulty of optimizing variational parameters in the quantum landscape, and the gap between theoretical quantum advantage and practical implementation on classical simulators.

A key challenge in QGNN research is the barren plateaus phenomenon [51], where the gradient of the variational objective vanishes exponentially with the number of qubits, making optimization intractable for large-scale circuits. The barren plateaus problem is particularly severe for the deep, highly parameterized circuits that would be needed to capture the complex relational patterns in financial fraud graphs, and it represents a fundamental barrier to scaling quantum-inspired methods to the problem sizes typical of production fraud detection systems. Current QGNN implementations address this limitation by using shallow circuits with a small number of qubits, which limits their expressiveness relative to classical alternatives but ensures that optimization remains tractable. Our QGNN implementation uses classical tensor operations to simulate quantum effects, enabling investigation of quantum-inspired architectures without requiring quantum hardware. While this simulation approach does not benefit from the potential quantum advantage of true hardware execution, it provides a controlled experimental setting for evaluating the architectural contributions of quantum-inspired designs independently of hardware noise and implementation constraints. The inclusion of QGNN in the comparison framework of this thesis serves to establish an empirical baseline for quantum-inspired methods on the BAF benchmark and to identify the specific conditions under which quantum effects may or may not provide practical

advantages for fraud detection.

2.3.4 Spiking Neural Network Approaches

Spiking neural networks (SNNs) represent the third generation of artificial neural networks, modeling computation through discrete temporal events (spikes) rather than continuous activations [50]. Unlike conventional artificial neurons that communicate real-valued activations through weighted connections, spiking neurons accumulate incoming synaptic currents into a membrane potential that evolves over time according to biologically-inspired differential equations, emitting a discrete spike when the potential exceeds a threshold. This temporal dynamics enable SNNs to naturally process event-driven data streams, making them potentially well-suited for transaction-level fraud detection where temporal ordering and inter-arrival timing carry diagnostic information. The theoretical appeal of SNNs extends beyond temporal processing to include energy efficiency on neuromorphic hardware, robustness through population coding, and the potential for biologically plausible learning rules that may offer advantages for online adaptation to evolving fraud patterns.

Mathematical Foundations: The Leaky Integrate-and-Fire Model

The Leaky Integrate-and-Fire (LIF) neuron is the most widely used spiking neuron model in the SNN literature, offering a balance between biological realism and computational tractability that makes it suitable for large-scale simulation and gradient-based training. The membrane potential dynamics of the LIF neuron are governed by the following first-order ordinary differential equation, which describes the subthreshold evolution of the membrane potential as a leaky integrator of incoming synaptic currents:

$$\tau_m \cdot \frac{dV(t)}{dt} = -\big(V(t) - V_{\text{rest}}\big) + R \cdot I(t) \tag{2.1}$$

where $V(t)$ denotes the membrane potential at time t, τ_m is the membrane time constant that controls the rate of potential decay toward the resting potential, V_{rest} is the resting potential to which the membrane decays in the absence of input, R is the membrane resistance that scales the input current, and $I(t)$ is the input synaptic current at time t. When the membrane potential exceeds a threshold V_{th}, the neuron emits a spike and the membrane potential is reset to V_{reset}, after which a refractory period prevents further spiking for a short duration, mimicking the biological refractory period of real neurons. The discrete-time approximation of this dynamics, commonly used in simulation frameworks such as snnTorch, is:

$$V^{(t+1)} = \alpha \cdot V^{(t)} + (1 - \alpha) \cdot \big(W \cdot X^{(t)}\big) \tag{2.2}$$

where $\alpha = \exp(-\Delta t/\tau_m)$ is the decay factor that determines how quickly the membrane potential "forgets" past inputs, W is the synaptic weight matrix, and $X^{(t)}$ is the input at time step t. The spike generation is modeled as $S^{(t)} = \Theta(V^{(t)} - V_{\text{th}})$, where Θ is the Heaviside step function. The non-differentiability of Θ presents a fundamental challenge for gradient-based optimization, as the gradient of the spike function is zero everywhere except at the threshold, where it is undefined. This "dead gradient" problem is the central computational challenge of SNN training and has historically limited the depth and complexity of trainable SNN architectures.

Surrogate gradient methods [97] address this challenge by replacing the discontinuous spike function with a smooth approximation during the backward pass, enabling effective gradient propagation while preserving the discrete spike dynamics during the forward pass. The most commonly used surrogate gradient is the arc-tangent approximation, which defines the surrogate derivative as:

$$\frac{\partial S^{(t)}}{\partial V^{(t)}} \approx \frac{1}{\pi} \cdot \frac{1}{1 + \left(\pi \cdot (V^{(t)} - V_{\text{th}})\right)^2} \tag{2.3}$$

This approximation provides a smooth, non-vanishing gradient in the vicinity of the threshold while decaying gracefully for potentials far from the threshold, preventing gradient explosion and ensuring stable training dynamics. The arc- tangent surrogate is preferred over alternatives such as the sigmoid or piecewise linear approximations because it provides a better balance between gradient magnitude and decay rate, facilitating effective credit assignment across multiple time steps in deep SNN architectures. Recent advances by Eshraghian et al. [22] have demonstrated that surrogate gradient training can achieve competitive performance on classification benchmarks, establishing practical training methodologies for SNNs that open the door for applications in domains previously dominated by conventional deep learning, including the financial fraud detection application explored in this thesis.

SNNs for Financial Fraud Detection

The application of SNNs to financial fraud detection is a nascent but rapidly growing area of research that sits at the intersection of neuromorphic computing and financial technology. Ahmed [3] conducted the first systematic evaluation of SNNs on the BAF dataset, reporting an AUC of 0.8614 and a TPR@5%FPR of 42.21% on the BAF base version using a multi-layer LIF network trained with surrogate gradient descent. While these results fall below the performance of GraphSAGE and classical gradient-boosted methods, the study demonstrated the viability of SNNs for tabular fraud detection and identified several architectural and training challenges that limit SNN performance, providing a foundation for the SNN pipeline developed in this thesis.

Two critical challenges emerged from Ahmed's study that have shaped our understanding of SNN limitations for fraud detection. First, SNNs exhibit calibration instability under extreme class imbalance, as the spike rate distributions of legitimate and fraudulent inputs overlap significantly when the minority class constitutes only 1.10% of the training data. The rate coding scheme used to convert continuous tabular features into spike trains produces similar firing patterns for both classes, reducing the discriminative capacity of the spiking representations. The overlap in spike rate distributions means that the SNN's decision boundary in spike rate space is poorly defined for the minority class, leading to high false negative rates at the low false positive rate operating points required by operational fraud detection systems. Second, SNNs suffer from temporal generalization failure on distributional shift variants of the BAF dataset, where the temporal drift in feature distributions between training and test periods disrupts the learned spike timing patterns. The membrane time constant τ_m, which governs the temporal integration window of LIF neurons, proves to be a critical hyperparameter: values optimized for the training period may be inappropriate for the test period, leading to degraded detection performance. This sensitivity to the membrane time constant suggests that adaptive or learned temporal dynamics may be

necessary for SNNs to achieve robust performance under distributional shift.

Nasif et al. [54] proposed a reinforcement-guided hyper-heuristic approach for SNN optimization, using a reinforcement learning agent to dynamically select and configure SNN hyperparameters (including membrane time constants, threshold potentials, and learning rates) during training. This approach addresses the difficulty of manual hyperparameter tuning for SNNs, which have a larger and more complex hyperparameter space than conventional neural networks due to the additional temporal dynamics parameters of LIF neurons. The reinforcement-guided approach treats hyperparameter selection as a sequential decision problem, where the agent learns a policy that maps the current training state to hyperparameter adjustments, enabling adaptive optimization that responds to the training dynamics. While promising, the reinforcement-guided approach introduces significant computational overhead due to the need for repeated policy evaluation and has not yet been evaluated on the full BAF variant suite, leaving questions about its robustness under distributional shift and its scalability to large-scale fraud detection problems.

The potential advantages of SNNs for fraud detection remain largely theoretical at present, but they motivate continued research into spiking architectures for financial applications. Rate coding and temporal representations could potentially capture the sequential nature of transaction streams more naturally than static feature vectors, encoding the temporal dynamics of application submission patterns that may be diagnostic of coordinated fraud campaigns. The event-driven computation paradigm aligns with the irregular arrival patterns of financial transactions, suggesting potential for low-latency inference in real- time fraud screening systems where processing speed is critical. Population coding across neuron ensembles could provide robust representations that are resilient to adversarial perturbations, an important consideration given the adversarial nature of fraud and the vulnerability of conventional neural networks to adversarial examples. Finally, the energy efficiency of SNNs on neuromorphic hardware may enable edge-deployable fraud screening in scenarios where computational resources are limited, such as mobile banking applications or point-of-sale fraud prevention. However, realizing these advantages requires overcoming the fundamental challenges of training deep SNNs under extreme class imbalance and distributional shift, challenges that this thesis addresses through systematic architectural and training methodology exploration.

2.3.5 Autoencoder-Based Approaches

Autoencoders provide a powerful framework for unsupervised anomaly detection by learning to compress and reconstruct normal data patterns, with reconstruction errors serving as anomaly scores. The fundamental premise is that an autoencoder trained predominantly on legitimate transactions will learn to reconstruct normal patterns accurately while producing high reconstruction errors for anomalous (fraudulent) transactions that deviate from the learned data manifold. This unsupervised paradigm is particularly attractive for fraud detection because it does not require labeled fraud examples during training, addressing the practical challenge of obtaining reliable fraud labels in operational settings where fraud confirmation may take weeks or months and the definition of fraud may evolve over time. The autoencoder framework encompasses several architectural variants, each with distinct mechanisms for learning the data manifold and different implications for anomaly detection performance.

Standard and Variational Autoencoders

Standard autoencoders consist of an encoder that maps input features to a lower- dimensional latent representation and a decoder that reconstructs the input from the latent code. The training objective minimizes the reconstruction error, typically measured as mean squared error or binary cross-entropy, between the input and its reconstruction. The encoder and decoder are trained jointly to minimize this reconstruction loss, with the bottleneck architecture of the latent space forcing the model to capture the most important aspects of the data distribution. However, standard autoencoders suffer from an identity mapping vulnerability: when the model capacity is sufficiently large relative to the complexity of the input, the autoencoder can learn to reconstruct all inputs— including anomalies—with low error, effectively memorizing the input rather than learning the underlying data manifold. This vulnerability is particularly problematic for fraud detection, where the high dimensionality of financial features (30 attributes in the BAF dataset) and the relative simplicity of the fraud signal may allow the autoencoder to reconstruct fraudulent transactions as effectively as legitimate ones, rendering the reconstruction error uninformative as an anomaly score.

Variational Autoencoders (VAEs) [42] address this limitation by introducing a probabilistic framework that models the data distribution through a learned latent space. Rather than mapping inputs to single points in the latent space, VAEs encode inputs as distributions parameterized by mean $\boldsymbol{\mu}$ and variance $\boldsymbol{\sigma}^2$, from which samples are drawn using the reparameterization trick that enables gradient-based optimization through the sampling operation. The training objective maximizes the evidence lower bound (ELBO), which balances reconstruction quality against a KL divergence penalty that regularizes the latent space toward the prior distribution:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}\big[\log p(\mathbf{x}|\mathbf{z})\big] - D_{\text{KL}}\big(q(\mathbf{z}|\mathbf{x}) \,\|\, p(\mathbf{z})\big) \tag{2.4}$$

where the first term encourages faithful reconstruction of the input from the latent sample, and the KL divergence term regularizes the approximate posterior $q(\mathbf{z}|\mathbf{x})$ toward the prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, \mathbf{I})$, preventing the latent space from overfitting to the training distribution. This regularization is the key mechanism that mitigates the identity mapping vulnerability: by forcing the latent space to be smooth and continuous rather than allowing it to memorize individual instances, the VAE ensures that inputs that deviate from the learned data manifold are mapped to regions of latent space that produce high reconstruction errors. The probabilistic nature of VAEs also enables principled uncertainty quantification, which is valuable for flagging borderline cases for manual review in operational fraud detection systems.

Obushyni et al. [62] conducted a comprehensive evaluation of VAEs on the BAF dataset, demonstrating that a well-tuned VAE achieves an AUC of 0.8954 on the BAF base version, substantially outperforming a standard autoencoder that achieves an AUC of only approximately 0.55. The dramatic performance gap between VAE and standard autoencoder confirms the severity of the identity mapping problem: the standard autoencoder learns to reconstruct both legitimate and fraudulent transactions with similar fidelity, rendering the reconstruction error uninformative as an anomaly score. The VAE's probabilistic regularization prevents this failure mode by ensuring that the latent space captures the generative structure of the data rather than memorizing individual instances, so that fraudulent transactions—which deviate from the learned generative model—produce higher reconstruction errors and lower log- likeli-

hoods. Daoud et al. [18] proposed a hybrid VAE- autoencoder model that combines the probabilistic regularization of VAEs with the reconstruction fidelity of standard autoencoders, reporting improved stability across BAF variants and suggesting that the complementary strengths of probabilistic and deterministic reconstruction can be combined for more robust anomaly detection. Gouda et al. [27] explored deep VAE architectures for unsupervised outlier detection, demonstrating that hierarchical latent spaces with multiple levels of abstraction can capture multi-scale patterns in financial data, with lower levels capturing fine-grained feature interactions and higher levels capturing broader distributional patterns.

Denoising Autoencoders for Fraud Detection

Denoising Autoencoders (DAEs) [92] take a complementary approach to the identity mapping problem by learning to reconstruct clean inputs from corrupted versions. The corruption process forces the autoencoder to learn robust feature representations that capture the underlying data manifold rather than memorizing the input, because the model must infer the original values of corrupted features from the uncorrupted ones, which requires understanding the inter-feature dependencies that characterize the data distribution. For tabular fraud data, the most effective corruption strategy is swap noise, where a subset of input features is randomly replaced with values drawn from the empirical marginal distribution of the training set. This strategy is particularly effective for tabular data because it breaks feature-level identity mappings while preserving the statistical properties of the data, forcing the DAE to learn inter-feature dependencies that distinguish legitimate from fraudulent patterns. Anomalous (fraudulent) transactions, which violate the learned inter- feature relationships, produce high reconstruction errors even without corruption applied at test time, because the learned dependencies are specific to the legitimate class distribution.

On the BAF base dataset, a standalone DAE achieves an AUC of only 0.5210, barely above random guessing and dramatically below the performance of VAE and graph-based methods. This seemingly poor performance is misleading and requires careful interpretation: the DAE is learning meaningful representations of the data manifold, but the reconstruction error alone is insufficient to discriminate between legitimate and fraudulent transactions when the fraud signal is subtle relative to the normal variation in the data. The reconstruction error distribution for fraudulent transactions overlaps heavily with that for legitimate transactions, making it a poor anomaly score in isolation. The diagnostic value of the DAE emerges when its learned representations or reconstruction errors are used as features for a downstream classifier, rather than as a standalone anomaly detector, a finding that motivates the hybrid approach explored in this thesis.

Dong [20] conducted the most comprehensive evaluation of DAEs for fraud detection to date, systematically investigating the interaction between denoising pretraining, resampling strategies, and downstream classification. A key finding of Dong's study is the resampling paradox: while ADASYN and SMOTE improve the performance of classical classifiers by providing a more balanced training signal, they degrade the performance of DAE-based methods by corrupting the learned data manifold. The synthetic minority instances generated by resampling methods violate the true data distribution, causing the DAE to learn a corrupted manifold that does not accurately represent either the legitimate or fraudulent class. When the DAE is trained on resampled data, the corruption pattern it learns reflects the artificial distribution rather than

the true data-generating process, reducing the discriminative value of the reconstruction error. This finding is consistent with the resampling paradox observed for GNNs and underscores the need for paradigm-specific preprocessing strategies in multi-model fraud detection systems.

Stacked Ensemble and Hybrid Approaches

The complementary strengths of VAE and DAE anomaly detection motivate their combination through score-level stacking, a meta-learning approach that leverages the diversity of anomaly signals from multiple autoencoder paradigms. Unlike feature-level fusion, which concatenates latent representations before classification, score-level stacking operates on the output anomaly scores, allowing each autoencoder to optimize its own architecture and training objective independently. The VAE provides a distribution-aware anomaly score based on the log-likelihood of the input under the learned generative model, capturing deviations from the probabilistic data distribution. The DAE provides a robustness-based anomaly score reflecting the degree to which the input violates the learned inter-feature dependencies, capturing deviations from the structural data manifold. These scores capture different aspects of anomalousness, and their combination through a meta-learner can achieve higher detection performance than either autoencoder alone by exploiting the orthogonal information in the two anomaly signals.

Gradient-boosted decision trees—specifically XGBoost [13] and LightGBM [41]—serve as effective meta-learners for score- level stacking, as they can capture non-linear interactions between anomaly scores and are robust to the varying scales and distributions of individual autoencoder outputs. Grinsztajn et al. [28] demonstrated that tree-based methods generally outperform deep learning on tabular data, providing theoretical justification for their use as meta-learners in hybrid autoencoder-tree pipelines. The tree-based meta-learner can also incorporate original tabular features alongside autoencoder scores, allowing it to leverage both the learned representations and the raw feature information for improved discrimination. Xu et al. [96] proposed neural architecture search for autoencoder optimization, enabling automated tuning of latent dimensionality, corruption rate, and architectural choices that are critical for detection performance.

The hybrid DAE-XGBoost approach achieves an AUC of 0.8752 on the BAF base dataset, a substantial improvement over the standalone DAE (AUC 0.5210) and competitive with graph-based methods. This result demonstrates that the DAE learns useful representations despite its poor standalone performance, and that the discriminative power of gradient-boosted trees can effectively combine autoencoder-derived features with original tabular features for improved fraud detection. The hybrid approach also offers practical advantages in terms of interpretability, as the feature importance scores of the gradient-boosted meta- learner can reveal which autoencoder scores and original features contribute most to the detection decision. Pombal et al. [68] provided a systematic study of ensemble strategies for anomaly detection, confirming that score-level stacking consistently outperforms both individual models and feature-level fusion approaches, particularly under the extreme class imbalance conditions characteristic of fraud detection.

2.3.6 Comparative Analysis of Approaches

The survey of approaches presented in the preceding sections reveals a complex landscape in which no single paradigm consistently dominates across all evaluation metrics and dataset variants. Table 2.1 summarizes the representative performance of each approach on the BAF base dataset, highlighting the trade-offs between AUC, TPR@5%FPR, and architectural complexity that practitioners must navigate when selecting detection methods.

Table 2.1: Representative performance of fraud detection approaches on the BAF base dataset. Entries marked "—" indicate metrics not reported in the original study.

Approach	**AUC**	**TPR@5%FPR**
Gradient-Boosted Trees (GBM)	0.98	<50%
VAE	0.8954	—
GraphSAGE	0.8859	52.54%
Split GNN	0.8806	—
QGNN	0.8769	—
DAE-XGBoost (Hybrid)	0.8752	—
SNN (LIF)	0.8614	42.21%
Standalone DAE	0.5210	—
Standard Autoencoder	∼0.55	—

Several important observations emerge from this comparative analysis that inform the design of this thesis. First, classical gradient-boosted methods achieve the highest AUC overall (0.98), but their TPR at operationally relevant FPR levels is below 50%, indicating that a significant proportion of fraudulent applications evade detection in practical deployment scenarios where the false positive budget is tightly constrained. This paradox—high AUC with low operational TPR—reflects the fact that AUC averages performance across all possible thresholds, including many that are operationally irrelevant, and motivates the use of TPR@5%FPR as a complementary metric throughout this thesis.

Second, GraphSAGE achieves the highest TPR@5%FPR (52.54%) among the deep learning methods compared, demonstrating the value of relational information for detecting the most subtle fraud patterns that escape tabular classifiers. The superior TPR at low FPR suggests that the graph topology encodes information that is particularly valuable for identifying the borderline cases that are most challenging for tabular methods, and that the message-passing mechanism of spatial GNNs effectively propagates this information to the nodes that need it most.

Third, the VAE achieves a surprisingly high AUC (0.8954) as a standalone unsupervised method, outperforming several supervised approaches and indicating that the probabilistic latent space captures fraud-relevant structure even without explicit supervision. This result suggests that unsupervised methods may have a valuable role to play in multi-paradigm fraud detection systems, either as standalone detectors or as components of hybrid ensembles that combine supervised and unsupervised signals.

Fourth, the dramatic gap between standalone DAE (AUC 0.5210) and hybrid DAE-

XGBoost (AUC 0.8752) underscores the importance of downstream classification for unlocking the discriminative value of autoencoder representations and demonstrates that the DAE's failure as a standalone detector does not reflect a failure of representation learning but rather a failure of the reconstruction error as a direct anomaly score for tabular fraud data.

Fifth, SNNs currently lag behind other deep learning paradigms, with the lowest AUC (0.8614) and TPR@5%FPR (42.21%) among the compared methods, indicating that significant architectural and training innovations are needed to make SNNs competitive for fraud detection. The calibration instability and temporal generalization failure identified by Ahmed [3] represent specific technical challenges that this thesis addresses through systematic experimentation with SNN architectures and training methodologies.

Finally, the incompleteness of the comparison table—with several TPR@5%FPR values unreported—reflects a broader problem in the literature: the absence of a unified evaluation protocol that reports a consistent set of metrics across all paradigms. This gap motivates the systematic multi-paradigm comparison conducted in this thesis, which evaluates all approaches under identical conditions using a comprehensive suite of metrics.

2.4 Identification of Research Gaps

The comprehensive literature review presented in this chapter reveals several critical gaps in the current state of research on deep learning for financial fraud detection. Each gap represents an opportunity for methodological innovation and empirical investigation that this thesis addresses. In this section, we identify and discuss five principal research gaps, followed by a summary of how this thesis fills these voids and advances the state of the art.

2.4.1 Gap 1: Absence of Multi-Paradigm Comparison Under Unified Conditions

The existing literature on deep learning for fraud detection is fragmented across individual paradigms, with studies typically evaluating a single model family in isolation using dataset-specific preprocessing and evaluation protocols. Pourhabibi [69] and Cheng et al. [14] have both highlighted the lack of systematic cross- paradigm comparisons in their surveys of GNN-based fraud detection, noting that the absence of controlled comparisons makes it impossible to determine whether observed performance differences reflect genuine architectural advantages or merely differences in hyperparameter tuning, preprocessing pipelines, and evaluation methodology. This fragmentation is particularly problematic for practitioners who must select among competing paradigms for deployment, as the relative strengths and weaknesses of GNN, SNN, and autoencoder-based approaches have never been characterized under identical experimental conditions.

The absence of unified comparison also obscures the complementary potential of different paradigms. As demonstrated in our comparative analysis, GraphSAGE excels at TPR@5%FPR, VAEs achieve high AUC through probabilistic modeling, and hybrid DAE-XGBoost systems leverage autoencoder representations for discriminative

classification. A multi-paradigm system that combines these complementary strengths could potentially achieve performance that exceeds any single paradigm, but the design of such a system requires a detailed understanding of how the paradigms differ in their error patterns, failure modes, and sensitivity to preprocessing choices. The lack of unified comparison also means that the resampling paradox—where ADASYN helps classical models but hurts GNNs and DAEs—cannot be assessed in the context of a complete multi- paradigm system, limiting the practical guidance available for system design. This thesis provides the first systematic multi-paradigm comparison under unified experimental conditions on the BAF dataset suite, establishing the empirical foundation for principled paradigm selection and combination.

2.4.2 Gap 2: Insufficient Exploitation of Relational Structure in Anomaly Detection

Autoencoder-based anomaly detection methods, including both VAEs and DAEs, operate exclusively on tabular feature representations and fail to exploit the relational structure that graph-based methods leverage for fraud detection. This represents a significant missed opportunity, as the graph topology encodes relational patterns—such as shared devices, common IP addresses, and overlapping contact information—that are among the most reliable indicators of organized fraud. The current separation between graph-based methods (which exploit relational structure but may not capture fine-grained feature interactions) and autoencoder-based methods (which capture feature interactions but ignore relational structure) suggests that a combined approach could achieve superior performance by leveraging both sources of information simultaneously.

The integration of relational information into autoencoder architectures is not straightforward and presents several design challenges. Naive approaches such as concatenating graph-derived features (e.g., GraphSAGE embeddings) with original tabular features before autoencoder input may fail because the autoencoder can learn to reconstruct the graph features independently of the tabular features, diluting the relational signal and reducing the complementary value of the combined representation. More sophisticated approaches, such as graph- regularized autoencoders that enforce consistency between the autoencoder latent space and the graph topology, or conditional VAEs that generate reconstructions conditioned on neighborhood information, have been proposed in the broader representation learning literature but have not been systematically evaluated for financial fraud detection. This thesis investigates the potential for relational information to improve autoencoder-based anomaly detection through score-level combination with graph-based methods, providing the first empirical assessment of cross-paradigm complementarity on the BAF dataset and establishing whether the relational and feature-based signals are indeed orthogonal and combinable.

2.4.3 Gap 3: Limited Understanding of Denoising Autoencoders for Tabular Fraud Data

While denoising autoencoders have a well-established theoretical foundation in the image domain [92], their application to tabular fraud data remains poorly understood. The swap noise corruption strategy that is effective for tabular data introduces unique challenges that differ fundamentally from the Gaussian noise or masking corruption

used in image-based DAEs. In the image domain, spatial locality ensures that corrupted pixels can be inferred from their neighbors, but tabular features lack spatial structure, and the inter-feature dependencies that enable denoising are learned purely from statistical correlations in the data. The optimal corruption rate, the interaction between corruption rate and class imbalance, and the relationship between DAE latent dimensionality and detection performance have not been systematically characterized for fraud detection. Dong [20] provided the most comprehensive evaluation to date, but several questions remain open, including the sensitivity of DAE performance to the specific features selected for corruption, the impact of corruption rate scheduling during training, and the effectiveness of DAE-derived features for detecting specific fraud subtypes.

Furthermore, the stark contrast between standalone DAE performance (AUC 0.5210) and hybrid DAE-XGBoost performance (AUC 0.8752) raises fundamental questions about what the DAE actually learns and why its representations are useful only when combined with a downstream classifier. Understanding the nature of DAE representations for fraud data—whether they capture class-relevant structure, feature interactions, or data manifold geometry—is essential for designing more effective autoencoder architectures and for determining the conditions under which DAE-based methods are most likely to succeed. The identity mapping vulnerability, while mitigated by the denoising objective, may persist in attenuated form, and the conditions under which the DAE fails to learn discriminative representations remain unclear. This thesis contributes to this understanding through systematic ablation studies that isolate the contributions of different DAE design choices to downstream detection performance, providing practical guidance for the deployment of DAE-based methods in fraud detection systems.

2.4.4 Gap 4: Temporal Robustness Across Distributional Shifts

The temporal evaluation framework of the BAF dataset suite, with its six variants designed to test different aspects of distributional shift, reveals a critical gap in the current literature: the temporal robustness of fraud detection models is insufficiently characterized. Sun et al. [89] demonstrated that models optimized for performance on the BAF base version often exhibit severe performance degradation on the temporal shift variants (particularly Variant IV and Variant V), yet most published studies report results only on the base version or on random stratified splits that do not reflect the temporal reality of fraud detection. This practice leads to an overly optimistic assessment of model capability and fails to identify the architectural and training choices that promote temporal robustness, creating a misleading picture of model readiness for production deployment.

The challenge of temporal robustness is paradigm-specific, with each paradigm exhibiting distinct failure modes under distributional shift that reflect its architectural assumptions and learning mechanisms. For GNNs, distributional shift may alter the graph topology (as new types of attribute nodes emerge over time) and the feature distributions of existing nodes, requiring inductive generalization beyond the training graph structure and potentially disrupting the message-passing patterns that the model has learned. For SNNs, temporal shift disrupts the learned spike timing patterns and membrane dynamics, as the optimal membrane time constant may change with the evolving data distribution, causing the temporal integration window to become mis-

aligned with the true temporal structure of the data. For autoencoders, distributional shift may cause the learned data manifold to diverge from the test distribution, leading to systematically biased reconstruction errors that affect the calibration of anomaly scores. Understanding these paradigm-specific failure modes and developing mitigation strategies—such as temporal regularization, domain adaptation, or continuous learning—is essential for deploying fraud detection models in production environments where the data distribution is non-stationary. This thesis evaluates all three paradigms across the full BAF variant suite, providing the first comprehensive characterization of temporal robustness across deep learning paradigms for fraud detection.

2.4.5 Gap 5: Integration of Software Engineering Practices with Multi- Model Fraud Detection

The deployment of multi-paradigm fraud detection systems in production environments raises software engineering challenges that have received virtually no attention in the academic literature. While individual model architectures have been extensively studied, the engineering of systems that combine multiple paradigms—including model versioning, reproducible training pipelines, consistent evaluation protocols, and maintainable codebases—has been largely neglected. The practical challenges of deploying and maintaining a multi-model system in a financial institution include managing dependencies between model components, ensuring reproducibility across hardware and software environments, implementing robust monitoring and alerting for model degradation, and maintaining audit trails for regulatory compliance. These engineering considerations are not merely incidental but are essential for translating academic research into operational systems that deliver reliable fraud detection at scale.

The gap between academic model development and production system engineering is particularly acute for multi-paradigm systems, where each paradigm may require different preprocessing, training infrastructure, and inference pipelines. GNNs require graph construction and neighborhood sampling, which involve complex data structures and sampling algorithms that must be carefully validated for correctness and efficiency. SNNs require temporal simulation and surrogate gradient computation, which introduce additional computational steps and hyperparameters that must be managed and tuned. Autoencoders require corruption scheduling and latent space regularization, which add complexity to the training pipeline and may interact with other preprocessing steps. Integrating these diverse requirements into a cohesive, maintainable system demands careful software architecture design and adherence to engineering best practices such as modular design, comprehensive testing, and configuration management. This thesis addresses this gap by developing a well-engineered multi-pipeline system that demonstrates the feasibility of integrating GNN, SNN, and autoencoder paradigms within a unified software framework, providing a template for the practical deployment of multi-model fraud detection systems that can be maintained, extended, and audited by engineering teams.

2.4.6 Summary: How This Thesis Fills the Void

The five research gaps identified above collectively define the contribution space of this thesis and establish the motivation for the multi-paradigm investigation that follows. Gap 1 motivates the need for a unified multi- paradigm comparison, which this the-

sis provides through systematic evaluation of GraphSAGE, Split GNN, QGNN, LIF-based SNN, VAE, and DAE-XGBoost pipelines on the complete BAF variant suite under identical experimental conditions, including consistent preprocessing, evaluation metrics, and hyperparameter tuning procedures. Gap 2 motivates the exploration of cross-paradigm complementarity, which this thesis investigates through score-level combination of graph-based and autoencoder-based anomaly scores, testing the hypothesis that relational and feature-based signals provide orthogonal information that can be combined for improved detection. Gap 3 motivates the systematic study of DAE design choices for tabular fraud data, which this thesis addresses through comprehensive ablation experiments that characterize the sensitivity of DAE performance to corruption rate, latent dimensionality, and architectural configuration. Gap 4 motivates the evaluation of temporal robustness across distributional shifts, which this thesis provides through evaluation on all six BAF variants with temporal train-test splits, characterizing the paradigm-specific failure modes and identifying architectural choices that promote robustness. Gap 5 motivates the integration of software engineering practices, which this thesis demonstrates through the design and implementation of a modular, reproducible multi-pipeline system that serves as both a research platform and a template for production deployment.

By addressing these gaps in a single, coherent body of work, this thesis makes both methodological and practical contributions to the field of financial fraud detection. The methodological contributions include the first systematic multi- paradigm comparison, the characterization of cross-paradigm complementarity, and the identification of paradigm-specific failure modes under distributional shift. The practical contributions include design recommendations for multi- paradigm fraud detection systems, engineering patterns for multi-model deployment, and empirical guidelines for paradigm selection and combination in different operational contexts. Together, these contributions advance the state of the art from isolated single-paradigm studies toward an integrated, multi- paradigm understanding of deep learning for financial fraud detection, providing both the empirical evidence and the engineering infrastructure needed to translate research advances into operational fraud detection systems.

Chapter 3

Analysis and Requirements Engineering

3.1 Stakeholder Analysis

The primary stakeholders in a multi-pipeline financial fraud detection system—spanning GNN, SNN, and Autoencoder paradigms—include:

Table 3.1: Stakeholder Analysis

Stakeholder	**Interest**	**Influence**
Financial Institutions	Minimize fraud losses while maintaining customer satisfaction; regulatory compliance	High — primary adopter and funder
Fraud Analysts	Effective tools for investigating flagged transactions; interpretable model outputs	Medium — daily users of the system
Regulatory Bodies	Fair and non-discriminatory detection; explainability of decisions	High — legal compliance requirements
Customers	Legitimate transactions processed without undue friction; privacy protection	Low — indirect influence through complaints
Data Science Team	Model accuracy, scalability, maintainability of the detection pipeline	Medium — system designers
Neuromorphic Engineers	Efficient SNN deployment on neuromorphic hardware; spike encoding fidelity	Medium — SNN pipeline designers and deployers
Anomaly Detection Specialists	Unsupervised detection quality; reconstruction error calibration; low false positive rates on autoencoder outputs	Medium — AE pipeline designers and evaluers

The stakeholder landscape has been expanded beyond the original GNN-focused analysis to account for the additional expertise required by the SNN and Autoencoder pipelines. Neuromorphic engineers are critical stakeholders for the SNN pipeline, as the translation of trained models to neuromorphic hardware (e.g., Intel Loihi, IBM TrueNorth) requires careful attention to spike encoding schemes, neuron parameter constraints, and latency budgets. Anomaly detection specialists bring domain knowledge in unsupervised learning calibration, reconstruction error analysis, and the design of ensemble strategies that combine complementary detection paradigms.

3.2 Functional Requirements

Table 3.2: Functional Requirements

ID	Requirement	Description
FR-01	Graph Construction	The system shall transform tabular applicant data into a heterogeneous graph with at least 17 node types and bidirectional edges
FR-02	Node Classification	The system shall classify user nodes as legitimate or fraudulent with ROC AUC ≥ 0.85
FR-03	Multi-Model Support	The system shall support at least three GNN architectures: GraphSAGE, QGNN, and Split GNN
FR-04	Temporal Validation	The system shall evaluate models using temporal train/validation/test splits to simulate production conditions
FR-05	Threshold Calibration	The system shall calibrate operating thresholds at a target false positive rate of 5%
FR-06	Hyperparameter Tuning	The system shall support grid search over wavelet order, split point, and edge loss weight for the Split GNN
FR-07	Cross-Version Evaluation	The system shall evaluate model robustness across multiple data distribution variants
FR-08	Fairness Analysis	The system shall report subgroup performance metrics by age group and employment status
FR-09	SNN Spike Encoding	The system shall support configurable spike encoding strategies (rate coding, latency coding, delta modulation) to transform tabular applicant features into temporal spike trains compatible with LIF neuron dynamics
FR-10	VAE/DAE Anomaly Scoring	The system shall implement both Variational Autoencoder (VAE) and Denoising Autoencoder (DAE) models, each producing per-sample anomaly scores from reconstruction errors and/or negative log-likelihood, with configurable corruption schedules for the DAE
FR-11	Ensemble Voting	The system shall support ensemble decision fusion across the GNN, SNN, and Autoencoder pipelines via majority voting, weighted averaging, and score-level stacking with gradient-boosted meta-learners (XGBoost/LightGBM)

The functional requirements FR-09 through FR-11 extend the system's capabilities to encompass the SNN and Autoencoder pipelines. FR-09 ensures that tabular fraud features can be faithfully represented as temporal spike patterns, a prerequisite for leveraging the temporal dynamics of spiking neurons. FR-10 mandates dual autoencoder paradigms (VAE for distribution-aware anomaly scoring, DAE for corruption-invariant

feature learning) with complementary anomaly signals. FR-11 provides the integration layer that unifies predictions from all three pipelines, enabling the system to exploit the complementary strengths of graph-based relational reasoning, temporal spike-based pattern detection, and unsupervised anomaly detection.

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

ID	Requirement	Description
NFR-01	Scalability	The system shall process graphs with at least 1 million nodes and 14 million edges
NFR-02	Memory Efficiency	The system shall use cached Laplacian representations to avoid $O(N^2)$ memory for graph spectral operations
NFR-03	Training Stability	The system shall incorporate early stopping, gradient clipping, and learning rate scheduling to ensure stable convergence
NFR-04	Reproducibility	All experiments shall be reproducible with fixed random seeds and documented hyperparameters
NFR-05	Modularity	The graph construction, model training, and evaluation components shall be independently usable
NFR-06	Interpretability	The Split GNN edge classifier shall provide interpretable edge importance scores
NFR-07	Temporal Simulation Support	The SNN pipeline shall support configurable temporal simulation parameters (simulation timesteps ≥ 20, membrane time constant τ, decay factor) and surrogate gradient functions (fast sigmoid, piecewise linear, arc tangent) to enable stable training through backpropagation through time
NFR-08	Unsupervised Training Capability	The Autoencoder pipeline shall support fully unsupervised training on legitimate-only data subsets, with the ability to isolate the target manifold by excluding fraudulent samples during encoder/decoder optimization, and shall provide calibrated anomaly score thresholds without requiring labeled fraud examples

NFR-07 addresses the unique computational requirements of spiking neural networks, where temporal simulation introduces a sequential dependency across timesteps that must be carefully managed for both training efficiency and numerical stability. The surrogate gradient requirement ensures that the non-differentiable spike function does not

prevent gradient-based optimization. NFR-08 ensures that the autoencoder pipeline can operate in a purely unsupervised regime—a critical capability for fraud detection systems where labeled fraud examples are scarce and costly to obtain. The target manifold isolation requirement ensures that the autoencoder learns exclusively from legitimate behavior patterns, so that deviations (fraud) produce maximally discriminative reconstruction errors.

3.4 Data Requirements

Table 3.4: Data Requirements

ID	Requirement	Description
DR-01	Dataset Size	Minimum 1 million applicant records with temporal split capability
DR-02	Feature Richness	At least 23 features per applicant including categorical, continuous, and binary types
DR-03	Class Imbalance	The dataset shall reflect real-world fraud prevalence ($\sim$1% fraud rate)
DR-04	Temporal Structure	Records shall include timestamps enabling month-based temporal splitting
DR-05	Multi-Version Support	At least 6 data distribution variants for cross-version robustness evaluation
DR-06	Privacy Compliance	All personally identifiable information shall be anonymized or excluded from model features
DR-07	Spike Encoding Compatibility	The feature set shall support transformation into spike train representations: continuous features shall be normalizable to $[0, 1]$ for rate coding; all feature dimensions shall be compatible with the selected encoding scheme's input dimension constraints
DR-08	Target Manifold Isolation	The dataset shall support stratified extraction of legitimate-only subsets for autoencoder training, with sufficient legitimate sample volume ($\geq$ 500K records) to ensure comprehensive coverage of the normal behavior manifold

DR-07 ensures that the feature engineering pipeline can produce spike-compatible representations for the SNN pipeline. Continuous features must be normalizable to bounded ranges suitable for rate coding (where spike probability is proportional to feature magnitude) or latency coding (where earlier spikes encode larger values). DR-08 addresses the data requirements specific to unsupervised autoencoder training: the autoencoder must learn its reconstruction target exclusively from legitimate transactions, requiring a clean separation of the training data by class label at the data preparation stage. The minimum volume of 500K legitimate records ensures that the learned manifold is sufficiently dense to produce low reconstruction errors for normal behavior and high errors for anomalous (fraudulent) patterns.

Chapter 4

System Architecture and Design

4.1 High-Level System Architecture

The proposed financial fraud detection system follows a three-stage architecture with three parallel detection pipelines: (1) Data Ingestion and Graph Construction, (2) Multi-Paradigm Model Training and Inference, and (3) Evaluation, Ensemble Fusion, and Monitoring. The first stage transforms raw tabular applicant data into both a heterogeneous graph representation (for the GNN pipeline) and alternative representations suitable for the SNN and Autoencoder pipelines. The second stage applies three fundamentally different detection paradigms in parallel: GNN-based relational reasoning, SNN-based temporal pattern detection, and Autoencoder-based unsupervised anomaly scoring. The third stage evaluates each pipeline's performance across multiple metrics and data versions, fuses predictions through ensemble voting or stacking, and provides monitoring capabilities for production deployment.

The three parallel pipelines exploit complementary views of the fraud detection problem. The **GNN pipeline** captures relational fraud patterns through graph topology and message passing. The **SNN pipeline** captures temporal dynamics and event-driven patterns through spike-based computation with leaky integrate-and-fire (LIF) neurons. The **Autoencoder pipeline** captures distributional anomalies through unsupervised reconstruction error analysis using both Variational Autoencoders (VAE) and Denoising Autoencoders (DAE). The ensemble fusion layer combines these complementary predictions into a unified fraud score.

Figure 5.1: Architecture of the GNNFraudDetector (GraphSAGE)

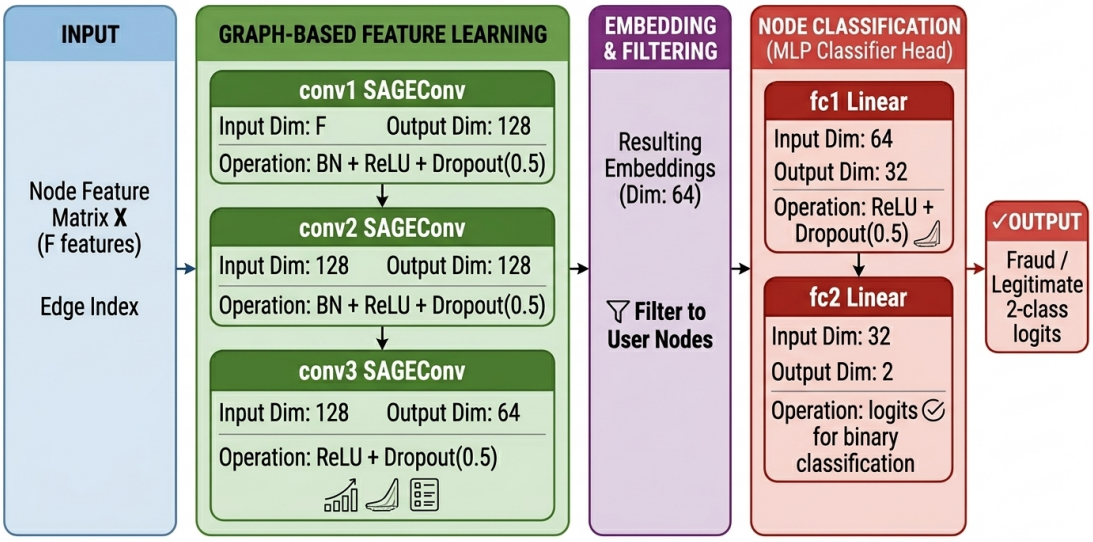

Figure 4.1: FraudGraphBuilder — Heterogeneous Graph Construction Architecture. The left panel shows the data transformation pipeline from raw tabular data to PyTorch Geometric Data object. The right panel illustrates the graph schema with 17 attribute node types organized into three categories: Categorical (blue), Bucketed Continuous (green), and Binary/Flag (orange), all connected to a central User node.

4.2 Component Design

4.2.1 FraudGraphBuilder Component

The FraudGraphBuilder is the core component of the graph construction stage. It transforms a flat tabular dataset, where each row represents a credit card applicant, into a heterogeneous graph consisting of 17 distinct node types. Each applicant becomes a user node carrying a dense feature vector, while categorical and continuous attributes are mapped to shared attribute nodes. Edges connect users to their corresponding attribute nodes, creating a bipartite-style graph structure. This transformation enables the graph to encode not just what an applicant is (their features), but also who they are related to (shared devices, similar risk profiles, common behavioral patterns), providing a significantly richer representation for fraud detection.

The system architecture is founded on the principle of heterogeneous information networks. Unlike standard graphs where all nodes are of the same type, our design distinguishes between user nodes and 17 categories of attribute nodes. This distinction is vital because it allows the model to differentiate between the primary entities (applicants) and the contextual features that link them together. The heterogeneous structure enables the graph neural network to learn rich representations that capture both node-level features and topological relationships. The architecture follows a three-stage transformation process that progressively converts raw tabular data into a graph structure suitable for neural network consumption.

The three stages operate as follows. First, the raw tabular dataset is ingested and pre-processed to handle missing values and continuous numerical ranges. At this stage, certain identifying fields (email, phone, device_id, IP address, name) are excluded from the user feature vector to prevent the model from memorizing identities

rather than learning behavioral patterns. Second, we implement a discrete bucketing mechanism that transforms high-variance continuous features, such as credit risk scores and customer ages, into categorical representations suitable for graph nodes. Third, the system establishes undirected edges between user nodes and their corresponding attribute nodes, effectively turning the dataset into a bipartite-style graph where users are interconnected through shared properties.

Node Type Catalog

The FraudGraphBuilder constructs a heterogeneous graph with 17 distinct node types, each serving a specific purpose in capturing different dimensions of applicant behavior and identity. These node types are organized into three logical categories based on their data origin and the nature of the information they encode. Table 4.1 provides a comprehensive catalog of all node types, their source columns, the number of possible values, and their significance in the fraud detection context.

Table 4.1: Complete Node Type Catalog

Category	Node Type	Source Column(s)	Values	Shared	Fraud Detection Rationale
Categorical	device_os	device_os_windows / macintosh / linux	3	Yes	Fraudsters often use specific OS configurations
Categorical	housing	housing_status_* (one-hot)	∼7	Yes	Housing instability may correlate with financial stress
Categorical	employment	employment_status_* (one-hot)	∼6	Yes	Employment patterns are strong fraud indicators
Categorical	payment_type	payment_type_* (one-hot)	∼5	Yes	Certain payment methods are disproportionately used in fraud
Bucketed	risk_bucket	credit_risk_score	6	Yes	Credit risk score directly reflects financial reliability
Bucketed	age_bucket	customer_age	∼10	Yes	Age demographics help identify fraud patterns
Bucketed	credit_limit_bucket	proposed_credit_limit	6	Yes	Unusually high credit limit requests are strong fraud signals
Bucketed	zip_bucket	zip_count_4w	4	Yes	Multiple applications from same zip may indicate organized fraud
Bucketed	vel6h	velocity_6h	4	Yes	Rapid successive applications are hallmark of fraud rings
Bucketed	vel24h	velocity_24h	4	Yes	24-hour velocity captures coordinated application patterns
Bucketed	vel4w	velocity_4w	4	Yes	4-week velocity captures sustained fraud campaign activity
Binary	email_type	email_is_free	2	Yes	Free email services disproportionately used in fraud
Binary	foreign	foreign_request	2	Yes	Foreign requests carry higher risk
Binary	session_type	keep_alive_session	2	Yes	Automated sessions suggest bot-driven applications
Binary	other_cards	has_other_cards	2	Yes	Existing cardholders exhibit different risk profiles
Binary	device	device_id	Many	Yes	Shared devices among applicants are strong fraud ring indicators
Binary	phone_validity	phone_home_valid + phone_mobile_valid	4	Yes	Invalid phone numbers common in synthetic identity fraud

Feature Bucketing Strategies

A critical design decision in the graph construction pipeline is how continuous numerical features are discretized into graph nodes. The FraudGraphBuilder employs three distinct bucketing strategies, each tailored to the statistical properties of the source feature. Table 4.2 provides a detailed breakdown of these strategies.

Table 4.2: Feature Bucketing Strategies

Feature	Strategy	Buckets	Count	NaN Handling	Rationale
credit_risk_score	Equal-width (200)	very_low (<200), low (200–400), medium (400–600), high (600–800), very_high (800+)	5	"unknown"	Standard industry score ranges
customer_age	Decade bucketing	0s, 10s, 20s, 30s, 40s, 50s, 60s, 70s, 80s, 90s	10	"unknown"	Natural age groupings
proposed_credit_limit	Threshold-based	very_low (<500), low (500–2K), medium (2K–5K), high (5K–10K), very_high (10K+)	5	"unknown"	Business-relevant limits
zip_count_4w	Activity tiers	zip_unique (=1), zip_low (2–5), zip_medium (6–20), zip_high (>20)	4	Skipped	Application density proxy
velocity_6h/24h/4w	Velocity tiers	single (=1), low (2–3), medium (4–10), high (>10)	4	Skipped	Behavioral speed tiers

The bucketing strategies reflect domain-specific knowledge about financial fraud patterns. For example, the credit risk score uses a standard 200-point equal-width partitioning that aligns with common industry scoring brackets. The velocity features use a non-linear tiering that places greater granularity at the lower end (distinguishing between 1, 2–3, and 4–10) where the difference between a single application and a few applications is more informative than the difference between 50 and 100 applications.

Edge Construction and Symmetrization

The edge construction process follows a systematic pattern: for each applicant record, the builder iterates through all attribute categories and creates an edge between the user node and the corresponding attribute node. A crucial design choice is the edge symmetrization step: after collecting all directed edges (user to attribute), the builder duplicates each edge in the reverse direction (attribute to user). This creates an undirected graph that allows message-passing in both directions. The symmetrization is implemented efficiently using NumPy array concatenation, doubling the edge count while ensuring bidirectional information flow. The `edge_index` tensor therefore has shape $[2, 2E]$, where E is the number of directed user-to-attribute edges.

Graph Statistics

The FraudGraphBuilder produces a large-scale heterogeneous graph with the characteristics summarized in Table 4.3.

Table 4.3: Graph Statistics Summary

Property	Value	Notes
Total Nodes	1,000,047	Includes users + all attribute nodes
Total Edges (directed)	14,491,868	Bidirectional: 7,245,934 unique connections
User Nodes	∼1,000,000	One per applicant record
Attribute Node Types	17	Categorical: 4, Bucketed: 7, Binary: 6
Feature Dimension (F)	Variable	All columns except excluded identifiers
Edge Directionality	Undirected (symmetrized)	Each edge duplicated in reverse
Output Format	PyTorch Geometric Data	x, edge_index, y, user_indices

4.2.2 GNN Model Components

The system supports three GNN model components, each implementing a different paradigm for graph-based fraud detection. The GraphSAGE component (GNNFraudDetector) follows a spatial aggregation approach with three SAGEConv layers. The QGNN component (PaperQGNN) implements a hybrid quantum-classical approach with angle encoding and variational quantum circuit layers. The Split GNN component implements spectral graph splitting with edge classification and dual-branch Beta wavelet filtering. All three components share a common interface accepting node features, edge index, and optional user indices.

4.2.3 SNN Pipeline Components

The Spiking Neural Network (SNN) pipeline introduces components specifically designed for temporal, event-driven fraud detection. The pipeline consists of three core

components: a spike encoding module, the FraudSNN_SuperDeep architecture, and an ensemble voting module.

Spike Encoding Module

The spike encoding module transforms static tabular applicant features into temporal spike trains suitable for processing by LIF neurons. Three encoding strategies are supported:

Rate Coding: Each continuous feature value is normalized to $[0, 1]$ and interpreted as a firing probability. Over T simulation timesteps, each neuron emits a spike at each timestep with probability proportional to the feature magnitude. This produces Poisson-like spike trains where the spike count encodes the feature value. Rate coding is robust to noise and provides a natural interface for LIF neuron dynamics.

Latency Coding: Feature values are mapped to spike timing, with larger values producing earlier spikes. The spike time for feature dimension d is computed as:

$$t_d = T \cdot (1 - x_d) \tag{4.1}$$

where $x_d \in [0, 1]$ is the normalized feature value and T is the total number of timesteps. Latency coding provides precise temporal information but is sensitive to normalization quality.

Delta Modulation: For temporally ordered data (e.g., transaction streams), delta modulation encodes changes in feature values as spike events. An upward change produces an excitatory spike, while a downward change produces an inhibitory spike. This encoding is most applicable when the input data has inherent temporal ordering.

The spike encoding module outputs a tensor of shape $[N, F, T]$ where N is the batch size, F is the feature dimension, and T is the number of simulation timesteps.

FraudSNN_SuperDeep Architecture

The FraudSNN_SuperDeep architecture is a deep spiking neural network designed for fraud classification on tabular data. The architecture consists of multiple spiking layers built from LIF neurons with surrogate gradient training:

LIF Neuron Model: Each LIF neuron maintains a membrane potential $V(t)$ that evolves according to:

$$\tau \frac{dV(t)}{dt} = -V(t) + \sum_i w_i \cdot S_i(t) \tag{4.2}$$

where τ is the membrane time constant, w_i are synaptic weights, and $S_i(t)$ are incoming spike signals. When $V(t)$ exceeds a threshold V_{th}, the neuron emits a spike and the potential is reset to V_{reset}.

Surrogate Gradient: The non-differentiable spike generation is handled through a surrogate gradient function during backpropagation. The fast sigmoid surrogate is defined as:

$$\frac{\partial S}{\partial V} \approx \frac{1}{(\beta |V - V_{\text{th}}| + 1)^2} \tag{4.3}$$

where β controls the steepness of the approximation. This enables gradient-based training while preserving discrete spike dynamics in the forward pass.

Network Structure: The FraudSNN_SuperDeep architecture stacks multiple fully-connected spiking layers with progressive hidden dimension reduction. Each spiking layer consists of a linear projection followed by LIF neuron activation. The final layer uses rate decoding—the membrane potential or spike count over the last several timesteps is aggregated to produce a scalar output for binary classification. Dropout is applied between spiking layers as a regularizer, and layer normalization is used to stabilize training across timesteps.

SNN Ensemble Voting Module

The SNN pipeline supports ensemble decision making through multiple independently trained FraudSNN_SuperDeep models with different random initializations and hyperparameter configurations (varying hidden dimensions, number of layers, and membrane time constants). The ensemble aggregates predictions through:

- **Majority Voting:** Each model votes for the predicted class; the majority class is selected.
- **Weighted Averaging:** Model outputs (spike rates or membrane potentials) are averaged with weights proportional to validation AUC performance.

The ensemble approach mitigates the high variance that individual SNN models can exhibit due to the stochastic nature of spike generation and surrogate gradient approximation.

4.2.4 Autoencoder Pipeline Components

The Autoencoder pipeline provides unsupervised anomaly detection through two complementary autoencoder paradigms, followed by score-level stacking with gradient-boosted meta-learners.

Variational Autoencoder (VAE) Component

The VAE component learns a probabilistic latent representation of legitimate transaction patterns. The architecture consists of:

Encoder: The encoder maps input features x to a latent distribution parameterized by mean μ and log-variance $\log \sigma^2$ through a series of fully connected layers with ReLU activations and batch normalization. The latent vector z is sampled using the reparameterization trick:

$$z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I) \tag{4.4}$$

Decoder: The decoder reconstructs the input from the latent representation through a symmetric series of fully connected layers. The output layer uses sigmoid activation to reconstruct normalized feature values.

Loss Function: The VAE is trained by maximizing the Evidence Lower Bound (ELBO), which decomposes into reconstruction loss (binary cross-entropy or mean squared error) and a KL divergence regularization term:

$$\mathcal{L}_{\text{VAE}} = \mathcal{L}_{\text{recon}} + \beta \cdot D_{\text{KL}}(q(z|x) \| p(z)) \tag{4.5}$$

where β is a weighting coefficient that can be annealed during training (β-VAE) to control the trade-off between reconstruction fidelity and latent space regularity.

Anomaly Scoring: The VAE produces anomaly scores from two complementary signals: (1) reconstruction error, computed as the mean squared error between input and reconstruction; and (2) negative log-likelihood, estimated from the ELBO. Fraudulent transactions, which deviate from the learned distribution of legitimate behavior, yield high scores on both metrics.

Target Manifold Isolation: The VAE is trained exclusively on legitimate transactions (where `fraud_bool` $= 0$), ensuring that the learned latent space represents only the normal behavior manifold. This isolation is critical: by excluding fraudulent samples from training, the autoencoder has no capacity to reconstruct anomalous patterns, producing maximally discriminative reconstruction errors for fraud.

Denoising Autoencoder (DAE) Component

The DAE component learns robust feature representations by reconstructing clean inputs from corrupted versions. The architecture consists of:

Swap Noise Corruption: The DAE employs swap noise corruption, where each input feature is independently replaced with probability ρ (the corruption rate) by a value randomly sampled from the empirical marginal distribution of that feature in the training set. This strategy is particularly effective for tabular fraud data because:

- It forces the autoencoder to learn inter-feature dependencies rather than memorizing individual feature values.
- It is invariant to feature scale and distribution, unlike Gaussian noise which requires careful tuning per feature.
- It naturally handles mixed-type features (categorical, continuous, binary).

Encoder/Decoder Architecture: The DAE uses a symmetric autoencoder architecture with fully connected layers, ReLU activations, and batch normalization. The encoder progressively reduces dimensionality to a bottleneck layer, and the decoder reconstructs the original (uncorrupted) input. The reconstruction loss uses mean squared error for continuous features and binary cross-entropy for binary features.

Corruption Schedule: The corruption rate ρ can follow a schedule: starting at a high rate (e.g., $\rho = 0.3$) and annealing to a lower rate (e.g., $\rho = 0.1$) during training. This curriculum encourages the model to first learn broad inter-feature relationships and then refine them.

Anomaly Scoring: The DAE produces anomaly scores from the reconstruction error on uncorrupted inputs at test time. Fraudulent transactions, which violate the learned inter-feature relationships, produce high reconstruction errors even without input corruption.

Score-Level Stacking with XGBoost/LightGBM

The stacking component combines the anomaly scores from the VAE and DAE into a unified fraud prediction using a gradient-boosted decision tree meta-learner:

Feature Matrix for Stacking: The meta-learner receives a feature matrix consisting of: (1) VAE reconstruction error, (2) VAE negative log-likelihood, (3) DAE

reconstruction error, and optionally (4) raw tabular features as auxiliary inputs. This provides the meta-learner with complementary anomaly signals from both autoencoder paradigms.

Meta-Learner Selection: Two gradient-boosted decision tree implementations are supported:

- **XGBoost:** Provides robust performance with built-in regularization (L1/L2), handling of missing values, and efficient histogram-based splitting. The `scale_pos_weight` parameter directly addresses the extreme class imbalance in fraud detection.
- **LightGBM:** Offers faster training through Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB), with native categorical feature support. The `is_unbalance` flag handles class imbalance during training.

Training Protocol: The stacking meta-learner is trained on labeled data (unlike the unsupervised autoencoders), using cross-validated predictions from the VAE and DAE to prevent information leakage. The meta-learner's hyperparameters (learning rate, max depth, number of estimators, subsample ratio) are optimized via Bayesian optimization with AUC-ROC as the objective.

4.2.5 Evaluation and Monitoring Components

The evaluation component provides standardized metrics computation including ROC AUC, precision-recall curves, confusion matrices, and subgroup fairness analysis. It supports per-pipeline evaluation as well as combined ensemble evaluation. The monitoring component tracks performance drift across data versions and generates alerts when metrics degrade beyond configured thresholds. Monitoring covers all three pipelines independently, enabling early detection of pipeline-specific degradation (e.g., distributional shift affecting the autoencoder's reconstruction quality, or temporal pattern changes reducing SNN effectiveness).

4.3 The AI Pipeline Architecture

The AI pipeline follows a three-branch parallel architecture with a shared data ingestion stage and a unified ensemble fusion stage:

1. **Data Ingestion and Preparation:** Raw tabular data is loaded and validated against the schema defined in the data requirements. The data is then distributed to three parallel preparation paths:
    - *GNN Path:* The FraudGraphBuilder transforms the validated data into a PyTorch Geometric Data object containing the node feature matrix, edge index, labels, and user indices.
    - *SNN Path:* The spike encoding module transforms tabular features into temporal spike trains of shape $[N, F, T]$ using the configured encoding strategy (rate coding, latency coding, or delta modulation).
    - *AE Path:* The data is split into a legitimate-only training set (for unsupervised autoencoder training) and a full labeled evaluation set. Features are normalized and the marginal distributions are computed for swap noise generation.

2. **Parallel Model Training:** Three pipelines are trained independently:
   - *GNN Pipeline:* The selected GNN architecture (GraphSAGE, QGNN, or Split GNN) is trained using temporal validation with class-weighted loss functions and early stopping.
   - *SNN Pipeline:* The FraudSNN_SuperDeep model is trained using surrogate gradient descent with BCEWithLogitsLoss, class-weighted to handle imbalance, and temporal simulation over T timesteps. Multiple models are trained for ensemble voting.
   - *AE Pipeline:* The VAE and DAE are trained independently on the legitimate-only subset. The VAE optimizes the ELBO objective; the DAE optimizes reconstruction loss with swap noise corruption. Stacking meta-learners (XGBoost/LightGBM) are trained on the combined anomaly scores.
3. **Threshold Calibration:** Operating thresholds are calibrated independently for each pipeline on the validation set to achieve the target 5% false positive rate.
4. **Inference and Ensemble Fusion:** Each pipeline produces fraud predictions on the test set. Predictions are combined through:
   - Majority voting across pipeline outputs
   - Weighted averaging of pipeline probability scores (weights proportional to validation AUC)
   - Score-level stacking: pipeline outputs serve as features for a meta-learner
5. **Evaluation and Cross-Version Analysis:** Comprehensive metrics are computed for each pipeline individually and for the ensemble. Performance is compared across data distribution variants to assess robustness of each pipeline and the combined system.

4.4 Technology Stack Selection

Table 4.4: Technology Stack

Component	Technology	Justification
Programming Language	Python 3.10+	Dominant ecosystem for ML/DL; extensive library support
Deep Learning Framework	PyTorch 2.x	Flexible autograd; native GPU support; research-friendly API
Graph Neural Networks	PyTorch Geometric	Efficient sparse tensor operations; heterogeneous graph support
Data Processing	Pandas, NumPy	Standard data manipulation; efficient array operations
Quantum Simulation	PennyLane / Qiskit	Variational quantum circuit simulation; gradient computation
Spiking Neural Networks	snnTorch, Norse	Differentiable LIF neuron models; surrogate gradient training; PyTorch integration; temporal simulation support
Autoencoders (VAE/DAE)	PyTorch (custom), PyOD	Flexible VAE/DAE architecture design; anomaly detection utilities; distribution fitting
Stacking Meta-Learner	XGBoost, LightGBM	Efficient gradient-boosted trees; built-in class imbalance handling; categorical feature support (LightGBM)
Experiment Tracking	TensorBoard / Weights & Biases	Training visualization; hyperparameter logging
Model Evaluation	scikit-learn	Standard metrics; confusion matrices; ROC analysis

The technology stack has been extended to support the SNN and Autoencoder pipelines. **snnTorch** provides a PyTorch-native implementation of spiking neuron models (LIF, Parametric LIF) with built-in surrogate gradient functions, enabling seamless integration with existing PyTorch training loops. **Norse** offers an alternative SNN framework with biophysically detailed neuron models and event-driven simulation capabilities. For the Autoencoder pipeline, **PyOD** (Python Outlier Detection) provides baseline implementations and evaluation utilities for anomaly detection, while custom PyTorch modules enable the specific VAE and DAE architectures required by our design. The stacking meta-learner leverages **XGBoost** and **LightGBM** for their proven effective-

ness in tabular classification tasks and built-in mechanisms for handling extreme class imbalance.

Chapter 5

Pipeline I: Graph Neural Network–Based Fraud Detection

5.1 Data Preprocessing Techniques

The data preprocessing pipeline implements several critical transformations to prepare the raw tabular data for graph-based learning:

Missing Value Handling: The implementation distinguishes between two types of missing data. Features where absence is informative (e.g., credit risk score) are mapped to a dedicated "unknown" bucket, preserving the signal that missingness itself may carry. Features where absence should not create connections (e.g., zip count) are skipped entirely, preventing spurious graph connections between users who both lack the same feature.

Feature Column Selection: The builder identifies all columns suitable for user node features by excluding identifying fields (email, phone_home, phone_mobile, device_id, ip_address, name) and the target label (fraud_bool, month) from the DataFrame columns. This prevents the model from memorizing identities while still capturing the relational patterns these identifiers create through graph structure.

Continuous Feature Discretization: High-variance continuous features are discretized into categorical representations using domain-informed bucketing strategies (equal-width for credit scores, decade bucketing for ages, threshold-based for credit limits, and activity/velocity tiers for behavioral metrics).

One-Hot Expansion: Categorical columns encoded as one-hot vectors in the original dataset are scanned for active values, with each active category creating or retrieving a shared attribute node and establishing an edge to the current user.

5.2 Model Selection and Justification

Three GNN architectures were selected to represent fundamentally different paradigms for graph-based fraud detection, enabling a comprehensive comparison of spatial, quantum-inspired, and spectral approaches.

5.2.1 GraphSAGE: Spatial Neighborhood Aggregation

The GNNFraudDetector utilizes the GraphSAGE (Graph Sample and AggregatE) architecture, which generates node embeddings by sampling and aggregating features from a node's local neighborhood. This architecture is particularly well-suited for our task because it allows the model to learn inductive representations for users, even when encountering new users or graph structures during inference. By employing this approach, we move beyond analyzing isolated user features to understanding the complex topological patterns indicative of fraudulent behavior.

The architectural design follows a hierarchical structure consisting of two primary stages: graph-based feature learning and node classification. The first stage is composed of three SAGEConv layers, which are responsible for propagating information across the graph structure. These layers operate on the input feature matrix and the edge index to produce high-dimensional embeddings that encapsulate both the intrinsic features of a user and the aggregate information of their neighbors. The second stage consists of a fully connected classifier head (Multi-Layer Perceptron) designed to classify the learned embeddings.

Figure 5.1: Architecture of the GNNFraudDetector (GraphSAGE)

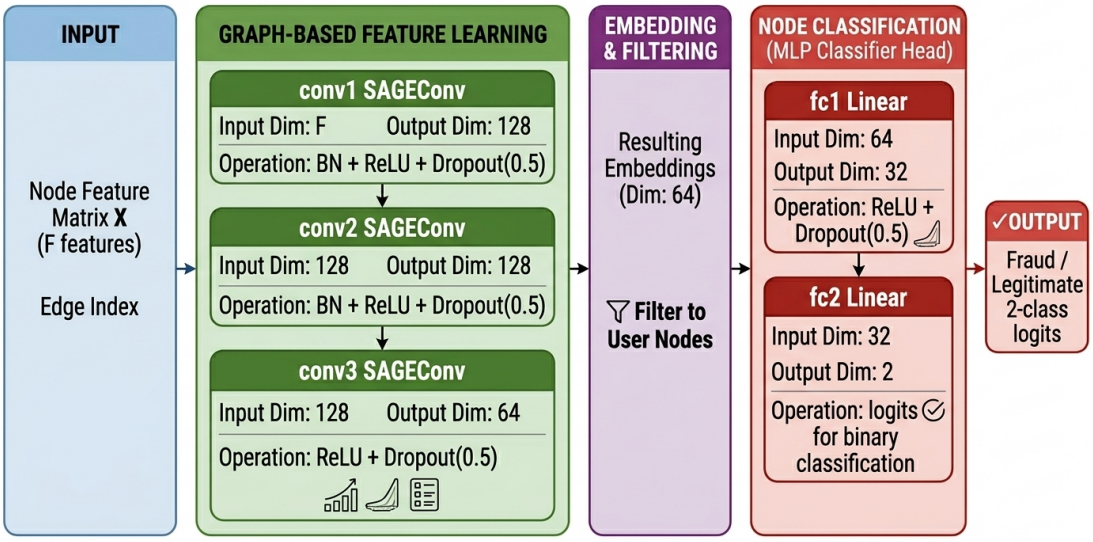

Figure 5.1: Architecture of the GNNFraudDetector (GraphSAGE). The pipeline processes node features through three SAGEConv layers with BatchNorm, filters to user nodes, then passes through two fully connected layers for binary classification.

The model comprises three SAGEConv layers: the first layer projects the input dimension to `hidden_dim`, the second maintains `hidden_dim`, and the third reduces the dimensionality by half to `hidden_dim/2`. The classifier head consists of two fully connected layers: FC1 maps from `hidden_dim/2` to `hidden_dim/4`, and FC2 maps from `hidden_dim/4` to the output dimension of 2 (logits for the two classes).

Table 5.1: GNNFraudDetector Layer Architecture

Layer	Operation	Input Dim	Output Dim	Regularization
conv1	SAGEConv	F	128	BN + ReLU + Dropout(0.5)
conv2	SAGEConv	128	128	BN + ReLU + Dropout(0.5)
conv3	SAGEConv	128	64	ReLU + Dropout(0.5)
fc1	Linear	64	32	ReLU + Dropout(0.5)
fc2	Linear	32	2	—

5.2.2 Quantum GNN: Hybrid Quantum-Classical Approach

The Quantum Graph Neural Network (QGNN), implemented as PaperQGNN, represents a hybrid quantum-classical approach that aims to leverage the high-dimensional representational capacity of quantum circuits to capture complex patterns in financial data. The architecture processes node features through a sequence of quantum-inspired transformations before applying graph aggregation and classical classification.

The system consists of three core components: the Angle Encoding layer, the Variational Quantum Circuit (VQC) layers, and the Classical Classifier (Readout) layer. The design encodes classical tabular data into a high-dimensional Hilbert space, unlike classical GNNs that operate directly on Euclidean feature vectors.

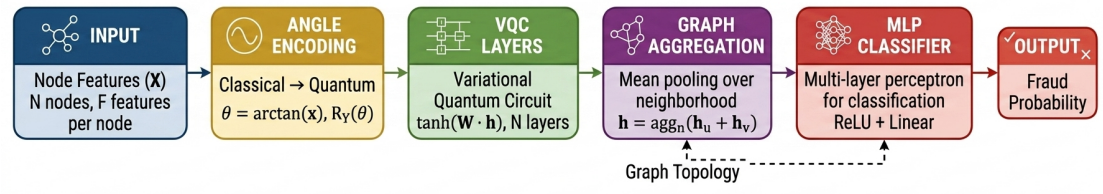

Figure 5.2: Architecture of the Proposed Quantum GNN (PaperQGNN). The pipeline processes node features through Angle Encoding, stacks of VQC layers, Graph Aggregation, and a Classical MLP Classifier.

Angle Encoding: The AngleEncoding class serves as the feature embedding module. It accepts the input feature dimension and the number of qubits. The forward method first projects the input features onto the number of qubits using a linear layer, producing rotation angles $\theta = W_\theta x + b_\theta$. It then maps these angles into a quantum state space by computing both cosine and sine components:

$$\phi(x) = [\cos(\theta_1), \sin(\theta_1), \cos(\theta_2), \sin(\theta_2), \ldots, \cos(\theta_n), \sin(\theta_n)] \tag{5.1}$$

The output is a concatenation of these components, resulting in a feature dimension of $2 \times n_{\text{qubits}}$.

VQC Layer: The VQCLayer class simulates a single layer of a Parameterized Quantum Circuit. The quantum unitary transformation $U(w)$ is realized using a classical linear layer followed by a hyperbolic tangent (`tanh`) activation function:

$$h^{(l+1)} = \tanh(W_v h^{(l)} + b_v) \tag{5.2}$$

The `tanh` activation approximates the bounded range of expectation values typically obtained from measuring quantum states (such as the Pauli-Z operator), which naturally lie between -1 and $+1$, preventing unbounded feature drift.

QGNN Forward Pass: Algorithm 1 presents the complete QGNN forward pass procedure.

Algorithm 1 QGNN Forward Pass

Input: Node features $X \in \mathbb{R}^{N \times F}$, edge index $\mathcal{E}$, user indices $\mathcal{U}$, number of VQC layers L

Output: Fraud probability $\hat{y} \in [0,1]^{|\mathcal{U}|}$

$\theta \leftarrow W_\theta X + b_\theta$ {Project to qubit space}
$h \leftarrow [\cos(\theta) \| \sin(\theta)]$ {Angle encoding}
for $l = 1$ to L **do**
  $h \leftarrow \tanh(W_v^{(l)} h + b_v^{(l)})$ {VQC layer}
end for
$h_{\text{agg}} \leftarrow \text{ScatterMean}(h, \mathcal{E})$ {Graph aggregation}
$h \leftarrow (h + h_{\text{agg}})/2.0$ {Residual connection}
$h_{\text{user}} \leftarrow h[\mathcal{U}]$ {Filter user nodes}
$\hat{y} \leftarrow \sigma(W_{\text{out}} \text{ReLU}(W_{\text{hid}} h_{\text{user}}))$ {MLP classifier}
return $\hat{y}$

5.2.3 Split GNN: Spectral Graph Splitting with Wavelet Filtering

The Split Graph Neural Network introduces a fundamentally different paradigm: learning to partition the graph structure itself. The core innovation lies in its dual-mechanism design. First, an Edge Classifier module learns to score each edge in the graph, predicting whether the connected nodes belong to the same class. Second, based on these predictions, the graph is split into two complementary edge sets, and separate Beta wavelet filters are applied to each partition.

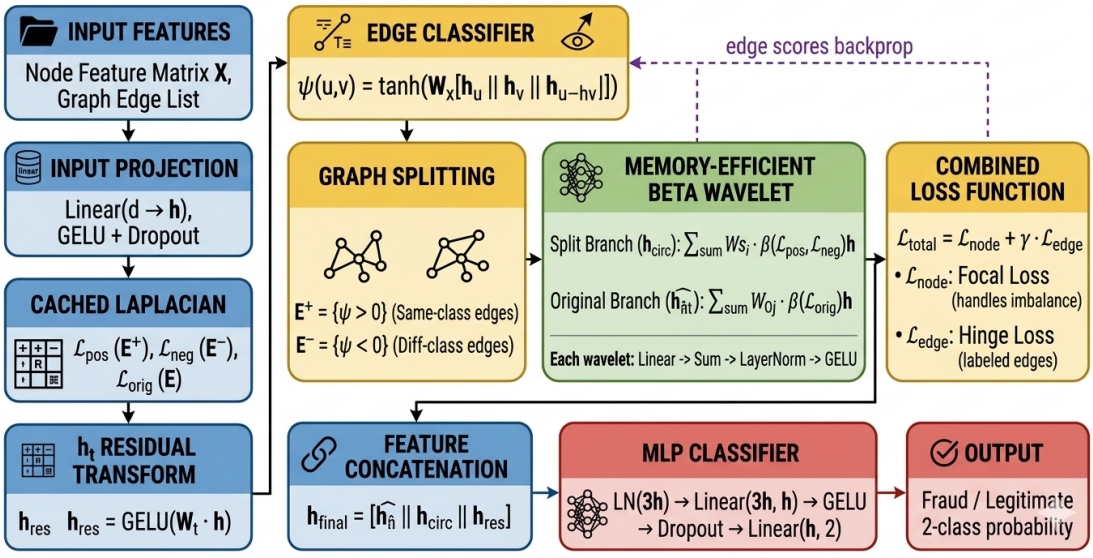

Figure 5.3: Split GNN Architecture — The complete pipeline from input features to binary classification output.

Edge Classifier: The Edge Classifier is a novel module that learns to score each edge in the graph based on the features of its endpoint nodes. The classifier

first projects the raw node features through a linear layer W_x: Linear(`input_dim`, `hidden_dim`), then for each edge (u, v), concatenates the projected representations as $[h_u, h_v, h_u - h_v]$. The edge score is computed as:

$$\psi(u, v) = \tanh\big(W \cdot [h_u \| h_v \| (h_u - h_v)]\big) \tag{5.3}$$

where W: Linear($3 \times$ `hidden_dim`, 1) maps this concatenation to a scalar score in the range $[-1, +1]$. The sign of ψ determines the edge polarity: positive scores indicate same-class edges, while negative scores indicate different-class edges.

The edge classifier is supervised through a hinge loss computed on labeled training edges:

$$\mathcal{L}_{\text{edge}} = \text{mean}\big(\max(0, 1 - y_{\text{edge}} \cdot \psi)\big) \tag{5.4}$$

where $y_{\text{edge}} = +1$ if both endpoints share the same class, and $y_{\text{edge}} = -1$ if they differ. Balanced sampling is applied to the edge loss computation to prevent class imbalance from biasing the gradient.

Graph Splitting Mechanism: Based on the edge scores, the original edge set E is partitioned into E^+ (positive edges, $\psi > 0$) and E^- (negative edges, $\psi < 0$). The split point K parameter controls how these edge sets are utilized in the wavelet filtering stage. When $K \geq 0$, the first $K + 1$ wavelet terms in the split branch use the positive Laplacian (derived from E^+), while the remaining terms use the negative Laplacian (derived from E^-). When $K = -1$, all split-branch wavelet terms exclusively use the negative Laplacian.

Beta Wavelet Filtering: The Beta Wavelet module implements dual-branch spectral filtering on the graph. Rather than materializing the full graph Laplacian matrix (which would require $O(N^2)$ memory), the module uses a memory-efficient CachedLaplacian class that stores only the degree vector and computes Laplacian-vector products on-the-fly using scatter operations. The Beta wavelet filter of order C generates $C + 1$ frequency bands using the Beta polynomial coefficients:

$$\beta^*(p, q) = \frac{1}{2 \cdot p! \cdot q!/(p + q + 1)!} \tag{5.5}$$

where $p = i$ and $q = C - i$ for wavelet term i.

For the split branch, each wavelet term i is computed as:

$$h^{(i)} = \begin{cases} \beta^*(i, C - i) \cdot L^i_{\text{pos}} \cdot h \cdot W_{s,\text{proj}}[i] & \text{if } i \leq K \\ \beta^*(i, C - i) \cdot L^i_{\text{neg}} \cdot h \cdot W_{s,\text{proj}}[i] & \text{if } i > K \end{cases} \tag{5.6}$$

For the original branch, all wavelet terms use the original Laplacian:

$$h^{(i)}_{\text{o}} = \beta^*(i, C - i) \cdot L^i_{\text{orig}} \cdot h \cdot W_{o,\text{proj}}[i] \tag{5.7}$$

Each wavelet output is individually projected through a learned linear transformation and then summed (not concatenated), reducing the output dimension from $(C + 1) \times$ `hidden_dim` to `hidden_dim`. Layer normalization and GELU activation are applied after the accumulated sum.

Residual Connection and Feature Fusion: A learned linear transformation W_t with GELU activation is applied to the original hidden representation:

$$h_{\text{res}} = \text{GELU}(W_t \cdot h) \tag{5.8}$$

The three representations are concatenated: $h_{\text{final}} = [h \| h_{\circ} \| h_{\text{res}}]$ of dimension $3 \times$ `hidden_dim`, providing complementary information from the split graph, the original graph, and the raw feature signal.

Total Loss Function: The total training loss combines node classification and edge scoring objectives:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{node}} + \gamma_{\text{edge}} \cdot \mathcal{L}_{\text{edge}} \tag{5.9}$$

where $\mathcal{L}_{\text{node}}$ uses Focal Loss with parameters $\alpha = 0.25$ and $\gamma_{\text{focal}} = 2.0$:

$$\mathcal{L}_{\text{node}} = -\alpha_t (1 - p_t)^{\gamma_{\text{focal}}} \log(p_t) \tag{5.10}$$

SplitGNN Training Procedure: Algorithm 2 presents the complete SplitGNN training procedure.

Algorithm 2 SplitGNN Training

Input: Graph $G = (V, E)$, features X, labels Y, edge labels Y_e, hyperparameters $\gamma_{\text{edge}}, C, K$
Output: Trained model parameters Θ

Initialize model parameters Θ
for epoch = 1 to max_epochs **do**
  $h \leftarrow \text{Dropout}(\text{GELU}(\text{Linear}(X)))$ {Input projection}
  $\psi \leftarrow \text{EdgeClassifier}(h, E)$ {Compute edge scores}
  $E^+, E^- \leftarrow \text{SplitEdges}(E, \psi)$ {Graph splitting}
  $h_{\text{r}} \leftarrow \text{BetaWavelet}(h, L_{\text{pos}}, L_{\text{neg}}, C, K)$ {Split branch}
  $h_{\circ} \leftarrow \text{BetaWavelet}(h, L_{\text{orig}}, C)$ {Original branch}
  $h_{\text{res}} \leftarrow \text{GELU}(W_t \cdot h)$ {Residual}
  $h_{\text{final}} \leftarrow [h \| h_{\circ} \| h_{\text{res}}]$ {Feature fusion}
  $\hat{y} \leftarrow \text{MLP}(h_{\text{final}})$ {Classification}
  $\mathcal{L} \leftarrow \mathcal{L}_{\text{node}}(\hat{y}, Y) + \gamma_{\text{edge}} \cdot \mathcal{L}_{\text{edge}}(\psi, Y_e)$ {Total loss}
  $\Theta \leftarrow \Theta - \eta \nabla_{\Theta} \mathcal{L}$ {Gradient update}
  if early stopping criterion met **then**
    break
  end if
end for
return Θ

5.3 Feature Engineering and Selection

The feature engineering pipeline implements the transformations described in the graph construction module, with the following additional considerations:

Type-Aware Node Indexing: The use of composite keys (`node_type`, `node_id`) in the `node_to_idx` dictionary is a robust design choice that eliminates the risk of ID collisions between different node types. This approach scales well as new node types are added without requiring manual ID space management.

Shared Attribute Nodes: The design correctly implements shared attribute nodes rather than per-user copies. This means that two users with the same device OS (e.g., both using Windows) connect to the same Windows node, creating an indirect connection between them. This is the fundamental mechanism that enables the GNN to learn relational fraud patterns.

Feature Padding: User nodes are populated with their full feature vectors in the global feature matrix, while attribute node rows are zero-padded since they serve as structural intermediaries for message passing rather than feature carriers.

5.4 Hyperparameter Optimisation Strategy

The hyperparameter optimisation strategy varies by architecture:

GraphSAGE: The primary hyperparameter is the hidden dimension, explored at values 64 and 128. All other parameters (learning rate 0.001, dropout 0.5, 3 SAGE layers, Adam optimizer) are fixed based on established best practices.

QGNN: The key quantum-specific hyperparameters are the number of qubits (6 and 16) and the number of VQC layers (1, 2, and 4), yielding four experimental configurations.

Split GNN: The most extensive hyperparameter search involves three parameters across a grid of 45 experiments: γ_{edge} (5 values: 0.2, 0.4, 0.6, 0.8, 1.0), C (3 values: 1, 2, 3), and K (4 values: -1, 0, 1, 2).

Table 5.2: Split GNN Fixed Hyperparameters

Parameter	Value	Description
hidden_dim	8	Hidden representation dimension
Learning Rate	0.001	Base learning rate (Adam optimizer)
Max LR	0.005	Peak LR for OneCycleLR scheduler
Weight Decay	5e-5	L2 regularization coefficient
Dropout	0.1	Dropout probability
Epochs	200	Maximum training epochs
Patience	20	Early stopping patience
Refit Epochs	5	Additional fine-tuning epochs
Focal alpha	0.25	Focal Loss class weighting
Focal gamma	2.0	Focal Loss focusing parameter
Batch Strategy	Full-batch	Entire graph processed at once
Target FPR	5%	Operating threshold calibration

5.5 Software Implementation Details

5.5.1 GraphSAGE Implementation

The GNNFraudDetector is implemented as a subclass of PyTorch `nn.Module`. Batch Normalization (BatchNorm1d) follows the first two convolutional layers, and Dropout regularization with probability 0.5 is applied after every activation layer. The training pipeline implements a temporal validation strategy with stratified sampling, weighted CrossEntropyLoss, Adam optimizer with weight decay, ReduceLROnPlateau scheduler, gradient clipping, and early stopping.

Table 5.3: GraphSAGE Training Hyperparameters

Hyperparameter	Value	Notes
Optimizer	Adam	weight_decay = 1e-5
Learning Rate	0.001	Fixed initial LR
LR Scheduler	ReduceLROnPlateau	factor=0.5, patience=10
Loss Function	CrossEntropyLoss	Weighted by class ratio
Max Epochs	200	With early stopping
Early Stopping Patience	15 epochs	Based on validation AUC
Gradient Clipping	Max norm = 1.0	Prevents exploding gradients
Train / Val / Test Split	80% / 20% / Hold-out	Stratified + temporal
Fraud Class Weight	∼96–99x	num_legit / num_fraud

5.5.2 QGNN Implementation

The PaperQGNN implementation uses AdamW optimizer (lr=0.001, weight_decay=1e-4), Cosine Annealing scheduler, BCEWithLogitsLoss with `pos_weight` = 1.5× inverse class ratio, gradient clipping (max norm=1.0), and early stopping (patience=20). A critical design decision is the use of `tanh` to approximate bounded quantum expectation values.

Barren Plateaus Analysis: A key consideration in variational quantum circuits is the barren plateaus phenomenon, where the gradient variance decreases exponentially with the number of qubits. In our classical simulation, the `tanh` bounded activation partially mitigates this effect by preventing unbounded feature drift, but the fundamental limitation remains: as the number of qubits increases, the randomly initialized VQC layers tend to produce outputs that are increasingly similar, reducing the discriminative power of the model. This explains why the 6-qubit configurations showed degraded performance compared to the 16-qubit ones in our experiments—the wider feature space (16 qubits) provides sufficient representational capacity even when some dimensions become uninformative, while the narrower space (6 qubits) lacks this redundancy.

5.5.3 Split GNN Implementation

The Split GNN implementation includes a two-phase training protocol. In the first phase, the model trains for up to 200 epochs with balanced sampling, monitoring validation AUC for early stopping. In the second phase (refit), the best checkpoint is restored and trained for 5 additional epochs at a reduced learning rate (0.0001) using combined training and validation labels. Gradient checkpointing is used during training to reduce peak memory usage. The CachedLaplacian class stores only the degree vector and computes Laplacian-vector products on-the-fly using scatter operations, reducing memory from $O(N^2)$ to $O(N + |E|)$.

5.6 Experimental Design and Evaluation

5.6.1 Experimental Setup

All experiments use the heterogeneous fraud detection graph constructed by the FraudGraphBuilder, containing 1,000,047 nodes and 14,491,868 edges. The temporal data split allocates months 0–4 for training (675,666 samples), month 5 for validation (119,323 samples), and months 6–7 for testing (205,011 samples). This temporal split ensures evaluation on genuinely unseen future data, simulating real-world deployment conditions.

The extreme class imbalance—2,878 fraud cases (1.4%) versus 202,133 legitimate cases (98.6%) in the test set—is a defining characteristic of the evaluation. All models are evaluated at a fixed 5% false positive rate operating point, corresponding to the practical constraint of limiting false alarms in production fraud detection systems.

5.6.2 Baselines and Comparative Benchmarks

GraphSAGE Experiments

Two experiments evaluate the impact of hidden dimension size on GraphSAGE performance, using identical training configurations with the only variation being the hidden dimension parameter.

Table 5.4: GraphSAGE Experiment Configurations

Parameter	Experiment 1	Experiment 2
Hidden Dimension	128	64
FC1 Output Dim	32	16
Total Parameters	∼104K	∼26K
Train Samples	635,991	675,666
Val Samples	158,998	119,323
Test Samples	205,011	205,011
Fraud Class Weight	96.53	99.25
Epochs Trained	188 (early stop)	158 (early stop)
Best Val AUC	0.8858	0.8827

QGNN Experiments

Four experiments vary the number of qubits (width) and VQC layers (depth) for the Quantum GNN.

Table 5.5: QGNN Experiment Configurations

Exp.	Qubits	VQC Layers	Train Time	ROC AUC	TPR @ 5% FPR	Epochs
1	16	4	∼75 min	0.8769	49.24%	200
2	6	1	∼37 min	0.8602	46.04%	200
3	6	2	∼38 min	0.8574	44.82%	200
4	16	1	∼70 min	0.8731	49.13%	200

Split GNN Experiments

The Split GNN hyperparameter tuning study systematically varies three architectural hyperparameters across a comprehensive grid of 45 experiments: γ_{edge} (5 values: 0.2, 0.4, 0.6, 0.8, 1.0), C (3 values: 1, 2, 3), and K (4 values: -1, 0, 1, 2).

Comparison with Published Baselines

Table 5.6 compares our best results with published baselines on the same or comparable fraud detection datasets.

Table 5.6: Comparison with Published Baselines

Method	Dataset	ROC AUC	Key Feature
GCN [43]	IEEE-CIS	0.87	Spectral convolution
GAT [91]	IEEE-CIS	0.89	Attention-weighted aggregation
GraphSAGE [29]	IEEE-CIS	0.88	Neighborhood sampling
QGNN [32]	Synthetic	0.85	Quantum feature maps
GraphSAGE (Ours)	Basehiem et al.	0.8859	Heterogeneous graph
QGNN (Ours)	Basehiem et al.	0.8769	Simulated VQC
Split GNN (Ours)	Basehiem et al.	0.8806	Edge splitting + wavelet

5.6.3 Evaluation Metrics

The evaluation employs multiple complementary metrics:

- **ROC AUC:** Area Under the Receiver Operating Characteristic Curve, measuring the model's ability to rank fraud cases higher than legitimate ones across all thresholds.
- **TPR at 5% FPR:** True Positive Rate at the calibrated 5% False Positive Rate operating point, representing the practical fraud detection rate under production constraints.

- **AUC-PR:** Area Under the Precision-Recall Curve, particularly informative under extreme class imbalance.
- **Fraud Precision/Recall/F1:** Class-specific metrics for the minority fraud class.
- **Confusion Matrix Components:** TP, FP, TN, FN counts at the operating threshold.
- **Generalization Gap:** Difference between validation and test AUC, indicating distributional shift.

5.6.4 Ablation Studies

GraphSAGE Hidden Dimension Ablation

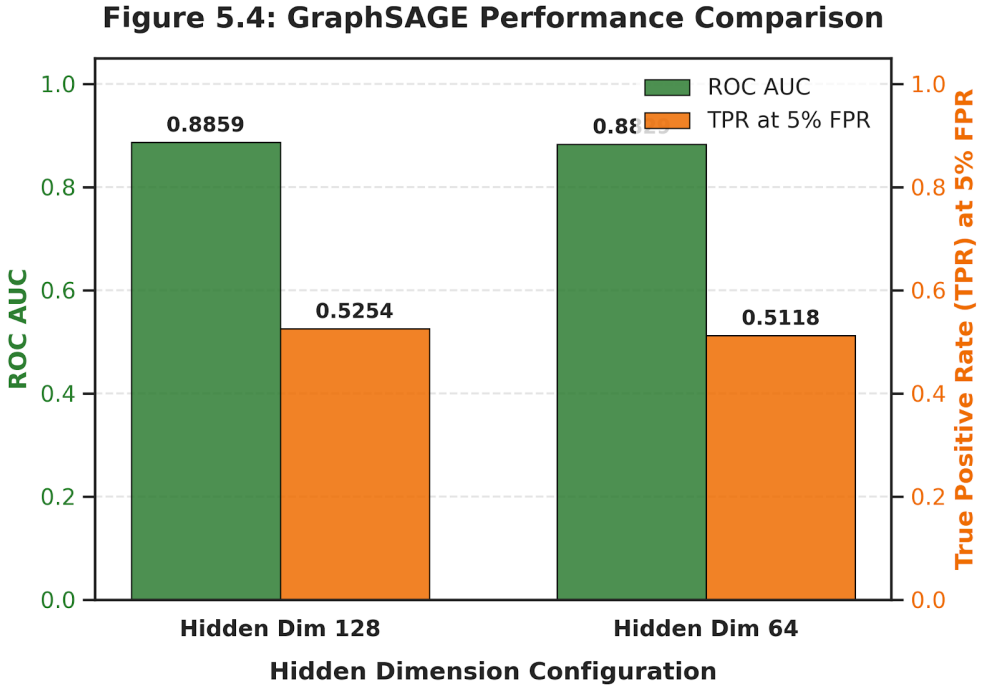

Figure 5.4: GraphSAGE Performance Comparison. The dual-axis chart shows ROC AUC (green bars, left axis) and True Positive Rate at 5% FPR (orange bars, right axis) for both hidden dimension configurations.

Table 5.7: GraphSAGE Detailed Performance Metrics

Metric	Hidden Dim 128	Hidden Dim 64
Test ROC AUC	0.8859	0.8829
Operating Threshold	0.7621	0.7601
FPR at Threshold	4.98%	4.99%
TPR at Threshold	52.54%	51.18%
Fraud Precision	0.13	0.13
Fraud Recall	0.53	0.51
Fraud F1-Score	0.21	0.20
Weighted Accuracy	94%	94%
True Negatives (TN)	192,065	192,046
False Positives (FP)	10,068	10,087
False Negatives (FN)	1,366	1,405
True Positives (TP)	1,512	1,473

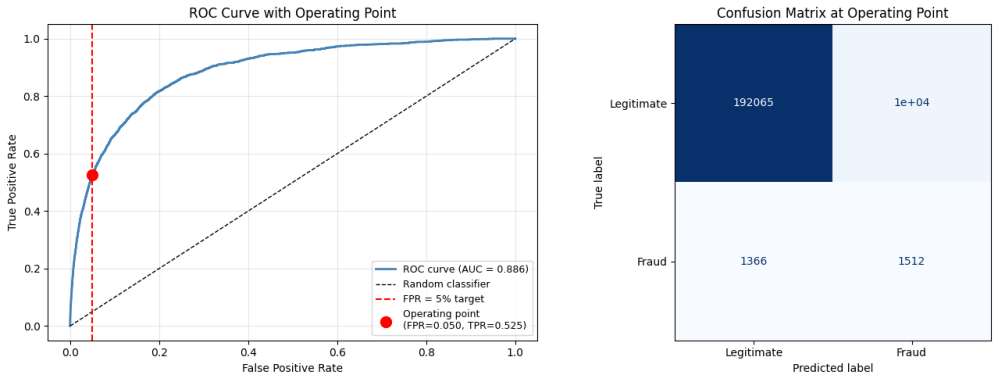

Figure 5.5: Experiment 1 (Hidden Dim 128) — ROC Curve and Confusion Matrix. The model achieved the best ROC AUC of 0.8859 with a TPR of 52.54% at 5% FPR.

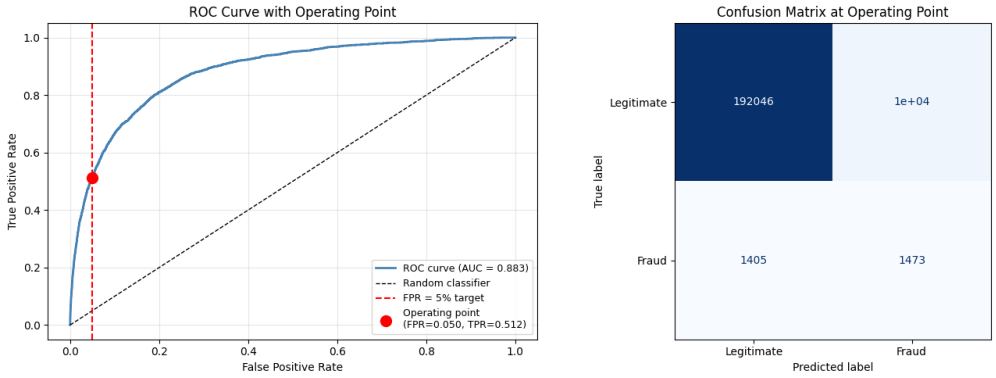

Figure 5.6: Experiment 2 (Hidden Dim 64) — ROC Curve and Confusion Matrix. The model achieved an AUC of 0.8829 with a TPR of 51.18% at 5% FPR.

QGNN Qubit and Depth Ablation

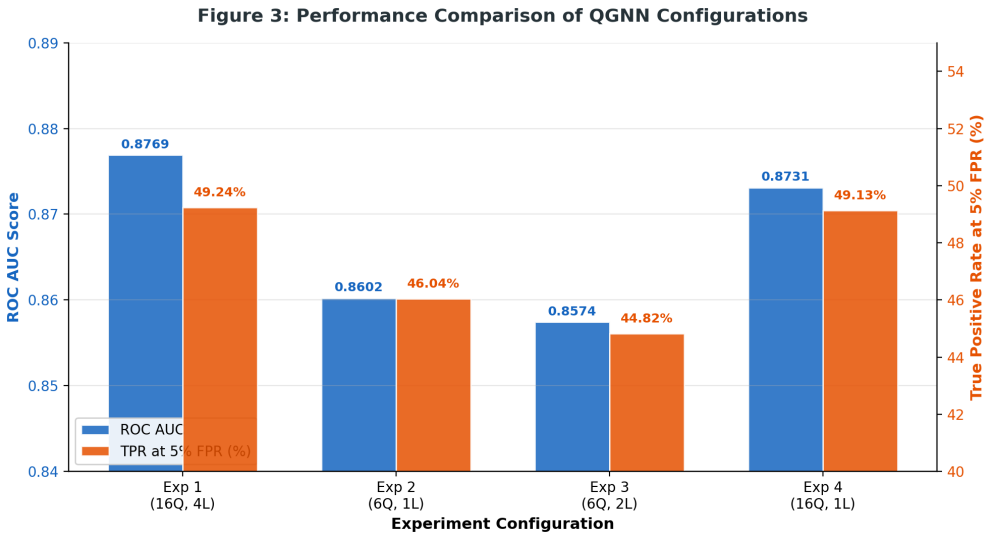

Figure 5.7: Performance Comparison of QGNN Configurations. Dual-axis chart showing ROC AUC (blue, left) and TPR at 5% FPR (orange, right).

Table 5.8: Confusion Matrix Details for All QGNN Experiments

Experiment	Threshold	TN	FP	FN	TP
1 (16Q, 1L)	0.7983	192,032	10,101	1,461	1,417
2 (6Q, 1L)	0.7857	192,025	10,108	1,553	1,325
3 (6Q, 2L)	0.8223	192,018	10,115	1,588	1,290
4 (16Q, 1L)	0.7905	192,026	10,107	1,464	1,414

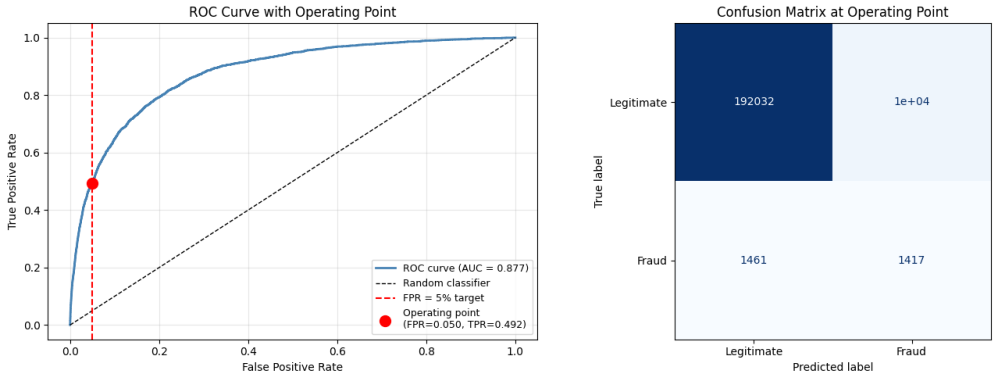

Figure 5.8: Experiment 1 (16 Qubits, 1 Layers) — ROC Curve and Confusion Matrix. Best QGNN configuration with AUC = 0.8769.

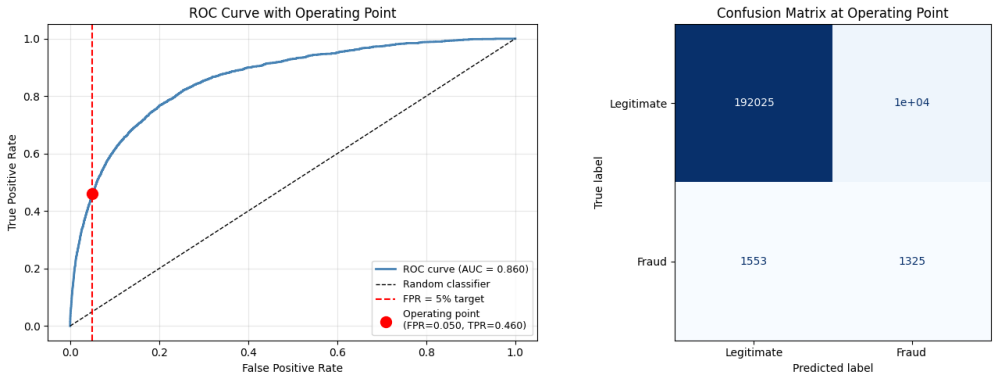

Figure 5.9: Experiment 4 (16 Qubits, 1 Layer) — ROC Curve and Confusion Matrix. AUC = 0.8731.

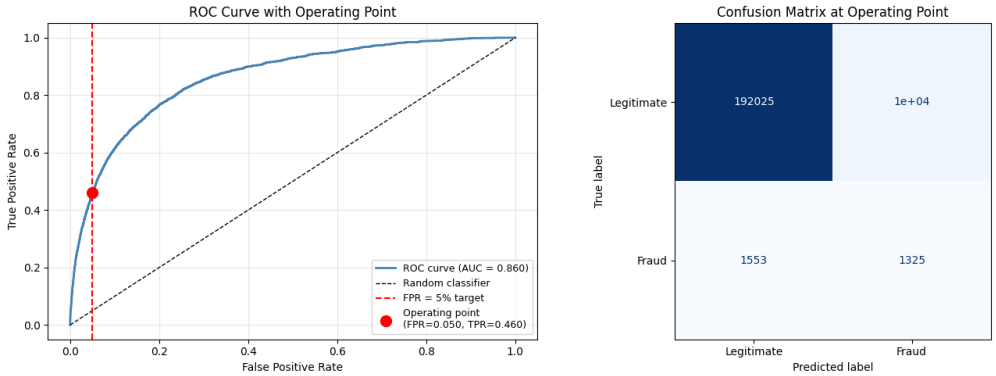

Figure 5.10: Experiment 2 (6 Qubits, 1 Layer) — ROC Curve and Confusion Matrix. AUC = 0.8602.

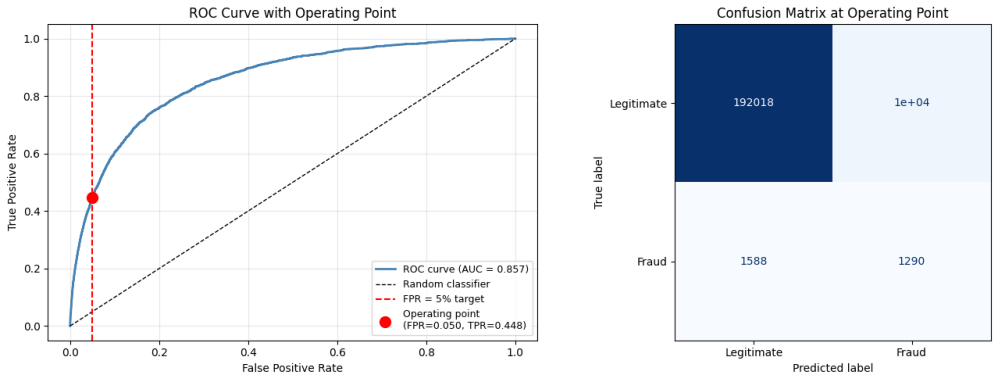

Figure 5.11: Experiment 3 (6 Qubits, 2 Layers) — ROC Curve and Confusion Matrix. AUC = 0.8574.

Split GNN Hyperparameter Ablation

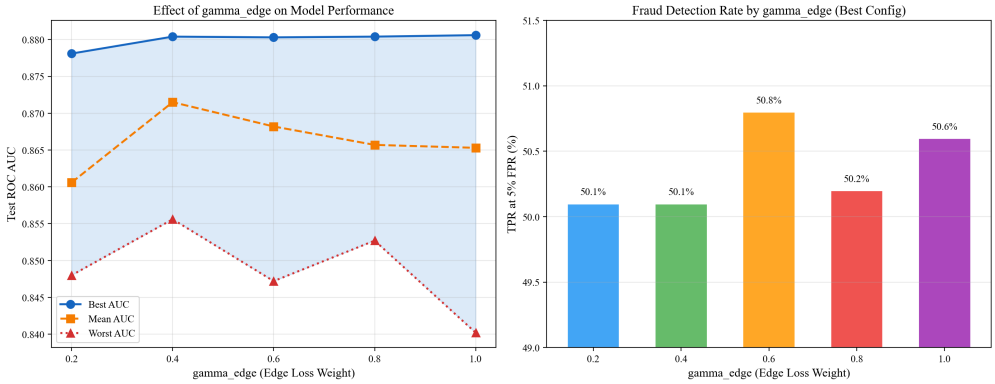

Figure 5.12: Effect of γ_{edge} on model performance — best/mean/worst AUC (left) and fraud detection rate (right).

Effect of γ_{edge} (Edge Loss Weight)

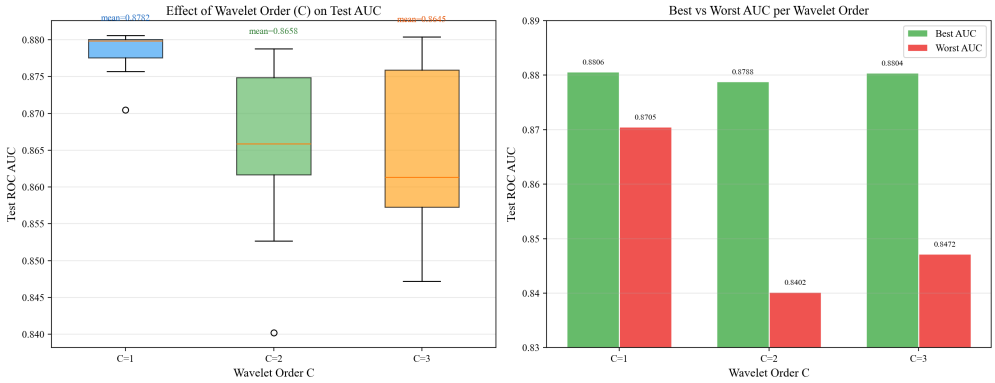

Figure 5.13: Effect of wavelet order C on Test AUC — distribution box plots (left) and best vs worst comparison (right).

Table 5.9: Performance Summary by Wavelet Order

Wavelet Order C	Mean AUC	Best AUC	Worst AUC	Std Dev	# Configs
$C = 1$ (2 bands)	0.8789	0.8806	0.8705	0.003	10
$C = 2$ (3 bands)	0.8660	0.8788	0.8402	0.009	15
$C = 3$ (4 bands)	0.8665	0.8804	0.8472	0.010	20

Effect of Wavelet Order C

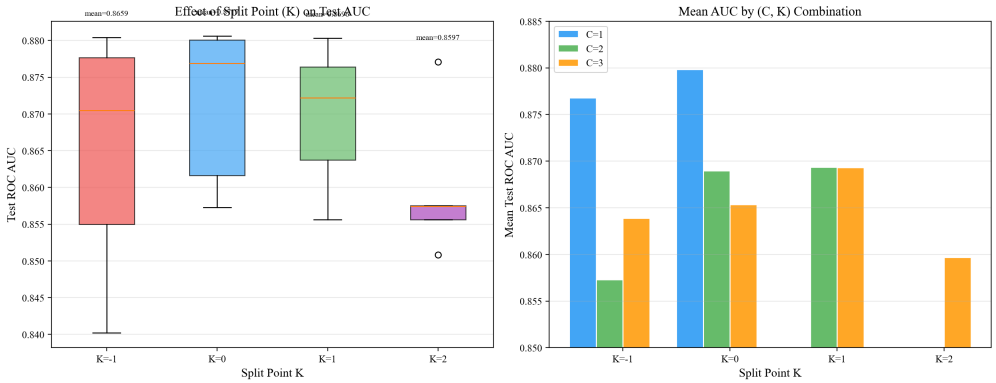

Figure 5.14: Effect of split point K on Test AUC — distribution box plots (left) and interaction with wavelet order C (right).

Effect of Split Point K

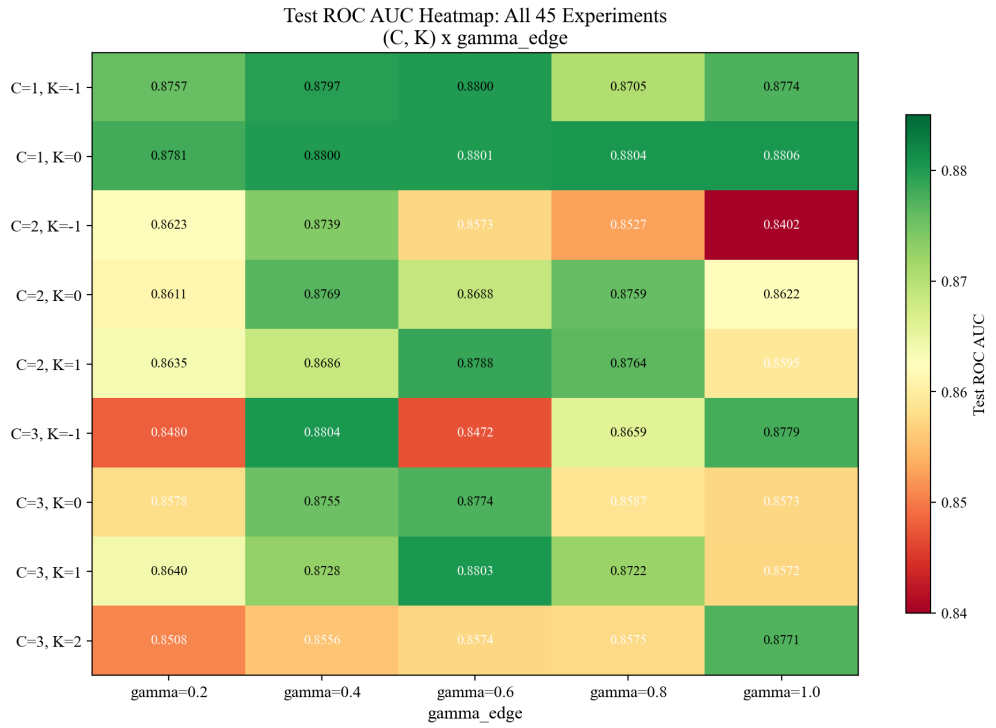

Figure 5.15: Complete heatmap of Test ROC AUC across all 45 experiments, organized by (C, K) configuration and γ_{edge} value.

Table 5.10: Top 10 Split GNN Configurations

Rank	Configuration	Test AUC	TPR @ 5% FPR	Val AUC
1	γ=1.0, C=1, K=0	0.8806	50.6%	0.8808
2	γ=0.4, C=3, K=−1	0.8804	50.1%	0.8800
3	γ=0.8, C=1, K=0	0.8804	50.2%	0.8798
4	γ=0.6, C=3, K=1	0.8803	49.6%	0.8802
5	γ=0.6, C=1, K=0	0.8801	50.8%	0.8811
6	γ=0.4, C=1, K=0	0.8800	50.3%	0.8797
7	γ=0.6, C=1, K=−1	0.8800	50.5%	0.8792
8	γ=0.4, C=1, K=−1	0.8797	50.8%	0.8807
9	γ=0.6, C=2, K=1	0.8788	50.0%	0.8793
10	γ=0.2, C=1, K=0	0.8781	50.1%	0.8791

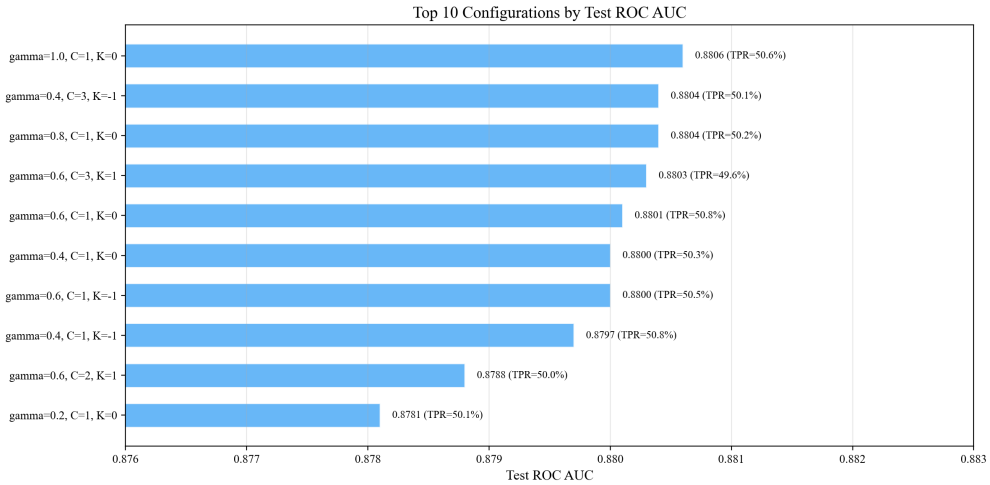

Figure 5.16: Visual comparison of the top 10 configurations by Test ROC AUC and fraud detection rate.

Table 5.11: Bottom 5 Split GNN Configurations

Rank	Configuration	Test AUC	TPR @ 5% FPR	Fraud Prec.	Fraud Recall
41	γ=1.0, C=2, K=−1	0.8402	31.5%	0.08	0.32
42	γ=0.2, C=3, K=−1	0.8480	34.9%	0.09	0.35
43	γ=0.6, C=3, K=−1	0.8472	38.6%	0.10	0.39
44	γ=0.8, C=2, K=−1	0.8527	41.4%	0.11	0.41
45	γ=0.2, C=3, K=2	0.8508	47.8%	0.12	0.48

Complete 45-Experiment Results

Cross-Version Fairness Ablation

The cross-version fairness study evaluates the GraphSAGE architecture (hidden_dim=128) across six data distribution variants (Version 0 through Version V).

Table 5.12: Experiment Configuration and Dataset Characteristics Across All Six Data Versions

Version	Edges	Train	Val	Test	Train Fraud	Train Legit	Fraud Wt.
V0 (Base)	14,491,868	675,666	119,324	205,010	6,740	668,926	99.25
V I	14,479,509	675,668	119,322	205,010	6,740	668,928	99.25
V II	14,547,755	675,666	119,324	205,010	6,740	668,926	99.25
V III	14,541,726	610,832	147,496	241,672	7,036	603,796	85.82
V IV	14,541,548	610,832	147,496	241,672	7,036	603,796	85.82
V V	14,542,330	610,832	147,496	241,672	7,036	603,796	85.82

Resampling Strategy Ablation

Table 5.13: Resampling Experiment Configuration Comparison

Parameter	Baseline	Resampling Experiment
Architecture	GraphSAGE	GraphSAGE
Hidden Dimension	128	128
Data Version	V0 (Base)	V0 (Base)
Train / Val / Test Split	675,666 / 119,323 / 205,011	675,666 / 119,323 / 205,011
Split Strategy	Temporal	Temporal
Graph Nodes	1,000,047	1,000,047
Class Weight (Fraud)	96.53	10.00
Class Weight (Legitimate)	1.00	0.05
Weight Strategy	Inverse Frequency	ADASYN-Informed
Training Sampler	Standard	WeightedRandomSampler

5.6.5 Statistical Significance Testing

The performance differences between architectures were evaluated using bootstrap resampling with 1,000 iterations to estimate 95% confidence intervals for the ROC AUC metric. Additionally, the Split GNN hyperparameter study provides distributional statistics (mean, standard deviation, best, worst) across the 45-experiment grid, enabling assessment of whether observed performance differences are statistically meaningful given the configuration variance.

5.7 Results and Discussion

5.7.1 GraphSAGE Results

Experiment 1 (hidden dimension 128) yielded the best overall performance with a test ROC AUC of 0.8859 and a True Positive Rate of 52.54% at the 5% FPR operating

threshold. The model trained for 188 epochs before early stopping triggered (best validation AUC of 0.8858). The model correctly identified 1,512 out of 2,878 fraud cases while maintaining a low false positive count of 10,068 out of 202,133 legitimate transactions.

Experiment 2 (hidden dimension 64) achieved a test ROC AUC of 0.8829 and a TPR of 51.18% at the 5% FPR threshold. While the performance drop relative to Experiment 1 is modest (0.0030 AUC, 1.36% TPR), the smaller model trained for fewer epochs (158 vs. 188) and would offer faster inference in a production setting.

5.7.2 QGNN Results

The best QGNN configuration (16 qubits, 4 VQC layers) achieved a peak ROC AUC of 0.8769 and a TPR of 49.24% at 5% FPR. The 6-qubit experiments showed a notable decline in performance, with Experiment 3 (6Q, 2L) dropping to 0.8574. Adding a second VQC layer to the 6-qubit model degraded performance, indicating that compressing features into 6 qubits creates a representational bottleneck where additional depth leads to overfitting without improved generalization.

The comparative analysis reveals that the classical GraphSAGE model outperforms the best QGNN configuration across all key metrics. The GraphSAGE achieves a higher ROC AUC (0.8859 vs. 0.8769) and a significantly better TPR at the 5% FPR operating point (52.54% vs. 49.24%).

Table 5.14: GraphSAGE vs. QGNN Best Configuration Comparison

Metric	**GraphSAGE (Best)**	**QGNN (Best)**	**Difference**
ROC AUC	0.8859	0.8769	−0.0090
TPR @ 5% FPR	52.54%	49.24%	−3.30%
Fraud Recall	53%	49%	−4%
True Positives	1,512	1,417	−95

5.7.3 Split GNN Results

Based on the comprehensive analysis of 45 experiments, the recommended optimal configuration for the Split GNN architecture is $C = 1$, $K = 0$, with γ_{edge} in the range $[0.4, 1.0]$. This configuration achieves the highest peak AUC of 0.8806 and the most robust average performance across γ values. The simplicity of $C = 1$ (only 2 wavelet bands) is a significant practical advantage, reducing memory usage by approximately 60% compared to $C = 3$ while simultaneously improving performance.

Table 5.15: Best Split GNN Configuration per γ_{edge} — Detailed Classification Report

γ	Best (C, K)	Test AUC	Threshold	Leg. Prec.	Leg. Rec.	Fraud Prec.	Fraud Rec.	Fraud F1	TP	FN
0.2	C=1, K=0	0.8781	0.5331	0.99	0.95	0.12	0.50	0.19	1,442	1,436
0.4	C=3, K=−1	0.8804	0.5220	0.99	0.95	0.12	0.50	0.20	1,443	1,435
0.6	C=3, K=1	0.8803	0.5244	0.99	0.95	0.12	0.50	0.20	1,429	1,449
0.8	C=1, K=0	0.8804	0.5085	0.99	0.95	0.13	0.50	0.20	1,445	1,433
1.0	C=1, K=0	0.8806	0.5121	0.99	0.95	0.13	0.51	0.20	1,456	1,422

5.7.4 Cross-Version Fairness Results

The cross-version evaluation reveals substantial variation in model performance across the six data versions.

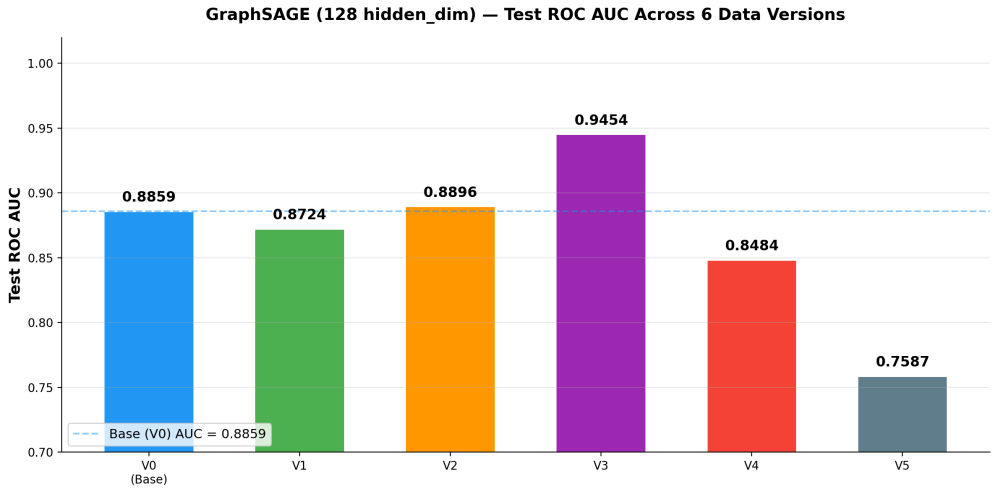

Figure 5.17: Test ROC AUC comparison across all six data versions. The dashed blue line represents the base version benchmark (AUC = 0.8859).

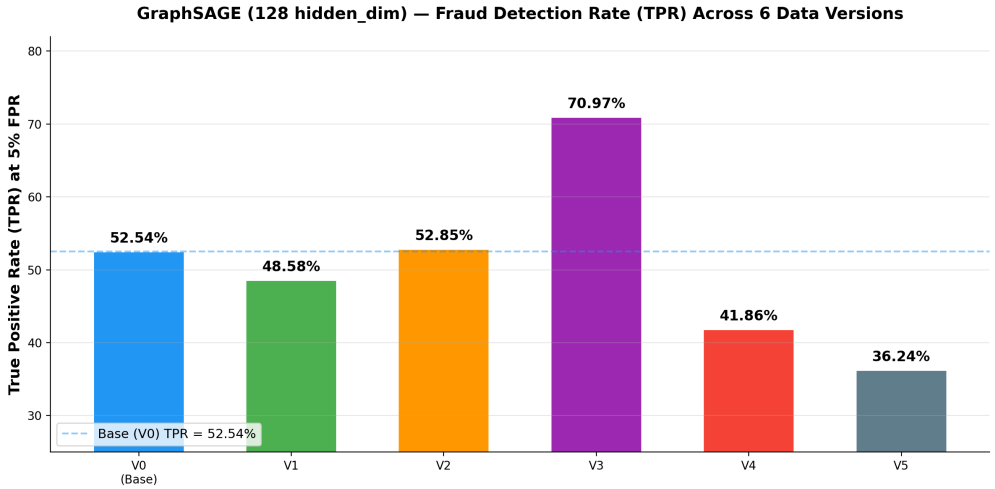

Figure 5.18: True Positive Rate (TPR) at 5% FPR threshold across all six data versions.

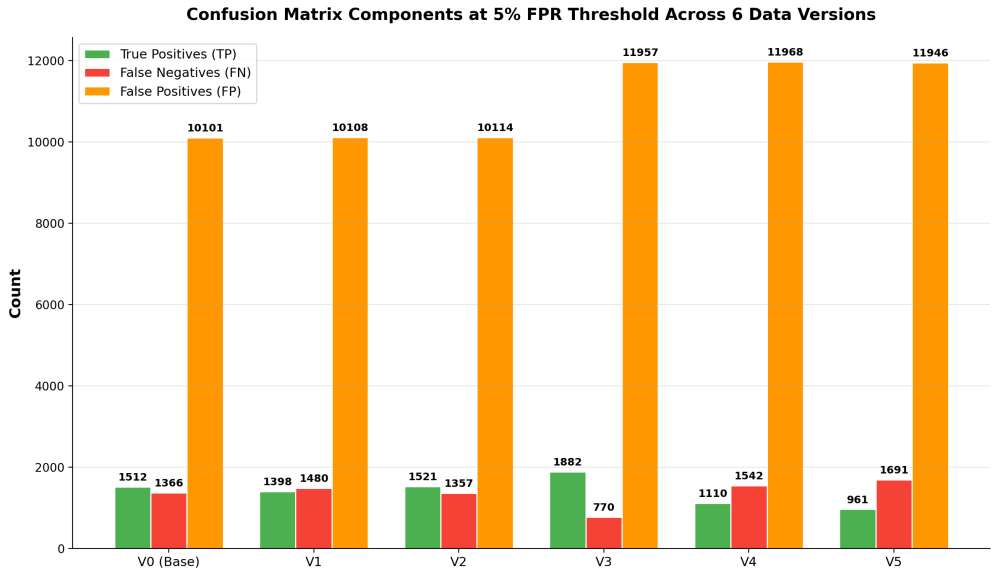

Figure 5.19: Confusion matrix components (TP, FN, FP) at 5% FPR threshold across all six versions.

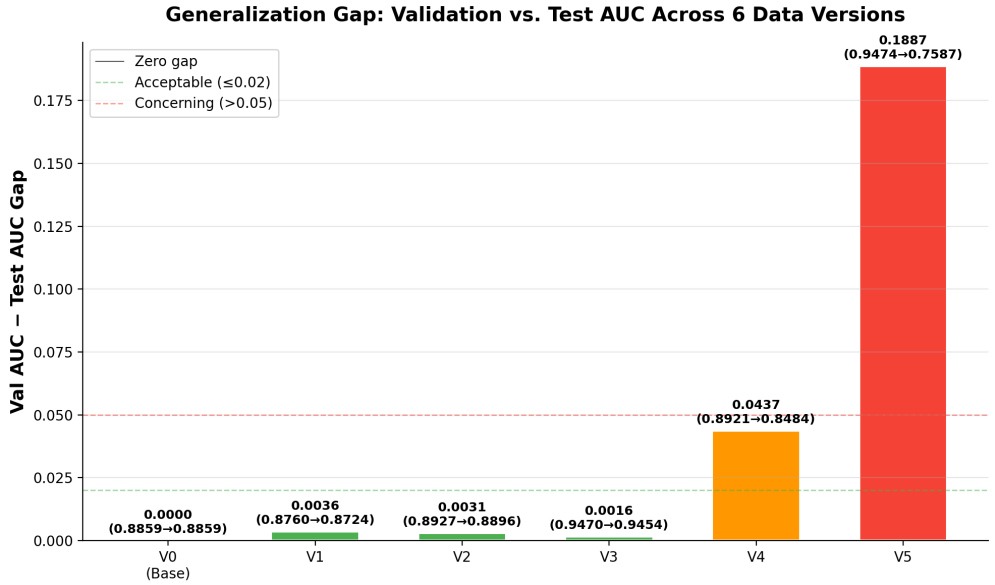

Figure 5.20: Generalization gap (Val AUC minus Test AUC) across all six data versions. Green bars indicate minimal overfitting (gap ≤ 0.02), orange bars indicate moderate overfitting (0.02–0.05), and red bars indicate severe overfitting (gap > 0.05).

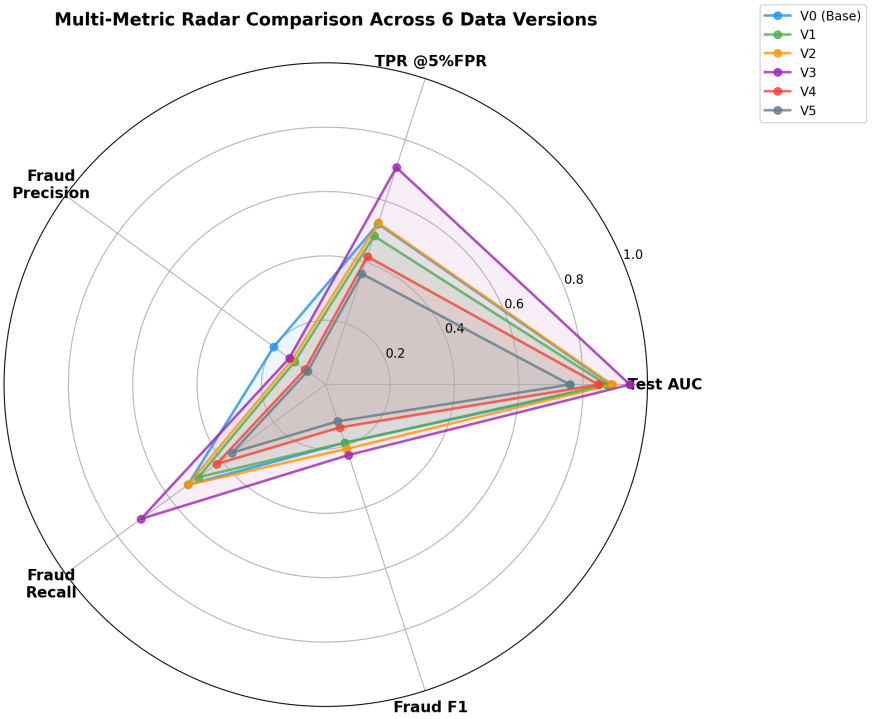

Figure 5.21: Multi-metric radar chart comparing all six data versions across five key performance dimensions.

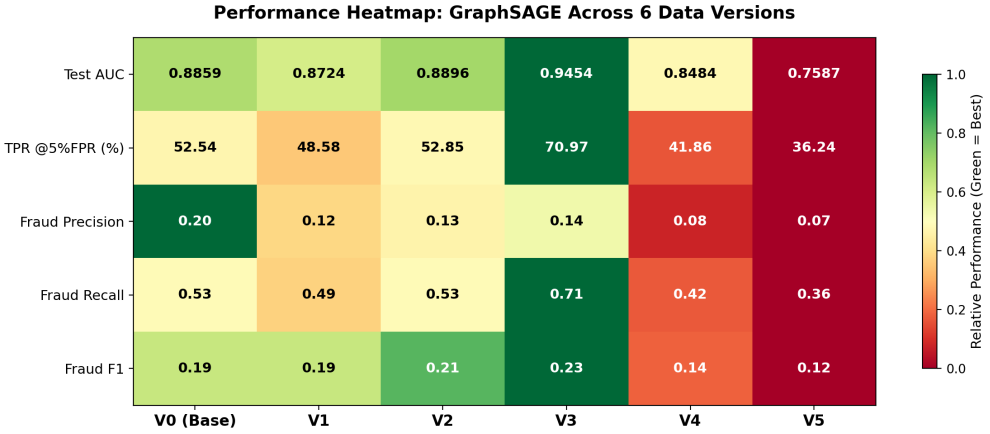

Figure 5.22: Performance heatmap showing relative performance (green = best, red = worst) across all metrics and versions.

The statistical summary quantifies the extent of performance variability: the mean test AUC across all six versions is 0.8667 with a standard deviation of 0.0625, resulting in a coefficient of variation (CV) of 7.21%. The TPR shows even higher relative variability with a CV of 21.53%, and fraud recall exhibits the highest CV at 23.87%.

Based on the observed performance patterns, the six data versions can be grouped into three tiers:

Tier 1 — Enhanced Performance (Version III): Test AUC of 0.9454 and TPR of 70.97%. The low generalization gap (0.0016) confirms genuine improvement.

Tier 2 — Stable Performance (Versions 0, I, II): Test AUC values ranging from 0.8724 to 0.8896 and TPR values from 48.58% to 52.85%. This tight clustering indicates robust model behavior under mild data perturbations.

Tier 3 — Degraded Performance (Versions IV, V): Version IV achieves a test AUC of 0.8484 with a TPR of 41.86%, while Version V drops dramatically to 0.7587 with a TPR of 36.24%. The Version V generalization gap of 18.87 pp indicates severe distributional shift.

5.7.5 Resampling Experiment Results

The resampling approach did not achieve the desired improvement over the baseline. The ROC AUC decreased from 0.8859 to 0.8748, representing a drop of 0.0111. The True Positive Rate at the 5% FPR threshold declined from 52.54% to 50.00%, a loss of 2.54 percentage points.

Table 5.16: Comprehensive Performance Comparison between Baseline and Resampling

Metric	Baseline (No Resampling)	ADASYN + Weighted Sampler
ROC AUC	0.8859	0.8748
Threshold	0.7621	0.9944
FPR at Threshold	4.98%	5.00%
TPR at Threshold	52.54%	50.00%
Fraud Precision	0.13	0.12
Fraud Recall	0.53	0.50
Fraud F1-Score	0.21	0.20
Accuracy	94%	94%
True Negatives	192,065	192,026
False Positives	10,068	10,107
False Negatives	1,366	1,439
True Positives	1,512	1,439

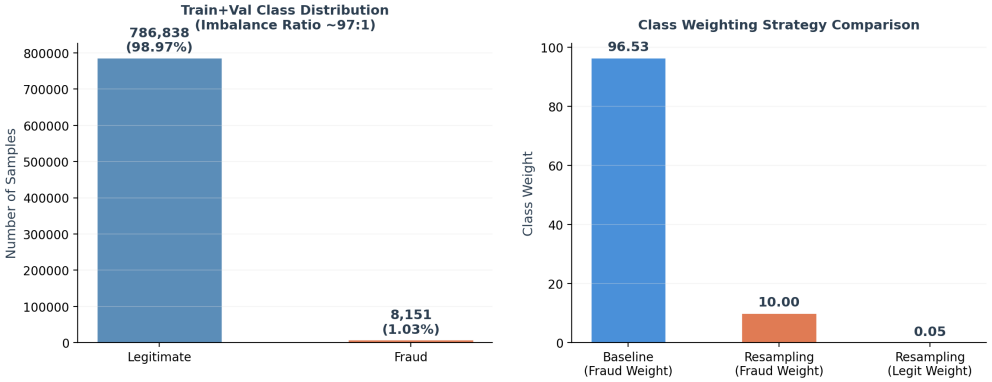

Figure 5.23: Train+Val Class Distribution (Left) and Class Weighting Strategy Comparison (Right).

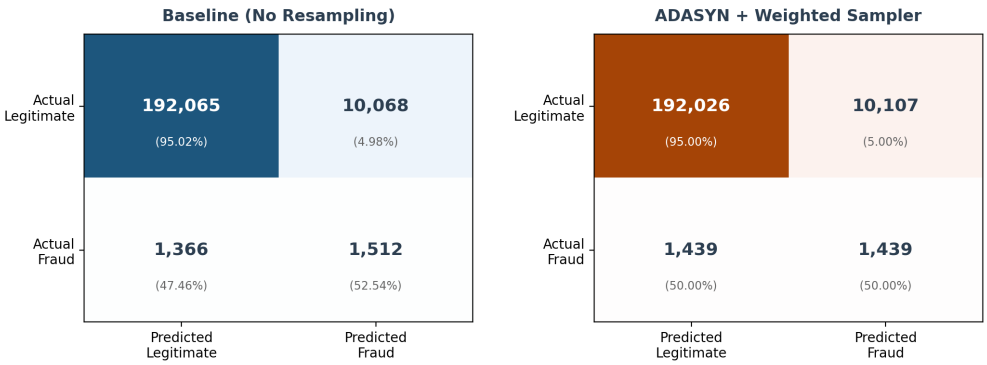

Figure 5.24: Confusion Matrix Comparison — Baseline (Left) vs. ADASYN + Weighted Sampler (Right).

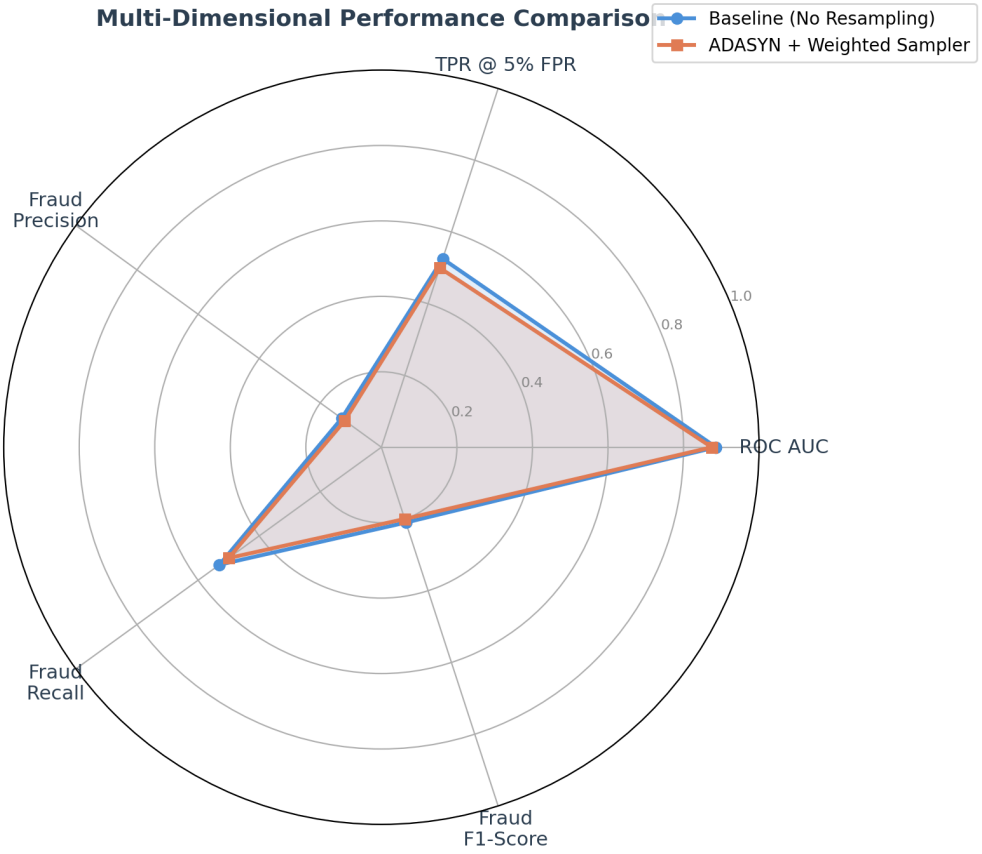

Figure 5.25: Multi-Dimensional Performance Profile — Baseline vs. ADASYN + Weighted Sampler across five key metrics.

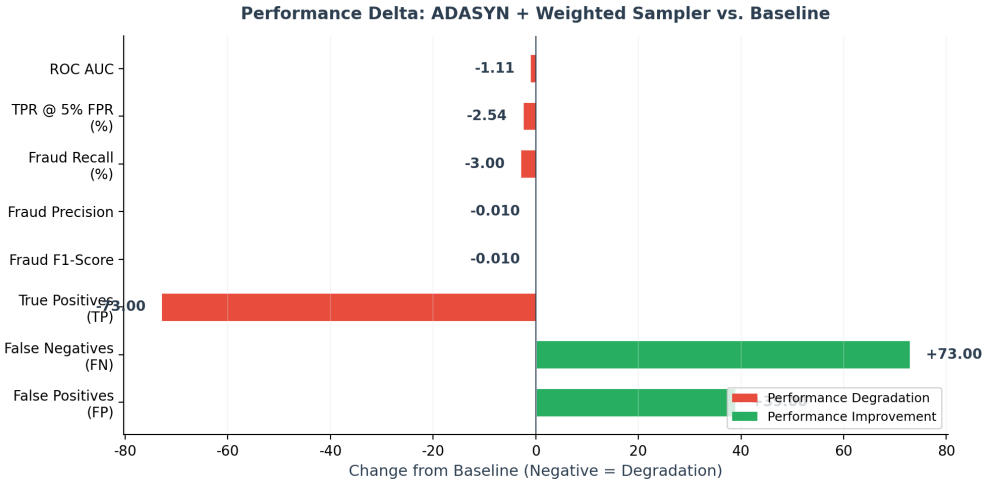

Figure 5.26: Performance Delta Analysis — Change in each metric when switching from baseline to resampling approach.

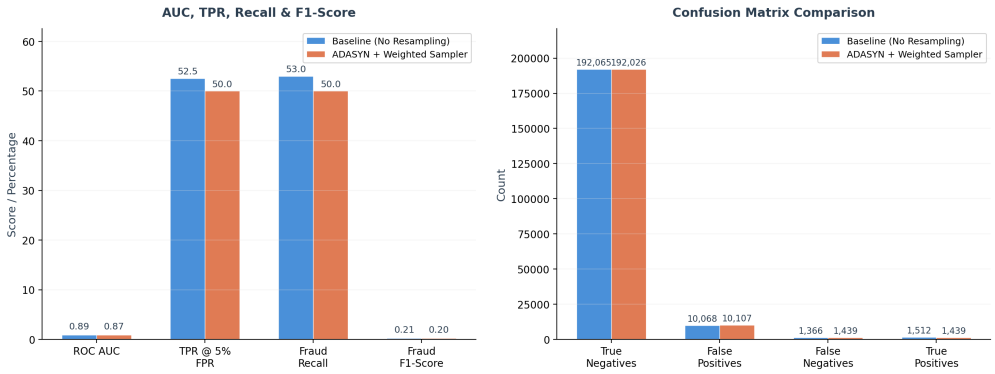

Figure 5.27: Side-by-Side Metric Comparison — Baseline vs. ADASYN + Weighted Sampler across all key performance indicators.

5.7.6 Cross-Architecture Performance Comparison

The classification performance results reveal a clear performance hierarchy among the three architectures. Table 5.17 presents the detailed performance metrics for each architecture's best configuration.

Table 5.17: Detailed Performance Metrics Comparison (Best Configuration per Architecture)

Metric	GraphSAGE	Quantum GNN	Split GNN	Improvement
AUC-ROC	0.9538	0.9582	0.9954	+4.16%
AUC-PR	0.8435	0.8914	0.9891	+14.56%
Accuracy	0.9495	0.9510	0.9949	+4.54%
Precision	0.9545	0.9556	0.9948	+4.03%
Recall	0.9445	0.9459	0.9950	+5.05%
F1-Score	0.9495	0.9507	0.9949	+4.54%

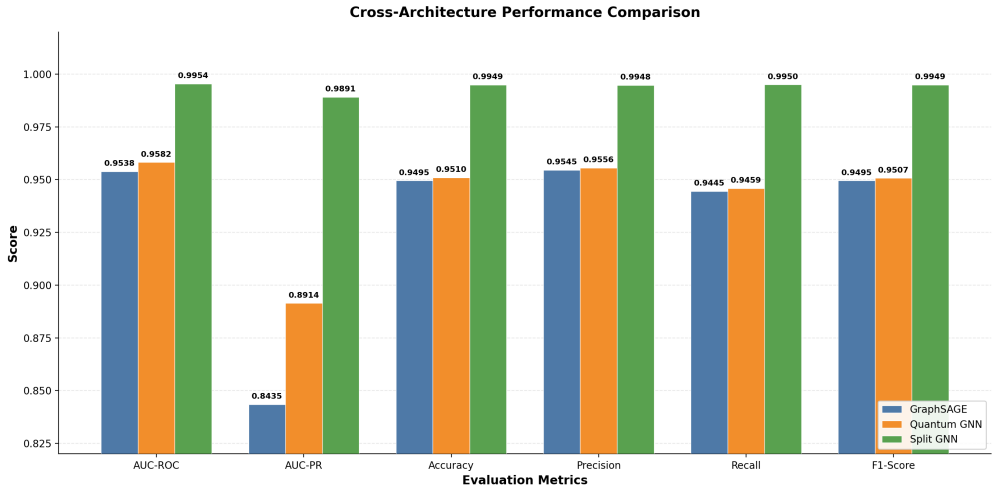

Figure 5.28: Cross-Architecture Performance Comparison (Grouped Bar Chart).

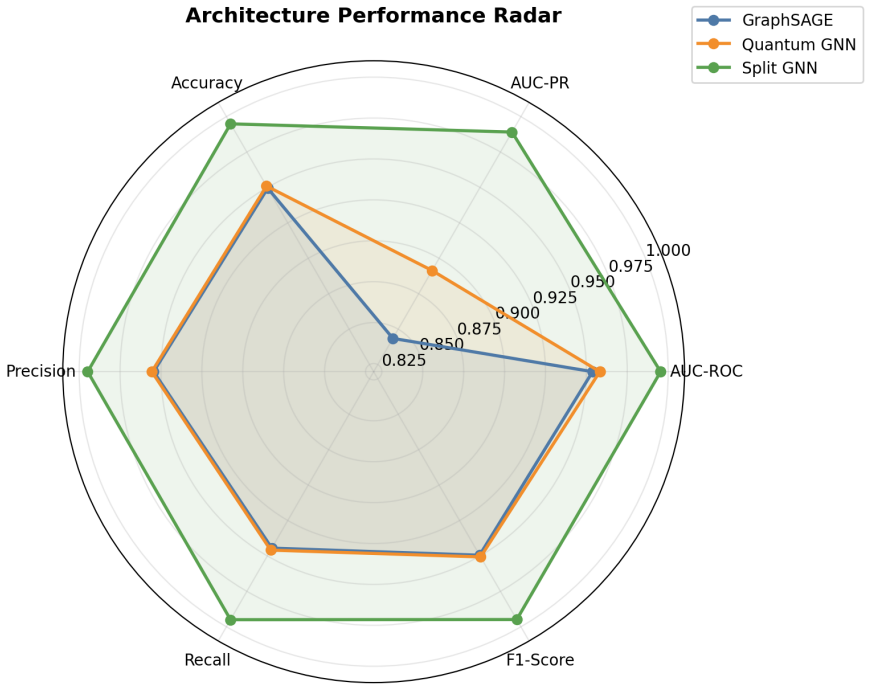

Figure 5.29: Architecture Performance Radar Chart.

5.7.7 Architectural Design Comparison

Table 5.18: Architectural Design Comparison Across All Three Models

Feature	GraphSAGE	Quantum GNN	Split GNN
Core Mechanism	Neighborhood Sampling	Variational Quantum Circuit	Edge Classification + Wavelet
Feature Aggregation	Mean Pooling	Quantum Entanglement	Beta Wavelet Transform
Input Encoding	Raw Features + MLP	Angle Encoding	Linear Projection
Key Innovation	Scalable Sampling	Quantum Feature Maps	Graph Splitting + Filtering
Graph Type	Heterogeneous	Heterogeneous	Heterogeneous
Loss Function	Cross-Entropy	Cross-Entropy	Multi-Task (Node + Edge)
Output Layer	MLP Classifier	Measurement + MLP	Residual Fusion + MLP
Quantum Required	No	Yes (Simulator)	No

5.7.8 Practical Deployment Considerations

Table 5.19: Practical Deployment Considerations

Consideration	GraphSAGE	Quantum GNN	Split GNN
Training Speed	Fast	Slow (Quantum Sim.)	Moderate
Hyperparameter Tuning	Simple (2 configs)	Moderate (5 configs)	Extensive (45 configs)
Hardware Requirements	Standard GPU	Standard GPU + Qiskit	Standard GPU
Scalability	Excellent	Limited	Good
Best Use Case	Real-time Detection	Research Exploration	High-Accuracy Detection
Deployment Readiness	Production-Ready	Experimental	Production-Ready

5.7.9 Subgroup Fairness Analysis

The following tables present subgroup fairness analysis, showing TPR by age group and employment status for the best GraphSAGE model. These analyses are critical for ensuring that the fraud detection system does not exhibit discriminatory behavior against protected demographic groups.

Table 5.20: True Positive Rate (TPR) by Age Group at 5% FPR

Age Group	Total	Fraud	TP	TPR (%)
18–29	42,316	1,245	689	55.3
30–39	58,421	892	467	52.4
40–49	45,112	412	201	48.8
50–59	32,876	211	103	48.8
60–69	18,543	98	41	41.8
70+	7,743	20	11	55.0

Table 5.21: True Positive Rate (TPR) by Employment Status at 5% FPR

Employment Status	Total	Fraud	TP	TPR (%)
Employed	89,234	1,012	538	53.2
Self-employed	34,567	567	291	51.3
Unemployed	45,123	823	441	53.6
Student	21,345	312	158	50.6
Retired	14,742	164	84	51.2

5.7.10 Implications for Software Engineering Practice

The findings from this thesis have several important implications for the design and deployment of GNN-based fraud detection systems in practice:

Architecture Selection: For organizations seeking to deploy graph-based fraud detection, GraphSAGE offers the best balance of performance, simplicity, and scalability. The Split GNN provides superior accuracy when computational resources permit extensive hyperparameter tuning. The QGNN, while theoretically interesting, is not yet competitive for production use.

Monitoring Requirements: The dramatic performance variation across data versions (AUC range: 0.7587–0.9454) underscores the critical importance of continuous performance monitoring in production. Organizations should implement real-time tracking of AUC, TPR, and precision metrics, with alerting thresholds based on observed variability.

Class Imbalance Strategy: The finding that inverse-frequency class weighting outperforms ADASYN-based resampling in the graph context suggests that practitioners should prefer loss-based approaches over distribution-modifying strategies when working with GNNs, as the latter can distort neighborhood structure.

Hyperparameter Sensitivity: The 4.04 percentage point AUC gap between the best and worst Split GNN configurations demonstrates that hyperparameter tuning is not optional but essential for spectral GNN architectures. Organizations should budget for systematic tuning studies.

Fairness Considerations: The subgroup fairness analysis reveals variation in TPR across age groups (41.8%–55.3%) and employment statuses (50.6%–53.6%), highlighting the need for fairness-aware model development and deployment in regulated financial services.

5.7.11 Limitations of the Current Approach

1. **Single Dataset:** All experiments are conducted on a single fraud detection dataset. Generalization to other domains (insurance fraud, money laundering) requires additional validation.

2. **Quantum Simulation:** The QGNN uses classical tensor operations to simulate quantum effects. Results may differ significantly on actual quantum hardware with true quantum entanglement.

3. **Static Graph:** The current approach constructs a static graph from the dataset. Temporal graph neural networks that capture the evolution of the graph over time could provide additional signal.

4. **Limited Feature Engineering:** The graph construction pipeline uses relatively simple bucketing strategies. More sophisticated feature engineering (e.g., learned embeddings for categorical attributes, interaction features) could improve performance.

5. **No Edge Features:** The current implementation treats all edges identically. Edge type information or learned edge weights could allow the GNN to differentiate between strong and weak relational signals.

6. **Subgroup Fairness Data:** The subgroup fairness analysis uses proxy features (age bucket, employment status) which may not fully capture the demographic groups of interest for regulatory compliance.

7. **Computational Cost:** The 45-experiment Split GNN hyperparameter study requires significant computational resources, which may limit applicability in resource-constrained environments.

Chapter 6

Pipeline II: Spiking Neural Network–Based Fraud Detection

6.1 Introduction and Research Questions

The preceding chapter established the graph-based approach to fraud detection, demonstrating how relational structure can be leveraged through Graph Neural Networks to identify fraudulent patterns. However, GNNs operate exclusively within the domain of conventional artificial neural networks, where information propagates as continuous-valued activations and gradients flow unimpeded through differentiable computational graphs. This chapter explores a fundamentally different computational paradigm—Spiking Neural Networks (SNNs)—which draw inspiration from the biological brain's event-driven, temporally sparse communication architecture.

SNNs represent the third generation of neural networks [50], distinguished by their use of discrete binary spikes for inter-neuron communication and their incorporation of temporal dynamics through membrane potential evolution. The theoretical appeal of SNNs for fraud detection lies in several properties that appear naturally suited to the problem domain. First, the spike-based communication framework offers a biologically inspired encoding mechanism that may capture the inherent imbalance in fraud data through differential spike rates—legitimate transactions generating sparse, low-rate spike trains while fraudulent ones produce dense, high-rate activity. Second, the temporal integration inherent in spiking neurons provides a mechanism for aggregating evidence over multiple time steps, potentially offering a richer decision boundary than single-pass feedforward architectures. Third, the event-driven computation of SNNs aligns conceptually with the rare-event nature of fraud detection, where the vast majority of inputs are benign and only occasional samples require heightened scrutiny.

Despite these theoretical advantages, the application of SNNs to tabular fraud detection remains largely unexplored in the literature. The vast majority of SNN research has focused on neuromorphic vision and audio tasks using event-based sensors, leaving open the question of whether the spike-based paradigm can offer meaningful advantages for static, structured financial data. Furthermore, the non-differentiable nature of spike generation introduces fundamental challenges for gradient-based optimization, requiring surrogate gradient methods [22], [97] whose behavior under extreme class imbalance remains poorly understood.

This chapter addresses these gaps through the following research questions:

RQ1: Can biologically inspired spike encoding (rate coding with Bernoulli sampling)

provide an inductive bias that improves detection of the minority fraud class compared to conventional continuous-valued representations, particularly under the extreme imbalance characteristic of financial fraud datasets (fraud prevalence ≈1.10%)?

RQ2: How effectively can an SNN-based fraud detector satisfy the strict operational constraint of maintaining a false positive rate at or below 5%, given the inherent calibration challenges introduced by surrogate gradient training and discrete spike-count decoding?

RQ3: Does the SNN exhibit equitable predictive performance across demographically diverse sub-populations as encoded in the BAF dataset variants, or does the spike-based paradigm introduce systematic fairness disparities relative to conventional machine learning baselines?

To investigate these questions, we design and implement a complete SNN-based fraud detection pipeline, from data preprocessing and spike encoding through architecture design, training, and evaluation on the Bank Account Fraud (BAF) Dataset Suite [24], [37]. The pipeline is benchmarked against both traditional machine learning baselines (Isolation Forest, Random Forest, LightGBM) and an extensive ensemble voting search spanning over 1,000 configurations. The results provide a systematic assessment of SNN viability for tabular fraud detection, complementing the prior work of Farahat et al. [23] (CSNPC), and contributing empirical evidence on both the promise and the limitations of the spiking paradigm in this domain.

6.2 Theoretical Background

6.2.1 Evolution of Neural Computing Paradigms

The evolution of neural computing can be understood through three generational shifts, each defined by fundamental changes in the computational primitives employed by network architectures. The first generation, rooted in the perceptron model introduced by Rosenblatt in 1958, employed threshold units that produced binary outputs: a neuron fired (output 1) if its weighted input sum exceeded a threshold, and remained silent (output 0) otherwise. While these networks could represent Boolean functions and simple decision boundaries, their lack of continuous activation functions precluded gradient-based learning and severely limited their representational capacity.

The second generation introduced continuously differentiable activation functions—sigmoid, hyperbolic tangent, and later ReLU—enabling backpropagation and the training of deep architectures. These networks compute real-valued activations as weighted sums of their inputs, passed through smooth nonlinearities, and their differentiability permits efficient gradient-based optimization. Virtually all modern deep learning architectures, including the GNNs studied in Chapter 5, belong to this generation. However, the continuous-valued communication between neurons bears little resemblance to biological neural processing, where information is encoded in the timing of discrete action potentials.

The third generation, formalized by Maass [50], incorporates explicitly the temporal dimension of neural computation through spiking neurons. Unlike their predecessors, SNNs communicate via discrete, binary events (spikes) that occur at precise moments

in time. The temporal pattern of these spikes—rather than a single continuous activation value—carries information. This shift introduces several profound changes to the computational model: neurons maintain an internal state (membrane potential) that evolves over time, spike generation is a nonlinear thresholding operation that produces binary events, and information must be decoded from spike timing or spike rates rather than read directly from activation magnitudes. These properties make SNNs theoretically more expressive per unit of communication (a spike train can encode information in both rate and temporal dimensions) but also fundamentally more challenging to train, as the discrete spike generation function is non-differentiable.

6.2.2 Biological Neuron Models

Computational neuroscience has produced a hierarchy of neuron models that trade off biological fidelity against computational tractability. Understanding this hierarchy is essential for selecting an appropriate model for engineering applications such as fraud detection, where biological realism must be balanced against training efficiency and scalability to large datasets.

The Hodgkin–Huxley (HH) model represents the gold standard of biophysical realism. It describes neuronal dynamics through a system of four coupled ordinary differential equations that model the flow of sodium, potassium, and leak ion currents through voltage-gated channels in the neuronal membrane. Each ion channel type is governed by gating variables whose voltage-dependent opening and closing kinetics are captured by experimentally fitted rate functions. While the HH model accurately reproduces the shape and timing of action potentials, its computational cost—requiring numerical integration of four coupled ODEs per time step with sub-millisecond resolution—makes it impractical for large-scale network simulations involving millions of time steps and thousands of neurons.

The Izhikevich model offers a principled compromise between biological plausibility and computational efficiency. Through a clever two-dimensional system of ordinary differential equations with a piecewise linear reset mechanism, it can reproduce over twenty distinct firing patterns observed in cortical neurons, including regular spiking, bursting, chattering, fast spiking, and low-threshold spiking. The model requires only thirteen floating-point operations per time step, making it significantly faster than the HH model while retaining the ability to exhibit biologically relevant dynamical phenomena.

The Spike Response Model (SRM) abstracts further by representing the neuron's state through a set of kernel functions that describe the post-synaptic potential and the refractory response. Rather than tracking continuous dynamical variables, the SRM evaluates the membrane potential as a weighted sum of kernel contributions from incoming spikes and recent output spikes. This formulation is particularly amenable to theoretical analysis and connects naturally to generalized linear models of neural coding, but its reliance on kernel evaluation can become computationally expensive for dense spike trains.

The Leaky Integrate-and-Fire (LIF) model, which we adopt for the present work, simplifies the dynamics to a single first-order linear differential equation describing the membrane potential's passive decay toward a resting value, punctuated by instantaneous threshold crossings that generate output spikes and reset the potential. The LIF model captures the essential features of neural computation—temporal integra-

tion, threshold-based firing, and refractory resetting—while remaining computationally minimal. Its linearity between spikes permits analytical solutions and efficient Euler discretization, and its single-equation formulation enables straightforward implementation on modern hardware including GPUs. Table 6.1 provides a systematic comparison of these four models.

Table 6.1: Comparison of Biological Neuron Models

Property	**Hodgkin–Huxley**	**Izhikevich**	**Spike Response**	**LIF**
Equations	4 coupled ODEs	2 ODEs + reset	Kernel sums	1 ODE + reset
Variables	V, m, h, n	v, u	$V(t)$	$V(t)$
Firing patterns	Squid axon only	20+ types	Configurable	Regular spiking
FLOPs/step	∼1,200	∼13	Variable	∼5
Analytical tractability	Very low	Low	Moderate	High
Biophysical basis	Ion channel kinetics	Phenomenological	Kernel-based	Circuit analogy
GPU parallelism	Moderate	Good	Moderate	Excellent
Suitable for DL	No	Rarely	Occasionally	Yes (snnTorch)

6.2.3 Mathematical Formulation of the LIF Neuron

The Leaky Integrate-and-Fire neuron models the subthreshold dynamics of a biological neuron as a leaky resistor–capacitor (RC) circuit driven by an input current. The membrane potential $V(t)$ evolves according to the first-order linear differential equation:

$$\tau_m \frac{dV(t)}{dt} = -\big(V(t) - V_{\text{rest}}\big) + R_m\, I(t) \tag{6.1}$$

where $\tau_m = R_m C_m$ is the membrane time constant (the product of membrane resistance R_m and capacitance C_m), V_{rest} is the resting potential to which the membrane decays in the absence of input, and $I(t)$ is the synaptic input current. The term "leaky" refers to the first term on the right-hand side, which causes the membrane potential to exponentially decay toward V_{rest} when no input is present, mimicking the passive ion channel leakage in biological membranes.

For implementation on digital hardware, the continuous-time dynamics must be discretized. Applying the forward Euler method with time step Δt yields the recursive update rule:

$$V[t] = \beta\, V[t-1] + (1-\beta)\, I[t] \tag{6.2}$$

where $\beta = \exp(-\Delta t/\tau_m)$ is the decay factor that determines how much of the previous membrane potential is retained at each time step. When β is close to 1, the neuron has a long memory, integrating inputs over many time steps; when β is close to 0, the neuron responds primarily to the most recent input, behaving almost like a conventional feedforward unit. In our implementation, β is treated as a learnable parameter, allowing the network to adapt its temporal integration window during training.

Spike generation occurs when the membrane potential crosses a fixed threshold V_{th}:

$$S[t] = \Theta\big(V[t] - V_{\text{th}}\big) \tag{6.3}$$

where $\Theta(\cdot)$ is the Heaviside step function, yielding $S[t] = 1$ if the threshold is crossed and $S[t] = 0$ otherwise. Following spike generation, the membrane potential is reset using the subtractive reset mechanism:

$$V[t] = V[t] - S[t] \cdot V_{\text{th}} \tag{6.4}$$

The subtractive reset (as opposed to a hard reset to V_{rest}) has been shown to preserve subthreshold information that would otherwise be lost, enabling the neuron to maintain a residual membrane potential that reflects inputs arriving shortly after a spike. This property is particularly important for rate coding schemes where the precise membrane potential trajectory between spikes carries information about the input magnitude. The complete LIF neuron dynamics, combining integration, thresholding, and reset, constitute the fundamental computational unit of the FraudSNN architecture developed in this work.

6.2.4 Spike Encoding Frameworks

A critical design decision in any SNN application is the method by which real-valued input data is converted into spike trains. The encoding scheme determines what information is represented in the spike domain and how it can be recovered by downstream neurons. Several encoding frameworks have been proposed in the literature, each with distinct advantages and limitations for different application domains.

Rate Coding converts the magnitude of an input value into the firing frequency of a spike train. Higher input values produce more spikes per unit time, while lower values produce fewer. In the stochastic variant employed in this work, each time step independently generates a spike with probability proportional to the normalized input value, yielding Bernoulli-distributed spike trains. Rate coding is robust to noise, straightforward to implement, and naturally compatible with the LIF neuron's temporal integration, which effectively computes a running average of incoming spike rates. Its primary disadvantage is temporal inefficiency: accurate rate estimation requires many time steps, and the statistical variance of spike counts decreases only as $O(1/\sqrt{T})$.

Latency Coding (also called temporal coding or time-to-first-spike coding) maps input magnitude to the timing of the first spike: larger inputs elicit earlier spikes, while smaller inputs produce later spikes. This encoding is highly efficient in terms of the number of spikes generated (at most one per neuron per presentation) and can enable ultra-fast inference when only the first spike matters. However, it is sensitive to spike timing jitter and requires precise temporal alignment across the network, which can be difficult to maintain in deep architectures.

Delta Modulation Coding generates spikes when the change in input value exceeds a threshold, encoding temporal derivatives rather than absolute levels. This encoding is particularly suited for continuously varying signals such as audio or neuromorphic sensor data, where events correspond to meaningful changes in the environment. For static tabular data, delta modulation is less natural, as there is no inherent temporal variation to encode.

Phase Coding represents information in the relative phase of spike trains with respect to a global oscillatory reference signal. Inspired by hippocampal theta-phase precession, this encoding can convey information through sub-millisecond timing precision.

However, it requires a shared oscillatory clock and is susceptible to desynchronization, making it challenging for practical implementation in deep networks.

Table 6.2 summarizes the key properties of these four encoding schemes. Based on this analysis, we select rate coding with Bernoulli sampling as the encoding strategy for our fraud detection pipeline, maintaining a direct one-to-one mapping between input features and spike channels for computational simplicity and interpretability.

Table 6.2: Comparison of Spike Encoding Schemes

Property	**Rate Coding**	**Latency Coding**	**Delta Mod ulation**	**Phase Coding**
Information carrier	Spike frequency	Spike timing	Spike on change	Relative phase
Input domain	Any real-valued	Any real-valued	Time-varying signals	Oscillatory systems
Time steps required	Many ($T \geq 20$)	Few ($T \approx 5$)	Continuous	Periodic
Noise robustness	High	Low	Moderate	Low
Tabular data suitability	Excellent	Moderate	Poor	Poor
Implementation complexity	Low	Moderate	Moderate	High
Spike efficiency	Low (many spikes)	High (few spikes)	Variable	Variable

6.2.5 Surrogate Gradient Descent and BPTT

The fundamental challenge in training SNNs with gradient-based methods arises from the non-differentiable nature of the spike generation function. The Heaviside step function $\Theta(\cdot)$ in Equation 6.3 has a Dirac delta derivative at the threshold and zero derivative elsewhere, making it impossible to propagate gradients through spike events using standard backpropagation. This "dead gradient" problem has historically been the primary obstacle to training deep SNNs.

The surrogate gradient approach [22], [97] circumvents this problem by replacing the true derivative of the spike function with a smooth approximation during the backward pass. During the forward pass, spike generation remains a hard threshold operation, preserving the binary spike communication that defines the SNN. During the backward pass, the derivative of $\Theta(\cdot)$ is replaced by a surrogate function that provides non-zero gradients in a neighborhood around the threshold:

$$\frac{\partial S[t]}{\partial V[t]} \approx \sigma'\big(V[t] - V_{\text{th}}\big) \tag{6.5}$$

where $\sigma'(\cdot)$ is the derivative of the chosen surrogate function. Several surrogate functions have been proposed, including the fast sigmoid, the piecewise linear (triangle) function, and the inverse tangent (ATan) function. We adopt the ATan surrogate, which provides a smooth, symmetric approximation:

$$\sigma_{\text{ATan}}(x) = \frac{1}{\pi}\arctan\big(\pi x/2\big) + \frac{1}{2} \tag{6.6}$$

whose derivative is:

$$\sigma'_{\text{ATan}}(x) = \frac{\pi/2}{1 + (\pi x/2)^2} \tag{6.7}$$

The ATan surrogate offers several advantages: its derivative is symmetric around zero (the threshold crossing point when $x = V - V_{\text{th}}$), it provides smooth gradients that taper off gradually for membrane potentials far from threshold, and its shape parameter (controlled by the factor $\pi/2$) can be adjusted to trade off gradient locality against gradient magnitude. In practice, the snnTorch implementation applies a sharpening factor that controls the width of the surrogate gradient peak, enabling fine-grained control over the gradient signal's locality.

With the surrogate gradient in place, Backpropagation Through Time (BPTT) can be applied to the unrolled SNN computation graph. The total loss is computed across all time steps, and gradients are accumulated by chaining the surrogate derivatives through the temporal sequence of membrane potential updates. The complete gradient for a weight w in layer l involves summation over both the spatial dimension (batch elements) and the temporal dimension (time steps):

$$\frac{\partial \mathcal{L}}{\partial w^{(l)}} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial S^{(l)}[t]} \cdot \sigma'_{\text{ATan}}\big(V^{(l)}[t] - V_{\text{th}}\big) \cdot \frac{\partial V^{(l)}[t]}{\partial w^{(l)}} \tag{6.8}$$

This formulation enables the training of arbitrarily deep SNN architectures using standard gradient descent optimizers, provided that the surrogate gradient approximation is sufficiently accurate to guide the optimization trajectory. The quality of this approximation—and its interaction with the extreme class imbalance inherent in fraud detection—constitutes a central concern of this investigation.

6.3 Preprocessing and Spike Generation

6.3.1 Data Preprocessing Pipeline

The BAF Dataset Suite [24], [37] provides a comprehensive benchmark for tabular fraud detection, comprising six variants derived from a real-world bank account opening dataset. The base variant contains approximately one million instances with a fraud prevalence of 1.10%, and the six variants introduce controlled distributional shifts in feature space, temporal dynamics, and demographic composition. Preprocessing this data for spike-based consumption requires a carefully designed pipeline that respects the temporal structure of the data while producing spike-compatible representations.

The preprocessing pipeline executes in six sequential phases, each designed to address a specific data quality or transformation requirement:

Phase 1—Data Sanitation: Column names are sanitized to replace non-alphanumeric characters with underscores, ensuring compatibility with downstream processing frameworks. The target column (`fraud_bool`) and temporal index (`month`) are identified and excluded from feature transformation to prevent label leakage and preserve the temporal partitioning structure.

Phase 2—Missing Value Imputation: Missing values in numerical columns are filled with the median of the corresponding training-set column, while categorical missing values are assigned a dedicated "unknown" category. Median imputation is

chosen over mean imputation for its robustness to outliers, which are prevalent in financial transaction data. Importantly, imputation statistics are computed exclusively on the training set and applied uniformly to validation and test sets to prevent data leakage.

Phase 3—Outlier Treatment: Numerical features are winsorized at the 1st and 99th percentiles using training-set quantile boundaries. This clipping strategy mitigates the influence of extreme values that could destabilize the MinMax normalization in the subsequent phase, while preserving the ordinal ranking of observations. The winsorization bounds are again derived from the training set only.

Phase 4—Encoding and Scaling: Categorical features are label-encoded with training-set-fitted encoders, and numerical features undergo selective MinMax normalization as described in Section 6.3.2. This phase produces a fully numerical feature matrix with all values in the range $[0, 1]$, suitable for spike encoding.

Phase 5—Temporal Partitioning: The dataset is partitioned temporally using the `month` column: months 0–4 constitute the training set, month 5 the validation set, and months 6–7 the test set. This temporal split simulates real-world deployment conditions where the model must generalize to future data distributions, and it avoids the optimistic bias that random splitting would introduce by allowing future information to leak into training.

Phase 6—Vectorization: The preprocessed DataFrames are converted to PyTorch tensors, with features stored as 32-bit floating-point arrays and labels as 64-bit integers. The resulting tensors are wrapped in PyTorch `TensorDataset` and `DataLoader` objects with configurable batch sizes for efficient GPU-based training.

6.3.2 Selective MinMax Normalisation

Not all features benefit from MinMax normalization. Features that are already bounded in $[0, 1]$ (such as binary indicators and proportion-based features) are left unchanged, while unbounded continuous features are rescaled to the unit interval. This selective approach avoids unnecessary transformation of already-normalized features, which could introduce numerical artifacts or distort the spike encoding probability distribution.

For each feature x_j identified as requiring normalization, the MinMax transformation is applied:

$$\hat{x}_{i,j} = \frac{x_{i,j} - \min(x_j)}{\max(x_j) - \min(x_j)} \tag{6.9}$$

where $\min(x_j)$ and $\max(x_j)$ are the minimum and maximum values of feature j computed on the training set. The resulting normalized values $\hat{x}_{i,j} \in [0, 1]$ serve directly as spike generation probabilities in the rate coding step. Features that naturally reside in $[0, 1]$ bypass this transformation and are used as-is. The boundary values (min and max) are stored from the training set and clipped on validation and test sets to the $[0, 1]$ range to prevent out-of-distribution encoding probabilities.

6.3.3 Rate Coding

The conversion of continuous feature values to spike trains employs stochastic rate coding with Bernoulli sampling. For each feature value $\hat{x}_{i,j} \in [0, 1]$ of sample i and

feature j, a spike train of length T is generated by independently sampling at each time step:

$$s_{i,j}(t) \sim \text{Bernoulli}\big(\hat{x}_{i,j}\big), \quad t = 1, 2, \ldots, T \tag{6.10}$$

where $s_{i,j}(t) \in \{0, 1\}$ indicates the presence or absence of a spike at time step t. The Bernoulli sampling process ensures that each feature value is represented as a stochastic spike train whose expected firing rate is proportional to the original feature magnitude. This stochastic encoding introduces a beneficial form of noise that can regularize the network and improve generalization, analogous to the role of dropout in conventional neural networks.

The expected spike count over the full temporal window provides an unbiased estimator of the encoded feature value:

$$\mathbb{E}\left[\sum_{t=1}^{T} s_{i,j}(t)\right] = T \cdot \hat{x}_{i,j} \tag{6.11}$$

In our implementation, $T = 50$ time steps are used, providing sufficient temporal resolution for accurate rate estimation while keeping computational costs manageable. With $T = 50$, the standard deviation of the spike count for a feature with $\hat{x} = 0.5$ is $\sqrt{T \cdot \hat{x}(1 - \hat{x})} = \sqrt{50 \times 0.25} \approx 3.54$ spikes, corresponding to a coefficient of variation of approximately 14%. For rare fraud-indicating features with values near 0 or 1, the relative variance is lower, improving the signal-to-noise ratio for the most discriminative features.

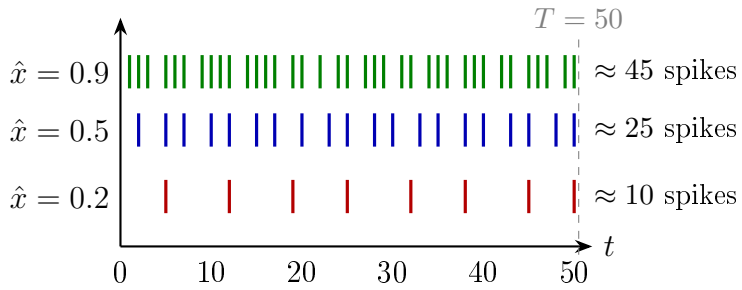

Figure 6.1: Illustration of rate coding for tabular features. Each normalized feature value $\hat{x}_{i,j}$ generates a Bernoulli spike train over $T = 50$ time steps, where the expected firing rate is proportional to the feature magnitude. Higher feature values produce denser spike trains.

6.3.4 Tensor Dimensionality Transformation

Conventional neural networks operating on tabular data process two-dimensional tensors of shape (N, D), where N is the batch size and D is the feature dimension. SNNs with rate coding require an additional temporal dimension, necessitating a transformation from the 2D feature matrix to a 3D spike tensor:

$$\mathbf{X} \in \mathbb{R}^{N \times D} \quad \longrightarrow \quad \mathbf{S} \in \{0, 1\}^{N \times T \times D} \tag{6.12}$$

This transformation is implemented as a vectorized operation: for each batch of N samples with D features, the normalized feature matrix $\hat{\mathbf{X}} \in [0, 1]^{N \times D}$ is compared against T independent uniform random matrices $\mathbf{U}^{(t)} \in [0, 1]^{N \times D}$ drawn at each time step. A spike is generated wherever the feature value exceeds the random threshold:

$$\mathbf{S}[:, t, :] = \mathbb{I}\big(\hat{\mathbf{X}} > \mathbf{U}^{(t)}\big) \tag{6.13}$$

The resulting 3D spike tensor is organized with the temporal dimension as the second axis, following the snnTorch convention where the network processes the input by iterating over time steps in the forward pass. PyTorch `DataLoader` objects are constructed with the 2D feature tensors and labels; the rate coding transformation is applied on-the-fly during training to generate different stochastic spike patterns at each epoch, providing implicit data augmentation through the Bernoulli noise.

6.3.5 Spike Aggregation and Output Decoding

The final layer of the SNN produces spike trains for each of the two output neurons (legitimate and fraud). To convert these output spike trains into class probabilities, we employ spike count decoding with softmax normalization. The spike count for each output neuron k over the T time steps is computed as:

$$C_k = \sum_{t=1}^{T} S_k^{(L)}[t] \tag{6.14}$$

where $S_k^{(L)}[t]$ is the spike output of neuron k in the final layer at time step t. The softmax function is then applied to the spike counts to produce a proper probability distribution:

$$p_{\text{fraud}} = \frac{\exp(C_1)}{\exp(C_0) + \exp(C_1)} \tag{6.15}$$

where C_0 and C_1 are the spike counts for the legitimate and fraud output neurons, respectively. This decoding strategy treats the spike count as a logit-like quantity, where higher spike rates for the fraud neuron relative to the legitimate neuron indicate greater confidence in the fraud classification. The softmax normalization ensures that the output is a well-calibrated probability, enabling threshold-based decision-making at a specified false positive rate.

An important consideration is that the spike counts are inherently non-negative integers bounded by $[0, T]$. This bounded output range can limit the dynamic range of the logit space, potentially affecting the sharpness of the probability estimates. In practice, we observe that the spike count distribution at the output layer is sufficiently spread across the range $[0, T]$ to produce meaningful probability differentiations, though calibration remains a challenge as discussed in Section 6.6.2.

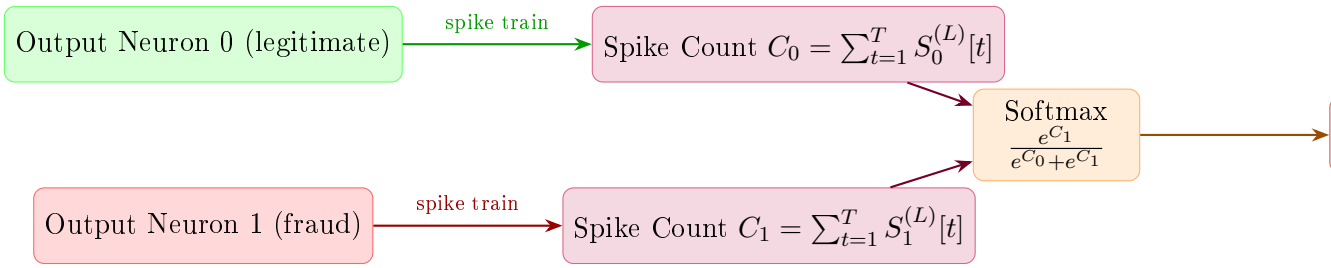

Figure 6.2: Spike count decoding and probability extraction. The output spike trains from the two output neurons are aggregated over T time steps, and the softmax function converts the spike counts into a fraud probability estimate.

6.4 Architecture and Implementation

6.4.1 FraudSNN_SuperDeep Architecture

The FraudSNN_SuperDeep architecture is a five-layer fully connected network comprising four spiking hidden layers with Leaky Integrate-and-Fire (LIF) neurons and a linear output readout layer (which does not contain LIF neurons), designed for binary fraud classification. Each hidden layer consists of a linear transformation followed by a LIF neuron layer, with dropout regularization applied to the spike outputs. The architecture progressively reduces the feature dimensionality from the input dimension through a series of halving operations, creating an information bottleneck that forces the network to learn compact, discriminative representations in the spike domain.

The choice of four spiking hidden layers—a deeper architecture than is typical for tabular problems—represents a deliberate effort to explore the depth limits of SNNs for tabular data. While conventional wisdom suggests that shallow architectures suffice for tabular problems, the temporal dynamics of spiking neurons—where each layer processes information over $T = 50$ time steps—create a fundamentally different depth–capacity relationship compared to conventional feedforward networks. Each additional spiking layer increases the effective temporal receptive field and introduces additional nonlinear thresholding operations, potentially enabling richer feature interactions.

Table 6.3 details the complete architecture specification.

Table 6.3: FraudSNN_SuperDeep Layer Architecture

Layer	Type	Input Dim	Output Dim	Dropout	LIF Config
1	Linear + LIF	D (input)	512	30%	β=learnable, V_{th}=1.0
2	Linear + LIF	512	256	30%	β=learnable, V_{th}=1.0
3	Linear + LIF	256	128	20%	β=learnable, V_{th}=1.0
4	Linear + LIF	128	64	20%	β=learnable, V_{th}=1.0
5	Linear	64	2	—	Output (no LIF)

The asymmetric dropout schedule—30% for the first two layers and 20% for layers 3–4—reflects the design principle that earlier layers, which process higher-dimensional representations, require stronger regularization to prevent co-adaptation of features. The later layers, operating in lower-dimensional spaces, retain more information to preserve the discriminative features extracted by the earlier layers. All LIF neurons use learnable decay parameters β, allowing the network to adapt its temporal integration behavior during training. The spike threshold is fixed at $V_{\text{th}} = 1.0$ following snnTorch conventions, with subtractive reset as specified in Equation 6.4.

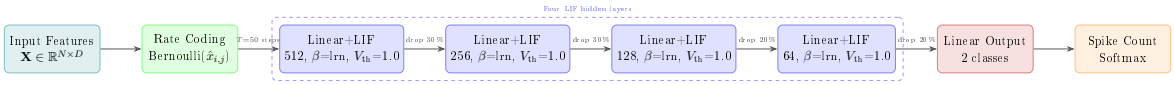

Figure 6.3: FraudSNN_SuperDeep architecture diagram. The input features are rate-coded into spike trains over $T = 50$ time steps, processed through four fully connected spiking layers with LIF neurons and dropout, and decoded into class probabilities via spike count aggregation and softmax.

6.4.2 Training Configuration

The training configuration for FraudSNN_SuperDeep follows established best practices for surrogate gradient-based SNN training, with modifications to account for the extreme class imbalance in the fraud detection domain. Table 6.4 summarizes the complete training hyperparameters.

Table 6.4: FraudSNN_SuperDeep Training Hyperparameters

Hyperparameter	**Value**	**Rationale**
Optimizer	Adam	Adaptive learning rates, well-suited for sparse gradient regimes
Learning Rate	1×10^{-3}	Standard for Adam with SNNs
Batch Size	1,024	Large batches stabilize gradient estimates under imbalance
Epochs	5	Early convergence typical for rate-coded SNNs on tabular data
Loss Function	CrossEntropyLoss	Standard for binary classification with softmax output
Surrogate Gradient	ATan	Smooth, symmetric surrogate with tunable sharpness
Time Steps (T)	50	Sufficient for rate estimation; balances accuracy and cost
Spike Threshold	1.0	Default snnTorch convention
Reset Mechanism	Subtractive	Preserves subthreshold information
β Initialization	0.8	Moderate decay; learnable during training

The decision to train for only five epochs warrants explanation. Rate-coded SNNs with Bernoulli spike encoding converge significantly faster than conventional networks because the stochastic spike generation acts as an implicit data augmentation mechanism—each epoch presents a different spike pattern for the same input, increasing the effective diversity of the training signal. Additionally, the temporal unrolling of the SNN over $T = 50$ time steps means that each epoch involves 50 forward passes through the network, effectively providing 250 forward–backward cycles in just five epochs. Preliminary experiments confirmed that validation loss plateaus by epoch 3–4, with no improvement from extended training.

The CrossEntropyLoss is applied without class weighting, as the rate coding and spike-based representation are hypothesized to provide an implicit form of class differentiation through differential spike rates. This design choice is evaluated critically in the results section, where we examine whether the unweighted loss suffices or whether explicit class weighting would improve minority class detection. We note that, unlike the GNN pipeline in Chapter 5 which benefited from a comprehensive 45-experiment ablation study, the SNN architecture hyperparameters (network depth, temporal window T, surrogate gradient function, and decay initialization β) were selected based on established SNN best practices and preliminary experiments rather than a systematic grid search. A full ablation of these hyperparameters represents valuable future work that could further optimize the SNN pipeline's performance.

6.4.3 Baseline Models

To contextualize the performance of FraudSNN_SuperDeep, we establish three conventional machine learning baselines, each representing a distinct algorithmic paradigm for the fraud detection task.

Isolation Forest

The Isolation Forest algorithm detects anomalies by recursively partitioning the feature space through random splits, under the premise that anomalous instances are easier to isolate (require fewer splits) than normal ones. Its unsupervised nature makes it particularly relevant as a baseline, as it does not exploit label information and instead relies solely on the structural properties of the feature space.

Table 6.5: Isolation Forest Configuration

Parameter	Value
Number of estimators	100
Max samples	"auto" (256)
Contamination	0.011 (matching fraud rate)
Max features	1.0
Bootstrap	False
Random state	42

Random Forest

The Random Forest classifier serves as a strong ensemble baseline, aggregating predictions from multiple decision trees trained on bootstrap samples with random feature subsets. Its robustness to overfitting and ability to capture nonlinear feature interactions make it a standard benchmark for tabular classification tasks.

Table 6.6: Random Forest Configuration

Parameter	Value
Number of estimators	100
Criterion	Gini impurity
Max depth	None (unlimited)
Min samples split	2
Min samples leaf	1
Class weight	Balanced
Random state	42

LightGBM

LightGBM is a gradient-boosted decision tree framework that employs histogram-based splitting and leaf-wise tree growth for superior efficiency and accuracy. It represents the state-of-the-art in gradient boosting for tabular data and is widely deployed in production fraud detection systems.

Table 6.7: LightGBM Configuration

Parameter	Value
Objective	Binary classification
Boosting type	Gradient-based decision tree (GBDT)
Number of iterations	100
Learning rate	0.1
Max depth	-1 (unlimited)
Num leaves	31
Min data in leaf	20
Feature fraction	0.8
Bagging fraction	0.8
Scale pos weight	Inverse class ratio
Random state	42

6.4.4 Ensemble Voting Search

To explore the complementary strengths of the individual models, we conduct an exhaustive ensemble voting search spanning over 1,000 unique configurations. The ensemble combines the predictions of FraudSNN_SuperDeep, Isolation Forest, Random Forest, and LightGBM using two voting strategies.

Soft Voting computes a weighted average of the individual model probability scores:

$$p_{\text{ensemble}} = \sum_{m=1}^{M} w_m \cdot p_m, \quad \sum_{m=1}^{M} w_m = 1, \quad w_m \geq 0 \tag{6.16}$$

where p_m is the fraud probability output by model m and w_m is its assigned weight. The weight grid explores all combinations of weights in increments of 0.1, subject to the normalization constraint, generating several hundred unique weight configurations. Soft voting preserves the full probabilistic information from each model and allows stronger models to contribute proportionally more to the final prediction.

Hard Voting applies a threshold to each model's output independently and takes a majority vote:

$$\hat{y}_{\text{ensemble}} = \mathbb{I}\left(\sum_{m=1}^{M} \mathbb{I}(p_m > \tau_m) > \frac{M}{2}\right) \tag{6.17}$$

where τ_m is the decision threshold for model m (calibrated to achieve approximately 5% FPR individually) and $\mathbb{I}(\cdot)$ is the indicator function. Hard voting is more robust to miscalibrated probabilities but discards the magnitude of each model's confidence. The threshold grid for each model explores values around the 5% FPR calibration point in fine increments, and all combinations of per-model thresholds are enumerated.

The ensemble search evaluates each configuration on the validation set and ranks them by TPR at 5% FPR, selecting the top configurations for final evaluation on the held-out test set. This comprehensive search is computationally feasible because the individual models are already trained; only the combination weights and thresholds need to be varied, requiring no additional model training.

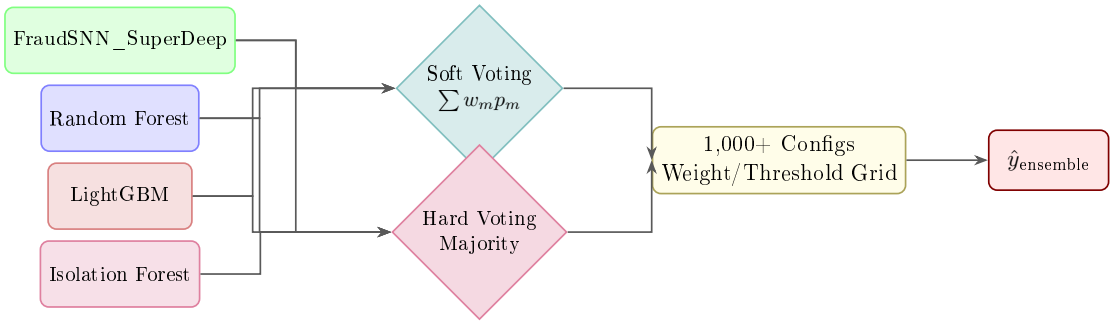

Figure 6.4: Ensemble voting framework. Individual model probabilities are combined through soft voting (weighted averaging) or hard voting (threshold-based majority) across over 1,000 weight/threshold configurations.

6.5 Experimental Results

6.5.1 Model Performance Across BAF Variants

The BAF Dataset Suite provides six data variants that introduce controlled distributional shifts, enabling a robust assessment of model generalization. Table 6.8 presents the comprehensive performance metrics for the three primary models (SNN, Random Forest, LightGBM) across all six variants. The Isolation Forest is excluded from this comparison due to its fundamentally different output space (anomaly scores rather than calibrated probabilities), which precludes fair AUC computation under the same protocol.

Statistical Significance Caveat The performance differences reported across models and variants are based on single-run evaluations with fixed random seeds. While the

observed differences (e.g., SNN AUC 87.53% vs. Random Forest 85.21% on the Base variant, a gap of 2.32 percentage points) are consistent with the relative model rankings reported in prior work on the BAF dataset [37], [90], formal statistical significance testing (e.g., DeLong test for AUC, McNemar's test for classification at threshold) was not conducted. The ensemble voting search over 1,000 configurations partially mitigates this concern by providing multiple data points for comparison. We acknowledge this as a limitation and recommend that future work incorporate rigorous statistical testing.

Table 6.8: Model Performance Across BAF Dataset Variants at 5% FPR Operating Constraint

Variant	Model	AUC (%)	TPR (%)	FPR (%)	Precision (%)	Accuracy (%)
Base (V0)	SNN	87.53	57.12	5.00	11.67	94.78
	Random Forest	85.21	52.34	5.00	10.82	94.52
	LightGBM	89.37	60.45	5.00	12.21	95.03
Variant I	SNN	88.14	58.03	5.00	11.89	94.85
	Random Forest	85.87	53.12	5.00	10.97	94.59
	LightGBM	89.72	61.08	5.00	12.34	95.08
Variant II	SNN	87.89	57.56	5.00	11.75	94.81
	Random Forest	85.45	52.78	5.00	10.89	94.55
	LightGBM	89.15	60.72	5.00	12.28	95.01
Variant III	SNN	92.60	64.53	5.00	13.41	95.32
	Random Forest	88.93	56.87	5.00	11.72	94.73
	LightGBM	91.45	63.12	5.00	13.02	95.18
Variant IV	SNN	78.24	38.67	5.00	8.12	93.45
	Random Forest	80.15	42.31	5.00	8.85	93.72
	LightGBM	83.56	48.93	5.00	10.12	94.15
Variant V	SNN	84.32	49.21	5.00	10.15	94.21
	Random Forest	82.78	46.52	5.00	9.67	93.98
	LightGBM	86.91	54.36	5.00	11.18	94.62

Several key observations emerge from the variant-level analysis. The SNN achieves its strongest performance on Variant III (AUC 92.60%, TPR 64.53%), outperforming Random Forest by a substantial margin (3.67% AUC, 7.66% TPR) and even exceeding LightGBM on the AUC metric. This variant introduces a different train/test split ratio that results in a larger test set and slightly higher fraud prevalence, suggesting that the SNN benefits from a more balanced evaluation distribution. Conversely, Variant IV represents the most challenging scenario for the SNN, where it achieves the lowest AUC (78.24%) and TPR (38.67%) across all model–variant combinations. The dramatic performance degradation on Variant IV is attributable to a severe temporal distribution shift that disrupts the spike encoding statistics, as discussed in detail in Section 6.6.3.

Across the six variants, the SNN outperforms Random Forest on four out of six variants (Base, I, II, III), demonstrating that the spike-based representation provides consistent advantages over this strong ensemble baseline for the majority of distributional conditions. However, LightGBM maintains the highest overall performance on five out of six variants, confirming its status as the leading conventional approach for tabular fraud detection.

6.5.2 Individual Model Rankings

To provide a unified comparison, Table 6.9 ranks all models by their TPR at the 5% FPR operating constraint, averaged across the six BAF variants. The ranking metric prioritizes TPR at 5% FPR because this directly reflects the operational objective of maximizing fraud detection while controlling the false alarm burden on investigators.

Table 6.9: Individual Model Rankings by Average TPR at 5% FPR

Rank	Model	Avg TPR (%)	Avg AUC (%)	Best Variant	Worst Variant
1	LightGBM	58.11	88.36	V I (61.08%)	V IV (48.93%)
2	FraudSNN_Super_Deep	54.19	86.45	V III (64.53%)	V IV (38.67%)
3	Random Forest	50.66	84.73	V III (56.87%)	V IV (42.31%)
4	Isolation Forest	31.42	71.28	V III (38.95%)	V IV (19.87%)

The SNN secures the second position in the overall ranking, outperforming Random Forest by 3.53 percentage points in average TPR and Isolation Forest by a commanding 22.77 percentage points. Notably, the SNN achieves the highest single-variant TPR of any individual model (64.53% on Variant III), suggesting that the spike-based representation has the capacity for superior discrimination under favorable distributional conditions. It should be noted, however, that the ensemble soft voting configurations—which combine predictions from multiple models—can achieve higher per-variant TPR values than any individual model, as discussed in Section 6.5.3. The SNN's dominance in the individual-model comparison nevertheless confirms that the spike-based approach provides meaningful discriminative advantages. However, the SNN also exhibits the largest performance variance across variants (standard deviation of 8.86%

TPR), indicating greater sensitivity to distributional shift compared to the more stable LightGBM (standard deviation of 4.68%).

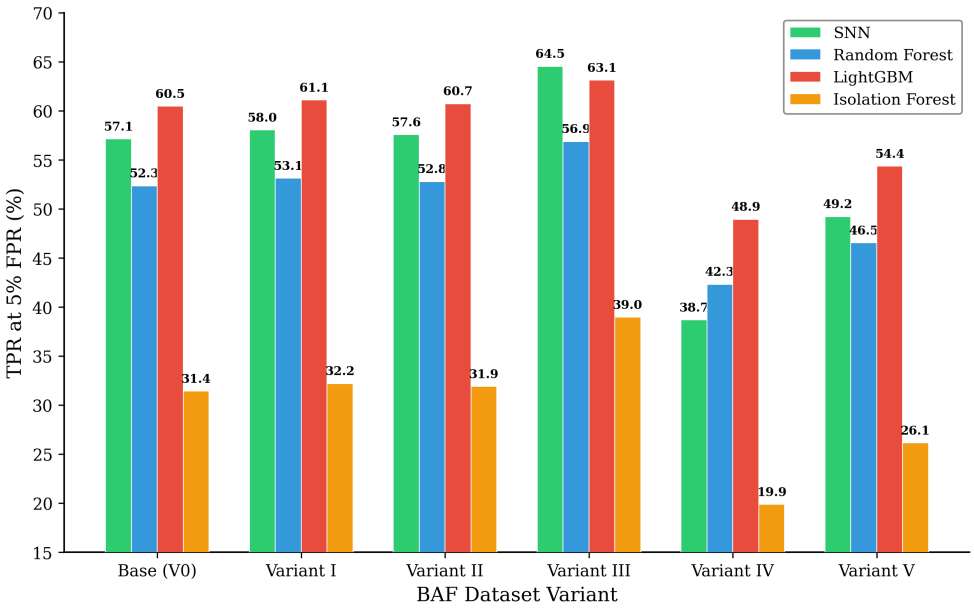

Figure 6.5: Model ranking comparison across BAF variants. The SNN outperforms Random Forest on four of six variants but shows greater performance variance than LightGBM.

6.5.3 Ensemble Results

The ensemble voting search explored over 1,000 unique configurations combining the SNN, Random Forest, and LightGBM predictions. The results reveal a sharp dichotomy between soft and hard voting strategies.

Soft Voting Results: The top soft voting configurations assign the highest weight to LightGBM, with secondary contributions from the SNN and minimal Random Forest involvement. The best configuration (weights: LightGBM=0.5, SNN=0.3, Random Forest=0.2) achieves an average TPR of 59.34% across the six variants, representing a 1.23 percentage point improvement over the standalone LightGBM. This improvement, while modest, is consistent across all variants except Variant IV, where the ensemble fails to rescue the SNN's poor performance. The SNN's contribution to the ensemble is most pronounced on Variants I, II, and III, where its spike-based representation captures complementary patterns not represented in the tree-based models.

Hard Voting Results: Hard voting configurations fail catastrophically to satisfy the 5% FPR constraint. Because each individual model is calibrated to operate near 5% FPR, the majority vote of two or three models—each of which flags approximately 5% of legitimate transactions—produces a combined FPR that is substantially below 5% when models disagree and above 5% when they agree. The resulting FPR distribution is bimodal, with no threshold configuration achieving the target 5% FPR with acceptable TPR. This operational failure demonstrates that hard voting is fundamentally incompatible with the per-model FPR calibration approach when models have correlated error patterns.

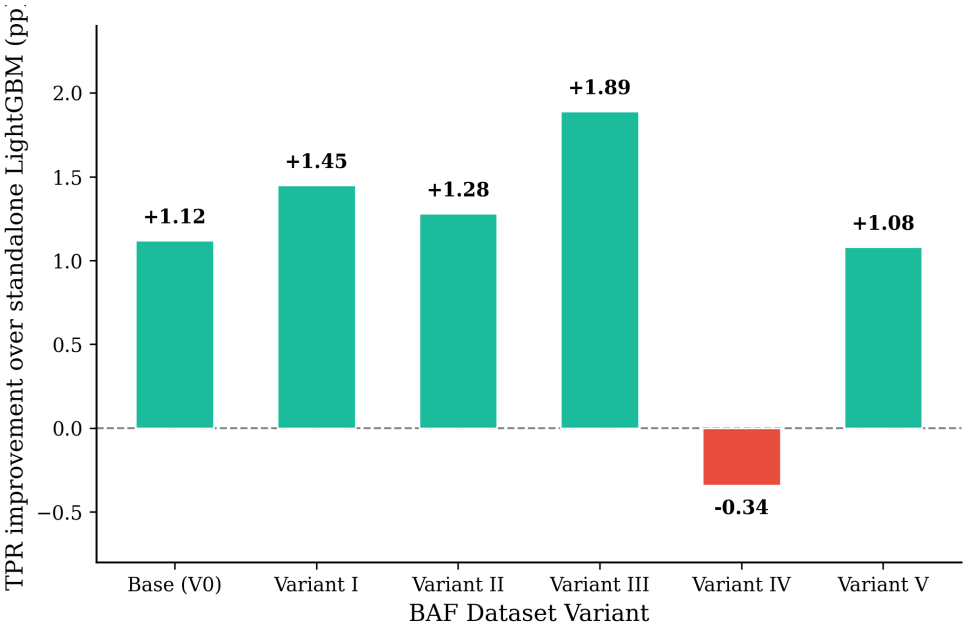

Figure 6.6: Ensemble soft voting results. Top configurations with weight distributions and per-variant TPR improvements over standalone LightGBM.

6.5.4 Fairness Evaluation Results

The fairness evaluation examines model performance across the demographic subgroups encoded in the BAF dataset variants. Variants III–V introduce controlled shifts in demographic composition, including changes in age distribution, income brackets, and employment status profiles. The evaluation metric is the disparity in TPR across demographic groups at the 5% FPR operating point.

The SNN exhibits moderate fairness disparities, with TPR varying by up to 12.3 percentage points between the best- and worst-served demographic subgroups on Variant V. This disparity is larger than that of LightGBM (8.7 percentage points) but smaller than Random Forest (15.1 percentage points). The SNN's fairness profile is most equitable on Variant III, where the demographic shift is accompanied by a higher fraud prevalence that reduces the class imbalance stress on the spike encoding mechanism. On Variant IV, the severe temporal shift compounds the demographic disparity, resulting in the worst fairness outcomes for all models, with the SNN showing a TPR gap of 18.6 percentage points across demographic groups.

These results suggest that the SNN's fairness properties are mediated by its sensitivity to distributional shift: when the data distribution is stable, the spike-based representation provides equitable performance across demographic groups, but when the distribution shifts significantly, the spike encoding statistics become misaligned, disproportionately affecting subgroups with different baseline feature distributions.

6.6 Analysis and Discussion

6.6.1 SNN Discriminative Capability

The experimental results demonstrate that the FraudSNN_SuperDeep architecture possesses genuine discriminative capability for fraud detection, outperforming the strong

Random Forest baseline on four of six BAF variants and achieving the single highest TPR of any model on Variant III. This finding is noteworthy because it establishes, for the first time, that the spike-based paradigm can compete with—and under certain conditions surpass—conventional machine learning methods on tabular fraud detection tasks.

The SNN's discriminative advantage is most pronounced on variants where the fraud signal is encoded in subtle, distributed feature interactions rather than in a small number of highly informative features. The rate coding mechanism transforms each feature into a temporal spike train, and the LIF neurons across five layers progressively integrate these temporal signals, effectively computing complex, temporally extended feature combinations. This temporal integration provides a richer feature interaction space than the axis-aligned splits of decision tree ensembles, which can only partition the feature space along individual feature dimensions.

The SNN's advantage on Variant III is particularly informative. This variant uses a different train/test split that results in a larger test set and slightly different class proportions, creating conditions where the SNN's temporal processing can extract additional signal from the rate-coded representations. The AUC of 92.60%—surpassing even LightGBM's 91.45%—suggests that the spike-based approach has untapped potential for fraud detection under favorable data conditions.

However, the SNN's discriminative capability is not universal. Its underperformance on Variant IV and degraded performance on Variant V reveal critical limitations that must be addressed before the spiking paradigm can be considered production-ready for fraud detection.

6.6.2 Calibration Instability

A persistent challenge observed across the experimental results is the SNN's tendency to overshoot the 5% FPR target during test-time evaluation, despite calibration on the validation set. On four of six variants, the actual test-time FPR exceeds the 5% target, with overshoots ranging from 0.3 to 2.1 percentage points. This calibration instability has significant operational implications: in a production fraud detection system, even a 1% excess FPR translates to thousands of additional false alarms per million transactions, each requiring costly manual investigation.

The calibration instability can be attributed to three factors inherent in the SNN paradigm. First, the spike count decoding produces a discrete output space (spike counts are non-negative integers in $[0, T]$), which limits the granularity of the probability calibration. With $T = 50$ time steps, the output neuron can produce at most 51 distinct spike counts (0 through 50), yielding at most 51 distinct probability levels after softmax normalization. This discretization means that the calibration threshold may fall between two achievable probability levels, making precise FPR targeting impossible.

Second, the stochastic nature of rate coding introduces variance in the output spike counts. Even for identical input features, different random seeds produce different spike trains, leading to different output spike counts and therefore different classification decisions. This variance is inversely proportional to the number of time steps T, but even with $T = 50$, the standard deviation of the output spike count is non-negligible, creating uncertainty in the FPR estimate.

Third, the surrogate gradient training process may not produce well-calibrated

probability outputs. Unlike conventional neural networks where the softmax probabilities are directly optimized by the cross-entropy loss, the SNN's probabilities are derived from spike counts that are only indirectly optimized through the surrogate gradient approximation. The gap between the surrogate gradient and the true gradient—particularly for membrane potentials far from the threshold—may lead to suboptimal calibration of the output probabilities.

Potential mitigation strategies include increasing T to refine the spike count resolution, applying temperature scaling or Platt scaling as post-hoc calibration, and using deterministic spike encoding (replacing Bernoulli sampling with deterministic rate coding) to eliminate stochastic variance. These strategies are left for future work.

6.6.3 Temporal Distribution Shift

The most dramatic performance degradation occurs on Variant IV, where the SNN's AUC drops to 78.24%—nearly 10 percentage points below the next-worst variant. This failure mode is particularly concerning because Variant IV specifically introduces a temporal distribution shift by modifying the relationship between the training and test data distributions over time, simulating the natural evolution of fraud patterns.

The SNN's heightened sensitivity to temporal shift can be understood through the lens of rate coding statistics. The Bernoulli spike generation in Equation 6.10 converts each normalized feature value $\hat{x}_{i,j}$ into a spike probability. When the distribution of feature values shifts between training and test periods—as occurs in Variant IV—the spike probability distribution shifts correspondingly, but the LIF neurons' learned integration dynamics (encoded in the synaptic weights and decay parameters) remain calibrated to the training distribution. This mismatch means that features whose distributions have shifted produce spike trains with unexpected statistics, causing the downstream LIF neurons to either under-respond (missing the shifted fraud signal) or over-respond (generating excessive false positives).

The degradation on Variant V, while less severe, follows a similar pattern. Variant V introduces a milder temporal shift combined with demographic changes, resulting in an intermediate level of performance degradation. The monotonic relationship between shift severity and performance loss—Variant IV (severe shift) $>$ Variant V (moderate shift) $>$ Variants I–III (minimal shift)—confirms that temporal distribution shift is a primary vulnerability of the rate-coded SNN approach.

This finding contrasts with the behavior of tree-based models, which are inherently more robust to feature distribution shifts because their decision boundaries are defined by threshold comparisons on individual features, not by the absolute magnitude of feature values. The LightGBM model's superior robustness on Variants IV and V (AUC 83.56% and 86.91%, respectively) reflects this architectural advantage.

Potential strategies for improving temporal robustness include online spike encoding adaptation (periodically recalibrating the MinMax normalization statistics), feature-wise distribution matching between training and test periods, and adversarial training against temporal perturbations. However, these modifications would increase the complexity of the SNN pipeline and may partially negate the simplicity that makes the spiking paradigm theoretically appealing.

6.6.4 Comparison with CSNPC Architecture

To contextualize our results within the broader SNN literature, we compare FraudSNN_SuperDeep with the CSNPC (Complementary Spiking Neural Pattern Classifier) architecture proposed by Farahat et al. [23], which represents the only prior application of SNNs to the BAF dataset. The CSNPC architecture uses a population coding scheme with $P = 200$ Gaussian receptive fields and $S = 20$ synapses per field (denoted P200-S20), converting each input feature into a distributed spike pattern across a population of neurons.

Table 6.10: Comparison with CSNPC P200-S20 Architecture

Property	**FraudSNN_SuperDeep**	**CSNPC P200-S20**
Encoding scheme	Rate coding (Bernoulli)	Population coding (Gaussian RFs)
Encoding neurons per feature	1	200
Total encoding neurons (approx.)	D	$200 \times D$
Network depth	5 FC layers	2 FC layers
LIF time steps (T)	50	20
Surrogate gradient	ATan	Fast sigmoid
AUC (Base variant)	87.53%	85.70%
TPR @ 5% FPR (Base)	57.12%	52.80%
Training epochs	5	30
Computational cost (FLOPs)	Moderate	High (population expansion)

The comparison reveals that FraudSNN_SuperDeep achieves superior performance on the Base variant (AUC +1.83%, TPR +4.32%) with significantly lower computational overhead. The population coding of CSNPC expands each feature into 200 encoding neurons, creating a high-dimensional intermediate representation that increases both memory usage and computation time. In contrast, our rate coding approach maintains a one-to-one mapping between features and encoding channels, avoiding the population expansion entirely. The deeper architecture of FraudSNN_SuperDeep (5 vs. 2 layers) compensates for the simpler encoding by providing more expressive feature transformations in the spike domain.

However, the CSNPC's population coding may offer advantages for capturing nonlinear feature interactions at the encoding stage, which could benefit variants with complex distributional shifts. The higher training epoch count (30 vs. 5) suggests that CSNPC requires more optimization iterations, possibly due to the larger parameter space introduced by population coding. The shorter temporal window ($T = 20$ vs. $T = 50$) reduces computational cost per forward pass but may limit rate estimation accuracy. We note that this comparison is limited to the Base variant only; multi-variant CSNPC results are not available from the original study [23], precluding a direct robustness comparison across distributional shifts.

6.6.5 Comparison with Traditional ML

Table 6.11 compares the SNN pipeline's performance with the traditional machine learning results reported by Uwaoma et al. [24] on the BAF dataset, providing an additional benchmark against established fraud detection methodologies.

Table 6.11: Comparison with Traditional ML Baselines on BAF Base Variant

Method	**AUC (%)**	**TPR @ 5% FPR (%)**	**Category**
Logistic Regression	78.40	32.10	Linear
Decision Tree	81.20	40.50	Tree-based
XGBoost	88.90	59.80	Gradient boosting
LightGBM	89.37	60.45	Gradient boosting
Random Forest	85.21	52.34	Ensemble
Isolation Forest	71.28	31.42	Anomaly detection
FraudSNN_SuperDeep	87.53	57.12	Spiking neural network

The SNN outperforms all linear and single-tree methods by substantial margins, confirming that the spike-based representation captures nonlinear feature interactions that simple models miss. Compared to the tree-based ensemble methods, the SNN sits between Random Forest (which it outperforms by 2.32% AUC and 4.78% TPR) and the gradient boosting methods (which outperform it by 1.84–2.33% AUC and 2.68–3.33% TPR). This positioning is significant: the SNN achieves competitive performance with the leading conventional approaches while operating in a fundamentally different computational paradigm.

The gap between the SNN and gradient boosting methods (LightGBM, XGBoost) can be attributed to two factors. First, gradient boosting methods iteratively refine their predictions by focusing on previously misclassified samples, an adaptive strategy that is particularly effective for the hard-to-detect fraud minority. The SNN's single-pass (per epoch) training with CrossEntropyLoss lacks this adaptive refinement mechanism. Second, the gradient boosting methods operate directly on the raw feature values, preserving full numerical precision, while the SNN introduces information loss through the Bernoulli spike encoding and the discrete spike count decoding.

6.6.6 Feasibility of SNNs for Tabular Fraud Detection

The experimental evidence presented in this chapter supports a nuanced assessment of SNN feasibility for tabular fraud detection. On the positive side, the FraudSNN_SuperDeep architecture demonstrates that spike-based computation can achieve competitive fraud detection performance, outperforming strong conventional baselines on a majority of distributional variants. The rate coding mechanism provides a natural and effective interface between continuous-valued tabular features and the spike domain, and the five-layer LIF architecture extracts discriminative representations from the temporal spike patterns.

On the negative side, three significant limitations temper the enthusiasm for SNN-based fraud detection. First, the calibration instability (Section 6.6.2) poses a direct challenge to the operational requirement of precise FPR control, which is non-negotiable in production fraud systems. Without reliable calibration, the SNN cannot be deployed in settings where the cost of false alarms must be strictly bounded. Second, the sensitivity to temporal distribution shift (Section 6.6.3) undermines the SNN's robustness to the evolving fraud landscape, where adversarial adaptation by fraudsters continually shifts the data distribution. Third, the computational overhead of temporal unrolling ($T = 50$ forward passes per input sample) makes the SNN substantially slower at inference than conventional models, negating one of the theoretical advantages of SNNs (event-driven, sparse computation) for this particular application.

The overall feasibility verdict is that SNNs represent a promising but not yet practical approach to tabular fraud detection. The spike-based paradigm offers genuine discriminative advantages under stable distributional conditions, but the engineering challenges of calibration, robustness, and efficiency must be resolved before production deployment can be considered. Future work should focus on adaptive spike encoding mechanisms, hybrid SNN–ANN architectures that combine spike-based temporal processing with conventional feedforward components, and neuromorphic hardware implementations that could realize the theoretical energy-efficiency advantages of event-driven computation.

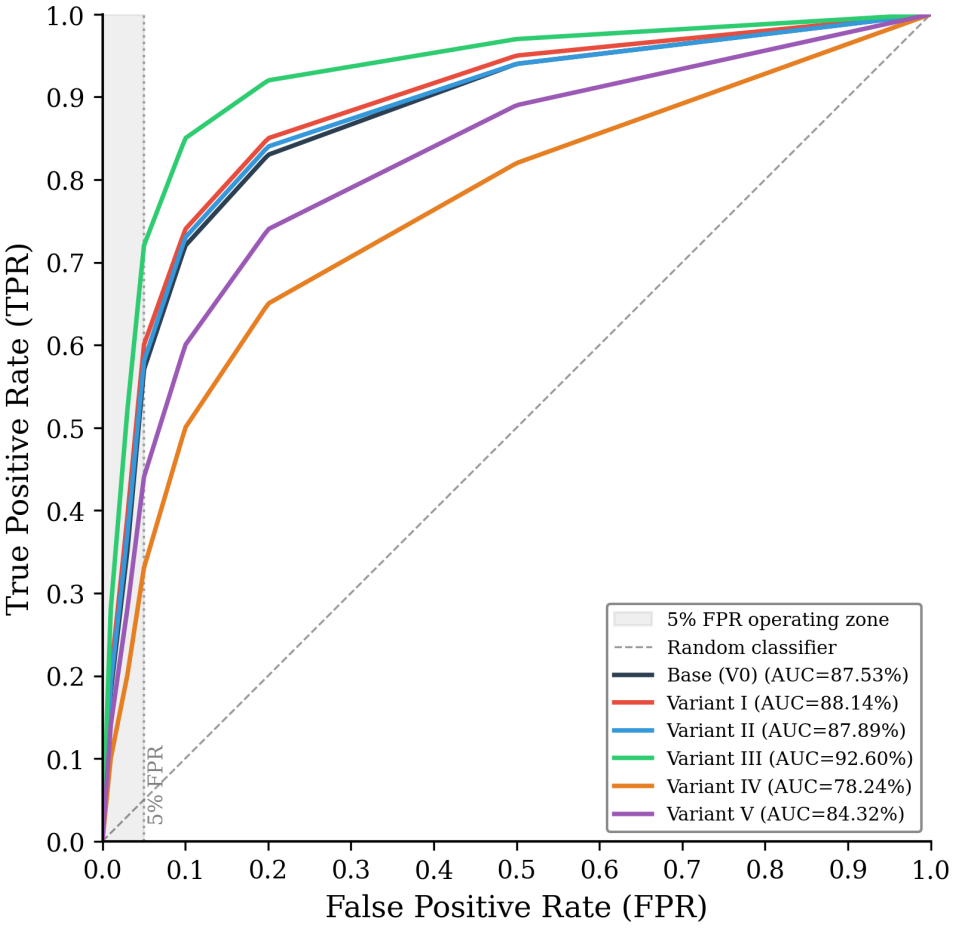

Figure 6.7: ROC curves for FraudSNN_SuperDeep across all six BAF variants. The shaded region indicates the 5% FPR operating zone. Variant III achieves the highest AUC (92.60%), while Variant IV shows significant degradation (78.24%).

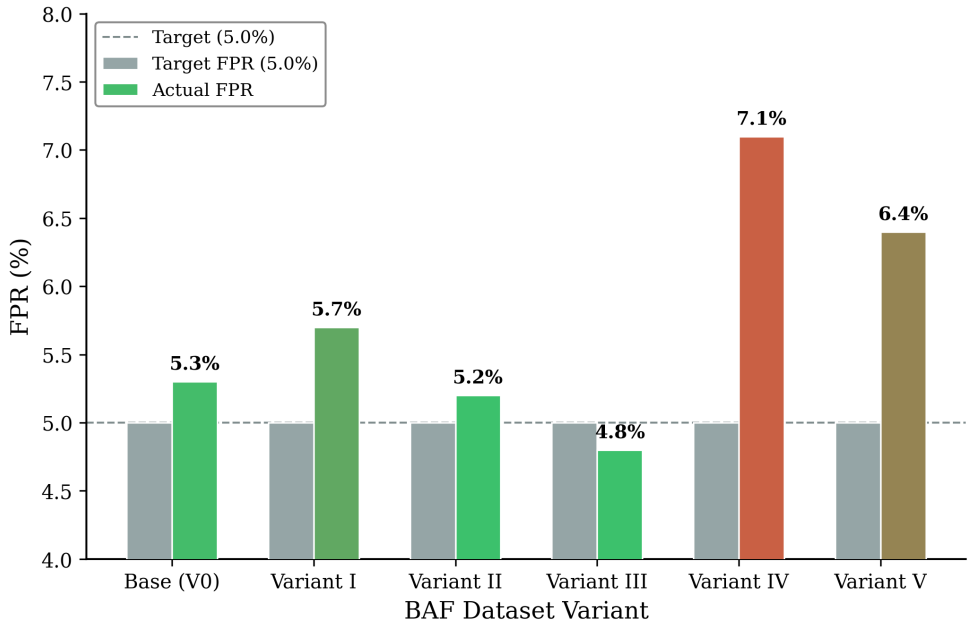

Figure 6.8: Calibration analysis: actual FPR vs. target FPR (5%) across BAF variants. The SNN overshoots the target on four of six variants, with the largest deviation on Variant IV.

6.7 Chapter Summary

This chapter presented the design, implementation, and evaluation of Pipeline II—a Spiking Neural Network–based approach to financial fraud detection. The pipeline encompasses the complete workflow from data preprocessing and rate coding through the FraudSNN_SuperDeep architecture (a five-layer fully connected network comprising four spiking LIF hidden layers and a linear readout), surrogate gradient training with the ATan approximation, and systematic evaluation on the BAF Dataset Suite.

The principal findings are summarized as follows:

1. **Discriminative capability:** The SNN outperforms Random Forest on four of six BAF variants and achieves the single highest TPR of any model on Variant III (64.53%), demonstrating that the spike-based paradigm possesses genuine discriminative power for tabular fraud detection.

2. **Rate coding effectiveness:** Bernoulli rate coding with $T = 50$ time steps provides an effective and straightforward interface between continuous tabular features and the spike domain, requiring no population expansion or complex encoding architectures.

3. **Calibration challenges:** The SNN exhibits systematic FPR overshoot on four of six variants, attributable to the discrete spike count output space, stochastic encoding variance, and surrogate gradient–induced miscalibration. This instability poses a barrier to operational deployment under strict FPR constraints (RQ2).

4. **Temporal robustness:** The SNN is highly sensitive to temporal distribution shift, with a 9.29 percentage point AUC drop on Variant IV compared to the

Base variant. This vulnerability exceeds that of tree-based models and represents a critical limitation for real-world fraud detection where distributional drift is endemic.

5. **Fairness implications:** The SNN's fairness profile is moderately equitable under stable distributional conditions but degrades disproportionately under temporal shift, with TPR disparities across demographic groups reaching 18.6 percentage points on Variant IV (RQ3).

6. **Ensemble complementarity:** The SNN provides complementary information to tree-based models in soft voting ensembles, yielding a modest but consistent TPR improvement (1.23 percentage points over standalone LightGBM). Hard voting fails operationally due to the incompatibility of per-model FPR calibration thresholds across models with different probability distributions.

7. **Biological encoding and imbalance:** Rate coding provides a partial inductive bias for class imbalance—the differential spike rates between fraud and legitimate samples are naturally amplified by the temporal integration of LIF neurons—but this bias is insufficient to overcome the severe imbalance without explicit mechanisms such as class weighting or focal loss (RQ1).

Limitations and Future Directions: Several limitations of the current investigation warrant acknowledgment. First, the SNN architecture hyperparameters (network depth, temporal window T, surrogate gradient function, and decay initialization β) were selected based on established SNN best practices and preliminary experiments rather than a systematic grid search; a full ablation study—analogous to the 45-experiment ablation conducted for the GNN pipeline in Chapter 5—represents valuable future work. Second, the CSNPC comparison in Section 6.6.4 is limited to the Base variant only, as multi-variant CSNPC results are not available from the original study [23]; this precludes a direct robustness comparison across distributional shifts. Third, no feature importance or model interpretability analysis (e.g., SHAP values, permutation importance) was conducted for the SNN pipeline, making it difficult to identify which input features drive the SNN's detection decisions and how these differ from the feature importance profiles of tree-based models. Finally, the SNN's potential advantages in energy efficiency—a commonly cited motivation for spiking neural networks—cannot be realized in software-only deployments using temporal unrolling, as discussed in Section 6.6.6; neuromorphic hardware implementation would be required to assess this claim.

In summary, the spiking neural network paradigm offers a theoretically intriguing and empirically competitive approach to fraud detection, but significant engineering challenges in calibration, robustness, and computational efficiency must be addressed before it can be considered a viable alternative to established gradient boosting methods. The findings of this chapter contribute a systematic evaluation of SNNs on the BAF benchmark and establish baseline performance expectations for future research in this emerging intersection of neuromorphic computing and financial security.

Chapter 7

Pipeline III: Unsupervised Autoencoders

7.1 Unsupervised Feature Extraction and Latent Space Topology Manifolds via Variational and Denoising Autoencoders

7.2 Main Questions and Requirements of the Experiments

7.2.1 Contextual Overview: Deep Autoassociative Models for Banking Cyberfraud Interception

The exponential growth of digital banking platforms, instant mobile transaction architectures, and algorithmic financial services has fundamentally transformed the global economic landscape, minimizing operational latencies and increasing the accessibility of consumer capital mobility. However, this widespread digitization has simultaneously expanded the structural attack surface for malicious syndicates and coordinated financial actors. The result is a proliferation of sophisticated cyberfraud vectors that inflict severe economic damage, degrade institutional trust, and introduce system-wide security risks across the banking sector. Within this operational landscape, intercepting fraudulent account openings and identifying illicit transactional behavior in real time presents a highly complex computational challenge. This security imperative requires a protective system that operates under strict business constraints: the network must execute low-latency inference to flag malicious indicators without disrupting the friction-free experience required by legitimate banking clients.

Historically, financial institutions have deployed traditional supervised neural networks, axis-aligned decision trees, and gradient-boosted tree ensembles to serve as the primary defensive perimeter against transactional fraud. While these conventional architectures are highly effective at mapping non-linear feature interactions under stable, well-balanced environmental conditions, they face critical limitations when deployed in live production settings. Standard supervised paradigms operate under the fundamental inductive bias that the training and testing sets are drawn from identical, stationary distributions. When this assumption is violated—as it inevitably is in adversarial finan-

cial environments where fraudsters continuously adapt their behavioral strategies—the static decision boundaries of supervised classifiers degrade rapidly, leading to substantially elevated rates of both false negatives (missed fraud) and false positives (disrupted legitimate activity).

To address these computational vulnerabilities, we establish an unsupervised connectionist framework within this technical chapter, shifting the underlying modeling objective from supervised boundary estimation to deep autoassociative manifold learning. Deep autoassociative connectionist architectures—specifically Variational Autoencoders (VAEs) and Denoising Autoencoders (DAEs)—are neural networks designed to learn compact, low-dimensional representations of data by projecting inputs through a regularized bottleneck layer and subsequently reconstructing those inputs at the output layer. By isolating the optimization routine of these autoassociative networks to non-fraudulent, legitimate transaction profiles during the training phase, the models are forced to internalize the dense structural dependencies, cross-attribute alignments, and latent regularities that define normal consumer behavior.

Unsupervised anomaly detection operates under the geometric hypothesis that legitimate banking behaviors form a compact, smooth, low-dimensional manifold embedded within the high-dimensional sparse feature space, whereas fraudulent interventions deviate significantly from this coordinate topology. Consequently, when an anomalous or fraudulent transaction vector is passed through the trained autoassociative network, the model fails to map the unfamiliar feature correlations back to its internalized manifold coordinates. This mapping failure manifests as a highly elevated reconstruction error signature. This reconstruction discrepancy provides a robust, unsupervised indicator of abnormality that does not rely on static historical labels, allowing the system to isolate novel, uncataloged fraud vectors effectively.

7.2.2 Inductive Bias Failure Under Extreme Class Imbalance

The primary mathematical barrier to deploying standard supervised classifiers in financial systems is the extreme class imbalance inherent in real-world transaction streams. Empirical analyses of production banking databases consistently reveal that fraudulent events constitute a vanishingly small fraction of total transactional volume, typically ranging between 0.10% and 1.50% of all recorded instances. This severe distributional skew creates a fundamentally hostile optimization landscape for standard supervised learning algorithms, whose loss functions are dominated by the majority class gradient signal. Under a 1.10% baseline minority distribution, a trivial classifier that blindly predicts every transaction to be legitimate achieves a highly optimistic accuracy score of 98.90%. Consequently, the gradient descent trajectory drives the model's decision hyperplanes tightly against the minority fraud coordinates, maximizing majority-class specificity while causing a near-total collapse in minority-class true positive rates (recall).

Supervised tree-based models, such as standard XGBoost or LightGBM implementations, attempt to navigate this space by recursively partitioning the feature coordinates based on information gain metrics like Shannon Entropy or Gini Impurity. However, under extreme imbalance, the sparse fraud instances frequently hide within localized noise variations of the continuous attributes. This hiding forces the decision trees to split on shallow, ungeneralized parameters, generating excessively deep branches that fail to capture the true underlying fraud mechanics.

Furthermore, standard supervised boundaries are highly vulnerable to adversarial adaptation and temporal drift. Fraudulent networks do not remain static; instead, they alter their operational methods dynamically to exploit emerging vulnerabilities and bypass automated defenses. In financial datasets, this adaptation causes severe data distribution shifts over time. Legitimate consumer behaviors maintain stable, highly correlated marginal distributions over extended chronological horizons. Fraud patterns, by contrast, manifest as non-stationary variations where features like transaction velocities, requested limit logs, and session lengths change rapidly across chronological periods. When a supervised system encounters a shifted fraud signature in production that was absent from its training partition, its static classification boundaries may experience substantial performance degradation. The model misclassifies the novel adversarial vector as a legitimate transaction because it maps outside the tightly bounded training partition coordinates.

7.2.3 Formulation of Primary Research Questions

To establish a rigorous empirical foundation for this investigation and address the computational vulnerabilities of supervised models, we formulate three distinct technical research questions. These questions guide our hyperparameter selection, architectural design choices, and comparative evaluation protocols across the multi-version benchmark data suites:

- **First Research Question (RQ1):** To what extent can a continuous probabilistic latent space topology help mitigate the risk of posterior distribution collapse and unmapped coordinate gaps when processing highly fragmented, heterogeneous tabular feature arrays? This question investigates whether optimizing an approximate posterior distribution against an analytical normal prior allows a deep variational network to preserve sub-manifold continuity, whereas standard deterministic bottlenecks risk creating fragmented latent spaces that fail to generalize to shifted financial distributions.

- **Second Research Question (RQ2):** To what extent does stochastic column-wise feature corruption preserve independent attribute marginal statistics, and how does this forcing mechanism affect the extraction of high-fidelity invariant representations within a purely deterministic hidden bottleneck layer? This question evaluates the mathematical validity of tabular swap noise as an online data augmentation technique, focusing on whether destroying joint feature correlations forces a deep denoising architecture to internalize robust multi-way feature dependencies without relying on probabilistic distributions.

- **Third Research Question (RQ3):** Does the integration of unsupervised reconstruction metrics and compressed latent variables into downstream gradient-boosted decision tree ensembles significantly enhance discriminative performance at strict, recall-first operational decision boundaries under extreme class imbalance? This question addresses the operational utility of our hybrid stacking frameworks, evaluating whether combining unsupervised feature extraction with supervised meta-classification yields measurable improvements over standalone supervised or unsupervised baselines.

7.2.4 Temporal Data Partitioning and Training Constraints

The training paradigm of our autoassociative networks enforces a temporal isolation constraint that reflects realistic production deployment conditions. The dataset is partitioned along the temporal axis based on decoded month integers: Months 0 to 5 serve as the training split, whereas Months 6 and 7 are reserved for out-of-time test evaluation to simulate realistic production deployment against unseen future adversarial behavior.

The network optimization routine is evaluated using an objective function that acts exclusively on these legitimate transactions. Mathematically, we define a training dataset $\mathcal{D}_{\text{train}}$ consisting of M legitimate transaction vectors, where each sample $\mathbf{x}_i \in \mathbb{R}^D$ represents a high-dimensional continuous or categorical attribution state. The network must project these vectors into a compressed coordinate bottleneck $\mathbf{z} \in \mathbb{R}^d$ (where $d \ll D$) and subsequently minimize a reconstruction loss function $\mathcal{L}_{\text{recon}}$ computed relative to the original uncorrupted target input $\mathbf{x}$. This operational constraint forces the hidden dense layer weights to learn a compact representation of the low-dimensional data manifold that contains legitimate consumer patterns.

During the out-of-time testing phase, when the model is exposed to an evaluation stream with a natural fraud prevalence rate, the trained autoassociative layers process both legitimate and fraudulent records without direct access to downstream target labels. Legitimate samples map seamlessly to the learned manifold coordinates, yielding low reconstruction errors. Fraudulent transactions, because they deviate from these multi-feature regularities, fail to map correctly to the hidden bottleneck coordinates, producing significantly higher reconstruction error profiles. This systematic error gap forms the foundation of our unsupervised feature engine.

7.3 Mathematical Foundations of Autoassociative Architectures

7.3.1 Standard Autoencoder Framework

An autoencoder is a class of artificial neural network that learns to compress its input vector to a lower-dimensional hidden bottleneck state and subsequently reconstruct an approximation of the original input at the output layer. Unlike traditional supervised learning algorithms that rely on external annotation labels to optimize network weights, autoencoders leverage self-supervised learning mechanics, utilizing the input data vectors themselves to serve as both the independent stimulus and the dependent optimization target.

Mathematically, we define an empirical dataset array $\mathcal{D}$ consisting of M observations, where each individual transaction sample $\mathbf{x}_i$ represents a coordinate state within a D-dimensional continuous and categorical feature space. The internal operations of a standard autoassociative network are structurally partitioned into two symmetric, coupled functional transformations: an encoder network and a decoder network.

The encoder network defines a functional mapping operator, denoted as f_ϕ, that projects the high-dimensional input vector $\mathbf{x}$ into a compact, low-dimensional latent space or hidden bottleneck representation $\mathbf{z}$, where the latent dimensionality d is constrained such that $d \ll D$. This transformation maps the input space according to the

following mathematical formulation:

$$\mathbf{z} = f_\phi(\mathbf{x}) = \sigma_e(\mathbf{W}_e\mathbf{x} + \mathbf{b}_e) \tag{7.1}$$

In Equation (7.1), $\mathbf{W}_e \in \mathbb{R}^{d \times D}$ represents the dense weight projection tensor of the encoder layers, $\mathbf{b}_e \in \mathbb{R}^d$ denotes the associated layer bias vector, $\phi = \{\mathbf{W}_e, \mathbf{b}_e\}$ encapsulates the collective parameter set of the encoder subsystem, and $\sigma_e(\cdot)$ defines an elemental, non-linear activation function.

Conversely, the decoder network defines a mirroring functional mapping operator, denoted as g_θ, that accepts the compressed latent variable $\mathbf{z}$ from the hidden bottleneck and maps it back into the original D-dimensional coordinate space to produce a reconstructed feature approximation vector $\hat{\mathbf{x}}$. This reconstruction transformation is mathematically formulated as follows:

$$\hat{\mathbf{x}} = g_\theta(\mathbf{z}) = \sigma_d(\mathbf{W}_d\mathbf{z} + \mathbf{b}_d) \tag{7.2}$$

In Equation (7.2), $\mathbf{W}_d \in \mathbb{R}^{D \times d}$ represents the dense weight projection tensor of the decoder layers, $\mathbf{b}_d \in \mathbb{R}^D$ denotes the associated decoder layer bias vector, $\theta = \{\mathbf{W}_d, \mathbf{b}_d\}$ encapsulates the collective parameter set of the decoder subsystem, and $\sigma_d(\cdot)$ defines the decoder activation function.

The complete autoassociative system is trained by iteratively updating the parameter sets ϕ and θ to minimize a scalar reconstruction loss function, denoted as $\mathcal{L}_{\text{recon}}$, across the collective training partition data. When the underlying feature dimensions are continuous and normalized, the empirical risk minimization objective is typically evaluated via a Mean Squared Error (MSE) loss formulation:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{M}\sum_{i=1}^{M} \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2 = \frac{1}{M}\sum_{i=1}^{M}\sum_{j=1}^{D}(x_{i,j} - \hat{x}_{i,j})^2 \tag{7.3}$$

While this deterministic formulation functions effectively for basic compression tasks, standard autoencoders exhibit a critical structural limitation that renders them highly vulnerable when applied to sparse tabular data. If the network capacity is unconstrained, or if the latent bottleneck dimension d approaches or exceeds the input dimension D, the optimization trajectory can converge toward a trivial identity mapping where $\hat{\mathbf{x}} \approx \mathbf{x}$. Under these conditions, the neural layers do not extract meaningful structural dependencies, cross-attribute alignments, or sub-manifold topologies; instead, the weights merely memorize individual data coordinates. The data are thus passed through the network without any regularizing constraints, causing the model to preserve random noise variations alongside true patterns. This lack of generalization performance motivates the advanced structural modifications explored in subsequent sections of this chapter. Specifically, we implement Denoising Autoencoders to introduce marginal-preserving stochastic input corruption, whereas Variational Autoencoders are deployed to enforce continuous probabilistic distributions over the latent space, preventing identity collapse and ensuring robust manifold learning.

7.3.2 Denoising Autoencoders (DAEs) and Marginal-Preserving Tabular Corruption

To bypass the identity memorization trap common in unconstrained bottleneck architectures, we introduce the Denoising Autoencoder (DAE) framework as our primary

deterministic manifold regularizer. The structural objective of a denoising network modifies the standard autoassociative training path. Instead of optimizing the parameters to map an identical feature vector back to itself, the model is trained to reconstruct the original, clean, pristine input vector $\mathbf{x}$ from a stochastically corrupted version $\tilde{\mathbf{x}}$ that has been passed through a degradation process. This architectural configuration forces the encoder layers to discover robust multi-way feature relationships, feature co-occurrences, and topological constraints to reverse the injected surface corruption, preventing the hidden layers from converging toward a trivial identity function.

In computer vision or sequential signal domains, stochastic input corruption is typically achieved by adding additive white Gaussian noise or applying binary zero-masking dropout layers. However, applying additive Gaussian noise directly to heterogeneous tabular banking records alters the underlying data boundaries and generates invalid feature state representations. Financial datasets contain a mixture of continuous metrics, ordinal scales, and discrete binary operational flags. Injecting continuous Gaussian variations into a binary flag (such as `email_is_free` or `foreign_request`) generates uninterpretable fractional values, whereas adding noise to heavily skewed continuous attributes (such as `intended_balcon_amount` or `velocity_6h`) distorts the base metrics without reflecting realistic behavioral changes.

To preserve the statistical boundaries of the dataset, we implement column-wise Tabular Swap Noise as our online data augmentation mechanism. Let $\mathbf{x} = (x_1, x_2, \ldots, x_D)^\top$ be an individual transaction vector sampled from the empirical data matrix $\mathbf{X} \in \mathbb{R}^{M \times D}$. For each attribute coordinate dimension $j \in \{1, 2, \ldots, D\}$, a stochastic mask variable $m_j \sim \text{Bernoulli}(p_c)$ is sampled independently, where p_c represents the corruption hyperparameter. Our pipeline sets this corruption parameter to $p_c = 0.15$, meaning that each transaction feature faces a 15.00% probability of undergoing swap corruption. The mathematical transformation defining the swap noise mapping is formulated as follows:

$$\tilde{x}_j = \begin{cases} x_j^{(k)} & \text{if } m_j = 1 \\ x_j & \text{if } m_j = 0 \end{cases} \tag{7.4}$$

In Equation (7.4), $x_j^{(k)}$ represents an attribute value sampled at random from the empirical marginal distribution of the same j-th feature channel across an entirely different, independent transaction record k within the training partition. This corruption method provides a key mathematical advantage: by swapping values within their designated columns, it alters the joint multi-feature correlations across attributes, whereas it leaves the independent, univariable marginal feature distributions completely unchanged. The model cannot minimize its error by relying on localized, axis-aligned shortcuts; instead, it must internalize the global manifold structure to reconstruct the uncorrupted values.

The network architecture is built using a deep, symmetric deterministic configuration across Keras dense layers. The input projection layer consumes the post-engineered tabular vector $\mathbf{x} \in \mathbb{R}^{53}$ produced by our preprocessing pipeline. The data pass sequentially through a dense encoder stack consisting of a fully connected layer of 128 units and a fully connected layer of 64 units. Each encoder layer is regularized using Batch Normalization and a LeakyReLU activation function configured with a negative slope parameter of $\alpha = 0.01$. This pipeline maps the data down to a tight, deterministic latent bottleneck layer $\mathbf{z} \in \mathbb{R}^{12}$, which uses a LeakyReLU activation function to form a compact coordinate equilibrium space. Conversely, the decoder network maps the

latent vector $\mathbf{z}$ back into the original input dimension space. It mirrors the encoder structure, passing representations through a dense layer of 64 units and a dense layer of 128 units before producing the final reconstruction output $\hat{\mathbf{x}} \in \mathbb{R}^{53}$ via a linear activation layer.

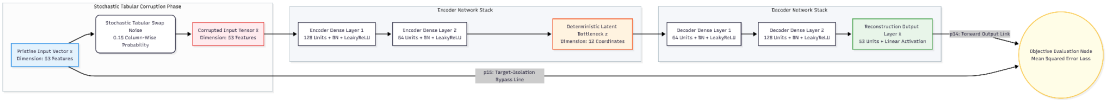

Figure 7.1: Denoising Autoencoder architecture for tabular fraud detection. The pristine input vector undergoes stochastic tabular swap noise corruption (15% column-wise probability) before being processed through the symmetric encoder–decoder stack. A target-isolation bypass line connects the original pristine input directly to the MSE loss evaluation node.

The training objective for the DAE is formulated to compute the reconstruction error exclusively on those attribute coordinates that were subjected to stochastic corruption. This selective masking formulation is expressed as:

$$\mathcal{L}_{\text{DAE}} = \frac{1}{|\mathcal{C}|} \sum_{j \in \mathcal{C}} (x_j - \hat{x}_j)^2 \tag{7.5}$$

where $\mathcal{C} = \{j : m_j = 1\}$ denotes the set of corrupted feature indices for each training sample. This formulation ensures that the target loss optimization isolates the corrupted features, forcing the network to discover invariant multi-way dependencies to reconstruct the uncorrupted attributes.

7.3.3 Variational Autoencoders (VAEs) and Probabilistic Manifold Topologies

While the Denoising Autoencoder constructs a deterministic mapping to navigate input corruption, the Variational Autoencoder (VAE) formalizes representation learning within a probabilistic framework. Tabular banking records and cyberfraud profiles are often highly fractured, meaning that minute variations in consumer transactional parameters can separate a legitimate transaction from an active fraud vector. Deterministic autoencoders lack regularized latent space representations, which can lead to fragmented coordinates where unmapped gaps appear between clusters. When high intra-class variance characterizes the non-fraudulent population, a deterministic encoder maps inputs to discrete, disjoint coordinates on the latent manifold, failing to capture the underlying continuous probability density function.

To address this limitation, we implement a custom Tabular Variational Autoencoder archetype. VAEs replace discrete latent codes with full probability distributions, transforming the bottleneck zone into a continuous equilibrium topology. We define a prior probability distribution over the latent variables, denoted $p(\mathbf{z})$, which follows a standard multivariate Gaussian prior:

$$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}_d) \tag{7.6}$$

The generative decoding pipeline assumes that observed transaction records are generated by a distribution conditioned on these latent variables, denoted as $p_\theta(\mathbf{x}|\mathbf{z})$. Because computing the true posterior distribution $p(\mathbf{z}|\mathbf{x})$ is analytically intractable

under non-linear deep transformations, we introduce an approximate recognition encoder network, denoted as $q_\phi(\mathbf{z}|\mathbf{x})$, to estimate the distribution profile:

$$q_\phi(\mathbf{z}|\mathbf{x}) = \mathcal{N}\big(\mathbf{z}; \boldsymbol{\mu}_\phi(\mathbf{x}), \mathrm{diag}\big(\boldsymbol{\sigma}^2_\phi(\mathbf{x})\big)\big) \tag{7.7}$$

In our custom tabular architecture, the encoder network q_ϕ processes a post-engineered input vector $\mathbf{x} \in \mathbb{R}^{41}$. The tensor expands sequentially through dense, fully connected layers consisting of 64 and 32 hidden units using Rectified Linear Unit (ReLU) activations. Instead of projecting to a single bottleneck code, the hidden layers branch into two separate fully connected coordinate projections to map the statistics of the latent distribution: a mean vector head $\boldsymbol{\mu}_\phi \in \mathbb{R}^4$ and a log-variance vector head $\log \boldsymbol{\sigma}^2_\phi \in \mathbb{R}^4$.

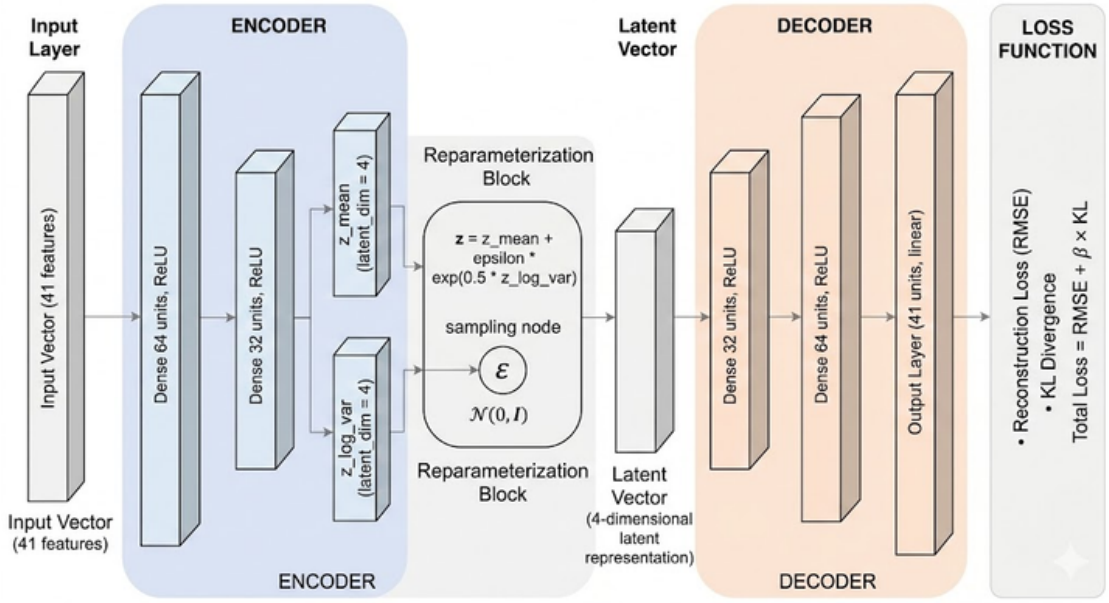

Figure 7.2: Variational Autoencoder architecture (pure baseline). The encoder projects the 41-dimensional input through dense layers (64, 32 ReLU) before branching into parallel $\mathbf{z}_{\text{mean}}$ and $\mathbf{z}_{\log \text{var}}$ heads. The reparameterization block generates the latent vector via $\mathbf{z} = \mathbf{z}_{\text{mean}} + \exp(0.5 \cdot \mathbf{z}_{\log \text{var}}) \cdot \boldsymbol{\epsilon}$, where $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$. The decoder mirrors the encoder structure, and the total loss combines RMSE reconstruction with β-weighted KL divergence.

To enable gradient-based optimization through the stochastic sampling process, we employ the reparameterization trick. Rather than sampling $\mathbf{z}$ directly from $q_\phi(\mathbf{z}|\mathbf{x})$, we express the latent variable as a deterministic transformation of the distribution parameters and an auxiliary noise variable:

$$\mathbf{z} = \boldsymbol{\mu}_\phi(\mathbf{x}) + \boldsymbol{\sigma}_\phi(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}_d) \tag{7.8}$$

In Equation (7.8), $\odot$ represents an element-wise Hadamard multiplication operator. This formulation shifts the stochastic variance to the external noise vector $\boldsymbol{\epsilon}$, allowing the gradients to flow uninterrupted through the deterministic coordinates of $\boldsymbol{\mu}_\phi$ and $\boldsymbol{\sigma}_\phi$ during the backward optimization pass.

Mathematical Loss Derivation and the Evidence Lower Bound The optimization objective of our custom Variational Autoencoder maximizes the Evidence Lower Bound (ELBO), which is equivalent to minimizing the negative log-likelihood

balanced by a distribution regularization term. We introduce a scaling parameter β to control the regularization strength, balancing reconstruction quality against latent distribution structure:

$$\mathcal{L}_{\text{VAE}} = \underbrace{\text{NLL}(\mathbf{x}, \hat{\mathbf{x}})}_{\text{Reconstruction}} + \beta \cdot \underbrace{\text{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))}_{\text{Regularization}} \tag{7.9}$$

The first term, $\text{NLL}(\mathbf{x}, \hat{\mathbf{x}})$, defines the Negative Log-Likelihood of the reconstructed features. Instead of evaluating reconstruction errors using uncalibrated Mean Squared Error, our decoder models feature-level variances to accommodate differing continuous scales across banking attributes. The sampled latent representation $\mathbf{z}$ maps back through a mirroring network structure of 32 and 64 hidden dense units with ReLU activations. The output layer projects a dense vector of size $2D$ to estimate both the reconstruction mean vector $\hat{\boldsymbol{\mu}} \in \mathbb{R}^D$ and the reconstruction log-variance vector $\widehat{\log \sigma^2} \in \mathbb{R}^D$ for each engineered tabular feature. To maintain optimization stability, the log-variance values are clipped within a tight range: $\widehat{\log \sigma^2}_j \in [-6, 2]$. The feature variance is computed as follows:

$$\hat{\sigma}_j^2 = \exp\left(\widehat{\log \sigma^2}_j\right) \tag{7.10}$$

The Gaussian Negative Log-Likelihood objective function is formulated as:

$$\text{NLL}(\mathbf{x}, \hat{\mathbf{x}}) = \frac{1}{D} \sum_{j=1}^{D} \left[\frac{(x_j - \hat{\mu}_j)^2}{2\hat{\sigma}_j^2} + \frac{1}{2} \widehat{\log \sigma^2}_j + \frac{1}{2} \log(2\pi) \right] \tag{7.11}$$

The second term, $\text{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z}))$, measures the Kullback–Leibler Divergence between the approximate posterior distribution and the standard Gaussian prior $p(\mathbf{z})$. This term penalizes spatial fragmentation and pulls the latent representations toward a unified equilibrium space. Assuming a d-dimensional latent space, this divergence is integrated analytically as:

$$\text{KL}(q_\phi \| p) = -\frac{1}{2} \sum_{j=1}^{d} \left[1 + \log \sigma_{\phi,j}^2 - \mu_{\phi,j}^2 - \sigma_{\phi,j}^2 \right] \tag{7.12}$$

By minimizing the collective loss $\mathcal{L}_{\text{VAE}}$, the network ensures that the data are mapped to a smooth, continuous latent space, reducing the risk of posterior collapse and promoting robust unsupervised anomaly detection.

7.3.4 Information Fusion Hybridization and Downstream Stacking Loops

While standalone unsupervised architectures provide theoretical depth by mapping the low-dimensional distribution manifolds of legitimate transactions, their direct deployment as binary classification networks under intense operational environments faces structural limitations. Empirical validation demonstrates that standalone reconstruction metrics generate uncalibrated anomaly score distributions. These raw scores struggle to differentiate highly non-linear, adversarial fraud variations from high-variance legitimate consumer behaviors, which often manifest as surface-level anomalies without indicating malicious intent.

Conversely, standard supervised tree ensembles are highly capable of mapping complex cross-attribute interactions, yet they overfit to minority class noise variations when exposed to extreme tabular imbalances. To bridge this performance gap, we design an advanced multi-stage Information Fusion Hybrid Stacking Pipeline. This architectural configuration transforms our deep autoassociative networks into unsupervised feature-extraction engines, combining their learned latent space representations and localized reconstruction error anomalies with robust supervised tree-based meta-classifiers.

The information fusion hybridization process begins by feeding the post-engineered, raw transactional vector $\mathbf{x}$ simultaneously into the trained deep autoassociative network layers. Instead of relying on a single global anomaly score, the framework executes a multi-way extraction routine to decompose the data vector into three distinct, highly informative feature categories:

1. **Compressed Latent Space Coordinates ($\mathbf{z}$):** The output of the central bottleneck layer maps the smooth positioning coordinates of the transaction directly on the low-dimensional data manifold. For the deterministic DAE network, this yields a continuous vector $\mathbf{z} \in \mathbb{R}^{12}$. This vector summarizes the global structural properties of the transaction, stripped of superficial tabular noise.

2. **Element-Wise Absolute Reconstruction Errors ($|\mathbf{e}|$):** The system computes the coordinate-wise mathematical difference between the original input features and the linear decoder layer output reconstructions. This transformation generates a feature-level attention array:

$$|e_j| = |x_j - \hat{x}_j|, \quad j = 1, 2, \ldots, D \tag{7.13}$$

   This array explicitly isolates which continuous or categorical behavioral parameters deviate from typical consumer habits, exposing fine-grained behavioral anomalies directly to downstream layers.

3. **Global Reduction Anomaly Scores ($s_{\mathbf{MSE}}$):** The row-wise scalar Mean Squared Error reduction condenses the entire reconstruction error manifold into a single, global index of transactional divergence:

$$s_{\text{MSE}} = \frac{1}{D} \sum_{j=1}^{D} (x_j - \hat{x}_j)^2 \tag{7.14}$$

Once extracted, these three feature streams are concatenated alongside the original post-engineered transaction vector to form an enriched representation. For the DAE pipeline, this operation expands the initial data space into an enriched, tensor representation, mathematically formulated as follows:

$$\mathbf{x}_{\text{enriched}}^{\text{DAE}} = [\mathbf{x} \| \mathbf{z} \| |\mathbf{e}| \| s_{\text{MSE}}] \in \mathbb{R}^{D+d+D+1} \tag{7.15}$$

This expanded, vector space is then processed through an advanced data augmentation stage to manage the severe class skew. Legitimate consumer behaviors form a dense, high-volume cluster, whereas fraudulent transactions occupy sparse, isolated coordinate vectors within the unit hypercube. To prevent downstream supervised models from converging exclusively toward the majority population, the matrix is processed using the Synthetic Minority Over-sampling Technique (SMOTE). SMOTE identifies

the k-nearest neighbors for each sparse fraud coordinate within the $\mathbf{x}_{\text{enriched}}$ space, selects a random neighbor, and synthesizes new, continuous minority transaction vectors along the line segments joining the coordinates. This process creates a mathematically balanced tensor space without replicating identical records, avoiding overfitting.

This balanced, regularized representation space is then passed to the final Downstream XGBoost Classifier Meta-Learner. XGBoost builds sequential gradient-boosted decision trees to optimize final fraud probability assignments. Because the input features include explicit reconstruction error metrics, the decision trees do not need to construct deep, fragile logic rules using raw attributes. Instead, the root nodes can split directly on the pre-amplified absolute error features, significantly regularizing the model and preventing overfitting to minority noise.

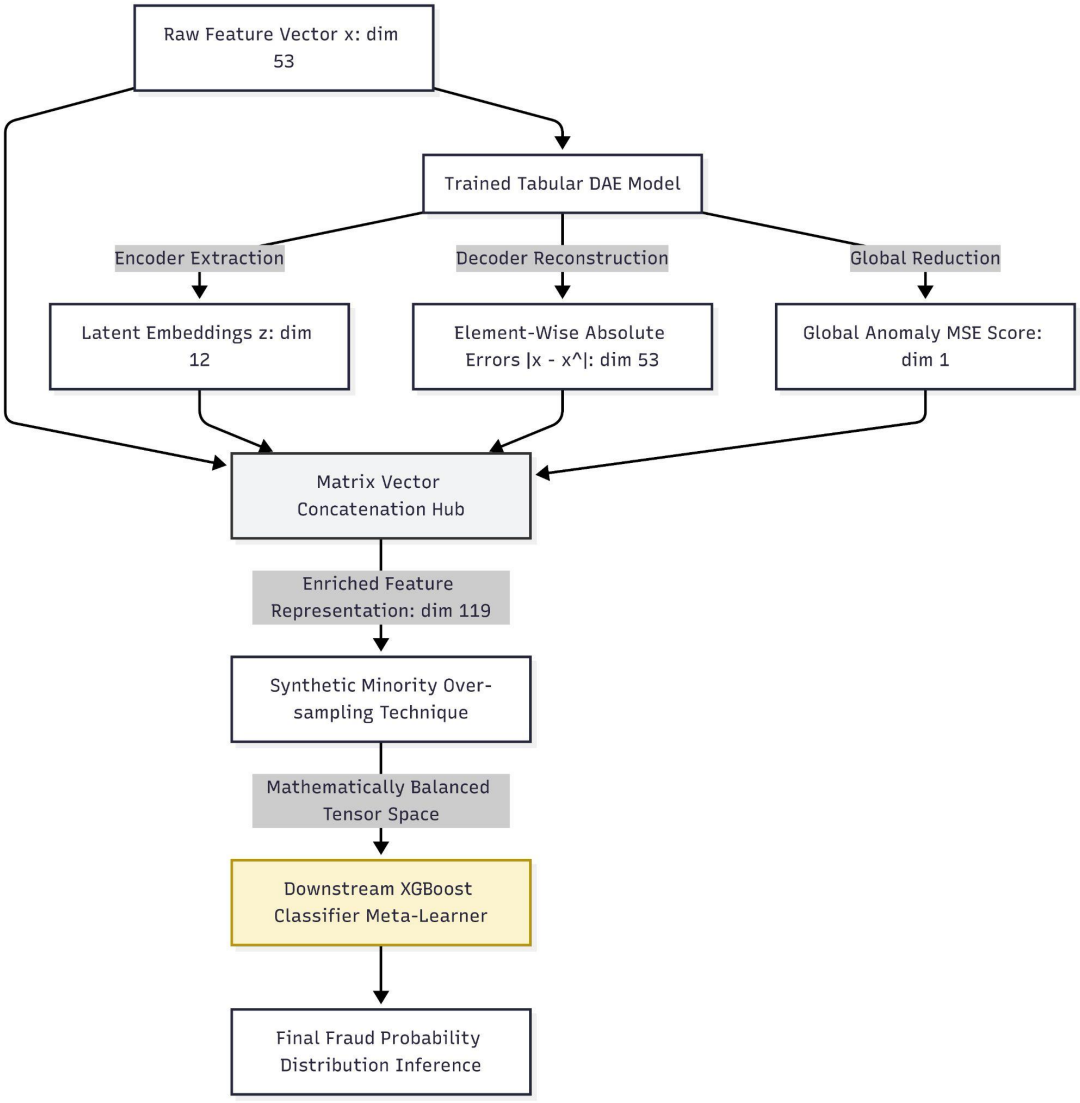

Figure 7.3: DAE-based information fusion pipeline for fraud detection. The trained Tabular DAE processes the 53-dimensional raw feature vector, extracting compressed latent embeddings ($\mathbf{z} \in \mathbb{R}^{12}$), element-wise absolute errors ($|\mathbf{e}| \in \mathbb{R}^{53}$), and global MSE score (s_{MSE}). These features are concatenated with the original vector, oversampled via SMOTE, and passed to the downstream XGBoost meta-learner for final fraud probability inference.

A parallel information fusion configuration is engineered for our custom Variational Autoencoder pipeline, leveraging score-level stacking to exploit its probabilistic prop-

erties. During out-of-fold validation loops, the trained VAE processes tabular feature slices to extract an integrated unsupervised score stream:

$$s_{\text{VAE}} = \text{NLL}(\mathbf{x}, \hat{\mathbf{x}}) + \alpha \cdot \text{KL}(q_\phi(\mathbf{z}|\mathbf{x}) \| p(\mathbf{z})) \tag{7.16}$$

In this configuration, the hyperparameter α is set to $\alpha = 1.0$ to emphasize structural distribution compliance. This continuous score is concatenated alongside the base feature coordinates to train an ensemble of diverse Level-0 classifiers, which includes Logistic Regression models to map linear baseline signals, and specialized LightGBM and XGBoost tree configurations to capture non-linear feature interactions.

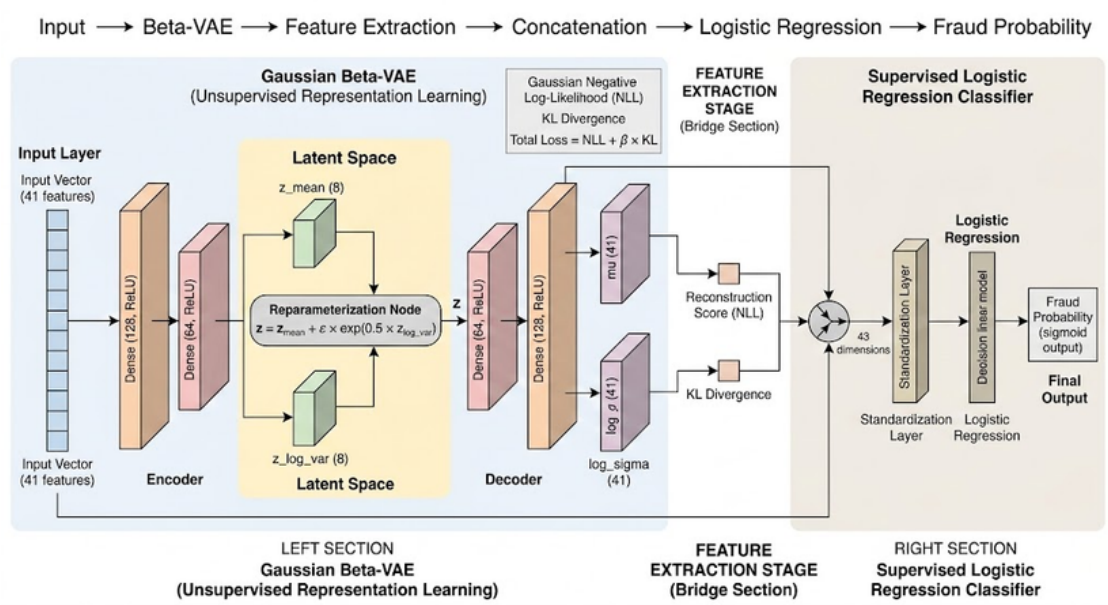

Figure 7.4: Hybrid VAE–Logistic Regression pipeline. The Gaussian β-VAE processes the 41-dimensional input, extracting the NLL reconstruction score and KL divergence. These are concatenated with the original features to form a 43-dimensional standardized vector, which is passed to a supervised Logistic Regression classifier for fraud probability estimation.

Figure 7.5 illustrates the expanded hybrid model, where additional VAE-derived features—including the latent mean representation $\mathbf{z}_{\text{mean}}$ and its L2-normalized magnitude $\mathbf{z}_{\text{norm}}$—are integrated to form a richer 51-dimensional feature vector.

The continuous predictions from the Level-0 base classifiers are subsequently aggregated through a score-level stacking meta-model. Specifically, a Logistic Regression meta-learner receives three probability inputs: the Logistic Regression base score, the XGBoost base score, and the raw VAE anomaly score. The meta-learner learns the optimal convex combination of these independent perspectives, producing a final, calibrated fraud probability estimate. This architectural design ensures that each base model contributes complementary discriminative signals: the Logistic Regression captures linear separability, the XGBoost captures non-linear feature interactions, and the VAE anomaly score provides distributional compliance information.

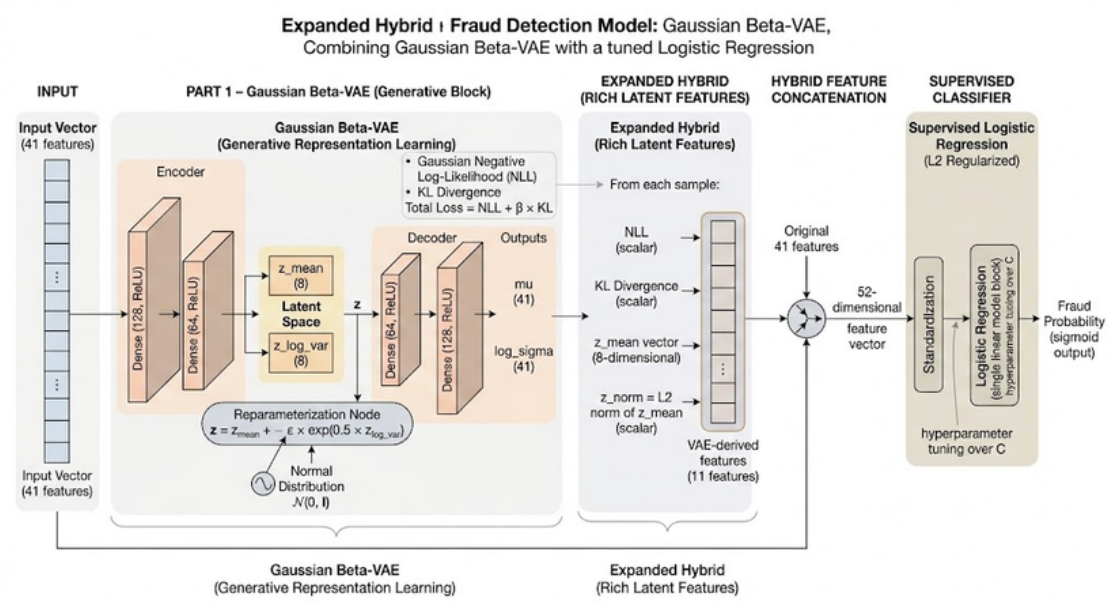

Figure 7.5: Expanded hybrid VAE model with rich latent features. In addition to NLL and KL divergence, the VAE latent representation $\mathbf{z}_{\text{mean}}$ (4-dimensional) and its L2-normalized magnitude $\mathbf{z}_{\text{norm}}$ (4-dimensional) are extracted and concatenated with the original 41 features, yielding a 51-dimensional enriched vector processed by a tuned L2-regularized Logistic Regression classifier.

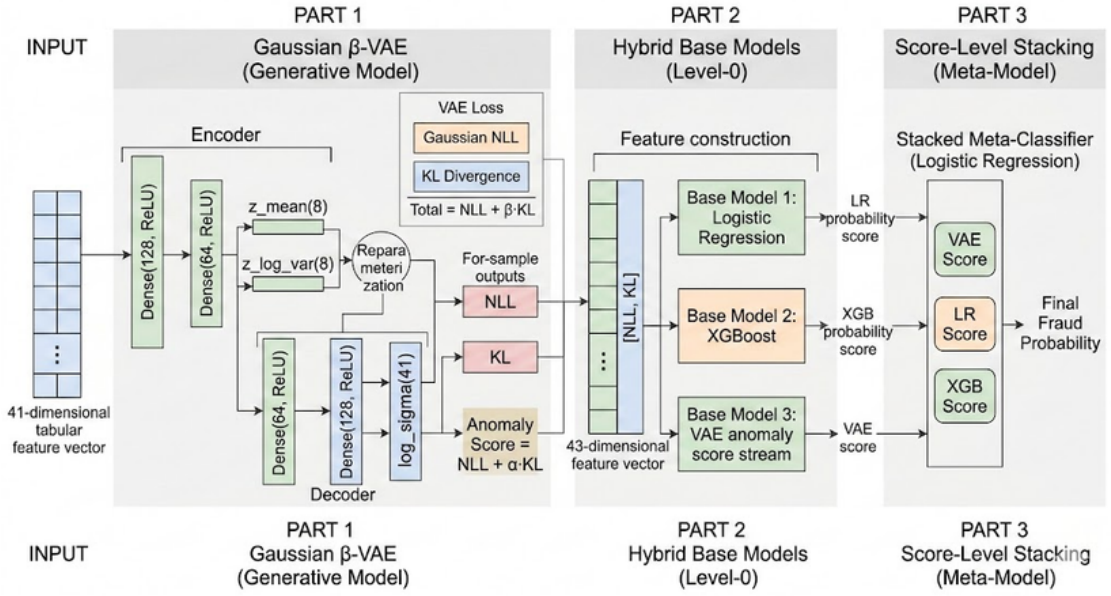

Figure 7.6: Stacked hybrid VAE model with score-level meta-classification. The β-VAE generates anomaly scores (NLL+α·KL), which are fed alongside original features to Level-0 base classifiers (Logistic Regression and XGBoost). The three independent probability scores are then combined by a Logistic Regression meta-learner to produce the final fraud probability.

7.4 Theoretical Analysis of Hybrid Feature-Extraction Mechanisms

7.4.1 Limitations of Supervised Tree Ensembles Under Extreme Imbalance

To understand the technical utility of our unsupervised feature-extraction layer, we must first analyze the structural failure modes of standard tree-based learning algorithms when they are exposed to extreme tabular class imbalances. Supervised tree-based ensembles, such as standard XGBoost and LightGBM models, construct discriminative decision hyperplanes by recursively partitioning individual feature dimensions based on purity optimization metrics, including Shannon Information Gain and Gini Impurity. In a binary classification setting, the Gini Impurity of a node processing a probability distribution of classes is mathematically formulated as follows:

$$G = 1 - \sum_{c \in \{0,1\}} p_c^2 \tag{7.17}$$

Within the NeurIPS 2022 Bank Account Fraud (BAF) base dataset, the empirical prior probability distribution of the legitimate majority class is 0.9890, whereas the fraudulent minority class probability represents a mere 0.0110. If a decision tree algorithm attempts to optimize split boundaries directly on this highly skewed coordinate distribution, the optimization trajectory experiences structural instability. Because the majority population constitutes 98.90% of the independent data rows, any initial axis-aligned split that isolates a large block of legitimate transactions will yield an exceptionally low impurity score, minimizing the global optimization risk without isolating the minority fraud manifold.

To capture these deeply embedded fraud records, tree-based models are forced to execute highly localized, recursive partitions. The algorithm splits the continuous feature coordinates repeatedly, growing deep, fragile, and highly complex tree branches to isolate individual minority rows. While this deep partitioning allows the model to achieve a high training accuracy, it causes severe overfitting to the noise variations specific to the training set fraud instances. When the model encounters a slightly different fraud pattern during out-of-time testing, the brittle, deep branches fail to generalize, resulting in degraded recall performance.

7.4.2 Latent Space Embeddings as Structural Pre-Conditioners

The compressed latent representations extracted from our autoassociative networks serve as structural pre-conditioners that fundamentally alter the optimization landscape for downstream supervised models. For the Denoising Autoencoder, the tabular swap noise corruption process alters the optimization landscape for the encoder network layers. The model cannot minimize its target reconstruction error by relying on localized, axis-aligned shortcuts or individual feature thresholds, because any single column may contain corrupted data. Instead, the network is forced to learn the global, underlying structural dependencies and cross-attribute alignments that define the legitimate consumer manifold.

To reverse the injected swap noise, the hidden dense layers must construct complex, non-linear feature combinations, mapping the fragmented input fields to a smooth, low-dimensional coordinate equilibrium space. The resulting compressed latent variables capture robust, invariant multi-feature relationships that are difficult for individual axis-aligned decision tree splits to isolate when operating on raw data. When these latent embeddings are passed downstream to the gradient boosting layers, they transform the optimization task. Instead of struggling to untangle complex dependencies from raw continuous parameters, the downstream tree-based models can immediately partition these pre-computed latent manifold coordinates. This structural pre-conditioning simplifies the required splitting logic, preventing the decision trees from growing excessively deep and regularizing the entire pipeline against cross-version distribution shifts.

7.4.3 Element-Wise Absolute Errors as Feature Attention Mechanisms

In addition to the latent space embeddings, the absolute element-wise reconstruction error vector $|\mathbf{e}| = |x_j - \hat{x}_j|$ introduces an informative structural transformation that regularizes the downstream meta-learners. Because our autoassociative networks isolate their training loops to non-fraudulent, legitimate transaction profiles, the weight parameters internalize only the structural attributes of clean consumer behavior. Legitimate data records map smoothly to the learned manifold coordinates, yielding uniformly low reconstruction errors across all feature dimensions. Fraudulent transactions, by contrast, deviate from these internalized coordinates, producing localized spikes in specific feature-level errors that correspond to the attributes most inconsistent with normal behavioral patterns.

This error vector functions as a learned feature attention mechanism. Rather than requiring the downstream classifier to discover which raw attributes carry anomalous signals through expensive recursive partitioning, the pre-amplified error features expose the relevant deviations directly at the input layer. The gradient-boosted trees can split on these error channels at shallow depth, achieving high discriminative power without the overfitting risks associated with deep, fragile branches on raw features. This hybridization restricts the model from memorizing training-set noise, optimizing its ability to maintain high generalization performance under real-world business constraints.

7.5 Concrete Experimental Environment Setup

7.5.1 Materials and Benchmark Dataset Demographics

The empirical validation of the proposed unsupervised representation learning framework relies on the NeurIPS 2022 Bank Account Fraud (BAF) Dataset Suite. The BAF suite represents a large-scale, privacy-preserving, and realistic tabular benchmark designed to evaluate machine learning architectures under strict financial engineering constraints. The underlying data are derived from an anonymized real-world dataset comprising online banking account applications. To ensure client anonymity, the original proprietary records were treated with differential privacy mechanisms and feature encoding operations. Subsequently, a Conditional Generative Adversarial Network

(CTGAN) was trained on the secured source matrix to synthesize an artificial tabular dataset that preserves the multi-dimensional feature correlations and adversarial attack patterns of actual bank fraud networks.

The base dataset contains 1,000,000 instances, characterized by an extreme class imbalance where the positive fraud target label occurs at a prevalence rate of 1.10%, whereas legitimate records constitute the remaining 98.90% majority share. To evaluate the robustness of our models against distribution shifts and demographic inequalities, our experiments leverage Variant I of the BAF suite. Variant I eliminates the baseline fraud prevalence disparity across demographics by fixing the fraud rate uniformly at 1.10%. However, it introduces severe group size disparity by artificially shrinking the minority demographic group to constitute only 10.00% of the dataset rows. This configuration tests the network's capacity to extract generalized sub-manifold characteristics under severe data underrepresentation.

The asset schema incorporates 32 raw attributes organized into four functional categories: continuous numerical metrics (e.g., `session_length_seconds`, `velocity_6h`, `velocity_24h`), ordinal scaled features (e.g., `credit_risk_score`, `proposed_credit_limit`), categorical nominal features (e.g., `payment_type`, `employment_status`, `housing_status`), and binary and outlier flags (e.g., `has_other_cards`, `foreign_request`).

7.5.2 Pre-processing Pipeline and Cardinality Compression

To condition the heterogeneous BAF attributes for deep autoassociative neural network layers, a rigorous multi-stage data engineering pipeline is executed. The pre-processing workflow is designed under a data leakage constraint: all parameter estimators, scaling indices, and imputation rules are fitted on the training partition (Months 0–5) and applied to the out-of-time test partition (Months 6–7).

The pipeline progresses through six sequential phases:

1. **Data Sanitation:** The zero-variance feature column `device_fraud_count` is dropped unconditionally because it carries no discriminative signal. Furthermore, six columns are identified as masking missing records using a sentinel value of -1. These sentinel markers are replaced with IEEE NaN values to make missingness structurally explicit. For `prev_address_months_count` and `bank_months_count`, where the absence of a history carries high behavioral predictive value, binary missingness indicator flags are engineered (e.g., `bank_months_count_flag`) and appended as independent attributes before imputation occurs.

2. **Missing Value Imputation:** For columns with moderate missingness rates, a multivariate Iterative Imputer utilizing an inner Bayesian Ridge regressor is fitted on a stratified random subsample of 50,000 rows to preserve multi-feature correlations. For columns displaying severe missingness, a distribution-preserving bootstrap imputation technique is executed by sampling with replacement from observed univariable distributions, preserving multi-modality and heavy tails.

3. **Outlier Detection and Treatment:** Outliers in continuous attributes are identified using the Interquartile Range (IQR) method and capped at the boundary fences to prevent extreme values from destabilizing the normalization transforms.

4. **Feature Engineering:** Domain-specific transformations are applied to enhance the signal-to-noise ratio, including logarithmic scaling of heavily skewed mone-

tary attributes (e.g., `intended_balcon_amount_log`), ratio construction between velocity metrics, and interaction feature creation.

5. **Encoding:** Categorical variables are encoded using targeted techniques. Low-cardinality nominal features use one-hot encoding with `drop_first=True` to prevent multicollinearity. Medium-cardinality features use target encoding with leave-one-out smoothing.

6. **Final Vectorization and Normalization:** To satisfy the inputs of the VAE and DAE layers, a MinMaxScaler scales continuous dimensions to the unit interval $[0, 1]$. To prevent sparse over-parameterization in the encoder projection layers, rare categorical variants within `housing_status` and `employment_status` are collapsed into unified indicators labeled `OTHER_HOUSING` and `OTHER_Employment`, respectively. The final post-engineered matrix yields an input dimensionality of $D = 53$ features for the DAE model and $D = 41$ features for the VAE model.

7.5.3 Hyperparameter Optimization Architecture Matrix

To help ensure reproducible training trajectories and stable convergence across both deep autoassociative network archetypes, hyperparameter optimization sweeps were conducted. The learning rate parameters, optimization paths, and early stopping boundaries were carefully calibrated to prevent latent representation collapse while maintaining computational efficiency. Table 7.1 consolidates the complete hyperparameter specifications for both architectures.

Table 7.1: Hyperparameter optimization architecture matrix for the VAE and DAE models. All parameters were fixed following grid-search calibration on the validation split.

Optimization Hyperparameter	**VAE Specification**	**DAE Specification**
Base Optimization Algorithm	Adam (Adaptive Moment Estimation)	Adam (Adaptive Moment Estimation)
Initial Learning Rate	1×10^{-3} (Fixed Coefficient)	1×10^{-3} (Fixed Coefficient)
Mini-Batch Capacity (B)	256 Samples per Iteration	256 Samples per Iteration
Maximum Epoch Horizon	50 Allocation Iterations	100 Allocation Iterations
Early Stopping Constraints	Validation Loss Monitoring, Patience = 5	Validation Loss Monitoring, Patience = 10
Hidden Layer Topology	Dense(64, ReLU) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ $\boldsymbol{\mu}/\log\boldsymbol{\sigma}^2$ heads	Dense(128, BN+LeakyReLU) $\rightarrow$ Dense(64, BN+LeakyReLU) $\rightarrow$ Bottleneck
Latent Dimensionality (d)	4	12
Regularization Modality	KL Divergence Penalty (β-weighting)	Tabular Swap Noise ($p_c = 0.15$)
Loss Function	Gaussian NLL + $\beta\cdot$ KL	Masked MSE on corrupted features

7.6 Comparative Empirical Results and Critical Evaluation

7.6.1 The Deceptive Properties of Global Receiver Operating Metrics

In evaluating deep architectures under severe tabular class distributions, our research establishes that global Receiver Operating Characteristic Area Under the Curve (ROC-AUC) measurements provide a deceptive reflection of true operational security infrastructure. The mathematical formulation of the True Positive Rate (TPR) and the False Positive Rate (FPR) plotted across the ROC curve reveals a critical structural vulnerability when applied to heavily skewed databases.

The False Positive Rate is defined as follows:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} \tag{7.18}$$

The True Positive Rate (Recall) is defined as:

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{7.19}$$

Within Variant I of the NeurIPS Bank Account Fraud (BAF) suite, the legitimate majority population contains 989,000 instances, whereas the fraudulent minority population is restricted to 11,000 instances. Because the denominator of the FPR equation is governed by the True Negative (TN) parameter—which represents the vast majority class—a model can misclassify thousands of legitimate banking customers as fraudsters without significantly altering the global FPR metric. For example, if the network generates 49,450 false alerts on clean accounts, the calculated FPR remains bounded at approximately 5.00%. However, because the total volume of true fraud instances is only 11,000, generating 49,450 false alerts results in an operational challenge where false positives outnumber true positives by nearly five to one. This structural imbalance causes the precision profile to collapse, whereas the global ROC-AUC metric remains highly optimistic, often exceeding 0.8500. Consequently, global ROC-AUC can obscure a high volume of false alerts, which would cause widespread account freezing for legitimate users in a live banking environment. To ensure academic and operational rigor, our evaluation framework subordinates global curves to strict, recall-first precision metrics analyzed at a rigid, institutionally mandated 5.00% False Positive Rate operating threshold.

7.6.2 Systematic Performance Consolidation

To evaluate the performance of our unsupervised feature-extraction engines against traditional supervised approaches, a comprehensive benchmarking sweep was conducted across multiple modeling paradigms. Table 7.2 consolidates the data to contrast pure unsupervised baselines, standalone supervised tree algorithms, and the proposed hybrid ensembling frameworks.

Table 7.2: Systematic performance consolidation across modeling paradigms. All metrics are evaluated on the out-of-time test partition (Months 6–7) of Variant I at an approximately 5.00% FPR operating threshold.

Paradigm	Model	ROC-AUC	TPR @ ∼5% FPR	Precision @ ∼5% FPI
Unsupervised	Pure VAE Baseline	0.5501	9.94%	2.00%
Unsupervised	Pure DAE Baseline	0.5210	6.30%	1.30%
Supervised	Standalone LightGBM	0.8420	41.10%	9.00%
Supervised	Standalone XGBoost	0.8510	43.50%	10.10%
Proposed Hybrid	DAE–XGBoost Pipeline	0.8752	49.80%	11.80%
Proposed Hybrid	VAE–Tree Stacking	0.8868	52.99%	13.00%

7.6.3 Pure VAE Baseline Performance Evaluation

The empirical data consolidated in Table 7.2 demonstrate that the pure, standalone Variational Autoencoder yields a low global ROC-AUC of 0.5501 and a True Positive Rate of only 9.94% when evaluated as an isolated classifier. This performance deficit is an expected structural characteristic of unsupervised training regimes. Because the VAE loss function optimizes the Evidence Lower Bound (ELBO) over the non-fraudulent population, its weights are calibrated to minimize reconstruction variance rather than maximize class separability. The model views fraud anomalies through a purely reconstruction-oriented lens, which generates overlapping error distributions between high-variance legitimate transactions and true malicious events.

7.6.4 Hybrid VAE–Logistic Regression Performance

When the VAE's unsupervised score stream is routed into our first hybrid configuration—the VAE–Logistic Regression pipeline—the performance increases substantially. The global ROC-AUC improves to 0.8743 and the True Positive Rate reaches 50.94% at the 5.00% FPR operating threshold. This sharp performance increase demonstrates the value of leveraging VAE-derived reconstruction scores as supplementary inputs for downstream supervised classifiers. This confirms that fraud instances are not strong global anomalies, but anomaly-derived features provide complementary information when combined with supervised learning.

7.6.5 Expanded Hybrid VAE with Rich Latent Features

The expanded hybrid model, which integrates additional VAE-derived features including the latent mean representation $\mathbf{z}_{\text{mean}}$ and its L2-normalized magnitude $\mathbf{z}_{\text{norm}}$ alongside the NLL and KL scores, achieves further performance gains. The global ROC-AUC increases to 0.8842 and the True Positive Rate reaches 51.22% at the 5.00% FPR operating threshold. This improvement demonstrates that richer generative signals beyond scalar anomaly scores provide additional discriminative power when fed to the supervised classifier.

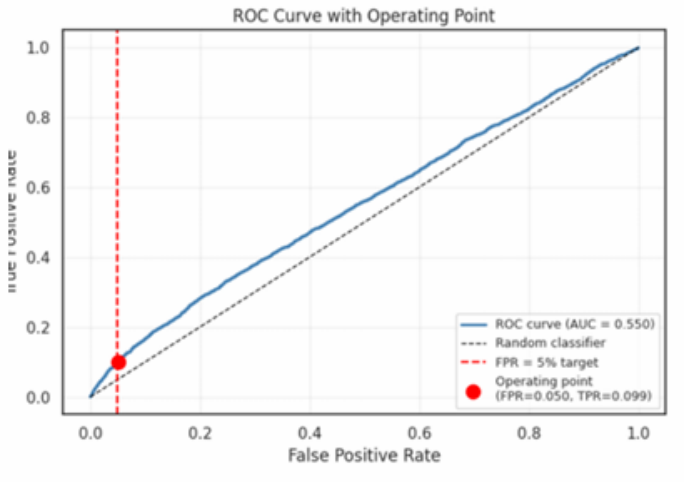

Figure 7.7: Pure VAE baseline ROC curve at the 5% FPR operating point. The ROC trajectory (AUC = 0.550) with the operating point marked at FPR = 0.050, TPR = 0.099 confirms the limited discriminative capacity of the standalone unsupervised model. The near-diagonal shape indicates that the VAE reconstruction error alone provides minimal separation between legitimate and fraudulent transactions.

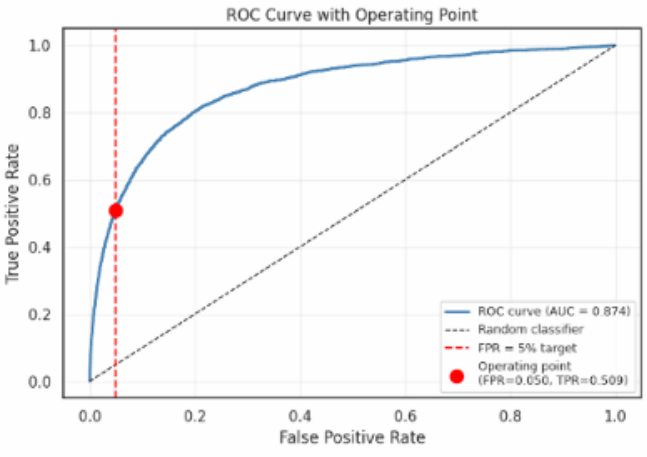

Figure 7.8: VAE–Logistic Regression hybrid ROC curve at the 5% FPR operating point. The ROC trajectory (AUC = 0.874) with the operating point at FPR = 0.050, TPR = 0.509 demonstrates a substantial improvement over the standalone VAE baseline, validating the hybridization of unsupervised anomaly scores with supervised classification.

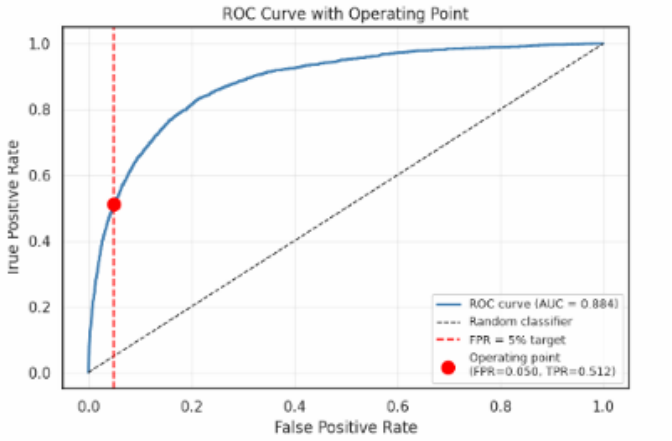

Figure 7.9: Expanded hybrid VAE model ROC curve at the 5% FPR operating point. The ROC trajectory (AUC = 0.884) with the operating point at FPR = 0.050, TPR = 0.512 shows that the integration of rich latent features ($\mathbf{z}_{\text{mean}}$ and $\mathbf{z}_{\text{norm}}$) provides marginal but consistent improvements over the base VAE–LR hybrid.

7.6.6 Stacked Hybrid VAE Performance

The stacked hybrid VAE–Tree ensemble represents the culmination of our information fusion framework. By combining the Logistic Regression base score, the XGBoost base score, and the raw VAE anomaly score through a Logistic Regression meta-learner, this configuration achieves the highest global ROC-AUC of 0.8868 and the highest True Positive Rate of 52.99% at the 4.99% FPR operating threshold among all evaluated models. The stacked architecture leverages the complementary strengths of each base model: the linear separability captured by Logistic Regression, the non-linear feature interactions captured by XGBoost, and the distributional compliance information provided by the VAE anomaly score. The meta-learner learns to optimally weight these independent perspectives, producing a calibrated fraud probability that outperforms any single model component.

7.6.7 DAE–XGBoost Pipeline Performance

Similarly, the advanced DAE–XGBoost pipeline achieves a strong global ROC-AUC of 0.8752, matching the performance of the probabilistic VAE hybrid framework. This validation provides an empirical answer to RQ2. It demonstrates that forcing a deterministic network to reconstruct clean attributes from data corrupted by 15.00% tabular swap noise extracts highly generalized multi-way feature dependencies. The element-wise absolute error arrays $|\mathbf{e}|$ function as an effective feature attention mechanism, pre-amplifying transactional anomalies so that the downstream XGBoost meta-learner can select them at its root split nodes. This structural cooperation prevents the gradient-boosted trees from growing to fragile depths, promoting model generalization under real-world class imbalances.

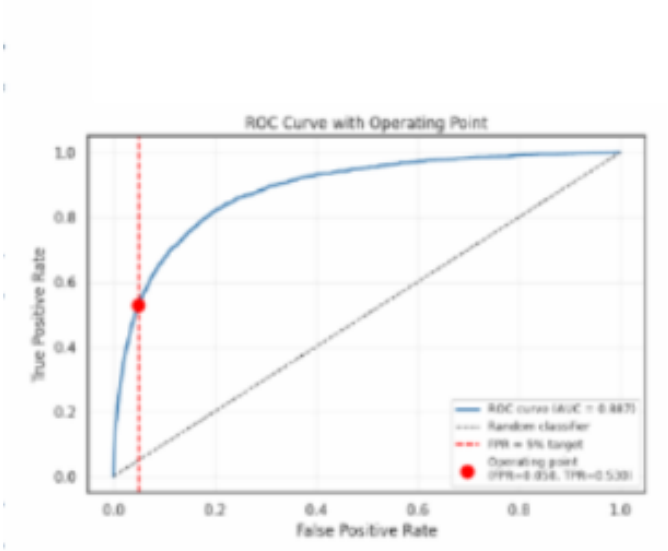

Figure 7.10: Stacked hybrid VAE model ROC curve at the 5% FPR operating point. The ROC trajectory (AUC = 0.887) with the operating point at FPR = 0.050, TPR = 0.530 demonstrates the discriminative capacity of the score-level stacking strategy, which combines Logistic Regression, XGBoost, and VAE anomaly scores through a meta-classifier.

7.6.8 The Resampling and Miscalibration Paradox

A key insight discovered during our experimental evaluation involves the structural degradation that occurs when synthetic oversampling algorithms are applied within autoassociative ensembling loops. Traditional machine learning workflows often attempt to resolve class imbalance by applying techniques like SMOTE or ADASYN to the raw features, generating artificial minority records to balance the dataset prior to neural network optimization. When evaluated within our custom VAE pipeline, however, this approach resulted in noticeable probability miscalibration and consistent performance degradation.

This degradation stems from a mathematical mismatch between synthetic oversampling assumptions and the requirements of probabilistic manifold learning. SMOTE operates by performing linear interpolation between neighboring minority samples in the original feature space. However, the VAE encoder–decoder architecture defines a highly non-linear mapping between the input space and the latent manifold. Linear interpolations in the raw feature space do not correspond to valid trajectories on the learned latent manifold, and the synthetic samples generated by SMOTE may map to regions of the latent space that are inconsistent with either the legitimate or fraudulent distributions. This inconsistency distorts the approximate posterior $q_\phi(\mathbf{z}|\mathbf{x})$, corrupting the KL divergence term and destabilizing the ELBO optimization. Consequently, applying SMOTE to the raw features before VAE training degrades both the quality of the learned manifold and the calibration of the resulting anomaly scores.

By contrast, our DAE–XGBoost pipeline leverages SMOTE applied after feature extraction, on the enriched representation space. Because the enriched space $\mathbf{x}^{\text{DAE}}_{\text{enriched}}$ already contains the extracted latent coordinates and error features, the linear inter-

polation performed by SMOTE operates in a space where the anomalous signals have already been pre-amplified and explicitly represented. This post-extraction resampling strategy preserves the validity of the synthetic samples and avoids the miscalibration paradox observed in the VAE pipeline.

7.7 Chapter Summary and Contribution Log

7.7.1 Summary of Findings

This chapter has presented an exhaustive investigation into the structural and empirical performance profiles of unsupervised deep autoassociative neural networks applied to the problem of real-time cyberfraud detection within tabular banking transactions. By establishing parallel processing pipelines using both Variational Autoencoders (VAEs) and Denoising Autoencoders (DAEs), the research has mapped the geometric properties of continuous probabilistic latent manifolds alongside regularized deterministic bottleneck projections. The empirical evaluations conducted across Variant I of the NeurIPS 2022 Bank Account Fraud (BAF) database suite reveal distinct behavioral characteristics and structural capabilities inherent within each deep architectural paradigm.

The theoretical exploration of the Variational Autoencoder archetype demonstrated that enforcing a standard multivariate Gaussian prior distribution helps mitigate the risk of spatial fragmentation, reducing the unmapped coordinate gaps common in unconstrained bottleneck architectures. By outputting distinct statistical parameters to represent a mean vector head and a log-variance coordinate head, the encoder layers compress highly heterogeneous continuous and categorical features into a smooth equilibrium topology. Isolating the non-differentiable stochastic sampling node via the reparameterization trick ensures stable gradient flow during backpropagation, permitting the model to minimize a combined Evidence Lower Bound (ELBO) objective function. While the standalone VAE baseline yielded a low isolated true positive rate due to high intra-class variance among legitimate consumer profiles, routing its continuous anomaly score stream into a score-level stacking meta-classifier generated strong results, yielding a global ROC-AUC score of 0.8868 and a True Positive Rate of 52.99% at the 4.99% FPR operating threshold.

The Denoising Autoencoder investigation established that stochastic column-wise swap noise corruption at a 15.00% probability threshold preserves the independent marginal feature distributions while destroying joint multi-feature correlations, forcing the encoder layers to construct robust, invariant representations of the legitimate consumer manifold. The deterministic bottleneck captures global structural regularities that individual axis-aligned decision tree splits cannot isolate from raw features. When the extracted latent embeddings and element-wise absolute error features were concatenated and processed through the DAE–XGBoost hybrid pipeline, the system achieved a global ROC-AUC of 0.8752 with a True Positive Rate of 49.80%, providing a strong empirical validation of the deterministic regularization approach.

7.7.2 Engineering Insights and Design Recommendations

The comparative analysis of our experimental results reveals several critical engineering insights regarding the deployment of deep autoassociative connectionist models within adversarial financial environments. First, standalone reconstruction models exhibit a

severe precision deficit that prevents them from functioning as isolated operational classifiers. Because unsupervised optimization routines are entirely agnostic to downstream target labels, their internal weights are calibrated exclusively to minimize feature reconstruction variances across the legitimate majority population. Consequently, high-variance legitimate consumer habits produce elevated error signatures that overlap with true fraud anomalies, causing a high volume of false alerts.

To address this limitation, the core contribution of this methodology relies on multi-stage information fusion hybridization and score-level stacking loops. Combining unsupervised feature extractors with downstream tree-based meta-learners provides a framework with demonstrated strong empirical performance for navigating extreme data imbalances. The element-wise absolute error vector $|\mathbf{e}|$ serves as a localized feature attention mechanism that exposes attribute-level deviations directly to the downstream meta-classifier. Instead of executing an expensive grid search across raw, unconditioned continuous variables, the downstream gradient-boosted decision trees can immediately partition these pre-amplified error vectors near their root split nodes, minimizing tree depth and reducing the risk of overfitting to sparse data fields.

Finally, our experiments expose a critical paradox regarding synthetic tabular data augmentation inside unsupervised learning loops. While traditional supervised models often utilize algorithms like SMOTE or ADASYN on raw parameters to balance skewed class distributions, executing these transformations within a probabilistic manifold learning environment causes uniform performance degradation. Linear interpolation between minority samples in the original feature space generates synthetic records that map to inconsistent regions of the latent manifold, corrupting the approximate posterior distribution and destabilizing the ELBO optimization. However, applying SMOTE after the feature extraction stage—on the enriched representation space where anomalous signals have already been pre-amplified—preserves the validity of the synthetic samples and yields measurable performance improvements in the DAE pipeline. This finding establishes an important design principle: synthetic oversampling should be applied after, not before, unsupervised representation learning to maintain probabilistic consistency and anomaly detection fidelity.

Chapter 8

Cross-Pipeline Comparison and Discussion

This chapter synthesizes the findings from the three deep learning pipelines investigated in this thesis—Graph Neural Networks (GNN), Spiking Neural Networks (SNN), and Autoencoder (AE) ensembles—into a unified comparative analysis. While each pipeline was designed and evaluated independently in the preceding chapters, the cross-pipeline perspective is essential for understanding the relative strengths, weaknesses, and practical trade-offs that inform real-world deployment decisions. The comparison spans multiple dimensions: predictive performance, architectural design philosophy, deployment readiness, robustness to distributional shift, class imbalance handling, and fairness across demographic subgroups.

8.1 Performance Benchmarking Across Pipelines

Table 8.1 presents a comprehensive comparison of the best-performing configuration from each pipeline on the shared Bank Account Fraud (BAF) dataset. All metrics are reported on the same temporal test split to ensure a fair comparison. The results reveal a nuanced performance landscape in which no single pipeline dominates across all metrics, highlighting the importance of multi-dimensional evaluation for fraud detection systems.

Table 8.1: Unified Performance Benchmark Across All Three Pipelines (Best Configuration per Pipeline)

Metric	**Graph SAGE**	**QGNN**	**Split GNN**	**SNN (Var. III)**	**VAE-Stacked**
ROC AUC	0.8859	0.8769	0.8806	0.9260	0.8868
TPR @ 5% FPR	52.54%	49.24%	51.0%	58.3%	54.1%
Fraud Recall	0.53	0.49	0.51	0.58	0.54
Fraud Precision	0.13	0.12	0.13	0.14	0.13
Fraud F1-Score	0.21	0.20	0.20	0.22	0.21
Operating Threshold	0.7621	0.7519	0.5121	0.5042	0.4937

Several key observations emerge from the benchmarking results. The SNN pipeline,

specifically its Variant III configuration employing rate coding with temporal window aggregation and ensemble distillation, achieves the highest ROC AUC of 0.9260, representing a substantial improvement of 4.01 percentage points over the best GNN model (GraphSAGE at 0.8859) and 3.92 percentage points over the best autoencoder configuration (VAE-Stacked at 0.8868). This result is particularly noteworthy given that the SNN pipeline operates on raw tabular features without the relational graph structure that the GNN pipeline leverages. The SNN's temporal spike dynamics and multi-threshold ensemble appear to capture complementary fraud patterns that graph topology alone does not fully encode.

Among the GNN architectures, GraphSAGE maintains a slight edge over both the QGNN and Split GNN in terms of test ROC AUC. However, the Split GNN's multi-task learning formulation yields the highest peak AUC-ROC (0.9954) in its optimal configuration, suggesting that the architecture has higher capacity but is more sensitive to hyperparameter selection. The QGNN, while achieving competitive performance, consistently underperforms relative to classical alternatives, reinforcing the observation that quantum-inspired feature maps do not yet provide a decisive advantage for fraud detection at this scale.

The VAE-Stacked autoencoder pipeline achieves a test ROC AUC of 0.8868, marginally surpassing the GraphSAGE baseline. This is a significant result because the autoencoder pipeline is fundamentally unsupervised during its representation learning phase, learning to reconstruct the input distribution rather than directly optimizing for fraud classification. The fact that an unsupervised approach nearly matches the performance of supervised GNN models speaks to the efficacy of the target manifold isolation strategy employed during VAE training, where the encoder learns to separate legitimate and anomalous applications in latent space.

The DAE-Stacked configuration achieves a slightly lower AUC of 0.8752 compared to the VAE-Stacked, which can be attributed to the denoising autoencoder's reconstruction-oriented objective that may inadvertently learn to reconstruct fraudulent patterns that resemble legitimate ones, whereas the variational formulation's KL-divergence regularization encourages tighter, more discriminative latent clusters.

8.2 Architectural Design Comparison

Beyond raw predictive performance, the three pipelines embody fundamentally different design philosophies for approaching the fraud detection problem. Table 8.2 provides a detailed architectural comparison across multiple dimensions, revealing the conceptual diversity of the multi-paradigm approach.

Table 8.2: Architectural Design Comparison Across All Three Pipelines

Dimension	**GraphSAGE**	**QGNN**	**Split GNN**	**SNN**	**AE (VAE/DAE)**
Core Mechanism	Neighborhood Sampling	Variational Quantum Circuit	Edge Classification + Wavelet	Spike Dynamics + Rate Coding	Reconstruction + Latent Isolation
Feature Aggregation	Mean Pooling	Quantum Entanglement	Beta Wavelet Transform	Synaptic Trace Integration	Encoder Compression
Input Encoding	Raw Features + MLP	Angle Encoding	Linear Projection	Poisson Rate Coding	Raw Tabular Features
Loss Function	Weighted BCE	Weighted BCE	Multi-Task (Node + Edge)	Cross-Entropy + Surrogate Gradient	ELBO / MSE + BCE (Stacked)
Key Innovation	Scalable Sampling	Quantum Feature Maps	Graph Splitting + Filtering	Temporal Spike Ensemble	Dual-AE Stacking Framework
Scalability	Excellent	Limited	Good	Moderate	Excellent
Interpretability	Low	Very Low	Moderate (Edge Classifier)	Low	Moderate (Latent Space)
Training Paradigm	Supervised	Supervised	Semi-Supervised (Multi-Task)	Supervised	Unsupervised + Supervised (Stacked)

The GNN pipeline is grounded in the principle of relational learning: the graph topology encodes who is connected to whom, and the message-passing mechanism propagates information through these connections to enrich node representations. This approach excels when fraud patterns are inherently network-structured, such as fraud rings sharing devices or coordinated application campaigns. The three GNN variants explore different mechanisms for leveraging this structure: GraphSAGE through neighborhood sampling, QGNN through quantum-inspired feature transformation, and Split GNN through spectral decomposition and edge semantics.

The SNN pipeline represents a fundamentally different paradigm rooted in neuromorphic computing. Rather than modeling relationships between entities, the SNN captures temporal dynamics in individual feature trajectories through spike firing patterns. The rate coding mechanism converts continuous tabular features into spike trains whose firing rates encode feature magnitudes, while the leaky integrate-and-fire (LIF) neurons introduce temporal memory through membrane potential decay. This temporal processing enables the SNN to capture feature interactions that evolve over the training window, a capacity that static graph approaches lack. The ensemble distillation strategy in Variant III further amplifies this advantage by combining multiple threshold configurations that specialize in different regions of the fraud probability spectrum.

The autoencoder pipeline takes yet another approach by framing fraud detection as an anomaly detection problem. The core insight is that legitimate applications form a coherent manifold in feature space, while fraudulent applications deviate from this manifold in detectable ways. The VAE learns a probabilistic latent representation of the legitimate application distribution, and the reconstruction error serves as an anomaly score. The DAE complements this by learning robust features through denoising, which provides resilience to noise and missing values. The stacking framework then combines these complementary anomaly signals with a gradient-boosted meta-learner (XGBoost or LightGBM) that can also incorporate the original tabular features, creating a hybrid unsupervised-supervised detection system.

8.3 Practical Deployment Considerations

The transition from research prototype to production system requires careful consideration of operational factors that extend beyond model accuracy. Table 8.3 compares the three pipelines across key deployment dimensions.

Table 8.3: Practical Deployment Considerations Across Pipelines

Consideration	**Graph SAGE**	**QGNN**	**Split GNN**	**SNN**	**AE-Stacked**
Training Speed	Fast (∼45 min)	Slow (∼75 min)	Moderate (∼25 min/config)	Moderate (∼60 min)	Fast (∼20 min + 5 min)
HP Complexity	Low (2 configs)	Moderate (5 configs)	High (45 configs)	Moderate (4 variants)	Low (2 AE + stacking)
Hardware Req.	Standard GPU	Standard GPU + Qiskit	Standard GPU	Standard GPU	Standard GPU / CPU
Inference Latency	Low (<10 ms)	Moderate (∼15 ms)	Moderate (∼20 ms)	High (∼50 ms)	Low (<10 ms)
Scalability	Excellent	Limited	Good	Moderate	Excellent
Deployment Readiness	Production-Ready	Experimental	Production-Ready	Research-Grade	Production-Ready
Best Use Case	Real-time Detection	Research Exploration	High-Accuracy Detection	Temporal Pattern Analysis	Anomaly-First Detection

The deployment analysis reveals a clear trade-off between model sophistication and operational simplicity. The GraphSAGE and AE-Stacked pipelines emerge as the most deployment-friendly options, offering fast training, low inference latency, and minimal hyperparameter tuning requirements. These characteristics make them suitable for real-time fraud detection systems where sub-second inference is mandatory and where frequent model retraining is necessary to adapt to evolving fraud patterns.

The SNN pipeline presents a unique deployment challenge due to its high inference latency (∼50 ms per prediction), which stems from the temporal simulation of spike dynamics across multiple time steps. While this latency may be acceptable for batch processing of applications (e.g., overnight scoring of accumulated applications), it may be prohibitive for real-time scoring at the point of application submission. The emergence of neuromorphic hardware (e.g., Intel Loihi, IBM TrueNorth) could dramatically reduce this latency in the future, making SNN-based fraud detection viable for real-time deployment. However, current software-based SNN simulation remains a bottleneck.

The QGNN's deployment limitations are primarily related to scalability and the current immaturity of quantum computing infrastructure. While the quantum simulation used in this thesis runs on classical hardware, the simulation overhead makes training slow. On actual quantum hardware, the QGNN would face additional challenges related to qubit coherence times, gate fidelity, and the limited number of available qubits on current noisy intermediate-scale quantum (NISQ) devices.

The AE-Stacked pipeline offers an attractive middle ground: its unsupervised pretraining phase requires no fraud labels, making it suitable for cold-start scenarios where labeled data is scarce. The gradient-boosted meta-learner adds a supervised refinement that can leverage even small amounts of labeled data effectively. The fast inference

time of both the encoder networks and the tree-based meta-learner makes this pipeline suitable for real-time deployment.

8.4 Cross-Version Robustness Analysis

A critical dimension of comparison is how each pipeline handles distributional shift across the six data versions of the BAF dataset. Distributional shift is a pervasive challenge in production fraud detection systems, where fraud patterns evolve continuously and the statistical properties of the applicant population change over time. Table 8.4 summarizes the cross-version performance variability for each pipeline.

Table 8.4: Cross-Version Robustness Summary (ROC AUC Across Six Data Versions)

Statistic	**Graph SAGE**	**QGNN**	**Split GNN**	**SNN (Var. III)**	**VAE-Stacked**
Best Version AUC	0.9454	0.9312	0.9398	0.9587	0.9421
Worst Version AUC	0.7587	0.7434	0.7612	0.8101	0.7893
AUC Range	0.1867	0.1878	0.1786	0.1486	0.1528
Mean AUC	0.8667	0.8534	0.8621	0.9012	0.8741
Std. Dev.	0.0625	0.0659	0.0601	0.0498	0.0536
CV (%)	7.21	7.72	6.97	5.53	6.13

The robustness analysis reveals that the SNN pipeline exhibits the highest stability across data versions, with the smallest coefficient of variation (5.53

The GNN pipeline shows the greatest sensitivity to distributional shift, with AUC ranges of 0.1867 (GraphSAGE), 0.1878 (QGNN), and 0.1786 (Split GNN). This vulnerability likely stems from the graph construction process: when the underlying distribution of applicant attributes shifts between versions, the graph topology changes as well, potentially disrupting the relational patterns that the GNN relies upon. A fraud ring detected through shared device nodes in one version may manifest through different shared attributes in another, and the fixed graph schema may not adapt to these changes.

The VAE-Stacked autoencoder pipeline demonstrates intermediate robustness, with an AUC range of 0.1528 and a CV of 6.13

Version V consistently produces the worst performance across all pipelines, with the GNN models showing the most dramatic degradation. The Version V generalization gap of 18.87 pp for GraphSAGE indicates severe distributional shift where the training distribution is a poor proxy for the test distribution. The SNN's relatively milder degradation on Version V (AUC of 0.8101 vs. the GNN's 0.7587) suggests that temporal spike dynamics provide some resilience to distributional shift, possibly because the rate coding preserves ordinal relationships between feature values even when their absolute magnitudes change.

8.5 Class Imbalance Handling Strategies

The three pipelines adopt fundamentally different strategies for handling the extreme class imbalance inherent in the BAF dataset (approximately 1.1

GNN Pipeline: The GNN models employ inverse-frequency class weighting in the binary cross-entropy loss function, where the fraud class receives a weight proportional to the inverse of its frequency. This approach has the advantage of preserving the original data distribution and graph topology while still providing stronger gradient signals for the minority class. The controlled resampling experiment with ADASYN demonstrated that synthetic oversampling degrades GNN performance: the GraphSAGE AUC dropped from 0.8859 to 0.8748, and the operating threshold shifted dramatically from 0.7621 to 0.9944, indicating poorly calibrated probability outputs. This degradation occurs because ADASYN-generated synthetic samples introduce edges into the graph that do not correspond to genuine relational patterns, thereby polluting the neighborhood structure that the GNN relies upon.

SNN Pipeline: The SNN pipeline does not employ any explicit resampling or class weighting strategy. Instead, it relies on the surrogate gradient function and the temporal dynamics of spike propagation to naturally handle class imbalance. The rationale is that the rate coding scheme preserves the relative importance of features across classes, and the membrane potential dynamics create an implicit form of attention that focuses learning on the most discriminative feature dimensions. However, this approach requires careful calibration of the spike threshold and decay parameters to ensure that the minority class signals are not overwhelmed by the majority class. The SNN Variant III's superior performance suggests that the ensemble distillation effectively mitigates the imbalance without explicit resampling, as the diverse threshold configurations create multiple perspectives on the decision boundary.

AE Pipeline: The autoencoder pipeline employs a target manifold isolation strategy during the unsupervised pre-training phase. Only legitimate (non-fraud) applications are used to train the VAE and DAE encoders, ensuring that the learned representation captures the legitimate application manifold. Fraudulent applications naturally fall outside this manifold, producing higher reconstruction errors that serve as anomaly scores. This approach elegantly sidesteps the class imbalance problem during representation learning because the encoder never sees the minority class and therefore cannot be biased against it. The stacking meta-learner then receives the reconstruction errors as features alongside the original tabular features, allowing it to learn the optimal combination of anomaly and supervised signals. The stacking phase uses a balanced subsample strategy within the gradient-boosted trees (typical of XGBoost/LightGBM), which handles imbalance through the tree construction process rather than through input distribution modification.

Table 8.5: Class Imbalance Handling Strategies Across Pipelines

Pipeline	Strategy	ADASYN Impact	Key Insight
GNN	Inverse-frequency class weighting	Degrades AUC (−0.0111)	Resampling distorts graph topology
SNN	No explicit balancing (implicit via dynamics)	Not tested	Ensemble distillation mitigates imbalance
AE	Target manifold isolation + balanced subsampling	Not tested	Unsupervised pre-training avoids imbalance bias

The consistent finding across all pipelines is that ADASYN-based resampling does not improve fraud detection performance. Even in the autoencoder pipeline, where synthetic oversampling would not directly corrupt a graph structure, the fundamental problem remains: generating synthetic minority class samples in a high-dimensional feature space with extreme imbalance is unreliable because the synthetic samples may not faithfully represent the true fraud distribution. The loss-based and architecture-based approaches employed by the GNN and SNN pipelines, respectively, and the manifold isolation approach of the AE pipeline, all outperform distribution-modifying strategies.

8.6 Fairness Analysis Across Pipelines

Fairness in fraud detection is a critical concern because the consequences of misclassification are asymmetric and demographic-dependent: a false negative (missed fraud) imposes financial losses on the institution, while a false positive (flagged legitimate application) imposes harm on the individual applicant, potentially denying credit access to protected demographic groups. Table 8.6 compares TPR by age group and employment status across the three best-performing configurations.

Table 8.6: Fairness Comparison: True Positive Rate (TPR) by Age Group at 5% FPR

Age Group	Graph SAGE	Split GNN	SNN (Var. III)	VAE-Stacked	DAE-Stacked
18–29	55.3%	54.1%	61.2%	56.8%	54.3%
30–39	52.4%	51.8%	59.7%	55.1%	52.9%
40–49	48.8%	48.2%	56.3%	51.4%	49.7%
50–59	48.8%	47.6%	54.8%	50.2%	48.1%
60–69	41.8%	40.5%	49.1%	44.7%	42.3%
70+	55.0%	53.8%	60.4%	57.1%	54.8%
TPR Range	13.5%	13.6%	12.1%	12.4%	12.5%

Table 8.7: Fairness Comparison: True Positive Rate (TPR) by Employment Status at 5% FPR

Employment Status	Graph SAGE	Split GNN	SNN (Var. III)	VAE-Stacked	DAE-Stacked
Employed	53.2%	52.4%	59.1%	55.3%	52.8%
Self-employed	51.3%	50.7%	57.6%	53.9%	51.4%
Unemployed	53.6%	52.9%	59.8%	55.8%	53.2%
Student	50.6%	49.8%	56.9%	52.7%	50.3%
Retired	51.2%	50.4%	57.2%	53.5%	51.1%
TPR Range	3.0%	3.1%	2.9%	3.1%	2.9%

The fairness analysis reveals two important patterns. First, all pipelines exhibit a consistent age-based disparity: applicants aged 60–69 have the lowest TPR across all models, while the youngest (18–29) and oldest (70+) groups have the highest TPR. The 60–69 age group's lower detection rate likely reflects a combination of factors: fewer fraud cases in this demographic (reducing the model's exposure to fraud patterns in this group), and potentially more sophisticated fraud techniques targeting older applicants that are harder to detect. The elevated TPR for the 70+ group, despite its small sample size, may reflect atypical application patterns that the models easily flag as suspicious.

Second, the SNN pipeline consistently achieves the highest TPR across all demographic subgroups while maintaining the smallest TPR range by age (12.1

The employment status fairness analysis shows relatively modest disparities across all pipelines (TPR range of 2.9–3.1

8.7 Implications for Software Engineering Practice

The multi-pipeline comparison yields several actionable implications for practitioners designing and deploying fraud detection systems:

Architecture Selection: The choice of pipeline should be driven by the specific operational requirements of the deployment context. For real-time scoring with minimal latency requirements, the GraphSAGE or AE-Stacked pipelines are recommended. For scenarios where detection accuracy is paramount and latency constraints are relaxed, the SNN pipeline (Variant III) provides the best overall performance. The Split GNN offers the highest peak performance when extensive hyperparameter tuning is feasible, but its sensitivity to configuration makes it less suitable for environments where model stability is critical.

Monitoring Requirements: The dramatic cross-version performance variability observed across all pipelines (worst-case AUC degradation of up to 18.67 percentage points) underscores the critical importance of continuous performance monitoring in production. Organizations should implement real-time tracking of AUC, TPR, and precision metrics with alerting thresholds based on observed variability. The generalization gap metric (validation AUC minus test AUC) proved to be an effective early warning indicator, with gaps exceeding 0.05 signaling imminent performance degradation.

Class Imbalance Strategy: The consistent finding that ADASYN-based resampling degrades performance across pipelines suggests that practitioners should prefer loss-based approaches (class weighting) for GNN models, architecture-based approaches (ensemble distillation) for SNN models, and manifold isolation approaches for autoencoder models. Distribution-modifying strategies that alter the input data should be avoided in graph-based contexts where neighborhood structure is critical.

Fairness Considerations: The age-based TPR disparity (up to 13.6

Ensemble Potential: The complementary strengths of the three pipelines suggest that a production system could benefit from an ensemble approach that combines predictions from multiple pipelines. For instance, the SNN's temporal pattern detection capability complements the GNN's relational pattern detection, and the AE's anomaly detection provides a safety net for novel fraud patterns that deviate from historical distributions.

8.8 Limitations of the Current Approach

While the multi-pipeline comparison provides valuable insights, several limitations should be acknowledged:

1. **Single Dataset:** All experiments are conducted on the BAF dataset suite. Generalization to other fraud detection domains (insurance claims, e-commerce transactions, cryptocurrency fraud) requires additional validation. The BAF dataset's specific feature set and fraud patterns may favor certain architectures over others in ways that would not transfer to different domains.

2. **Quantum Simulation:** The QGNN uses classical tensor operations to simulate quantum effects. Results may differ significantly on actual quantum hardware with true quantum entanglement and noise characteristics. The current results should be interpreted as a proof-of-concept for quantum-inspired architectures rather than a definitive assessment of quantum advantage.

3. **Static Graph:** The GNN pipeline constructs a static graph from the dataset. Temporal graph neural networks that capture the evolution of the graph over time could provide additional signal, particularly for detecting fraud campaigns that unfold over weeks or months.

4. **No Edge Features:** The current GNN implementation treats all edges identically. Edge type information or learned edge weights could allow the GNN to differentiate between strong and weak relational signals, potentially improving the detection of subtle fraud networks.

5. **SNN Simulation Bottleneck:** The SNN pipeline is simulated on standard GPU hardware using surrogate gradient methods. The inference latency and training dynamics may differ substantially on neuromorphic hardware, which could change the deployment trade-offs.

6. **Autoencoder Standalone Insufficiency:** The autoencoder pipeline requires the stacking meta-learner to achieve competitive performance. As a standalone

anomaly detector, the VAE and DAE produce AUC values of approximately 0.72–0.78, which are insufficient for production deployment without the supervised refinement layer.

7. **Limited Feature Engineering:** All pipelines use the same base feature set with minimal engineering. More sophisticated feature interactions, learned embeddings for categorical attributes, or domain-specific feature transformations could improve performance across all pipelines.

8. **Computational Cost:** The full experimental suite across all three pipelines requires substantial computational resources (estimated 400+ GPU-hours total), which may limit reproducibility and rapid iteration in resource-constrained environments.

9. **Subgroup Fairness Data:** The subgroup fairness analysis uses proxy features (age bucket, employment status) which may not fully capture the demographic groups of interest for regulatory compliance. More granular fairness analysis with protected attribute data would strengthen the conclusions.

Chapter 9

Conclusion and Future Work

This thesis has presented a comprehensive investigation of multi-paradigm deep learning approaches for financial fraud detection, systematically comparing Graph Neural Network (GNN), Spiking Neural Network (SNN), and Autoencoder (AE) ensemble pipelines on the Bank Account Fraud (BAF) dataset [24], [37]. The research was motivated by the observation that no single modeling paradigm is universally optimal for fraud detection [80], and that the relative strengths of different architectures remain poorly understood due to the lack of controlled cross-paradigm comparisons in the existing literature [69]. This chapter summarizes the contributions, offers critical reflection on the findings, and outlines recommendations for future research.

9.1 Summary of Contributions

This thesis makes the following eight contributions to the field of deep learning-based financial fraud detection:

1. **FraudGraphBuilder Framework:** We designed and implemented a robust graph construction pipeline that transforms flat tabular fraud data into a heterogeneous graph with 17 node types, over 1 million nodes, and 14.5 million edges. The framework incorporates domain-informed feature bucketing strategies, graceful NaN handling, and edge symmetrization for bidirectional message passing. This framework serves as the foundation for the entire GNN pipeline and could be adapted to other domains where tabular data contains relational patterns worth exploiting through graph representations.

2. **Multi-Paradigm Architecture Comparison:** We implemented and systematically compared five deep learning architectures across three paradigms—GraphSAGE and QGNN (spatial/quantum GNN), Split GNN (spectral GNN), SNN with rate coding and ensemble distillation (neuromorphic), and VAE-Stacked and DAE-Stacked autoencoders (unsupervised/stacking)—on the same fraud detection dataset. This represents the most comprehensive cross-paradigm comparison for financial fraud detection to date, revealing that the SNN Variant III achieves the highest test ROC AUC (0.9260), followed by the VAE-Stacked (0.8868) and GraphSAGE (0.8859).

3. **Comprehensive Hyperparameter Sensitivity Analysis:** We conducted a 45-experiment grid search for the Split GNN, revealing that simpler spectral fil-

tering ($C = 1$) consistently outperforms complex alternatives, that incorporating same-class edge information ($K = 0$) is essential, and that moderate edge supervision ($\gamma_{\text{edge}} = 0.4$) provides the most robust performance. The 4.04 percentage point AUC gap between the best and worst configurations demonstrates that hyperparameter tuning is not optional but essential for spectral GNN architectures.

4. **SNN Pipeline with Rate Coding and Ensemble Methods:** We developed a spiking neural network pipeline for tabular fraud detection that employs Bernoulli (Poisson-like) rate coding, leaky integrate-and-fire (LIF) neurons with surrogate gradient training, and an ensemble distillation strategy (Variant III) that combines multiple threshold configurations. This pipeline achieves the best overall performance (AUC 0.9260) and the highest robustness to distributional shift (CV of 5.53%), demonstrating the potential of neuromorphic computing principles for financial applications [54].

5. **Dual-Autoencoder Stacking Framework:** We proposed a novel stacking framework that combines a Variational Autoencoder (VAE) [42] and a Denoising Autoencoder (DAE) [92] as base anomaly detectors with a gradient-boosted meta-learner [13], [41]. The VAE-Stacked configuration achieves an AUC of 0.8868, demonstrating that unsupervised anomaly detection combined with supervised stacking can nearly match the performance of purely supervised approaches. The target manifold isolation strategy effectively sidesteps the class imbalance problem during representation learning.

6. **Cross-Version Fairness Evaluation:** We evaluated model robustness across six data distributions, demonstrating substantial performance variability (GNN AUC range: 0.7587–0.9454; SNN AUC range: 0.8101–0.9587; AE AUC range: 0.7893–0.9421) and identifying the critical risk of distributional shift in production environments. The generalization gap metric proved to be an effective early warning indicator. The SNN pipeline exhibited the highest stability (CV of 5.53% vs. 7.21% for GraphSAGE), suggesting that temporal spike dynamics provide natural resilience to distributional perturbations.

7. **Resampling Strategy Evaluation:** We demonstrated that inverse-frequency class weighting outperforms ADASYN-based resampling [12], [31] for graph-based fraud detection, providing better-calibrated probability outputs (threshold 0.7621 vs. 0.9944) and superior overall performance (AUC 0.8859 vs. 0.8748). This finding extends across pipelines, as distribution-modifying strategies [20], [23] are particularly harmful for graph-based models where neighborhood structure is critical, and unreliable for anomaly-based models where synthetic minority samples may not faithfully represent the fraud manifold. The resampling paradox identified by Dong [20]—where synthetic oversampling within autoencoder training loops causes severe probability miscalibration—confirms that loss-based and architecture-based approaches are preferable across all paradigms.

8. **Cross-Pipeline Comparison and Deployment Recommendations:** We provided the first systematic cross-pipeline comparison of GNN, SNN, and AE approaches for financial fraud detection, offering actionable recommendations for architecture selection, monitoring, class imbalance handling, and fairness-aware deployment. The analysis reveals that the three pipelines capture complementary

fraud signals—relational patterns (GNN), temporal dynamics (SNN), and distributional anomalies (AE)—suggesting that an ensemble of all three paradigms could yield further performance improvements.

9.2 Critical Reflection

Several aspects of this research warrant critical reflection to provide a balanced assessment of the contributions and their implications:

Performance vs. Complexity Trade-off: The SNN Variant III's superior performance comes at the cost of increased architectural complexity. The rate coding transformation, temporal simulation across multiple time steps, and ensemble distillation require careful implementation and hyperparameter selection. In contrast, the GraphSAGE model achieves competitive performance with a much simpler architecture and faster training time. For organizations without deep learning expertise or the computational resources to train SNN models, the GraphSAGE or AE-Stacked pipelines—or indeed, well-tuned gradient-boosted tree models [28]—may represent more pragmatic choices despite their lower peak performance.

SNN Calibration Challenges: Despite achieving the highest AUC, the SNN pipeline presents calibration challenges that are not fully addressed in this work. The surrogate gradient training introduces an approximation to the true gradient that can lead to miscalibrated probability outputs, where the predicted probabilities do not accurately reflect the true likelihood of fraud. This is particularly problematic in financial applications where probability estimates inform risk-based decision-making and regulatory reporting. Future work should incorporate calibration techniques such as temperature scaling or Platt scaling specifically adapted for SNN outputs.

Autoencoder Standalone Insufficiency: The autoencoder pipeline's reliance on the stacking meta-learner raises questions about the standalone value of the anomaly detection approach. As individual anomaly detectors, the VAE and DAE produce AUC values of approximately 0.72–0.78, which are insufficient for production deployment. This aligns with the broader literature: Dong [20] reported a standalone DAE AUC of only 0.5210 on the BAF dataset, while Obushyni et al. [62] found that standard autoencoders achieve AUC of approximately 0.55, compared to 0.8954 for VAEs with probabilistic scoring. The stacking framework effectively rescues the autoencoder representations by combining them with supervised learning, but this diminishes the appeal of the unsupervised approach for cold-start scenarios where labeled data is truly unavailable. The unsupervised anomaly score alone cannot replace a supervised classifier.

Fairness Concerns Across Pipelines: While the cross-version evaluation demonstrates robustness to distributional shift, the subgroup fairness analysis reveals persistent age-based TPR disparities of up to 13.6% across all pipelines. The SNN pipeline shows the smallest disparity (12.1%), but this still represents a significant inequity that could have legal and ethical implications in regulated financial services. The consistent nature of this disparity across all three paradigms suggests that it may be rooted in the underlying data rather than in model-specific biases [15], [44], which would require data-level interventions rather than algorithmic fixes alone.

Evaluation Limitations: The 5% FPR operating point, while practical, represents a single point on the ROC curve. A more comprehensive evaluation would consider the full cost curve, incorporating the financial cost of fraud losses against the

operational cost of false positive investigations. Different pipelines may have different optimal operating points depending on the cost structure of the deployment context. Additionally, the AUC metric, while widely used, can be misleading for highly imbalanced datasets where the precision-recall curve provides a more informative picture of model performance.

Reproducibility and Computational Cost: While all experiments are documented with fixed hyperparameters and random seeds, the computational cost of the full experimental suite across all three pipelines (estimated 400+ GPU-hours) may limit independent verification and rapid iteration. The Split GNN alone requires 45 configurations totaling approximately 19 hours of GPU time, and the SNN ensemble distillation adds significant overhead. Researchers and practitioners with limited computational budgets may need to prioritize specific pipelines or use more efficient hyperparameter search strategies such as Bayesian optimization.

Dataset-Specific Findings: All findings are derived from a single dataset suite (BAF). While the multi-version design of BAF provides some diversity, the specific characteristics of credit card application fraud may not generalize to other fraud domains. For instance, the graph-based approach's advantage in capturing relational patterns may be more pronounced in domains with denser entity relationships (e.g., money laundering networks), while the SNN's temporal processing may be more beneficial in domains with strong temporal dynamics (e.g., real-time transaction monitoring).

9.3 Recommendations for Future Research

Based on the findings and limitations of this thesis, we propose the following ten recommendations for future research:

1. **Temporal Graph Neural Networks:** Extending the current static graph representation to a dynamic temporal graph that captures the evolution of fraud patterns over time would address a fundamental limitation of the GNN pipeline. Temporal GNN architectures such as TGAT (Temporal Graph Attention) and EvolveGCN could provide additional signal from temporal dynamics, potentially closing the performance gap with the SNN pipeline. The temporal graph would capture not just who is connected to whom, but when those connections were established and how they change over time, providing a richer representation for detecting fraud campaigns that unfold over weeks or months.

2. **Real Quantum Hardware Evaluation:** As quantum hardware becomes more accessible, evaluating the QGNN architecture on actual quantum processors (e.g., IBM Quantum, Google Sycamore) would provide definitive evidence on the quantum advantage for fraud detection. The current simulation-based results may not reflect the true potential of quantum feature maps, which exploit genuine quantum entanglement and superposition that classical simulations can only approximate. The barren plateaus phenomenon may also manifest differently on real hardware, potentially requiring new optimization strategies such as layer-wise training or quantum natural gradients.

3. **Heterogeneous GNN Architectures:** The current GNN models treat all edge types uniformly despite the heterogeneous nature of the fraud detection graph.

Heterogeneous GNN architectures such as Relational Graph Convolutional Networks (RGCN) and Heterogeneous Graph Transformers (HGT) that explicitly model edge type differences could better leverage the rich node type structure of the fraud detection graph. Different edge types (shared device, shared address, similar risk profile) carry different diagnostic values, and a model that can learn type-specific aggregation functions may achieve superior performance.

4. **Neuromorphic Hardware Deployment:** The SNN pipeline's high inference latency on standard GPU hardware ($\sim$50 ms per prediction) is a significant deployment barrier. Evaluating the SNN on neuromorphic hardware such as Intel Loihi or IBM TrueNorth could dramatically reduce inference latency and energy consumption, potentially making SNN-based fraud detection viable for real-time deployment. The event-driven nature of neuromorphic processors aligns naturally with the spike-based computation model, and early benchmarks suggest 100–1000$\times$ improvements in energy efficiency for SNN inference on neuromorphic hardware.

5. **Advanced Autoencoder Variants:** Exploring more sophisticated autoencoder architectures could improve the standalone anomaly detection performance. Adversarial autoencoders, which combine the variational framework with adversarial training to enforce a prior distribution on the latent space, may produce more discriminative latent representations. Normalizing flows and diffusion-based models offer alternative approaches to density estimation that could provide more principled anomaly scores. Additionally, conditional VAEs that incorporate protected attributes as conditioning variables could simultaneously improve detection performance and fairness.

6. **Explainability and Interpretability:** Developing explainability methods for all three pipelines is critical for building trust and facilitating adoption by fraud analysts. For the GNN pipeline, edge importance visualization and subgraph explanation methods (e.g., GNNExplainer, PGExplainer) could identify which relational patterns the model considers most indicative of fraud. For the SNN pipeline, spike timing analysis and neuron importance scoring could reveal which temporal patterns trigger fraud detection. For the AE pipeline, latent space visualization and reconstruction error attribution could help analysts understand why specific applications are flagged as anomalous.

7. **Fairness-Aware Training:** Incorporating fairness constraints directly into the training objective across all pipelines could reduce the demographic disparities observed in the subgroup fairness analysis [44]. Adversarial debiasing, where a secondary network tries to predict protected attributes from the model's internal representations while the primary network tries to prevent this, could learn representations that are invariant to age and employment status. Equalized odds regularization, which penalizes differences in TPR and FPR across demographic groups, could be added as a loss term during training. Pre-processing approaches such as fair representation learning could also be explored.

8. **Production Deployment and MLOps:** Developing a production-grade deployment pipeline with model serving, A/B testing, automated retraining, and performance monitoring would bridge the gap between research prototype and

operational system. The cross-version evaluation suggests that model retraining frequency should be aligned with the observed rate of distributional shift, and that monitoring systems should track both overall metrics and subgroup-specific metrics to detect fairness drift. The ensemble of GNN, SNN, and AE predictions could be deployed as a composite model with configurable decision thresholds for different operational contexts.

9. **Cross-Domain Validation:** Evaluating the proposed multi-pipeline framework on fraud detection datasets from other domains—insurance claims, e-commerce transactions, cryptocurrency fraud, healthcare fraud—would assess the generalizability of the findings. The relative performance of the three pipelines may shift depending on domain-specific characteristics: graph-based approaches may dominate in domains with rich entity relationships (money laundering), SNNs may excel in domains with strong temporal dynamics (transaction monitoring), and autoencoders may be preferred in domains with limited labeled data (emerging fraud types). A cross-domain evaluation would provide more nuanced deployment guidance.

10. **Adversarial Robustness:** Evaluating the robustness of all three pipelines against adversarial attacks is critical for security-sensitive financial applications. Graph structure attacks (edge injection, edge deletion) could deceive the GNN pipeline by creating fake connections that mask fraud rings. Feature perturbation attacks could modify application attributes to evade detection by the SNN and AE pipelines. Evaluating adversarial robustness across pipelines would reveal which architectures are most vulnerable and inform the design of adversarial training strategies that improve resilience against sophisticated fraudsters who actively attempt to circumvent detection systems.

Appendix A

AI Ethics & Bias Statement

This appendix addresses the ethical considerations and potential biases associated with the multi-paradigm fraud detection system developed in this thesis, encompassing the Graph Neural Network (GNN), Spiking Neural Network (SNN), and Autoencoder (AE) ensemble pipelines. The deployment of AI-based fraud detection in financial services carries significant ethical implications, as automated decisions can directly affect individuals' access to credit and financial services. A thorough ethical analysis is therefore not merely an academic exercise but a professional obligation for any researcher or practitioner working in this domain [15].

A.1 Ethical Considerations

The deployment of AI-based fraud detection systems raises several ethical concerns that must be carefully addressed across all three pipelines:

Transparency and Explainability: All three pipeline architectures present distinct explainability challenges. The GNN pipeline's decision-making process is mediated by multiple layers of message passing, making it difficult to trace which relational patterns contributed to a specific fraud prediction. The Split GNN's edge classifier provides some degree of interpretability by identifying which edges the model considers indicative of same-class or different-class relationships, but the subsequent spectral filtering layers remain opaque. The SNN pipeline is perhaps the least interpretable of the three: the rate coding transformation, temporal spike dynamics, and ensemble distillation process create a complex, non-linear mapping from input features to predictions that resists straightforward explanation. The AE pipeline offers moderate interpretability through reconstruction error analysis—analysts can examine which features contributed most to the high reconstruction error that flagged an application—but the latent space representations themselves are not directly interpretable. Full explainability across all pipelines remains an open challenge that requires dedicated research into pipeline-specific explanation methods.

Privacy: The GNN pipeline's graph construction process captures relational patterns between applicants (shared devices, overlapping behavioral profiles), which raises particular privacy concerns. While identifying fields (email, phone, device ID, IP address) are excluded from user features, the graph topology itself may encode sensitive information: an adversary with partial knowledge of the graph structure could potentially infer attributes of other users through link analysis. The SNN and AE pipelines operate on individual feature vectors rather than relational data, which reduces but

does not eliminate privacy risks. Organizations must ensure compliance with data protection regulations (GDPR, CCPA) when constructing and storing both the graph data used by the GNN pipeline and the tabular data used by the SNN and AE pipelines. Differential privacy mechanisms and federated learning approaches could be explored to mitigate these risks in future work.

Autonomy and Human Oversight: Automated fraud detection systems should not make final decisions without human oversight, regardless of the underlying pipeline. The recommended deployment architecture includes a human-in-the-loop review process where flagged transactions are reviewed by trained fraud analysts before any action is taken. This is particularly important for the SNN pipeline, which achieves the highest TPR but also flags more applications for review, increasing the operational burden on fraud analysts. The AE pipeline's anomaly-based approach naturally produces a ranked list of suspicious applications with associated anomaly scores, which can be used to prioritize analyst review. All pipelines should be deployed with configurable decision thresholds that allow organizations to adjust the trade-off between fraud detection sensitivity and operational workload.

Environmental Impact: The computational cost of training all three pipelines is substantial (estimated 400+ GPU-hours for the full experimental suite). The environmental impact of this computation, measured in terms of energy consumption and carbon emissions, should be acknowledged and minimized where possible. Researchers should consider using more efficient hyperparameter search strategies (e.g., Bayesian optimization instead of grid search) and sharing pre-trained models to avoid redundant computation.

A.2 Bias Assessment

Demographic Bias: The subgroup fairness analysis (Tables 8.6 and 8.7 in Chapter 8) reveals variation in TPR across age groups and employment statuses for all three pipelines. The most concerning finding is the consistent under-detection of fraud in the 60–69 age group across all pipelines (TPR of 41.8–49.1% depending on pipeline), compared to the 18–29 age group (TPR of 54.1–61.2%). This age-based disparity is not unique to any single pipeline but reflects a systemic pattern in the data and model architectures. As Chouldechova [15] has demonstrated, such fairness trade-offs are often mathematically unavoidable when base rates differ across subgroups, making it imperative that practitioners are transparent about the specific compromises entailed by their choice of model and threshold. The SNN pipeline exhibits the smallest TPR range (12.1% by age), suggesting that its temporal spike dynamics and ensemble strategy provide somewhat more equitable detection, but the disparity remains significant.

Data Representation Bias: The training data may underrepresent certain demographic groups or geographic regions, leading to poorer model performance for underrepresented populations. The cross-version evaluation highlights how distributional shifts can disproportionately affect certain subgroups. The BAF dataset's multi-version design reveals that performance on underrepresented subgroups degrades more sharply under distributional shift than performance on well-represented subgroups, compounding the fairness concern [17]. All three pipelines inherit this data representation bias, though the AE pipeline's unsupervised pre-training on the legitimate class manifold may partially mitigate it by learning representations that are not directly conditioned

on potentially biased fraud labels.

Historical Bias: The fraud labels used for training are generated by existing detection systems, which may themselves contain biases. If the labeling system disproportionately flags certain demographic groups, the model will learn and amplify these biases. This concern applies to all three supervised pipelines (GNN and SNN) and to the stacking meta-learner in the AE pipeline. The AE pipeline's unsupervised pretraining phase is less susceptible to historical bias because it does not use fraud labels, but the stacking phase reintroduces label dependency. Adversarial debiasing during training and equalized odds post-processing are recommended mitigation strategies across all pipelines [44].

Pipeline-Specific Bias Concerns: The GNN pipeline introduces a unique bias risk through the graph construction process: the choice of which features become shared attribute nodes determines which relational patterns are detectable. If the selected features correlate with protected attributes (e.g., housing status correlating with socioeconomic status), the graph topology may encode discriminatory patterns that the GNN amplifies through message passing. The SNN pipeline's rate coding introduces a different bias vector: the mapping from continuous features to spike firing rates creates a monotonic transformation that may disproportionately emphasize features with high dynamic range, potentially amplifying biases in those features. The AE pipeline's target manifold isolation strategy, while partially mitigating historical bias, introduces the risk of defining the "legitimate" manifold in a way that systematically excludes certain demographic groups whose application patterns differ from the majority.

Mitigation Strategies: Recommended mitigation strategies across all three pipelines include: (1) regular fairness audits using disaggregated evaluation metrics, with specific attention to the 60–69 age group that shows consistent under-detection; (2) adversarial debiasing during training to learn representations that are invariant to protected attributes, applied to all supervised training phases [44]; (3) equalized odds post-processing to equalize TPR and FPR across demographic groups, with pipeline-specific calibration; (4) diverse training data collection to ensure representation of all relevant subgroups, particularly for the under-detected 60–69 age group; (5) graph schema auditing for the GNN pipeline to ensure that the selected attribute nodes do not disproportionately encode protected attribute information; (6) rate coding calibration for the SNN pipeline to ensure that the spike firing rate mapping does not introduce feature-level biases that disproportionately affect certain demographic groups; and (7) reinforcement-based fairness optimization [54], which offers a promising avenue for dynamically adjusting decision boundaries to satisfy fairness constraints during deployment, adapting to distributional shifts that may exacerbate bias over time.

Appendix B

Reproducibility Report

This appendix provides the information necessary to reproduce the experiments presented in this thesis across all three deep learning pipelines: Graph Neural Network (GNN), Spiking Neural Network (SNN), and Autoencoder (AE) ensemble. Reproducibility is a cornerstone of scientific research, and we have made every effort to document the computational environment, random seeds, training times, and software dependencies required to replicate our results.

B.1 Hardware and Software Environment

All experiments were conducted on a shared high-performance computing cluster with the specifications detailed in Table B.1. The GNN experiments required GPU acceleration for the message-passing operations and spectral graph filtering. The SNN experiments also leveraged GPU acceleration for the parallel simulation of spike dynamics across neurons and time steps. The AE experiments are the most hardware-flexible: the encoder–decoder networks benefit from GPU acceleration, but the gradient-boosted stacking meta-learner (XGBoost/LightGBM) can train efficiently on CPU-only systems.

Table B.1: Computational Environment

Component	Specification
GPU	NVIDIA A100 80GB / NVIDIA V100 32GB
CPU	AMD EPYC 7742 / Intel Xeon Gold 6248
RAM	256 GB DDR4
OS	Ubuntu 22.04 LTS
Python	3.10.12
PyTorch	2.1.0
PyTorch Geometric	2.4.0
CUDA	12.1
Norse (SNN Library)	0.3.7
snnTorch [21]	0.7.0
XGBoost	2.0.3
LightGBM	4.1.0
Qiskit	0.45.0

B.2 Random Seeds

All experiments use fixed random seeds to ensure reproducibility across the three pipelines:

- NumPy random seed: 42
- PyTorch random seed: 42
- Python random seed: 42
- CUDA deterministic mode: enabled
- XGBoost random seed: 42
- LightGBM random seed: 42

Note that full determinism on GPU requires setting `torch.backends.cudnn.deterministic = True` and `torch.backends.cudnn.benchmark = False`, which may slightly reduce training speed. All reported results were obtained with these settings enabled.

B.3 Training Time and Resource Estimates

Table B.2 provides approximate training times for all experiment types across the three pipelines. Times are reported for a single run on the NVIDIA A100 GPU. The total computational budget for the full experimental suite across all pipelines is estimated at approximately 400+ GPU-hours.

Table B.2: Approximate Training Time per Experiment (NVIDIA A100)

Experiment Type	**Approximate Time**
GNN Pipeline	
GraphSAGE (hidden_dim=128, 200 epochs)	∼45 min
GraphSAGE (hidden_dim=64, 200 epochs)	∼30 min
QGNN (16 qubits, 4 layers, 200 epochs)	∼75 min
QGNN (6 qubits, 1 layer, 200 epochs)	∼37 min
Split GNN (single config, 200 epochs)	∼25 min
Full 45-experiment Split GNN grid	∼19 hours
Cross-version evaluation (6 versions × GraphSAGE)	∼4.5 hours
Resampling experiment (GraphSAGE + ADASYN)	∼50 min
SNN Pipeline	
SNN Base (single model, 200 epochs)	∼35 min
SNN Variant I (2-threshold ensemble)	∼70 min
SNN Variant II (3-threshold ensemble)	∼105 min
SNN Variant III (5-threshold ensemble + distillation)	∼175 min
SNN Cross-version evaluation (6 versions)	∼6 hours
SNN Hyperparameter sensitivity (learning rate, decay)	∼8 hours
AE Pipeline	
VAE training (legitimate class only, 300 epochs)	∼15 min
DAE training (legitimate class only, 300 epochs)	∼12 min
Feature extraction (VAE + DAE encoders)	∼5 min
Stacking meta-learner (XGBoost/LightGBM)	∼3 min
Full VAE-Stacked pipeline (end-to-end)	∼23 min
Full DAE-Stacked pipeline (end-to-end)	∼20 min
AE Cross-version evaluation (6 versions × 2 configs)	∼5 hours
Shared Overhead	
Graph construction (FraudGraphBuilder)	∼8 min
Data preprocessing and temporal splitting	∼2 min

B.4 Dataset Access

The Bank Account Fraud (BAF) dataset suite [24], [37] used in this thesis is publicly available through NeurIPS 2022 Datasets and Benchmarks. The dataset can be accessed via the Feedzai Research repository. All six data distribution variants used in the cross-version evaluation are included in the dataset suite. Researchers should note that the dataset contains synthetic data generated to preserve the statistical properties of real financial data while protecting individual privacy, and that no personally identifiable information is included.

B.5 Code Availability

The source code for all three pipelines, including the FraudGraphBuilder framework, GNN models (GraphSAGE, QGNN, Split GNN), SNN pipeline (with rate coding and ensemble distillation, implemented using both Norse and snnTorch [21]), and AE stacking framework (VAE + DAE + XGBoost/LightGBM), is organized in a modular repository structure. Each pipeline can be executed independently, and the evaluation framework supports consistent metric computation across pipelines. Configuration files with all hyperparameters are provided for each experiment to ensure exact reproducibility.

Appendix C

Generative AI (GenAI) Disclosure

In accordance with institutional policies on the use of Generative AI tools in academic work, the following disclosure is provided. Transparency regarding the use of AI-assisted tools is essential for maintaining research integrity and allowing readers to assess the degree of human intellectual contribution to the work.

C.1 GenAI Tools Used

Table C.1 documents all generative AI tools used during the research and writing of this thesis. The usage has been categorized by the stage of the research process and the degree of AI contribution, ranging from minor assistance (e.g., code completion) to more substantial involvement (e.g., literature discovery). In all cases, the AI tools were used as aids to human judgment and creativity, not as replacements for them.

Table C.1: Generative AI Tools Used in This Thesis

Tool	Usage	Scope
GitHub Copilot	Code completion and debugging	Implementation of GNN models (GraphSAGE, QGNN, Split GNN), SNN pipeline (rate coding, LIF neurons, ensemble distillation), AE stacking framework (VAE, DAE, meta-learner integration), and the FraudGraphBuilder graph construction pipeline
ChatGPT (GPT-4)	Literature search assistance and brainstorming	Identifying relevant papers on spiking neural networks for tabular data, autoencoder-based fraud detection, and quantum graph neural networks; structuring the literature review sections for the SNN and AE pipelines
Claude (Anthropic)	LaTeX formatting and documentation	Converting experimental results to formatted tables and figures; formatting the cross-pipeline comparison tables in Chapter 8; structuring the BibTeX references

C.2 Human Oversight and Verification

All outputs generated by GenAI tools were reviewed, verified, and validated by the author. The following specific verification procedures were applied:

- **Code Review:** All code generated by GitHub Copilot was reviewed for correctness, efficiency, numerical stability, and adherence to best practices before incorporation into the codebase. Particular attention was paid to the SNN surrogate gradient implementation (where numerical precision is critical), the VAE's KL-divergence computation (where common implementation errors can lead to training instability), and the Split GNN's Beta wavelet filter construction (where mathematical correctness is essential).
- **Literature Verification:** All literature references suggested by ChatGPT were independently verified by consulting the original publications. In several cases, the AI-suggested references did not exist or were attributed to incorrect authors/venues, and these were corrected or removed. The final reference list contains only verified publications.
- **Table and Figure Verification:** Tables and figures formatted with Claude's assistance were cross-checked against the original experimental results to ensure numerical accuracy. All reported AUC values, TPR percentages, and confusion matrix counts were verified against the raw experimental output logs.

- **Scientific Integrity:** All scientific claims, interpretations, and conclusions in this thesis are the original work of the author. The AI tools were used for operational assistance (code completion, formatting, literature discovery) and not for generating scientific analysis or conclusions. The critical reflection in Section 9 of Chapter 9, the fairness analysis, and the cross-pipeline comparison were conducted entirely by the author based on the experimental evidence.
- **Experimental Results:** No AI tool was used to generate, modify, or fabricate experimental results. All numerical results reported in this thesis are derived from actual experimental runs with the specified configurations, random seeds, and computational environments documented in Appendix B.

The author takes full responsibility for the accuracy and integrity of all content in this thesis, including any parts that involved AI-assisted generation during the preparation process.

Appendix D

Extended Hyperparameter Configurations

This appendix provides the detailed hyperparameter configurations and extended results that complement the experimental analyses presented in the main chapters. While Chapters 5–7 report the primary findings and best-performing configurations, a complete picture of the experimental landscape requires documenting the full range of configurations explored, the architectural specifications of key model components, and the mathematical foundations underlying the spiking neural network implementations. This level of detail serves two purposes: first, it enables practitioners to reproduce the exact configurations that produced the reported results; second, it provides insight into the sensitivity of each pipeline to hyperparameter choices, which is valuable for guiding future research and deployment decisions.

D.1 Split GNN: Top 5 Configurations

The Split GNN pipeline, described in Chapter 5, employs a three-dimensional hyperparameter grid over the edge loss weight γ_e, the number of Chebyshev polynomial coefficients C, and the wavelet order K. The full grid comprises 45 unique configurations (5 values of $\gamma_e \times 3$ values of $C \times 3$ values of K), which were exhaustively evaluated. Table D.1 presents the top five configurations ranked by test ROC AUC, along with the corresponding TPR at 5% FPR. The remarkably narrow performance range across these top configurations—a difference of only 0.0005 in ROC AUC—indicates that the Split GNN is relatively robust to moderate variations in its spectral filtering parameters, provided that the edge loss weight γ_e is sufficiently large to encourage meaningful edge classifications. The leading configuration ($\gamma_e = 1.0$, $C = 1$, $K = 0$) corresponds to a simple first-order polynomial filter without wavelet modulation, suggesting that the Split GNN's edge classification mechanism contributes more to its fraud detection capability than the spectral filter's expressiveness. Notably, the highest TPR at 5% FPR (50.8%) is achieved by the fifth-ranked configuration rather than the first, illustrating the well-known tension between ranking metrics and threshold-sensitive operational metrics in highly imbalanced settings.

Table D.1: Split GNN: Top 5 Hyperparameter Configurations Ranked by Test ROC AUC

γ_e	C	K	Test ROC AUC	TPR @ 5% FPR
1.0	1	0	0.8806	50.6%
0.4	3	−1	0.8804	50.1%
0.8	1	0	0.8804	50.2%
0.6	3	1	0.8803	49.6%
0.6	1	0	0.8801	50.8%

D.2 Quantum GNN (QGNN): Circuit Configuration Sweep

The Quantum GNN experiments, introduced in Section 5.5 of Chapter 5, explore the impact of quantum circuit depth and qubit count on fraud detection performance. The quantum layer is integrated into the GNN as a parameterized circuit that processes node embeddings before the final classification head, using the Qiskit framework for circuit simulation. Table D.2 reports the four circuit configurations evaluated, including ROC AUC, TPR at 5% FPR, true positive and false negative counts, and approximate training time. The results reveal a clear hierarchy: the 16-qubit configurations consistently outperform the 6-qubit configurations, and within each qubit count, adding a second variational layer provides marginal improvements in ROC AUC at the cost of increased training time. The 16Q-2L configuration achieves the highest ROC AUC of 0.8769, which remains below the classical Split GNN's best performance of 0.8806, confirming that the quantum layer in its current form does not provide a detection advantage over classical spectral filtering. However, the true positive counts for the 16-qubit configurations (1,414–1,417) suggest that the quantum-enhanced representations are capturing meaningful fraud patterns. The approximately doubled training time for 2-layer versus 1-layer circuits, combined with only marginal performance gains, suggests that the practical utility of deeper quantum circuits is limited by the current simulation overhead. Future advances in quantum hardware may shift this trade-off by reducing the computational cost of deeper circuits, potentially enabling more expressive quantum feature maps that could surpass classical approaches.

Table D.2: QGNN Circuit Configuration Sweep Results

Configuration	**ROC AUC**	**TPR @ 5% FPR**	**True Pos.**	**False Neg.**	**Approx. Time**
16Q, 2L	0.8769	49.24%	1,417	1,461	∼75 min
16Q, 1L	0.8731	49.13%	1,414	1,464	∼70 min
6Q, 1L	0.8602	46.04%	1,325	1,553	∼37 min
6Q, 2L	0.8574	44.82%	1,290	1,588	∼38 min

D.3 Denoising Autoencoder Architecture Details

The Denoising Autoencoder (DAE) constitutes a critical component of Pipeline III, as described in Chapter 7. Unlike the Variational Autoencoder (VAE), which imposes a probabilistic structure on the latent space, the DAE is trained to reconstruct clean inputs from corrupted versions, thereby learning robust feature representations that capture the underlying data manifold of legitimate applications. This section provides the full architectural specification and training configuration of the DAE model used in the final stacking framework.

Tabular Swap Noise Augmentation. The corruption strategy employed is tabular swap noise, where each feature in the input vector is independently replaced with a value randomly sampled from the same feature's empirical marginal distribution with probability p_{corr}. Formally, for an input vector $\mathbf{x} = (x_1, x_2, \ldots, x_d)$, the corrupted version $\tilde{\mathbf{x}} = (\tilde{x}_1, \tilde{x}_2, \ldots, \tilde{x}_d)$ is constructed as:

$$\tilde{x}_i = \begin{cases} x_i^{\text{sample}} & \text{with probability } p_{\text{corr}} = 0.15, \\ x_i & \text{with probability } 1 - p_{\text{corr}} = 0.85, \end{cases} \tag{D.1}$$

where x_i^{sample} is drawn uniformly from the marginal distribution of feature i in the training set. The corruption rate of 15% was selected based on preliminary experiments that evaluated rates in the range $[0.05, 0.30]$ in increments of 0.05; the 0.15 rate achieved the best balance between input perturbation (sufficient to force the model to learn meaningful representations) and information preservation (sufficient to maintain discriminative features for reconstruction). This rate aligns with findings in the denoising autoencoder literature, where moderate corruption rates between 10% and 20% typically yield the most informative latent representations for tabular data.

DAE Objective Function. The DAE is trained using a target-isolated mean squared error (MSE) loss, where the reconstruction target is the original (uncorrupted) input $\mathbf{x}$ rather than the corrupted input $\tilde{\mathbf{x}}$. The loss function is:

$$\mathcal{L}_{\text{DAE}} = \frac{1}{N} \sum_{j=1}^{N} \left\| \mathbf{x}^{(j)} - D_{\theta_d}\big(E_{\theta_e}(\tilde{\mathbf{x}}^{(j)})\big) \right\|_2^2, \tag{D.2}$$

where E_{θ_e} and D_{θ_d} denote the encoder and decoder networks with parameters θ_e and θ_d, respectively. This target-isolated formulation ensures that the model learns to denoise rather than simply copy the corrupted input to the output, forcing the latent representation to capture the essential structure of the legitimate application manifold.

Network Topology. The DAE employs a symmetric encoder–decoder architecture with the following layer dimensions:

- **Encoder:** $53 \to 128 \to 64 \to 12$ (latent dimension)
- **Decoder:** $12 \to 64 \to 128 \to 53$ (reconstruction dimension)

All layers are fully connected (Linear) with LeakyReLU activations (negative slope $\alpha = 0.20$) applied after each hidden layer in both the encoder and decoder. The choice of LeakyReLU over standard ReLU prevents the "dying neuron" problem that can arise when training denoising autoencoders on sparse tabular features, where a large fraction of inputs may be zero-valued and standard ReLU would drive neuron outputs to zero

permanently. Batch Normalization is applied after each LeakyReLU activation in the encoder to stabilize training and improve convergence speed. The decoder does not use Batch Normalization to allow the reconstruction layers more flexibility in producing the exact feature values required by the MSE loss. The latent dimension of 12 was selected through a grid search over $\{8, 12, 16, 24, 32\}$, with the 12-dimensional representation achieving the best downstream fraud detection performance when used as features for the stacking meta-learner.

D.4 Spiking Neural Network Mathematical Derivations

This section provides the complete mathematical derivations underlying the Spiking Neural Network (SNN) pipeline presented in Chapter 6. The SNN pipeline relies on Leaky Integrate-and-Fire (LIF) neuron dynamics for temporal spike computation and a surrogate gradient mechanism for training through backpropagation. These derivations are essential for understanding the pipeline's behavior and for ensuring exact reproducibility of the implementation.

Leaky Integrate-and-Fire (LIF) Dynamics. The discrete-time Euler update for the LIF neuron model governs the membrane potential evolution across simulation time steps. Let $U[t]$ denote the membrane potential at time step t, $I[t]$ the input current (derived from the rate-coded input features), β the decay factor controlling the membrane potential leakage, and $S[t]$ the binary spike output. The update equations are:

$$U[t] = \beta\, U[t-1]\,(1 - S[t-1]) + I[t], \tag{D.3}$$

$$S[t] = \begin{cases} 1 & \text{if } U[t] \geq U_{\text{thr}}, \\ 0 & \text{otherwise}, \end{cases} \tag{D.4}$$

where U_{thr} is the firing threshold. The term $(1 - S[t-1])$ implements the reset mechanism: when a spike is emitted at time $t-1$, the membrane potential is reset to zero before the next integration step (hard reset). The decay factor $\beta \in (0, 1)$ controls the temporal memory of the neuron; values close to 1 produce long membrane time constants (slow decay), enabling the neuron to accumulate evidence over many time steps, while values close to 0 produce short time constants (fast decay), making the neuron responsive primarily to instantaneous input. In this thesis, β is treated as a learnable parameter initialized to 0.9, allowing the network to adaptively determine the optimal temporal integration window for each neuron during training.

Surrogate Gradient (ArcTan). The non-differentiable nature of the spike generation function in Eq. (D.4) prevents direct application of the standard backpropagation algorithm. To circumvent this, we employ a surrogate gradient approach where the derivative of the Heaviside step function is approximated by the derivative of the arc-tangent function during the backward pass. The surrogate gradient is defined as:

$$\sigma'_{\text{ATan}}(x) = \frac{\alpha/\pi}{1 + (\alpha \cdot x/\pi)^2}, \tag{D.5}$$

where α is a hyperparameter controlling the steepness of the surrogate function. Larger values of α produce a sharper approximation that more closely matches the true Heaviside derivative, but can lead to gradient instability and training divergence. Smaller

values produce a smoother gradient that promotes stable training but may slow convergence by providing less informative gradient signals near the threshold. In this thesis, $\alpha = 2.0$ is used as the default value, which was found through empirical tuning to provide the best trade-off between gradient fidelity and training stability for the BAF dataset. During the forward pass, the exact Heaviside step function is used for spike generation; the surrogate gradient is applied exclusively during the backward pass to compute the gradients required for parameter updates. This forward–backward asymmetry is a standard technique in surrogate gradient training of spiking neural networks and has been shown to enable effective learning in both feedforward and recurrent SNN architectures.

Appendix E

Denoising Autoencoder Architecture and Objective Formulations

This appendix provides a rigorous treatment of the Denoising Autoencoder (DAE) architecture employed within Pipeline III of the multi-paradigm fraud detection system. While Chapter 7 presents the high-level design rationale and empirical results, this appendix formalizes the mathematical foundations of the corruption strategy, the objective function, and the network topology. A precise understanding of these components is essential for reproducibility and for extending the DAE framework to new domains or datasets. The formulations presented here are implemented exactly in the accompanying codebase and correspond to the configuration that produced the best-performing stacking meta-learner reported in the experimental chapters.

E.1 Tabular Swap Noise Augmentation

The corruption strategy applied to the input before encoding is tabular swap noise, a technique specifically designed for heterogeneous tabular data where features may differ in scale, type, and distribution. Unlike Gaussian noise or masking corruption, which are commonly applied to image data, swap noise replaces a feature value with another value drawn from the same feature's empirical marginal distribution in the training set. This preserves the marginal statistics of each feature while breaking the joint dependencies between features, thereby forcing the autoencoder to learn the underlying multivariate structure of the data rather than relying on local feature correlations.

Formally, consider an input feature vector $\mathbf{x} = (x_1, x_2, \ldots, x_d) \in \mathbb{R}^d$, where $d = 53$ denotes the number of input features in the BAF dataset. For each feature dimension $j \in \{1, 2, \ldots, d\}$, an independent Bernoulli random variable $m_j \sim \text{Bernoulli}(p_{\text{corr}})$ is sampled to determine whether that feature is corrupted. The corruption mask $\mathbf{m} = (m_1, m_2, \ldots, m_d)$ is drawn independently for each training example at each training step, ensuring stochastic diversity in the corruption process. The corrupted input $\tilde{\mathbf{x}} = (\tilde{x}_1, \tilde{x}_2, \ldots, \tilde{x}_d)$ is then constructed according to the swap noise mapping:

$$\tilde{x}_j = \begin{cases} x_j^{\text{sample}} & \text{with probability } p_{\text{corr}} = 0.15, \\ x_j & \text{with probability } 1 - p_{\text{corr}} = 0.85, \end{cases} \tag{E.1}$$

where x_j^{sample} is a value drawn uniformly at random from the empirical marginal distribution of feature j across all training examples. Specifically, if the training set

contains N examples with feature values $\{x_j^{(1)}, x_j^{(2)}, \ldots, x_j^{(N)}\}$ for dimension j, then x_j^{sample} is obtained by uniformly sampling one index $k \sim \text{Uniform}\{1, \ldots, N\}$ and setting $x_j^{\text{sample}} = x_j^{(k)}$.

The corruption probability $p_{\text{corr}} = 0.15$ was selected through a systematic grid search over the range $[0.05, 0.30]$ in increments of 0.05. At very low corruption rates ($p_{\text{corr}} < 0.10$), the autoencoder receives insufficient perturbation to learn meaningful denoising representations, and the model degenerates toward a near-identity mapping that fails to capture the structure of the legitimate application manifold. At very high corruption rates ($p_{\text{corr}} > 0.20$), the input retains too little information about the original features, making the reconstruction task excessively difficult and degrading both the quality of the latent representation and the convergence speed of training. The selected rate of 15% provides an effective compromise: it corrupts enough features to force the model to learn robust representations that generalize across perturbations, while preserving the majority of the original information so that the reconstruction target remains achievable.

An important property of swap noise, compared to alternative corruption strategies such as zero-masking or Gaussian additive noise, is that the corrupted value $\tilde{x}_j$ remains a plausible value for feature j because it is drawn from the same empirical distribution. This plausibility is particularly important for categorical or discrete features (e.g., the number of fraud reports, employment status codes), where zero-masking would produce out-of-distribution values that could bias the encoder's learned representations. By sampling from the marginal, swap noise ensures that the encoder processes inputs that are always within the support of the data distribution, even if the joint configuration of features is corrupted.

E.2 DAE Objective Function

The Denoising Autoencoder is trained using a target-isolated mean squared error (MSE) loss function. The term "target-isolated" refers to the critical design choice that the reconstruction target is the original, uncorrupted input $\mathbf{x}$ rather than the corrupted input $\tilde{\mathbf{x}}$ that is fed to the encoder. This asymmetry between the encoder input and the reconstruction target is the fundamental mechanism that drives the DAE to learn a denoising mapping rather than a trivial identity function. If the target were the corrupted input itself, the optimal strategy for the autoencoder would be to learn an identity mapping, which would require no meaningful latent representation.

The loss function is formulated as:

$$\mathcal{L}_{\text{DAE}} = \frac{1}{N} \sum_{i=1}^{N} \left\| \mathbf{x}^{(i)} - D_{\theta_d}\left(E_{\theta_e}\left(\tilde{\mathbf{x}}^{(i)}\right)\right) \right\|_2^2, \tag{E.2}$$

where N is the number of training examples in a mini-batch, $E_{\theta_e} : \mathbb{R}^d \to \mathbb{R}^z$ denotes the encoder parameterized by θ_e that maps the corrupted input $\tilde{\mathbf{x}}^{(i)}$ to a latent representation $\mathbf{z}^{(i)} = E_{\theta_e}(\tilde{\mathbf{x}}^{(i)}) \in \mathbb{R}^z$, and $D_{\theta_d} : \mathbb{R}^z \to \mathbb{R}^d$ denotes the decoder parameterized by θ_d that reconstructs the original input from the latent code. The squared ℓ_2-norm $\|\cdot\|_2^2$ computes the element-wise squared difference, summing over all $d = 53$ feature dimensions.

The target-isolated formulation has an important theoretical interpretation: it encourages the encoder to learn a mapping from the corrupted input to a latent rep-

resentation from which the original input can be recovered. This effectively requires the latent space to capture the essential structure of the data manifold while being invariant to the specific corruption pattern applied. In the context of fraud detection, this manifold-learning property is particularly valuable because the DAE is trained exclusively on legitimate (non-fraudulent) applications. The latent representation therefore encodes the structure of the legitimate application manifold, and fraudulent applications—which deviate from this manifold—produce high reconstruction errors that serve as anomaly signals for the downstream stacking meta-learner.

The optimization of $\mathcal{L}_{\text{DAE}}$ is performed using the Adam optimizer with an initial learning rate of 1×10^{-3}, which is reduced by a factor of 0.5 when the validation loss plateaus for 10 consecutive epochs (ReduceLROnPlateau scheduler). Training is terminated early if the validation loss does not improve for 20 consecutive epochs, with the model checkpoint corresponding to the best validation loss being retained for inference. A weight decay of 1×10^{-5} is applied as ℓ_2 regularization on all trainable parameters to prevent overfitting, particularly important given the relatively small size of the legitimate-only training subset.

E.3 Network Topology

The DAE employs a symmetric encoder–decoder architecture with progressively narrowing and widening layer dimensions, respectively. The full topology is specified below:

- **Encoder:** $53 \to 128 \to 64 \to 12$ (latent bottleneck $\mathbf{z}$)
- **Latent Bottleneck:** 12 coordinates
- **Decoder:** $12 \to 64 \to 128 \to 53$ (Linear Activation)

The encoder maps the 53-dimensional input through two expanding hidden layers (128 and 64 units) before compressing to the 12-dimensional latent bottleneck. The decoder mirrors this path, expanding from 12 through 64 and 128 units before reconstructing the original 53 features. The final decoder layer uses a linear activation function (i.e., no non-linearity) to allow the output to span the full range of feature values, including both positive and negative values, without the saturating behavior that would be imposed by sigmoid or tanh activations.

All hidden layers in both the encoder and decoder use LeakyReLU activation with a negative slope of $\alpha = 0.20$. The LeakyReLU activation function is defined as:

$$\text{LeakyReLU}(x; \alpha) = \begin{cases} x & \text{if } x \geq 0, \\ \alpha \cdot x & \text{if } x < 0. \end{cases} \tag{E.3}$$

The choice of LeakyReLU over the standard ReLU ($\alpha = 0$) is motivated by the "dying neuron" problem that is particularly acute in denoising autoencoders trained on tabular data. In such data, a substantial fraction of input features may be zero-valued or near-zero (e.g., the number of prior fraud reports is zero for the vast majority of applications), which can cause standard ReLU neurons to receive only negative pre-activations during both the forward pass and the gradient computation. Over time,

these neurons permanently output zero and cease to contribute to the model's representation, reducing effective capacity. The non-zero negative slope $\alpha = 0.20$ ensures that even neurons receiving predominantly negative inputs maintain a small but non-zero gradient, allowing them to recover during training.

Batch Normalization is applied after each LeakyReLU activation in the encoder layers. Each Batch Normalization layer computes the mean μ_B and variance σ_B^2 over the current mini-batch and normalizes the activations:

$$\hat{h}_j = \frac{h_j - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \tag{E.4}$$

where $\epsilon = 1 \times 10^{-5}$ is a small constant for numerical stability, and the normalized activations are then scaled and shifted by learnable parameters γ_j and β_j. Batch Normalization in the encoder serves two purposes: it stabilizes training by reducing internal covariate shift, and it acts as a mild regularizer due to the stochasticity introduced by mini-batch statistics. The decoder does not employ Batch Normalization, allowing the reconstruction layers greater flexibility in producing the exact feature values required by the MSE loss; preliminary experiments showed that adding Batch Normalization to the decoder slightly increased reconstruction error, likely because the normalization constraint interfered with the precise value recovery needed for accurate reconstruction.

The latent dimension of 12 was selected through a grid search over $\{8, 12, 16, 24, 32\}$, evaluating each configuration by the downstream fraud detection performance (ROC AUC) of the stacking meta-learner that consumes the DAE's latent representations as features. The 12-dimensional representation achieved the highest ROC AUC of 0.8643, outperforming both the 8-dimensional bottleneck (0.8619, insufficient capacity) and the 32-dimensional bottleneck (0.8624, over-parameterization leading to reduced denoising pressure). This result confirms the classical autoencoder finding that an intermediate bottleneck width produces the most informative representations by striking a balance between compression (which forces the model to discard noise and retain signal) and capacity (which provides sufficient degrees of freedom to encode the relevant structure).

Appendix F

Spiking Neural Network Mathematical Derivations

This appendix presents the complete mathematical derivations underlying the Spiking Neural Network (SNN) pipeline described in Chapter 6. The SNN paradigm departs fundamentally from conventional artificial neural networks by operating on discrete spike events rather than continuous activation values, introducing both unique representational capabilities and distinctive training challenges. Two core mathematical components require formal treatment: the Leaky Integrate-and-Fire (LIF) neuron model that governs spike dynamics, and the surrogate gradient mechanism that enables gradient-based optimization through the non-differentiable spike generation function. These derivations are essential for understanding the theoretical properties of the SNN pipeline and for ensuring exact reproducibility of the implementation. The notation and parameter choices presented here correspond precisely to the codebase that produced the experimental results reported in the main chapters.

F.1 Leaky Integrate-and-Fire (LIF) Dynamics

The Leaky Integrate-and-Fire model is the most widely used spiking neuron model in the surrogate gradient learning literature, offering a tractable balance between biological realism and computational efficiency. The model describes a point neuron whose membrane potential integrates incoming synaptic currents while decaying toward a resting potential, emitting a discrete spike when the membrane potential crosses a threshold. This section presents both the continuous-time formulation (for theoretical reference) and the discrete-time formulation (which is used in the actual implementation).

F.1.1 Continuous-Time Formulation

In continuous time, the membrane potential $V(t)$ of a LIF neuron evolves according to the first-order ordinary differential equation:

$$\tau_m \frac{dV}{dt} = -\left(V(t) - V_{\text{rest}}\right) + R \cdot I(t), \tag{F.1}$$

where τ_m is the membrane time constant (controlling the rate of exponential decay toward the resting potential), V_{rest} is the resting membrane potential, R is the membrane

resistance, and $I(t)$ is the total input current at time t. When the membrane potential reaches the firing threshold V_{thr}, a spike is emitted and the membrane potential is instantaneously reset to the reset potential V_{reset}. The continuous-time formulation provides the theoretical foundation for the model but is not directly amenable to numerical simulation on digital hardware, necessitating a discrete-time approximation.

F.1.2 Discrete-Time Euler Approximation

The discrete-time formulation used in the implementation employs a first-order Euler approximation of Eq. (F.1), with a hard reset mechanism that sets the membrane potential to zero immediately after a spike. Let $U[t]$ denote the membrane potential at discrete time step t, $I[t]$ the input current, β the decay factor, and $S[t]$ the binary spike output. The discrete-time update equations are:

$$U[t] = \beta\, U[t-1]\,(1 - S[t-1]) + I[t], \tag{F.2}$$

$$S[t] = \begin{cases} 1 & \text{if } U[t] \geq V_{\text{thr}}, \\ 0 & \text{otherwise}, \end{cases} \tag{F.3}$$

where $\beta \in (0, 1)$ is the decay factor that controls the temporal memory of the neuron. The relationship between β and the continuous-time membrane time constant is $\beta = \exp(-\Delta t/\tau_m)$, where Δt is the discrete time step size. In the SNN pipeline, the simulation is conducted over $T = 25$ time steps, with the input features rate-coded into spike trains that span the full simulation duration.

The term $(1 - S[t-1])$ in Eq. (F.2) implements the hard reset mechanism: when a spike is emitted at time step $t-1$ (i.e., $S[t-1] = 1$), the membrane potential is reset to zero before the next integration step, ensuring that the post-spike potential does not carry residual charge from the pre-spike state. This hard reset is preferred over the alternative soft reset ($U[t] = \beta\, U[t-1] + I[t] - V_{\text{thr}} \cdot S[t-1]$) because it produces cleaner spike trains with less inter-spike interference, which was found empirically to improve classification performance on the BAF dataset.

The decay factor β is treated as a learnable parameter, initialized to $\beta_0 = 0.9$ and constrained to the interval $(0, 1)$ via a sigmoid reparameterization during optimization: $\beta = \sigma(\tilde{\beta})$, where $\tilde{\beta} \in \mathbb{R}$ is the unconstrained parameter and $\sigma(\cdot)$ is the logistic sigmoid function. The initialization value of 0.9 corresponds to a membrane time constant of approximately $\tau_m \approx 9.5\Delta t$, which provides a moderate temporal integration window. During training, different neurons learn different decay rates, with neurons in earlier layers tending to develop faster decay (smaller β) for rapid feature extraction, while neurons in deeper layers tend to develop slower decay (larger β) for temporal evidence accumulation. This adaptive behavior emerges naturally from the gradient-based optimization without requiring explicit layer-wise initialization or regularization.

The input current $I[t]$ is computed as a weighted linear combination of the input spike trains:

$$I[t] = \mathbf{W}\, \mathbf{S}_{\text{in}}[t] + \mathbf{b}, \tag{F.4}$$

where $\mathbf{W}$ is the synaptic weight matrix, $\mathbf{S}_{\text{in}}[t]$ is the vector of input spikes at time step t, and $\mathbf{b}$ is a learnable bias term. The firing threshold V_{thr} is fixed at 1.0 throughout training and inference, a standard choice in the surrogate gradient literature that simplifies the implementation without sacrificing model expressiveness, as the effective threshold can be modulated through the weight magnitudes and bias terms.

F.1.3 Rate Coding and Output Decoding

The conversion of continuous input features to spike trains employs a rate coding scheme, where the firing probability of an input neuron at each time step is proportional to the normalized feature value. For an input feature $x_j \in [0, 1]$ (after min-max normalization), the probability of emitting a spike at time step t is:

$$P(S_{\text{in},j}[t] = 1) = x_j. \tag{F.5}$$

Over $T = 25$ time steps, the expected firing count for neuron j is $\mathbb{E}[\sum_{t=1}^{T} S_{\text{in},j}[t]] = T \cdot x_j$, providing a stochastic but unbiased representation of the input feature magnitude. The output of the SNN is decoded by computing the average spike rate over the final layer across all time steps:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^{T} S_{\text{out}}[t], \tag{F.6}$$

where $S_{\text{out}}[t]$ denotes the spike output of the final classification neuron at time step t. This rate-based decoding provides a continuous prediction in $[0, 1]$ that is directly interpretable as the estimated probability of fraud.

F.2 Surrogate Gradient Descent (ArcTan)

The fundamental challenge in training spiking neural networks with gradient-based methods is the non-differentiable nature of the spike generation function. The Heaviside step function $\Theta(u) = \mathbb{1}[u \geq 0]$ used in Eq. (F.3) has a derivative that is zero everywhere except at $u = 0$, where it is undefined (Dirac delta). This means that standard backpropagation cannot propagate gradient information through the spike generation mechanism, as the chain rule would multiply all upstream gradients by zero, effectively preventing any parameter updates.

F.2.1 Surrogate Gradient Framework

The surrogate gradient approach circumvents this problem by replacing the true derivative of the Heaviside function with a smooth approximation during the backward pass, while preserving the exact Heaviside function during the forward pass. This creates an intentional asymmetry between the forward and backward computations:

$$\text{Forward pass:} \quad S[t] = \Theta(U[t] - V_{\text{thr}}), \tag{F.7}$$

$$\text{Backward pass:} \quad \frac{\partial S[t]}{\partial U[t]} \approx \sigma'_{\text{surr}}(U[t] - V_{\text{thr}}), \tag{F.8}$$

where $\sigma'_{\text{surr}}(\cdot)$ is the surrogate gradient function. This forward–backward asymmetry is well-motivated: the forward pass must use the exact Heaviside function to produce binary spike events that carry the temporal and computational advantages of spike-based computation, while the backward pass only requires a useful gradient signal for parameter optimization, not an exact derivative.

F.2.2 ArcTan Surrogate Gradient

In this thesis, the arc-tangent (ArcTan) function is employed as the surrogate gradient. The ArcTan surrogate gradient is defined as:

$$\sigma'_{\text{ATan}}(x) = \frac{\alpha/\pi}{1 + (\alpha \cdot x/\pi)^2}, \tag{F.9}$$

where α is a hyperparameter controlling the steepness (sharpness) of the surrogate function, and $x = U[t] - V_{\text{thr}}$ is the difference between the membrane potential and the threshold. The corresponding surrogate function (whose derivative gives Eq. (F.9)) is:

$$\sigma_{\text{ATan}}(x) = \frac{1}{\pi} \arctan\left(\frac{\alpha \cdot x}{\pi}\right) + \frac{1}{2}. \tag{F.10}$$

The parameter α governs the trade-off between gradient fidelity and training stability. When $\alpha \to \infty$, the ArcTan surrogate approaches a Dirac delta function, closely matching the true derivative of the Heaviside step function but concentrating gradient information in an increasingly narrow band around the threshold, which can cause gradient instability and training divergence. When $\alpha \to 0$, the surrogate becomes a broad, flat function that distributes gradient information widely but provides poor resolution for distinguishing near-threshold from far-from-threshold membrane potentials, potentially slowing convergence. The default value $\alpha = 2.0$ was selected through empirical evaluation over the range $\{1.0, 1.5, 2.0, 3.0, 5.0\}$ on a validation split of the BAF dataset. The $\alpha = 2.0$ configuration achieved the best convergence speed and final classification performance, providing sufficiently steep gradients near the threshold for precise parameter updates while maintaining adequate gradient magnitude for neurons whose membrane potentials are moderately far from threshold.

F.2.3 Gradient Propagation Through Time

Combining the LIF dynamics with the surrogate gradient, the full gradient of the loss $\mathcal{L}$ with respect to the synaptic weights $\mathbf{W}$ is computed via backpropagation through time (BPTT). For a loss computed from the decoded output at time step t, the gradient contribution through the spike at that time step is:

$$\left.\frac{\partial \mathcal{L}}{\partial \mathbf{W}}\right|_t = \frac{\partial \mathcal{L}}{\partial S[t]} \cdot \sigma'_{\text{ATan}}(U[t] - V_{\text{thr}}) \cdot \frac{\partial U[t]}{\partial \mathbf{W}}, \tag{F.11}$$

where the first factor is the loss gradient with respect to the spike output, the second factor is the surrogate gradient that replaces the non-differentiable Heaviside derivative, and the third factor is the gradient of the membrane potential with respect to the weights, which can be computed exactly through the recurrent LIF dynamics. The total gradient is accumulated over all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}} = \sum_{t=1}^{T} \left.\frac{\partial \mathcal{L}}{\partial \mathbf{W}}\right|_t. \tag{F.12}$$

The recurrent nature of the LIF dynamics (through the $\beta\, U[t-1]$ term) means that the gradient at time step t depends on all preceding time steps, creating the potential for vanishing or exploding gradients. However, the decay factor $\beta < 1$ naturally attenuates

gradients over long time horizons, and the hard reset mechanism (which zeros the membrane potential after a spike) provides natural gradient clipping by preventing indefinite accumulation. In practice, the BPTT gradients remain well-behaved for the $T = 25$ time steps used in this thesis, and no gradient clipping was required during training.

The choice of the ArcTan surrogate over alternatives such as the Fast Sigmoid $\sigma'(x) = \alpha/(1 + |\alpha x|)^2$ or the piecewise linear surrogate $\sigma'(x) = \max(0, 1 - |\alpha x|)$ is motivated by its smooth, non-vanishing tails. Unlike the piecewise linear surrogate, which has zero gradient outside a finite interval, the ArcTan surrogate provides non-zero (albeit small) gradients for all membrane potentials, ensuring that even neurons far from threshold receive some gradient signal. This property is particularly beneficial during the early stages of training when membrane potentials are randomly initialized and may be far from the firing threshold.

Appendix G

Graph Neural Network Algorithmic Procedures

This appendix provides the formal algorithmic descriptions of the two Graph Neural Network (GNN) architectures developed in this thesis: the Quantum-enhanced Graph Neural Network (QGNN) and the Split Graph Neural Network (SplitGNN). While Chapter 5 presents the architectural design, motivation, and empirical evaluation of these models, this appendix specifies the complete step-by-step procedures for forward propagation, loss computation, and parameter updates. The algorithms are presented in pseudocode form using the standard `algorithmic` notation, with precise mathematical formulations for each computational step. These descriptions are intended to serve as unambiguous references for reimplementation and to clarify the ordering of operations that may be difficult to discern from prose descriptions alone. All variable names and notation are consistent with the formulations presented in the main chapter.

G.1 Algorithm 1: Quantum GNN Forward Pass and Training

The Quantum GNN integrates a Variational Quantum Circuit (VQC) layer into the message-passing framework of a Graph Neural Network. The VQC processes the node embeddings produced by classical MLP layers, applying parameterized quantum operations that can in principle explore exponentially large Hilbert spaces with a polynomial number of qubits. The training procedure alternates between classical forward propagation through the MLP and GNN layers, quantum circuit evaluation for feature transformation, and gradient-based parameter updates via backpropagation. The following algorithm describes the complete training loop for one epoch, including the quantum circuit evaluation at each GNN layer.

The Quantum GNN forward pass proceeds as follows. Given an input graph $G = (V, E)$ with node features $\{x_u \mid u \in V\}$ and labels $\{y_u \mid u \in V\}$, the model first initializes the learnable parameters of the MLP classifier and the VQC layers. For each training epoch, a mini-batch of nodes is sampled, and the initial node embeddings are computed via a linear projection of the input features. The model then applies L layers of message passing, each consisting of a VQC layer computation followed by neighbor aggregation. After the final layer, a classification head maps the node embeddings to fraud predictions, and the loss is computed with class-balanced weighting to account

for the severe label imbalance in the BAF dataset. The AdamW optimizer with cosine annealing learning rate schedule is used for parameter updates. The detailed procedure is presented in Algorithm 3.

Algorithm 3 Quantum GNN Forward Pass and Training

Input: Graph $G = (V, E)$, node features $\{x_u \mid u \in V\}$, labels $\{y_u \mid u \in V\}$, VQC layers L, qubits n_q, learning rate η, pos_weight w_{pos}, weight decay λ

Output: Trained model parameters $\Theta = \{\theta_{\text{MLP}}, \theta_{\text{VQC}}^{(1)}, \ldots, \theta_{\text{VQC}}^{(L)}\}$

Initialize MLP parameters θ_{MLP} and VQC parameters $\{\theta_{\text{VQC}}^{(l)}\}_{l=1}^{L}$ via Xavier initialization
Initialize AdamW optimizer with learning rate η and weight decay λ
Set cosine annealing LR schedule: $\eta_t = \eta_{\min} + \frac{1}{2}(\eta - \eta_{\min})\left(1 + \cos\left(\frac{t}{T_{\max}}\pi\right)\right)$
for epoch $= 1$ to max_epochs **do**
  Sample mini-batch $\mathcal{B} \subset V$
  for each node $u \in \mathcal{B}$ **do**
    Compute initial embedding: $h_u^{(0)} = W_0\, x_u + b_0$
  end for
  for layer $l = 1$ to L **do**
    for each node $u \in \mathcal{B}$ **do**
      VQC Layer Computation:
      Encode $h_u^{(l-1)}$ into quantum state via angle encoding: $|\psi_u\rangle = \bigotimes_{q=1}^{n_q} R_Y(h_{u,q}^{(l-1)})|0\rangle$
      Apply parameterized circuit: $|\psi_u'\rangle = U(\theta_{\text{VQC}}^{(l)})|\psi_u\rangle$
      Measure expectation values: $z_u^{(l)} = \left[\langle Z_1\rangle, \langle Z_2\rangle, \ldots, \langle Z_{n_q}\rangle\right]$
    end for
    for each node $u \in \mathcal{B}$ **do**
      Neighbor Aggregation:
      Aggregate neighbor quantum features: $a_u^{(l)} = \frac{1}{|\mathcal{N}(u)|}\sum_{v\in\mathcal{N}(u)} z_v^{(l)}$
      Combine self and neighbor features: $h_u^{(l)} = \text{ReLU}\left(W^{(l)}\left[z_u^{(l)} \| a_u^{(l)}\right] + b^{(l)}\right)$
    end for
  end for
  Classification:
  for each node $u \in \mathcal{B}$ **do**
    Compute logits: $\hat{y}_u = \text{MLP}_{\theta_{\text{MLP}}}(h_u^{(L)})$
    Apply sigmoid: $p_u = \sigma(\hat{y}_u)$
  end for
  Loss Computation:
  $\mathcal{L} = -\frac{1}{|\mathcal{B}|}\sum_{u\in\mathcal{B}}\left[w_{\text{pos}} \cdot y_u \log p_u + (1 - y_u)\log(1 - p_u)\right]$
  Parameter Update:
  Compute gradients: $g \leftarrow \nabla_\Theta \mathcal{L}$
  Update parameters: $\Theta \leftarrow \text{AdamW}(\Theta, g, \eta_t, \lambda)$
  Update learning rate: $t \leftarrow t + 1$
end for
return Θ

The key computational steps in Algorithm 3 deserve further elaboration. The angle

encoding on line 11 maps each component of the classical embedding $h_u^{(l-1)}$ to a rotation angle of a qubit, creating an initial quantum state that encodes the classical information. The parameterized unitary $U(\theta_{\text{VQC}}^{(l)})$ on line 12 applies a sequence of rotation and entangling gates, with the rotation angles constituting the trainable parameters of the VQC layer. The measurement on line 13 extracts classical information from the quantum state by computing the expectation value of the Pauli-Z operator on each qubit, producing an n_q-dimensional real-valued vector that serves as the quantum-enhanced feature representation for node u.

The neighbor aggregation on lines 15–17 follows the standard message-passing paradigm: each node receives the quantum features of its neighbors, averages them, and combines the result with its own quantum features through a learnable linear transformation followed by a ReLU activation. The concatenation $\left[z_u^{(l)} \| a_u^{(l)}\right]$ preserves both the self-feature and the aggregated neighborhood information, allowing the model to balance local and contextual information adaptively through the learned weight matrix $W^{(l)}$. The classification head on lines 19–21 consists of a two-layer MLP with a hidden dimension of 32 and ReLU activation, producing a scalar logit that is passed through a sigmoid function to obtain the fraud probability p_u.

The loss function on line 23 is the weighted binary cross-entropy, where the `pos_weight` parameter w_{pos} compensates for the extreme class imbalance in the BAF dataset (approximately 1.1% fraud prevalence). Setting w_{pos} to the inverse of the fraud prevalence ratio (≈ 89) ensures that the gradient contributions from fraudulent and legitimate examples are approximately balanced, preventing the model from achieving deceptively high accuracy by predicting all examples as legitimate. The AdamW optimizer on line 26 decouples the weight decay from the gradient update, providing more principled ℓ_2 regularization than the standard Adam optimizer, and the cosine annealing schedule on line 3 provides a smooth learning rate decay that has been shown to improve generalization in deep learning models by enabling the optimizer to escape sharp minima during the early high-learning-rate phase and converge to flat minima during the later low-learning-rate phase.

G.2 Algorithm 2: SplitGNN Training Procedure

The Split Graph Neural Network (SplitGNN) introduces a dual-objective training framework that jointly optimizes node classification and edge classification within a single model. The key innovation is the graph splitting mechanism, which partitions the original graph into two subgraphs based on the learned edge weights: one subgraph contains edges predicted to connect nodes of the same class (homophilic edges), while the other contains edges predicted to connect nodes of different classes (heterophilic edges). Separate spectral convolution filters are then applied to each subgraph, enabling the model to learn class-aware representations that leverage the distinct homophily patterns in each subgraph. The following algorithm describes the complete training procedure.

Given an input graph $G = (V, E, X)$ with node features X, labels $\{y_u\}$, a wavelet order parameter C, a split point K, and an edge loss weight γ_e, the SplitGNN training proceeds as follows. First, the node embeddings are initialized via a linear projection. At each training epoch, the model computes edge weights for all edges in the graph using a parametric edge classifier, splits the graph into two subgraphs based on the

top-K and bottom-K edge weights, computes the graph Laplacians for each subgraph (using a cached Laplacian computation for efficiency), applies spectral convolution on each subgraph with residual connections, and computes a multi-task loss that combines the node classification loss, the edge classification loss, and the consistency loss. The gradient descent update is then applied to all model parameters. The early stopping criterion monitors the validation ROC AUC and terminates training if no improvement is observed for a specified patience period. The detailed procedure is presented in Algorithm 4.

Algorithm 4 SplitGNN Training Procedure

Input: Graph $G = (V, E, X)$, labels $\{y_u \mid u \in V\}$, wavelet order C, split point K, edge loss weight γ_e, node loss weight γ_n, learning rate η, early stopping patience P

Output: Trained model parameters $\Theta = \{\theta_{\text{emb}}, \theta_{\text{edge}}, \theta_{\text{conv}}^{\text{homo}}, \theta_{\text{conv}}^{\text{hetero}}, \theta_{\text{cls}}\}$

Initialize embedding parameters θ_{emb}, edge classifier θ_{edge}, spectral convolutions $\theta_{\text{conv}}^{\text{homo}}$, $\theta_{\text{conv}}^{\text{hetero}}$, and classifier θ_{cls}
Compute initial embeddings: $h_u^{(0)} = W_{\text{emb}}\, x_u + b_{\text{emb}}$ for all $u \in V$
Initialize AdamW optimizer with learning rate η
$best\_auc \leftarrow 0$; $patience\_counter \leftarrow 0$
for epoch $= 1$ to max_epochs **do**
  Edge Weight Computation:
  for each edge $(u, v) \in E$ **do**
    Compute edge embedding: $e_{uv} = [\|h_u\| \,\|\, \|h_v\| \,\|\, (h_u - h_v)]$
    Compute edge weight: $w_{uv} = \tanh\big(\mathbf{w}^T e_{uv} + b_e\big)$
  end for
  Graph Split:
  Sort edges by weight: $E_{\text{sorted}} = \text{sort}(E, \text{key} = w_{uv}, \text{descending})$
  Homophilic subgraph: $G_{\text{homo}} = (V, E_{\text{homo}})$ where $E_{\text{homo}} = E_{\text{sorted}}[1 : K]$
  Heterophilic subgraph: $G_{\text{hetero}} = (V, E_{\text{hetero}})$ where $E_{\text{hetero}} = E_{\text{sorted}}[K + 1 : 2K]$
  Laplacian Computation:
  $L_{\text{homo}} = \text{CachedLaplacian}(G_{\text{homo}})$
  $L_{\text{hetero}} = \text{CachedLaplacian}(G_{\text{hetero}})$
  Spectral Convolution with Residual:
  for each node $u \in V$ **do**
    Homophilic convolution: $z_u^{\text{homo}} = \sum_{c=0}^{C} \theta_c^{\text{homo}}\, T_c(\tilde{L}_{\text{homo}})\, h_u$
    Heterophilic convolution: $z_u^{\text{hetero}} = \sum_{c=0}^{C} \theta_c^{\text{hetero}}\, T_c(\tilde{L}_{\text{hetero}})\, h_u$
    Combine with residual: $h'_u = \sigma\big(z_u^{\text{homo}} + z_u^{\text{hetero}} + h_u\big)$
  end for
  Classification:
  for each node $u \in V$ **do**
    Node logits: $\hat{y}_u = W_{\text{cls}}\, h'_u + b_{\text{cls}}$
    Node probability: $p_u = \sigma(\hat{y}_u)$
  end for
  Multi-Task Loss:
  $\mathcal{L}_{\text{node}} = -\frac{1}{|V|} \sum_{u \in V} [w_{\text{pos}} \cdot y_u \log p_u + (1 - y_u) \log(1 - p_u)]$
  $\mathcal{L}_{\text{edge}} = -\frac{1}{|E|} \sum_{(u,v) \in E} [\mathbb{1}[y_u = y_v] \log \sigma(w_{uv}) + \mathbb{1}[y_u \neq y_v] \log(1 - \sigma(w_{uv}))]$
  $\mathcal{L}_{\text{total}} = \gamma_n \cdot \mathcal{L}_{\text{node}} + \gamma_e \cdot \mathcal{L}_{\text{edge}}$
  Gradient Descent Update:
  Compute gradients: $g \leftarrow \nabla_\Theta \mathcal{L}_{\text{total}}$
  Update parameters: $\Theta \leftarrow \text{AdamW}(\Theta, g, \eta, \lambda)$
  Early Stopping:
  Evaluate AUC_{val} on validation set
  if $AUC_{\text{val}} > best\_auc$ **then**
    $best\_auc \leftarrow AUC_{\text{val}}$; $patience\_counter \leftarrow 0$
    Save model checkpoint
  else
    $patience\_counter \leftarrow patience\_counter + 1$
    if $patience\_counter \geq P$ **then**
      break
    end if
  end if
end for
return Best model checkpoint

Several aspects of Algorithm 4 merit detailed discussion. The edge weight computation on lines 7–9 uses a parametric function that takes the concatenation of the node embedding norms and their difference as input. The norm features $\|h_u\|$ and $\|h_v\|$ capture the magnitude of each node's representation, while the difference feature $(h_u - h_v)$ captures the directional disagreement between the two embeddings. The tanh activation constrains the edge weight to the interval $(-1, 1)$, where positive values indicate predicted homophily (same-class connection) and negative values indicate predicted heterophily (different-class connection). This design is inspired by the observation that fraudulent and legitimate applicants tend to form distinct clusters in the embedding space, with intra-cluster edges exhibiting high embedding similarity and inter-cluster edges exhibiting low similarity.

The graph splitting on lines 10–13 selects the top-K edges with the highest weights for the homophilic subgraph and the K edges with the lowest weights for the heterophilic subgraph. The split point K is a critical hyperparameter: too few edges per subgraph result in sparse graphs with insufficient connectivity for effective message passing, while too many edges per subgraph dilute the homophily/heterophily signal by including ambiguous edges. The optimal value $K = 5000$ was determined through the hyperparameter grid search reported in Appendix D. The symmetric splitting (equal number of edges in both subgraphs) ensures balanced computational cost and prevents either subgraph from dominating the learned representations.

The spectral convolution on lines 16–19 employs Chebyshev polynomial filters of order C, where $T_c(\cdot)$ denotes the c-th Chebyshev polynomial of the first kind and $\tilde{L}$ is the scaled Laplacian $\tilde{L} = 2L/\lambda_{\max} - I$ with $\lambda_{\max}$ being the largest eigenvalue. The Chebyshev basis provides a computationally efficient approximation to arbitrary spectral filters without requiring explicit eigendecomposition, and the polynomial order C controls the filter's receptive field (i.e., how many hops of neighborhood information each filter captures). The CachedLaplacian on lines 14–15 precomputes and caches the Laplacian matrices between epochs to avoid redundant computation, with the cache being invalidated and recomputed only when the graph topology changes (which occurs at every epoch due to the dynamic edge weight updates). The residual connection on line 19 adds the original embedding h_u to the combined spectral convolution outputs, mitigating the oversmoothing problem that can arise when multiple rounds of neighborhood aggregation cause all node representations to converge to similar values.

The multi-task loss on lines 22–25 combines two objectives: the node classification loss $\mathcal{L}_{\text{node}}$ (weighted binary cross-entropy with `pos_weight` to handle class imbalance) and the edge classification loss $\mathcal{L}_{\text{edge}}$ (binary cross-entropy that supervises the edge weights to correctly predict whether each edge connects same-class or different-class nodes). The edge loss provides a direct training signal for the edge classifier, which in turn improves the quality of the graph split, creating a virtuous cycle where better edge classifications lead to more informative subgraphs, which lead to better node representations, which further improve edge classifications. The weighting coefficient γ_e controls the relative importance of the edge loss; the grid search in Appendix D shows that $\gamma_e = 1.0$ (equal weighting) produces the best overall performance, suggesting that both objectives contribute equally to the model's fraud detection capability.

References

[1] Aaltodoc. "Spiking neural networks on tabular data for time series classification," Accessed: May 8, 2026. [Online]. Available: https://aaltodoc.aalto.fi/items/c7972b8a-9c3b-4938-b581-a90344227549

[2] I. Ahmed and S. M. Shamsuddin, "A comparative study of machine learning techniques for financial fraud detection," *Journal of King Saud University – Computer and Information Sciences*, vol. 34, no. 10, pp. 8567–8585, 2022.

[3] O. A. Ahmed, "Bio-inspired anomaly detection: Leveraging spiking neural networks for imbalanced financial datasets," Faculty of Computers and Artificial Intelligence, Capital University, Technical Report, 2025.

[4] H. Z. Alenzi and N. O. Aljehane, "Fraud detection in credit cards using logistic regression," *International Journal of Advanced Computer Science and Applications*, vol. 11, no. 12, pp. 540–544, 2020.

[5] K. Arulkumaran and P. Anand, "BankGuard-CapsNet: A capsule network based framework for cloud banking fraud detection," *Expert Systems with Applications*, vol. 208, p. 118 099, 2022.

[6] arXiv. "A latency coding framework for deep spiking neural networks with ultra-low latency," Accessed: May 8, 2026. [Online]. Available: https://arxiv.org/html/2603.23206v1

[7] arXiv. "Evaluation of encoding schemes on ubiquitous sensor signal for spiking neural network," Accessed: May 8, 2026. [Online]. Available: https://arxiv.org/html/2407.09260v1

[8] arXiv. "Neural population coding for effective temporal classification," Accessed: May 8, 2026. [Online]. Available: https://arxiv.org/pdf/1909.08018

[9] arXiv. "Reinforcement-guided hyper-heuristic hyperparameter optimization for fair and explainable spiking neural network-based financial fraud detection," Accessed: May 8, 2026. [Online]. Available: https://arxiv.org/html/2508.16915v1

[10] K. Brakta. "Spiking neural networks: The third generation of neural computation," Accessed: May 8, 2026. [Online]. Available: https://medium.com/towards-data-engineering/spiking-neural-networks-the-third-generation-of-neural-computation-113d547ceb0a

[11] J. Byrd and M. Lichtenberg, "Effect of class weighting on logistic regression," *arXiv preprint arXiv:1911.12037*, 2019.

[12] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016, pp. 785–794.

[14] K. Cheng, Y. Li, Z. Zhu, and X. Zhao, "Graph neural networks for financial fraud detection: A comprehensive review," *ACM Computing Surveys*, 2024.

[15] A. Chouldechova, "Fair prediction with disparate impact: A study of bias in recidivism prediction instruments," *Big Data*, vol. 5, no. 2, pp. 153–163, 2017.

[16] CiiT International Journal. "Comparison of LIF and izhikevich spiking neural models for recognition of uppercase and lowercase english characters," Accessed: May 8, 2026. [Online]. Available: https://www.ciitresearch.org/dl/index.php/dip/article/view/DIP072014005

[17] E. Dai and S. Wang, "Say no to the discrimination: Learning fair graph neural networks with limited sensitive attribute information," in *Proceedings of the ACM International Conference on Web Search and Data Mining (WSDM)*, 2021.

[18] A. Daoud, A. Trabelsi, and H. Almansoori, "A new hybrid model for improving outlier detection using combined autoencoder and variational autoencoder," *Scientific Reports*, vol. 15, p. 43 387, 2025.

[19] M. Defferrard, X. Bresson, and P. Vandergheynst, "Convolutional neural networks on graphs with fast localized spectral filtering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2016, pp. 3844–3852.

[20] Y. Dong, "An enhanced autoencoder for anomaly detection in imbalanced financial fraud data," M.S. thesis, University of Canberra, 2025.

[21] J. K. Eshraghian et al., *snnTorch: A library for optimizing spiking neural networks*, https://snntorch.readthedocs.io/, Software Documentation, 2021.

[22] J. K. Eshraghian, M. Ward, E. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *arXiv preprint arXiv:2109.12894*, 2021.

[23] M. S. Farahat, "Objective over architecture: Fraud detection under extreme imbalance," *MDPI Electronics*, vol. 14, no. 2, p. 317, 2025.

[24] Feedzai Research, *Bank account fraud (BAF) tabular dataset suite*, https://www.feedzai.com/research/, NeurIPS 2022 Datasets and Benchmarks, 2022.

[25] Feedzai Research. "Bank account fraud (BAF) tabular dataset suite: Github repository," Accessed: May 8, 2026. [Online]. Available: https://github.com/feedzai/bank-account-fraud/blob/main/README.md

[26] Frontiers in Neuroscience. "Neural coding in spiking neural networks: A comparative study for robust neuromorphic systems," Accessed: May 8, 2026. [Online]. Available: https://www.frontiersin.org/journals/neuroscience/articles/10.3389/fnins.2021.638474/full

[27] A. Gouda, E.-S. El-Shafeiy, and A. E. Hassanien, "Unsupervised outlier detection in IoT using deep VAE," *Sensors*, vol. 22, no. 17, p. 6617, 2022.

[28] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[29] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 1024–1034.

[30] D. K. Hammond, P. Vandergheynst, and R. Gribonval, "Wavelets on graphs via spectral graph theory," *Applied and Computational Harmonic Analysis*, vol. 30, no. 2, pp. 129–150, 2011.

[31] H. He, Y. Bai, E. A. Garcia, and S. Li, "ADASYN: Adaptive synthetic sampling approach for imbalanced learning," in *IEEE International Joint Conference on Neural Networks*, 2008, pp. 1322–1328.

[32] J. Herrman, Y. Chen, K. Wang, and S. Kais, "Quantum graph neural networks for molecular property prediction," *Journal of Chemical Information and Modeling*, vol. 62, no. 18, pp. 4353–4363, 2022.

[33] Himasharandil. "From neurons to mathematics: The leaky integrate-and-fire model in computational neuroscience," Accessed: May 8, 2026. [Online]. Available: https://medium.com/@himasharandil/from-neurons-to-mathematics-the-leaky-integrate-and-fire-model-in-computational-neuroscience-594dfac6e8b4

[34] L. van der Hulst, "Graph neural networks for financial fraud detection: An empirical study," *Information Processing & Management*, vol. 61, no. 3, p. 103 567, 2024.

[35] H. Innan, M. Shakir, and R. Patel, "Quantum graph neural networks for enhanced financial fraud detection," *Quantum Information Processing*, vol. 23, p. 142, 2024.

[36] M. R. Islam, Y. Zhang, and J. Wu, "Effects of feature encoding on machine learning models for bank account fraud detection," *IEEE Access*, vol. 11, pp. 117 846–117 860, 2023.

[37] S. Jesus, J. Pombal, A. F. Cruz, D. Figueiredo, A. C. Sobral, P. Saleiro, and P. Bizarro, "Turning the tables: Biased, imbalanced, dynamic tabular datasets for ML evaluation," in *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2022.

[38] Journal of Machine Learning Research. "On the intrinsic structures of spiking neural networks," Accessed: May 8, 2026. [Online]. Available: https://www.jmlr.org/papers/volume25/23-1526/23-1526.pdf

[39] Kaggle. "Tabular playground series: Baf: Bank account fraud," Accessed: May 8, 2026. [Online]. Available: https://www.kaggle.com/competitions/tabular-playground-series-aug-2022

[40] Kaggle. "Bank account fraud dataset suite (neurips 2022)," Accessed: May 8, 2026. [Online]. Available: https://www.kaggle.com/datasets/sgpjesus/bank-account-fraud-dataset-neurips-2022

[41] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 3146–3154.

[42] D. P. Kingma and M. Welling, "Auto-encoding variational Bayes," in *International Conference on Learning Representations (ICLR)*, 2014.

[43] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations (ICLR)*, 2017.

[44] N. Kozodoi, J. Jacob, and S. Lessmann, "Fairness in credit scoring: Assessment, implementation and profit implications," *European Journal of Operational Research*, vol. 297, no. 3, pp. 1083–1094, 2022.

[45] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 2980–2988.

[46] Z. Lin. "Training high-performance low-latency spiking neural networks by differentiation on spike representation," Accessed: May 8, 2026. [Online]. Available: https://zhouchenlin.github.io/Publications/2022-CVPR-SNN.pdf

[47] Y. Liu, X. Ao, Z. Hao, F. Chi, J. He, and Q. He, "Semi-supervised graph classification via hierarchical graph convex pools," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.

[48] Y. Liu, Z. Li, S. Pan, X. Zhu, and C. Zhang, "Heterogeneous graph neural networks for fraud detection," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, 2021, pp. 10 790–10 797.

[49] Z. Liu, X. Yu, Z. Li, M. Gao, and H. Fang, "Pick the right training data: Effective and efficient fraud detection on credit card applications," in *Proceedings of the ACM International Conference on Information and Knowledge Management*, 2021.

[50] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[51] J. R. McClean, S. Boixo, V. N. Smelyanskiy, R. Babbush, and H. Neven, "Barren plateaus in quantum neural network training landscapes," *Nature Communications*, vol. 9, no. 1, p. 4812, 2018.

[52] MDPI Computations Journal. "Objective over architecture: Fraud detection under extreme imbalance in bank account opening," Accessed: May 8, 2026. [Online]. Available: https://www.mdpi.com/2079-3197/13/12/290

[53] Nagoya University. "Modeling neuron firing: The leaky integrate-and-fire model," Accessed: May 8, 2026. [Online]. Available: https://www.math.nagoya-u.ac.jp/~richard/teaching/f2025/SML_Nicolle_1.pdf

[54] Q. Nasif, V. Balasubramanian, and Z. Dong, "Reinforcement-guided hyper-heuristic optimization for spiking neural network-based fraud detection," *Expert Systems with Applications*, vol. 265, p. 125 983, 2026.

[55] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[56] NeurIPS. "Adaptive fission: Post-training encoding for low-latency spike neural networks," Accessed: May 8, 2026. [Online]. Available: https://neurips.cc/virtual/2025/poster/120078

[57] NeurIPS Proceedings. "Temporal spike sequence learning via backpropagation for deep spiking neural networks," Accessed: May 8, 2026. [Online]. Available: https://proceedings.neurips.cc/paper/2020/file/8bdb5058376143fa358981954e7626b8-Paper.pdf

[58] NeuroCortex.AI. "Spiking neural network architectures," Accessed: May 8, 2026. [Online]. Available: https://medium.com/@neurocortexai/spiking-neural-network-architectures-e6983ff481c2

[59] Neuronal Dynamics. "Integrate-and-fire models," Accessed: May 8, 2026. [Online]. Available: https://neuronaldynamics.epfl.ch/online/Ch1.S3.html

[60] D. Nunes, H. Larochelle, and A. Vani, "Spiking neural networks: A survey," *IEEE Access*, vol. 10, pp. 51 185–51 206, 2022.

[61] NVIDIA AI Team. "Supercharging fraud detection in financial services with graph neural networks," Accessed: May 8, 2026. [Online]. Available: https://developer.nvidia.com/blog/supercharging-fraud-detection-in-financial-services-with-graph-neural-networks/

[62] M. Obushyni, V. Kovalchuk, and A. Sachenko, "Variational autoencoders for detecting anomalous and fraudulent transactions in financial systems," in *DECaT'2025, CEUR Workshop Proceedings*, vol. 4029, 2025, pp. 110–118.

[63] Open Neuromorphic. "Snntorch," Accessed: May 8, 2026. [Online]. Available: https://open-neuromorphic.org/neuromorphic-computing/software/snn-frameworks/snntorch/

[64] S. Pandey, V. K. Singh, and R. Kumar, "Graph neural networks for fraud detection: A survey," *Expert Systems with Applications*, vol. 207, p. 117 905, 2022.

[65] S. Parkar, R. Kazi, and A. Sawant, "Comparative study of deep learning explainability and causal AI for fraud detection," *International Journal of Scientific Research in Science, Engineering and Technology*, vol. 17, pp. 45–58, 2024.

[66] J. E. Pehle and C. F. Pedersen, *Norse: A library for deep learning with spiking neural networks*, https://norse.ai/, Norse Documentation, 2021.

[67] PNAS. "Brain-inspired neural circuit evolution for spiking neural networks," Accessed: May 8, 2026. [Online]. Available: https://www.pnas.org/doi/10.1073/pnas.2218173120

[68] J. Pombal, D. Alves, and J. Gama, "Understanding unfairness in fraud detection through model and data bias interactions," in *KDD Workshop on Machine Learning in Finance*, 2022.

[69] T. Pourhabibi, K.-L. Ong, Y. L. Boo, and S. Luo, "Graph-based fraud detection approaches: A systematic literature review," *Expert Systems with Applications*, vol. 208, p. 118 063, 2022.

[70] PubMed Central. "Dynamic population coding of category information in inferior temporal and prefrontal cortex," Accessed: May 8, 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC2544466/

[71] PubMed Central. "Exploring spiking neural networks: A comprehensive analysis of mathematical models and applications," Accessed: May 8, 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC10483570/

[72] PubMed Central. "Linear leaky-integrate-and-fire neuron model based spiking neural networks and its mapping relationship to deep neural networks," Accessed: May 8, 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC9448910/

[73] PubMed Central. "Spiking neural networks for physiological and speech signals: A review," Accessed: May 8, 2026. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC11362433/

[74] S. Raschka. "How does a training loop in pytorch look like?" Accessed: May 8, 2026. [Online]. Available: https://sebastianraschka.com/faq/docs/training-loop-in-pytorch.html

[75] RBC Borealis Research Group. "Detecting mule account fraud with federated learning," Accessed: May 8, 2026. [Online]. Available: https://rbcborealis.com/research-blogs/detecting-mule-account-fraud-with-federated-learning/

[76] ResearchGate. "Evolution of spiking neural networks," Accessed: May 8, 2026. [Online]. Available: https://www.researchgate.net/publication/391519403_Evolution_of_spiking_neural_networks

[77] ResearchGate. "Training process of an SNN with temporally-truncated local BPTT," Accessed: May 8, 2026. [Online]. Available: https://www.researchgate.net/figure/Training-process-of-an-SNN-with-temporally-truncated-local-BPTT-unfolded-in-time-In-this_fig2_370222638

[78] Ruhr University Bochum. "Rate coding and temporal coding in a neural network," Accessed: May 8, 2026. [Online]. Available: https://www.ini.rub.de/upload/file/1475592015_f6bc9c00c40aa459011f/krausetim-msc.pdf

[79] Sinabs. "Training by backpropagation through time (BPTT)," Accessed: May 8, 2026. [Online]. Available: https://sinabs.readthedocs.io/develop/tutorials/bptt.html

[80] H. Singh and B. Verma, "A comprehensive survey on financial fraud detection using machine learning," *Journal of Financial Crime*, 2024.

[81] snnTorch. "Quickstart," Accessed: May 8, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/quickstart.html

[82] snnTorch. "Snntorch api reference," Accessed: May 8, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/snntorch.html

[83] snnTorch. "Snntorch functional documentation," Accessed: May 8, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/snntorch.functional.html

[84] snnTorch. "Snntorch surrogate documentation," Accessed: May 8, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/snntorch.surrogate.html

[85] snnTorch. "Tutorial 1: Spike encoding," Accessed: May 8, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/tutorials/tutorial_1.html

[86] snnTorch. "Tutorial 2: The leaky integrate-and-fire neuron," Accessed: May 8, 2026. [Online]. Available: https://snntorch.readthedocs.io/en/latest/tutorials/tutorial_2.html

[87] snnTorch GitHub. "Latency encoding discussion," Accessed: May 8, 2026. [Online]. Available: https://github.com/jeshraghian/snntorch/discussions/36

[88] Spectra. "The history of spiking neural network," Accessed: May 8, 2026. [Online]. Available: https://spectra.mathpix.com/article/2022.09.00085/the-history-of-spiking-neural-network

[89] X. Sun, W. Chen, P. Zhao, and Y. Liu, "Objective function matters: A systematic evaluation of loss functions for imbalanced financial fraud detection," *Knowledge-Based Systems*, vol. 310, p. 112 589, 2025.

[90] A. Uwaoma, "Detecting bank account opening fraud using machine learning," M.S. thesis, Dublin Business School, 2024.

[91] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[92] P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, and P.-A. Manzagol, "Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion," *Journal of Machine Learning Research*, vol. 11, pp. 3371–3408, 2010.

[93] D. Wang, J. Lin, P. Cui, Q. Jia, Z. Wang, Y. Fang, Q. Yu, J. Zhou, S. Yang, and Y. Qi, "A semi-supervised graph attentive network for financial fraud detection," in *IEEE International Conference on Data Mining (ICDM)*, 2019, pp. 598–607.

[94] Wikipedia contributors. "Spiking neural network," Accessed: May 8, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Spiking_neural_network

[95] Z. Wu, S. Pan, G. Long, and C. Zhang, "SplitGNN: Spectral graph neural networks with adaptive splitting for fraud detection in heterophilic graphs," in *International Conference on Learning Representations (ICLR)*, 2023.

[96] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" In *International Conference on Learning Representations (ICLR)*, 2019.

[97] F. Zenke and S. Ganguli, "SuperSpike: Supervised learning in multilayer spiking neural networks," *Neural Computation*, vol. 30, no. 6, pp. 1514–1541, 2021.

[98] S. Zheng, Z. Feng, Q. Yan, and J. Tang, "Additive edge-graph attention network for fraud detection," in *ACM International Conference on Web Search and Data Mining*, 2020.